

FR-3

Voice Entry

พูดจบ "ข้าว 50 บาท" แล้วบันทึกให้
เข้าใจระบบทั้งหมดตั้งแต่ศูนย์ จนอธิบายโค้ดและตอบ
คำถามอาจารย์ได้ เหมือนเป็นคนสร้างระบบนี้เอง

ภาค ก · พื้นฐานทั้งระบบ (สำหรับคนไม่เคยเขียนโค้ด)

ภาค ข · FR-3 ละเอียดทุกส่วน

โค้ดจริงพร้อมคำอธิบายทีละบรรทัด



วิธีใช้เล่มนี้

เล่มนี้ทำให้คนที่รับผิดชอบ FR-3 Voice Entry เข้าใจระบบทั้งหมด ตั้งแต่ "แอปนี้สร้างด้วยอะไร" ไปจนถึง "บรรทัดนี้ของโค้ดทำอะไร" แม้ไม่เคยเขียนโปรแกรมมาก่อน

เล่มนี้แบ่งเป็น 2 ภาค

i ภาค ก · พื้นฐานทั้งระบบ

เหมือนกันทุกเล่ม (FR-1 ถึง FR-6) สอนตั้งแต่ศูนย์: เครื่องมือที่ใช้ (VS Code, Node.js, Git), ภาษา TypeScript, React, การทำแอปและเว็บจากโค้ดชุดเดียว, เซิร์ฟเวอร์ Supabase, การเชื่อม AI, การออกแบบหน้าจอ (UX/UI), การทดสอบ และ GitHub

★ ภาค ข · FR-3 Voice Entry

เฉพาะ FR ของเล่มนี้ เขียนในมุม "ฉันเป็นคนสร้างระบบนี้" มีหน้าจอจริง หลักการ ตัวอย่างคิดมือ โค้ดจริงพร้อมคำอธิบายทีละบรรทัด การทดสอบ เหตุผล การออกแบบ คำถามอาจารย์ แบบฝึกหัด และสรุปปต์นำเสนอ

เปิดโค้ดใน VS Code ได้ทุกจุดที่เห็นในเล่ม

โค้ดทุกก้อนในเล่มนี้ ดึงมาจากไฟล์จริงในโปรเจกต์ ไม่ได้พิมพ์ใหม่ เลขบรรทัดทางซ้ายจึงตรงกับในไฟล์ทุกตัว ทุกก้อนมีแถบล่างแบบนี้:

src/domain/runway.ts

ตัวอย่าง · บรรทัด 17-23

```
17 * so the status is "no_spending" and no date is shown.
18 */
19 import { addDays, bkkDayKey, daysBetween } from './dates';
20 import { confirmedOnly } from './summary';
21 import type { Transaction } from './types';
22
23 export const DEFAULT_FLOOR_SATANG = 50_000; // ฿500
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:17

- 1 **แตะชื่อไฟล์ หรือ "เปิดใน VS Code (เว็บ)"** ระบบจะเปิด VS Code ในเบราว์เซอร์ (vscode.dev) พร้อมโปรเจกต์ MindPay และกระโดดไปที่บรรทัดนั้นเลย ไม่ต้องติดตั้งอะไร ใช้ได้ทั้งคอมและแท็บเล็ต ครั้งแรกอาจถามให้เข้าสู่ระบบ GitHub ก่อน
- 2 **แตะ "GitHub"** เปิดไฟล์เดียวกันบนเว็บไซต์ GitHub และไฮไลต์บรรทัดเหล่านั้น (ใช้ได้บนมือถือ)
- 3 **ถ้าโคลนโปรเจกต์ลงเครื่องแล้ว** (วิธีอยู่ในบท A3) เปิด VS Code กด **Ctrl+P** (Mac: **Cmd+P**) แล้วพิมพ์ข้อความตัวหน้าที่มุมขวา เช่น **src/domain/runway.ts:17** แล้วกด Enter จะเปิดไฟล์และไปที่บรรทัด 17 ทันที

ชื่อไฟล์สีเขียวขีดเส้นใต้ในเนื้อหา เช่น `src/domain/runway.ts` ก็แตะเปิดได้เหมือนกัน

i ลิงก์ล็อกไว้ที่เวอร์ชันของเล่มนี้

ทุกลิงก์ชี้ไปที่โค้ดเวอร์ชันวันที่ 3 ต.ค. 2569 (commit 646b485) ซึ่งเป็นเวอร์ชันที่ใช้เขียนเล่มนี้ ถ้าทีมแก้ไขโค้ดหลังจากนี้ ลิงก์ในเล่มยังเปิดเวอร์ชันเดิมที่เลขบรรทัดตรงกับในเล่ม ถ้าอยากดูเวอร์ชันล่าสุด ให้เปลี่ยนไปที่ branch main ใน VS Code หรือ GitHub

อ่านคำอธิบายโค้ดยังไ

หลังโค้ดแต่ละก้อนจะมีตาราง "อ่านทีละบรรทัด" คอลัมน์ซ้ายคือเลขบรรทัด คอลัมน์ขวาคือคำอธิบายเป็นภาษาคน ให้เปิดโค้ดกับตารางคู่กัน อ่านบรรทัดในโค้ดก่อน แล้วค่อยอ่านคำอธิบาย ถ้าเจอคำที่ไม่รู้จัก ให้กลับไปดูบท A4 ถึง A7 ซึ่งสอนพื้นฐานภาษาตั้งแต่ศูนย์

บรรทัด	ความหมาย
17	ตัวอย่าง: คำอธิบายของบรรทัด 17 จะอยู่แถวนี้
18–20	ถ้าหลายบรรทัดทำงานเรื่องเดียวกัน จะอธิบายรวมกันเป็นช่วง

กล่องสีต่าง ๆ ในเล่ม

≈ เปรียบเทียบ

เทียบกับของในชีวิตจริง ให้เห็นภาพก่อนลงรายละเอียด

★ จำให้ได้

เรื่องสำคัญที่อาจารย์มักถาม

! ระวัง

จุดที่คนมักเข้าใจผิด หรือนึกที่เคยเกิดขึ้นจริง

i เพิ่มเติม

รายละเอียดเสริม ข้ามได้ถ้ารีบ

ถ้ามีเวลาน้อย อ่านตามนี้

เวลาที่มี	อ่านอะไร
15 นาที	บท A1 (MindPay คืออะไร), บท A2 (แผนที่ระบบ), แล้วภาค ข บทที่ 1 และบทสุดท้าย (สรุปต้นน้ำเสนอ)
1–2 ชั่วโมง	ภาค ก บท A1, A2, A4, A9 แล้วอ่านภาค ข ทั้งหมดยกเว้นตารางโค้ดทีละบรรทัด
เตรียมตอบอาจารย์	ภาค ข บท "ทำไมฉันออกแบบแบบนี้", "ข้อจำกัด", "คำถามที่อาจารย์อาจถาม" และ "แบบฝึกหัด"
อยากอธิบายโค้ดได้จริง	อ่านทั้งเล่มตามลำดับ เปิด VS Code คู่กัน และลองทำแบบฝึกหัดโดยไม่ดูเฉลย

i ตัวเลขในเล่ม

ตัวอย่างขีดมือและเฉลยแบบฝึกหัดทุกข้อ ตรวจโดยรันฟังก์ชันจริงของแอปแล้ว ภาพหน้าจอตายจากเว็บเวอร์ชันจริงในโหมดทดลอง (ข้อมูลตัวอย่าง) วันที่ 9 ต.ค. 2569 ข้อมูลตัวอย่างสร้างตามวันที่เปิดแอป ถ้าเปิดดูเองตัวเลขจะต่างไปเล็กน้อย แต่หลักการเหมือนเดิมทุกอย่าง

สารบัญ

วิธีใช้เล่มนี้	1
เล่มนี้แบ่งเป็น 2 ภาค	1
เปิดโค้ดใน VS Code ได้ทุกจุดที่เห็นในเล่ม	1
อ่านคำอธิบายโค้ดอย่างไร	2
กล่องสีต่าง ๆ ในเล่ม	2
ถ้ามีเวลาน้อย อ่านตามนี้	2
พื้นฐานทั้งระบบ MINDPAY	9
MindPay คืออะไร และทีมเราสร้างอะไร	10
ปัญหาที่ทีมเราอยากแก้	10
6 ฟังก์ชันหลัก (Functional Requirements)	10
หน้าต่างแอปโดยรวม	11
ใช้แอปได้ 3 แบบ	12
ตัวเลขของโปรเจกต์	12
คำศัพท์ที่จะเจอบ่อยในเล่มนี้	13
แผนที่ระบบ: ชิ้นส่วนทั้งหมดต่อกันอย่างไร	14
ระบบมี 4 ส่วนใหญ่	14
ข้างในแอป: แบ่งโค้ดเป็น 5 ชั้น	14
ตามข้อมูล 1 รายการ: ตั้งแต่กดบันทึก จนถึงเลขบนหน้าหลักเปลี่ยน	15
ตามข้อมูลที่ใช้ AI: สแกนสลิป 1 ใบ	19
เครื่องมือ: เขียนโค้ดที่ไหน ใช้โปรแกรมอะไร และ VS Code ตั้งแต่ศูนย์	21
กล่องเครื่องมือของทีมเรา	21
ติดตั้งเครื่องมือ (ถ้าครั้งเดียว)	21
เอาโค้ด MindPay มาไว้ในเครื่อง	21
ไฟล์ package.json: บัตรประชาชนของโปรเจกต์	22
คำสั่งที่ใช้บ่อย	25
รู้จักหน้าจอส VS Code	25
คีย์ลัดที่ทำให้อ่านโค้ดเร็วขึ้น 10 เท่า	26
ส่วนเสริม (Extensions) ที่แนะนำ	27
ไม่อยากติดตั้งอะไร: ใช้ VS Code บนเว็บ	27
ฝึกใช้: ตามรอย Money Runway ใน 5 นาที	27
พื้นฐานการเขียนโปรแกรม สำหรับคนที่ไม่เคยเขียนเลย	28
โปรแกรมคืออะไร	28
1. ค่า (value) และชนิดของค่า	28
2. ตัวแปร: กล่องที่มีชื่อ	28
3. ตัวดำเนินการ: คำนวณและเปรียบเทียบ	29
4. เชื้อไข if: ถ้า... ให้ทำ...	29
5. ฟังก์ชัน: เครื่องจักรรับของเข้า ส่งของออก	30
6. Array: รายการของหลายค่า	30
7. Object: ของ 1 ชิ้นที่มีหลายช่อง	32
8. วงซ้ำ (loop)	33
9. ข้อความ (string) ที่ใช้บ่อย	33
10. คณิตศาสตร์ที่ใช้	33
แบบฝึกหัด A4	33

TypeScript: ภาษาที่ทีมเราใช้ และชนิดข้อมูลของ MindPay	35
ทำไมต้อง TypeScript	35
การบอชนิดข้อมูล (type annotation)	35
ไฟล์ชนิดข้อมูลกลาง: src/domain/types.ts	35
ชนิดข้อมูลแบบพิเศษที่เจอบ่อย	37
import และ export: แคร่โค้ดระหว่างไฟล์	38
งานที่ต้องรอ: async, await และ Promise	39
จัดการความผิดพลาด: try, catch, throw	39
ทำไมเก็บเงินเป็นสตางค์ (จำนวนเต็ม)	40
อ่านโค้ดจริง: แปลงข้อความที่ผู้ใช้พิมพ์เป็นสตางค์	40
Regular expression (regex) อ่านยังไ้	42
แปลงสตางค์กลับเป็นข้อความบาท	43
React: วาดหน้าจอจากข้อมูล	45
แนวคิดหลัก: หน้าจอเป็นผลลัพธ์ของข้อมูล	45
Component: ชิ้นส่วนหน้าจอที่เป็นฟังก์ชัน	45
State: ข้อมูลที่เปลี่ยนได้ และทำให้หน้าจอวาดใหม่	47
Hook ที่ใช้ในโปรเจกต์	48
Context: แคร่ข้อมูลให้ทุกหน้าโดยไม่ต้องส่งต่อเป็นทอด ๆ	50
ทุกอย่างประกอบกัน: หน้าจอใช้ข้อมูลกลางยังไ้	51
React Native และ Expo: โค้ดชุดเดียว ได้ทั้งแอปและเว็บ	53
React Native คืออะไร	53
Expo: ชุดเครื่องมือที่ทำให้ React Native ง่ายขึ้น	54
Expo Router: ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า	54
อ่านโค้ด: _layout.tsx โครงของทั้งแอป	56
แถบเมนูล่าง: (tabs)/_layout.tsx	60
โค้ดที่ต่างกันระหว่างมือถือกับเว็บ	61
จากโค้ด สู่เว็บและแอปที่ผู้ใช้ลง	63
วิธีอ่านโค้ด MindPay ด้วยตัวเอง	65
ขั้นตอนอ่านไฟล์ที่ไม่เคยเห็น	65
รูปแบบที่เจอซ้ำทั้งแอป (ตารางโกง)	65
ฟังก์ชันวันเวลาที่ต้องรู้ (ทุก FR ใช้)	66
แผนที่: อยากรู้อะไรนี้ ไปเปิดไฟล์ไหน	67
เซิร์ฟเวอร์และฐานข้อมูล: Supabase	69
ฐานข้อมูลคืออะไร	69
Supabase ให้อะไรทีมเราบ้าง	69
Migration: ไฟล์สร้างฐานข้อมูล	70
Row Level Security (RLS): หัวใจความปลอดภัย	73
กฎแฉ 2 แบบ: อันไหนอยู่ในแอปได้ อันไหนห้าม	75
แอปต่อ Supabase ยังไ้: src/data/supabase.ts	75
การสมัครและเข้าสู่ระบบ	77
Repo: ชั้นที่ซ่อนว่าข้อมูลอยู่ที่ไหน	78
เปิดแอปแล้วเกิดอะไรขึ้น: AppProvider เลือกโหมด	83
การเชื่อม AI: ทำไมต้องผ่านเซิร์ฟเวอร์ และทำงานยังไ้	86
AI ในแอปคืออะไร	86
ทำไมแอปไม่เรียก AI เอง	86
เส้นทางของคำถาม 1 ข้อถึงน้องกล่า	86

ฝั่งแอป: src/services/coach.ts	87
ฝั่งเซิร์ฟเวอร์: Edge Function coach ที่ละบัสกัถ	88
ตัวช่วยกลาง: supabase/functions/_shared/common.ts	90
เลือก AI ตัวไหน: Claude หรือ Gemini	93
ส่งคำขอไปที่ Claude: callClaude	95
ตั้งค่า AI (ผู้ดูแลทำครั้งเดียว)	98
หลักการใช้ AI อย่างรับผิดชอบที่ทีมเราใช้	98
UX/UI: การออกแบบหน้าจอและ Design System	99
หลักการออกแบบของ MindPay	99
Design System คืออะไร	99
สี: Green + Gold	99
ความอ่านง่ายของสี (WCAG) และเทสที่บังคับ	101
ตัวอักษร	103
ชิ้นส่วนสำเร็จรูป: ปุ่ม (Button)	103
Accessibility: ทุกคนใช้ได้	106
แอนิเมชัน: สวยแต่ไม่กินแบต	106
เช็กลิสต์ก่อนเพิ่มหน้าจอใหม่ (จาก DESIGN_SYSTEM.md)	106
การทดสอบ: พิสูจน์ว่าระบบทำงานถูก	108
ทำไมต้องเขียนเทส	108
Unit test: ตรวจสอบฟังก์ชันทีละตัว	108
E2E: ทดสอบทั้งแอปเหมือนคนใช้จริง	110
Manual test: ทดสอบบนมือถือจริง	113
Traceability: ผูก Requirement → โค้ด → เทส	114
ตรวจสอบคุณภาพอื่น ๆ	114
Git และ GitHub: เก็บโค้ด ทำงานเป็นทีม และ build อัตโนมัติ	116
Git: ระบบบันทึกประวัติ	116
ใช้ Git ประจำวัน	116
ประวัติของ MindPay UU GitHub	117
GitHub Actions: หุ่นยนต์ที่ทำงานทุกครั้งที่เรา push	117
Secrets UU GitHub	122
ไฟล์เอกสารใน repo	122
6 FR ทำงานร่วมกันอย่างไร และคำถามระดับระบบ	124
สายข้อมูลของทั้งระบบ	124
ใครใช้ของใคร	124
ไฟล์ที่หลาย FR ใช้ร่วมกัน (แก้แล้วกระทบหลายคน)	124
คำถามระดับระบบ ที่อาจารย์ถามใครก็ได้	125
คำศัพท์รวม	126
FR-3 VOICE ENTRY	128
FR-3 ในหน้าเดียว	129
เข้าใจได้แค่ไหน (ตัวอย่างที่รันกับโค้ดจริงแล้ว)	129
3 ข้อที่ต้องจำ	129
ปัญหาที่ฉันทอยากแก้	131
เป้าหมายที่ฉันทตั้งไว้ตอนออกแบบ	131
Requirement และเงื่อนไขผ่าน	132
ประวัติสั้น ๆ ใน Git	132

เส้นทางของ 1 ประโยค	133
แผนที่โค้ดของ FR-3	133
ทัวร์หน้าจอ: เปิดหน้าพุดจก และฟังเสียง	135
ทัวร์หน้าจอ: แอปเข้าใจประโยคยังไ	137
ทัวร์หน้าจอ: แกด้อนบันทึก แล้วบันทึก	139
ตัวเลขหลังบันทึก (ตรวจด้วยการคำนวณ)	139
ทัวร์หน้าจอ: เครื่องที่ฟังเสียงไม่ได้	141
ข้อความเมื่อฟังไม่สำเร็จ	141
หลักการ: จากประโยคเป็นรายการใน 6 ชั้น	142
หลักการ: ชั้นที่ 1–3 ทำให้ข้อความเป็นมาตรฐาน	143
หลักการ: อ่านเลขคำไทยแบบที่คนพูด	144
หลักการ: ชั้นที่ 4–6 หาเงิน แบ่งชื่อ ตัดสินรายการ	145
ชั้นที่ 4: จำนวนเงินคือเลขแบบไหน	145
ชั้นที่ 5: แบ่งชื่อ	145
ชั้นที่ 6: รายรับหรือรายจ่าย	145
ชั้นที่ 6 (ต่อ): ชื่อ หมวด และวัน	145
อ่านโค้ด: voice.ts ตอนที่ 1 – ชนิดข้อมูลและเลขคำไทย	147
ส่วนที่ 1: คอมเมนต์หัวไฟล์และชนิดข้อมูล (บรรทัด 1–30)	147
ส่วนที่ 2: คำตัวเลขไทย (บรรทัด 32–41)	148
ส่วนที่ 3: thaiNumberWords — แปลงคำเป็นเลข (บรรทัด 43–90)	148
ไล่โค้ดที่ละโทเคน (trace)	150
อ่านโค้ด: voice.ts ตอนที่ 2 — คำ หมวด และชื่อ	152
ส่วนที่ 1: หน่วย คำรายรับ คำจ่ายเงิน (บรรทัด 92–112)	152
ส่วนที่ 2: คำเชื่อม และคำลงท้าย (บรรทัด 113–120)	153
ส่วนที่ 3: หมวดจากคำพูด (บรรทัด 122–145)	154
ส่วนที่ 4: ทำความสะอาดชื่อ (บรรทัด 147–153)	155
อ่านโค้ด: voice.ts ตอนที่ 3 — ทั้งประโยค	156
ส่วนที่ 1: ป้ายวัน (บรรทัด 155–163)	156
ส่วนที่ 2: normalizeMarked — ทำความสะอาดข้อความ (บรรทัด 165–186)	156
ส่วนที่ 3: normalizeSpoken (บรรทัด 219–224)	158
ส่วนที่ 4: parseSpokenEntry — หาจำนวนเงิน (บรรทัด 226–241)	159
ส่วนที่ 5: parseSpokenEntry — แบ่งชื่อและสร้างรายการ (บรรทัด 242–268)	160
ไล่โค้ดทั้งประโยค: "เมื่อวานคำรต 40 วันนีข้าว 50"	161
อ่านโค้ด: หน้าจอ voice.tsx ตอนที่ 1 — ความจำและการกระทำ	162
ส่วนที่ 1: คอมเมนต์ import และค่าคงที่ (บรรทัด 1–43)	162
ส่วนที่ 2: แต่งเสียงที่เดินตามเสียงพูด (บรรทัด 45–78)	163
ส่วนที่ 3: ความจำของหน้า และรายการที่จะบันทึก (บรรทัด 80–114)	164
ส่วนที่ 4: แถบโมค (บรรทัด 116–141)	166
ส่วนที่ 5: บันทึกทั้งหมด (บรรทัด 143–178)	167
อ่านโค้ด: หน้าจอ voice.tsx ตอนที่ 2 — หน้าตา	169
ส่วนที่ 6: หัวหน้าและการ์ดโมค (บรรทัด 180–250)	169
ส่วนที่ 7: ข้อความแจ้ง และช่องพิมพ์ (บรรทัด 252–281)	171

ส่วนที่ 8: รายการที่จะบันทึก (บรรทัด 283–346)	171
อ่านโค้ด: ฟังเสียงบนมือถือ <code>speech.ts</code> ทั้งไฟล์	174
ส่วนที่ 1: หาโมดูลแบบไม่บังคับ (บรรทัด 1–33)	174
ส่วนที่ 2: คำช่วย มีตัวฟังใหม่ และขอสิทธิ์ (บรรทัด 35–55)	175
ส่วนที่ 3: <code>listen</code> — เริ่มฟัง (บรรทัด 57–96)	176
อ่านโค้ด: ฟังเสียงบนเว็บ <code>speech.web.ts</code> ทั้งไฟล์	178
อ่านโค้ด: การตั้งค่าใน <code>app.json</code>	181
การทดสอบ FR-3	182
Unit test: <code>voice.test.ts</code>	182
Unit test: TC-80 (เคสจากการตรวจสอบ repository และผู้ตรวจอิสระ)	184
E2E: พุดจดในเบราว์เซอร์จริง (ข้อ 19)	185
MT-12: ทดสอบบนมือถือจริง	187
FR-3 เชื่อมกับ FR อื่นอย่างไร	188
ทำไมฉันออกแบบแบบนี้	189
ข้อจำกัด และสิ่งที่จะทำต่อ	190
คำถามที่อาจารย์อาจถาม พร้อมคำตอบ	191
แบบฝึกหัดเช็คความเข้าใจ	192
เฉลย	192
สคริปต์นำเสนอ 1 นาที และคำศัพท์	193
คำศัพท์ของ FR-3	193

ภาค ก

พื้นฐานทั้งระบบ MindPay

สอนตั้งแต่ศูนย์ สำหรับคนที่ไม่เคยเขียนโค้ด ภาคนี้เหมือนกับทุกเล่ม เพราะไม่ว่าจะรับผิดชอบ FR ไหน ทุกคนต้องเข้าใจว่าแอปทั้งตัวสร้างด้วยอะไร ทำงานที่ไหน และชิ้นส่วนต่างๆ คอยกันยังไง

- เครื่องมือ ภาษา TypeScript และ React ตั้งแต่พื้นฐาน
- โค้ดชุดเดียวกลายเป็นทั้งเว็บและแอป Android ได้ยังไง
- เซิร์ฟเวอร์ Supabase ฐานข้อมูล และความปลอดภัย
- การเชื่อม AI, การออกแบบ UX/UI, การทดสอบ และ GitHub

MindPay คืออะไร และทีมเราสร้างอะไร

ก่อนดูโค้ดสักบรรทัด ต้องรู้ก่อนว่าแอปนี้แก้ปัญหอะไร ใครใช้ และมีส่วนไหนบ้าง

ปัญหาที่ทีมเราอยากแก้

- นักศึกษาและคนเพิ่งเริ่มทำงาน (First Jobber) ส่วนใหญ่รับและจ่ายเงินผ่านแอปธนาคารบนมือถือ ทุกครั้งที่โอนเงินจะได้ "สลิป" เป็นรูปเก็บไว้ในแกลเลอรี แต่ปัญหาคือ
- **ขี้เกียจจด** แอปจดรายรับรายจ่ายทั่วไปต้องพิมพ์เองทุกรายการ ทั้งที่ข้อมูลอยู่ในสลิปแล้ว คนจึงเลื่อนการจดไปเรื่อย ๆ จนข้อมูลไม่ครบ
 - **เห็นแค่ยอดเงิน แต่ไม่รู้ว่าพอหรือเปล่า** แอปธนาคารบอกว่าเหลือ ฿3,500 แต่ไม่ได้บอกว่าเงินก้อนนี้จะอยู่ได้ถึงวันไหน
 - **รู้ตัวตอนเงินหมดแล้ว** ไม่มีใครช่วยเตือนหรือช่วยคิดก่อนจ่าย

ทีมเราจึงสร้าง **MindPay** แอปการเงินส่วนตัวที่ "ไม่ได้แค่จดว่าใช้ไปเท่าไร แต่บอกว่าเงินที่มีจะพอใช้อีกกี่วัน และช่วยคิดก่อนจ่าย" (ประโยคแรกของไฟล์ `README.md` ในโปรเจกต์)

6 ฟังก์ชันหลัก (Functional Requirements)

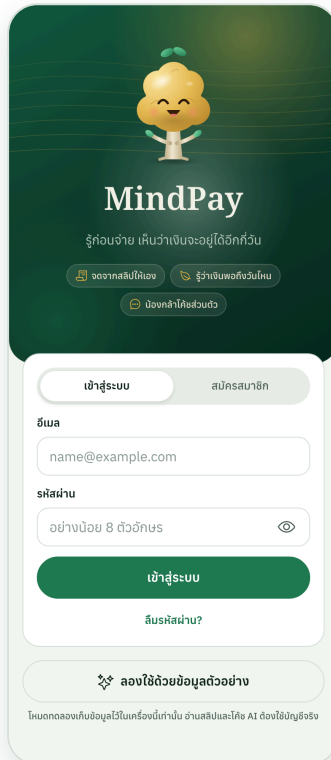
ในวิชา Software Engineering เราเรียกสิ่งที่ระบบ "ต้องทำได้" ว่า **Functional Requirement** หรือ **FR** แต่ละ FR มีเจ้าของ 1 คนในกลุ่ม MindPay มีทั้งหมด 6 ข้อ

FR	ชื่อ	ทำอะไร (ภาษาคน)	อยู่ตรงไหนในแอป
FR-1	Transaction Management	จดรายการเอง แก้ไข เลือกหมวด และลบรายการได้	แท็บ "รายการ" และหน้า "จดรายการ"
FR-2	Overview Dashboard	สรุปรายรับ รายจ่าย ยอดคงเหลือ และยอดแยกตามหมวด	แท็บ "หน้าหลัก"
FR-3	Voice Entry	พูดว่า "ข้าว 50 บาท" แล้วแอปจดให้	ปุ่มไมค์ทองบนหน้าหลัก
FR-4	Automatic Slip Detection	หาสลิปในแกลเลอรีเองแล้วให้ AI อ่านยอดเงิน วันที่ และชื่อผู้รับ	ปุ่มทวงกลางแถบล่าง "สแกนสลิป" และสแกนอัตโนมัติตอนเปิดแอป
FR-5	AI Coach (น้องกล้า)	โค้ช AI ที่อธิบายตัวเลขการเงินเป็นภาษาคน และพูดออกเสียงได้	แท็บ "โค้ช"
FR-6	Money Runway	คำนวณว่าเงินที่มีจะพอใช้อีกกี่วัน และหมดวันไหน	แท็บ "เงินพอถึง" และการ์ดเขี้ยวบนหน้าหลัก

★ **จำให้ได้: ทุก FR ต่อกันเป็นสายเดี่ยว**

FR-1, FR-3 และ FR-4 คือ **ทางเข้าของข้อมูล** (จุดเอง พุดจุด สแกนสลิป) ส่วน FR-2 และ FR-6 คือ **สมองที่คิดตัวเลข** (ยอดคงเหลือ และเงินพอถึงวันไหน) และ FR-5 คือ **ปากที่อธิบาย** ตัวเลขเหล่านั้นให้ผู้ฟัง ถ้า FR ไหนฟัง FR ที่อยู่ปลายน้ำจะได้ตัวเลขผิดตามไปด้วย

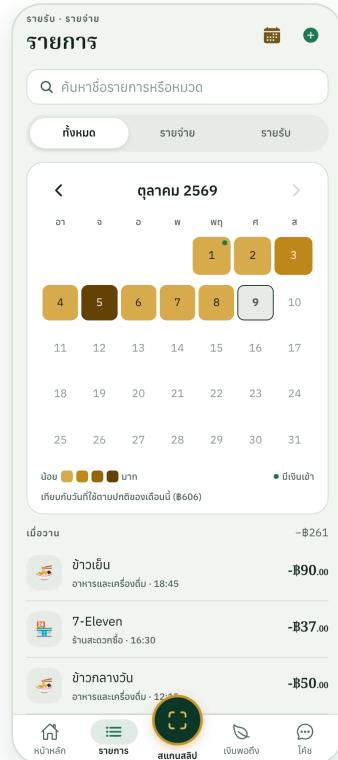
หน้าตาแอปโดยรวม



หน้าต้อนรับ: สมัคร เข้าสู่ระบบ หรือ "ลองใช้ด้วยข้อมูลตัวอย่าง"



หน้าหลัก (FR-2): ยอดคงเหลือ, การ์ดเงินพอถึง (FR-6), บอกลำไย พุดสรุป



แท็บรายการ (FR-1): รายการ ทั้งหมด ค้นหา กรองรายรับ/รายจ่าย



ด้านล่างของทุกหน้ามี **แถบเมนู 5 ช่อง**: หน้าหลัก, รายการ, ปุ่มทวง "สแกนสลิป" ตรงกลาง, เงินพอถึง และไอซ์ ปุ่มสแกนอยู่ตรงกลางและใหญ่ที่สุด เพราะเป็นสิ่งที่ผู้ใช้ทำบ่อยที่สุด (โค้ดอยู่ใน `src/app/(tabs)/_layout.tsx` ซึ่งจะอ่านทีละบรรทัดใน un A7)

ใช้แอปได้ 3 แบบ

แบบ	เปิดยังไ้	ต่างกันยังไ้
เว็บ	https://2550expo-tech.github.io/Socrates-and-Skeletons/ (เปิดในเบราว์เซอร์ได้ทั้งคอม iPhone และ Android)	ทำได้ทุกอย่าง ยกเว้นสแกนแคเลเลอร์ทั้งหมดเอง (บนเว็บต้องกดเลือกรูปสลิป)
แอป Android (.apk)	ดาวน์โหลดจากหน้า Releases ของ GitHub ชื่อ latest-apk	สแกนแคเลเลอร์อัตโนมัติ พุดจดย่อยด้วยไมค์ นื่องคล้ายพุดออกเสียง
Expo Go	แอปตัวช่วยตอนพัฒนา สแกน QR จากคอมของนักพัฒนา	ใช้ทดสอบระหว่างเขียนโค้ดเท่านั้น

และไม่ว่าเปิดแบบไหน ผู้ใช้เลือกได้ 2 โหมด

- **บัญชีจริง** สมัครด้วยอีเมล ข้อมูลเก็บบนเซิร์ฟเวอร์ (Supabase) เปิดจากเครื่องไหนก็เห็นข้อมูลเดียวกัน ใช้ AI อ่านสลิปและไอซ์ได้
- **โหมดทดลอง** แตะ "ลองใช้ด้วยข้อมูลตัวอย่าง" แอปสร้างรายการตัวอย่างประมาณ 75 วันให้ เก็บไว้ในเครื่องอย่างเดียว ไม่ต้องสมัคร ใช้ตอนฟรีเซนต์หรือลองเล่น (AI ยังใช้ไม่ได้ในโหมดนี้)

ตัวเลขของโปรเจกต์

~18,700

บรรทัดโค้ดของแอปและเซิร์ฟเวอร์ (ไม่นับ
เทสต์)

157

unit test ผ่านทั้งหมด

145

ข้อตรวจในเบราวเซอร์ (E2E) ผ่านทั้งหมด

โฟลเดอร์	มีอะไร	ไฟล์	บรรทัด
src/app	หน้าจอทุกหน้า	19	5,266
src/ui	ชิ้นส่วนหน้าตา: ปุ่ม การ์ด สี กราฟ ภาพน้องกล้า	31	6,042
src/domain	กฎและการคำนวณทั้งหมด (สมองของแอป)	24	3,457
src/data	การเก็บและโหลดข้อมูล	9	1,019
src/services	งานที่ต้องคุยกับโลกภายนอก: แกลเลอรี ไมค์ เสียง AI	15	1,776
supabase	ฐานข้อมูลและโค้ดฝั่งเซิร์ฟเวอร์	8	1,160
e2e, scripts	เทสต์ในเบราวเซอร์ และสคริปต์ช่วย (ไอคอน ภาพหน้าจอ)	10	1,867

นับจากโค้ดเวอร์ชัน 3 ต.ค. 2569 ไฟล์ .ts, .tsx, .sql และ .mjs ไม่รวมไฟล์เทสต์ (ไฟล์เทสต์อีกประมาณ 2,000 บรรทัด)

≈ เปรียบแอปเป็นร้านอาหาร

หน้าจอ (src/app, src/ui) คือหน้าร้านที่ลูกค้าเห็น **กฎการคำนวณ** (src/domain) คือสูตรอาหารของเชฟ **ข้อมูล** (src/data) คือตู้เย็นเก็บวัตถุดิบ **เซิร์ฟเวอร์** (supabase) คือโถงกลางที่ทุกสาขาใช้ร่วมกัน และ **AI** คือผู้เชี่ยวชาญภายนอกที่เราโทรถามเมื่อต้องอ่านสลิปหรืออธิบายตัวเลข บทต่อไปจะวาดแผนที่ว่าทั้งหมดนี้ต่อกันยังไง

คำศัพท์ที่จะเจอบ่อยในเล่มนี้

คำ	ความหมายแบบง่าย
รายการ (Transaction)	เงินเข้าหรือเงินออก 1 ครั้ง เช่น ข้าวมันไก่ ฿50
แบบร่าง (Draft)	รายการจากสลิปที่ AI อ่านแล้วแต่ยังไม่แน่ใจ รอผู้ตรวจ ยังไม่นับในยอดเงิน
ยืนยันแล้ว (Confirmed)	รายการที่นับในยอดเงินแล้ว
สตางค์ (satang)	หน่วยที่แอปใช้เก็บเงินในระบบ 100 สตางค์ = 1 บาท (เหตุผลอยู่บท A5)
เส้นเงินสำรอง (Floor)	เงินขั้นต่ำที่ผู้ใช้ไม่อยากให้ต่ำกว่า ค่าเริ่มต้น ฿500
น้องกล้า	มาสคอตของแอป เป็นต้นกล้าจากต้นไม้เงิน และเป็นโคชิ AI ใน FR-5
Supabase	บริการเซิร์ฟเวอร์ที่ให้ทั้งฐานข้อมูล ระบบบัญชีผู้ใช้ และที่รันโค้ดฝั่งเซิร์ฟเวอร์
Expo	เครื่องมือที่ทำให้โค้ดชุดเดียวเป็นได้ทั้งแอป Android, iPhone และเว็บ

แผนที่ระบบ: ชิ้นส่วนทั้งหมดต่อกันยังงี้

ภาพใหญ่ของ MindPay ตั้งแต่นิ้วแตะจอ จนข้อมูลไปถึงฐานข้อมูลบนคลาวด์และกลับมาเป็นตัวเลขบนหน้าจอ

ระบบมี 4 ส่วนใหญ่

① แอปบนมือถือหรือเว็บ (โค้ดในโฟลเดอร์ src/)

สร้างด้วย Expo + React Native ภาษา TypeScript โค้ดชุดเดียวได้ทั้งแอป Android, iPhone และเว็บ ทำหน้าที่แสดงหน้าจอ รับการแตะ คำนวณตัวเลขทั้งหมด (ยอดคงเหลือ, Runway) อ่าน QR บนสลิป และแปลงเสียงพูดเป็นรายการ

↕ อินเทอร์เน็ต (HTTPS) ด้วย "publishable key" ซึ่งเป็นกุญแจสาธารณะ

② Supabase: ฐานข้อมูล + ระบบบัญชีผู้ใช้ (โค้ดใน supabase/migrations/)

ฐานข้อมูล PostgreSQL เก็บตาราง profiles, transactions, savings_goals, ai_usage ทุกตารางเปิด Row Level Security (RLS) ทำให้แต่ละคนอ่านและแก้ไขได้เฉพาะแถวของตัวเอง ระบบ Auth จัดการสมัคร เข้าสู่ระบบ ยืนยันอีเมล

↕ เฉพาะงานที่ต้องใช้ AI

③ Supabase Edge Functions: โค้ดฝั่งเซิร์ฟเวอร์ (โค้ดใน supabase/functions/)

ฟังก์ชัน 2 ตัว: **parse-slip** (อ่านสลิป, FR-4) และ **coach** (โค้ชน้องกล้า, FR-5) ตรวจสอบว่าผู้ใช้เข้าสู่ระบบจริง นับโควตาต่อวัน แล้วคือ "กุญแจลับ" ของ AI ไว้เรียกแทนแอป

↕ API key ลับ (อยู่บนเซิร์ฟเวอร์เท่านั้น)

④ AI ภายนอก: Claude (Anthropic) หรือ Gemini (Google)

อ่านรูปสลิปแล้วตอบเป็น JSON และเขียนคำอธิบายของโค้ช ระบบเลือกใช้ตัวที่มีกุญแจ ถ้ามีทั้งสองตัวจะใช้ Claude ก่อน

★ **จำให้ได้: 4 กฎของสถาปัตยกรรม (จาก README)**

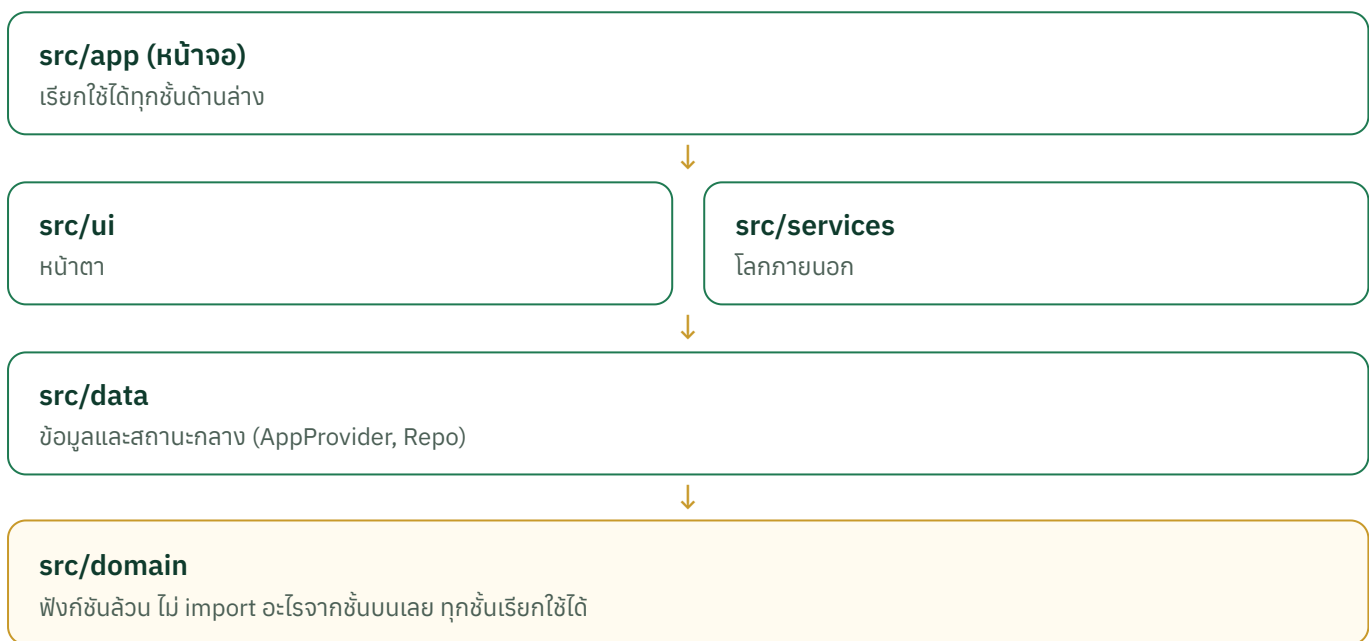
- เงินเก็บเป็นสตางค์ (จำนวนเต็ม) ไม่มีปัญหาทศนิยมเพี้ยน
- เวลาอิงเวลาไทย (UTC+7) ทุกจุด ทั้งการจัดกลุ่มรายวันและการอ่านวันที่บนสลิป
- สลิปที่ AI อ่านได้ไม่ชัดเป็น "แบบร่าง" จนกว่าผู้ใช้จะยืนยัน ไม่เข้ายอดเงินก่อน
- กุญแจ AI อยู่บนเซิร์ฟเวอร์เท่านั้น ในแอปไม่มีกุญแจลับเลยสักตัว

ข้างในแอป: แบ่งโค้ดเป็น 5 ชั้น

ถ้าเอาโค้ดทุกอย่างไปกองในไฟล์เดียว จะแก้ยากและทดสอบไม่ได้ ทีมเราจึงแยกตามหน้าที่ แต่ละโฟลเดอร์ใน src/ มีหน้าที่ชัดเจน และมีกฎว่าใครเรียกใครได้

ชั้น	โฟลเดอร์	หน้าที่	ตัวอย่างไฟล์
หน้าจอ	src/app/	แต่ละไฟล์คือ 1 หน้า (Expo Router) วางชิ้นส่วน รับการแตะ แล้วเรียกชั้นอื่นทำงาน	<code>src/app/transaction.tsx</code> หน้าจอดรายการ
ชิ้นส่วน หน้าตา	src/ui/	ปุ่ม การ์ด ช่องกรอก สี ตัวอักษร กราฟ ภาพวาด ใช้ซ้ำได้ทุกหน้า	<code>src/ui/components.tsx</code> , <code>src/ui/theme.ts</code>

ชั้น	โฟลเดอร์	หน้าที่	ตัวอย่างไฟล์
กฎและการคำนวณ	src/domain/	"สมอง" ของแอป เป็นฟังก์ชันล้วน รับตัวเลขเข้า ส่งตัวเลขออก ไม่ยุ่งกับหน้าจอหรืออินเทอร์เน็ท จึงทดสอบได้ง่าย มี unit test ทุกไฟล์	<code>src/domain/runway.ts</code> , <code>src/domain/money.ts</code>
ข้อมูล	src/data/	โหลดและบันทึกข้อมูล ทั้งบนคลาวด์ (Supabase) และในเครื่อง (โหมดทดลอง) และเก็บ "สถานะกลาง" ที่ทุกหน้าใช้ร่วมกัน	<code>src/data/repo.ts</code> , <code>src/data/AppProvider.tsx</code>
บริการภายนอก	src/services/	งานที่คุยกับโลกนอกแอป: แกลเลอรี่รูป ไมโครโฟน เสียงอ่าน และเรียก Edge Functions	<code>src/services/slips.ts</code> , <code>src/services/coach.ts</code>



≈ ทำไมแยก domain ออกมา

เหมือนแยก "สูตรคำนวณภาษี" ออกจาก "แบบฟอร์มกรอกภาษี" ถ้าสูตรเปลี่ยน แค่นี้เดียว แบบฟอร์มทุกฉบับได้ผลใหม่ทันที และเราตรวจสอบสูตรได้ด้วยการใส่ตัวเลขทดลองโดยไม่ต้องกรอกแบบฟอร์มจริง โค้ดใน `src/domain` ก็เช่นกัน มีเทสต์ 157 ข้อที่รันจบในไม่ถึง 2 วินาทีโดยไม่ต้องเปิดแอป

ตามข้อมูล 1 รายการ: ตั้งแต่กดบันทึก จนตัวเลขบนหน้าหลักเปลี่ยน

วิธีที่ดีที่สุดในการเข้าใจระบบ คือเลือกการกระทำหนึ่งอย่าง แล้วตามดูว่ามันวิ่งผ่านไฟล์ไหนบ้าง สมมติผู้ใช้กด "ข้ามมันไ้฿85" แล้วกด "บันทึกรายการ"

1. หน้าจอดรายการ [src/app/transaction.tsx](#)

ฟังก์ชัน `save()` แปลง "85" เป็น 8,500 สตางค์ ด้วย `parseBahtToSatang` แล้วเรียก `addTx(...)`



2. สถานะกลาง [src/data/AppProvider.tsx](#)

`addTx` ส่งต่อให้ Repo บันทึก รวบรวมกลับมา แล้วเพิ่มรายการใหม่เข้าไปในรายการทั้งหมด (`setTxs`)



3. Repo [src/data/repo.ts](#)

บัญชีจริง: แปลงชื่อช่องเป็นแบบฐานข้อมูล (`amountSatang` → `amount_satang`) แล้วส่งคำสั่ง `insert` ไปที่ Supabase · โหมดทดลอง: เก็บลงเครื่องแทน



4. ฐานข้อมูล Supabase [supabase/migrations/20260927000000_init.sql](#)

ตรวจกฎ: จำนวนเงินต้อง > 0, ผู้บันทึกต้องเป็นเจ้าของแถว (RLS) แล้วเพิ่มแถวใหม่ในตาราง `transactions` ส่งแถวที่บันทึกแล้วกลับไป



5. คำนวณใหม่อัตโนมัติ `useMoney()` ใน [src/data/AppProvider.tsx](#)

เพราะรายการเปลี่ยน React จึงคำนวณยอดคงเหลือ (FR-2) และ Money Runway (FR-6) ใหม่ด้วยฟังก์ชันใน `src/domain` แล้ววาดหน้าหลักใหม่ ตัวเลขลดลง ฿85 กันที

ดูโค้ดจริงของแต่ละชั้น (ยังไม่ต้องเข้าใจทุกคำ บท A4–A9 จะสอนทีละอย่าง ตอนนี้งูแค่ว่ามันต่อกันยังงี้)

ชั้น 1: หน้าจอดรายการเรียก `addTx`

`src/app/transaction.tsx`

บรรทัด 87–124

```
87  async function save(confirm: boolean) {
88    if (busy) return;
89    const satang = parseBahtToSatang(amount);
90    if (satang === null) {
91      setAmountError('ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50');
92      return;
93    }
94    setAmountError(null);
95    if (dayError || timeError) return;
96    const input = {
97      kind,
98      amountSatang: satang,
99      categoryKey: category,
100     title: title.trim(),
101     note: note.trim() || null,
102     occurredAt: bkkToIso(day, time),
103   };
104   setBusy(true);
105   try {
106     if (existing) {
107       await updateTx(existing.id, {
108         ...input,
109         ...(confirm ? { status: 'confirmed' as const, reviewFlags: [] } : {}),
110       });
111       toast({ message: confirm && isDraft ? 'ยืนยันรายการแล้ว รวมในยอดเงินแล้ว' : 'บันทึกการแก้ไขแล้ว' });
112     } else {
```

```

113     await addTx({
114         ...input,
115         source: 'manual',
116         status: 'confirmed',
117         slipRef: null,
118         slipImageHash: null,
119         ocrConfidence: null,
120         reviewFlags: [],
121     });
122     toast({ message: kind === 'income' ? 'เพิ่มเงินเข้าแล้ว' : 'บันทึกรายการแล้ว' });
123 }
124 leaveWhenDone();

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:87

บรรทัด	ความหมาย
87	ประกาศฟังก์ชัน save ซึ่งทำงานเมื่อกดปุ่มบันทึก คำว่า async แปลว่าข้างในมีงานที่ต้องรอ (เช่น รออินเทอร์เน็ต)
88	ถ้ากำลังบันทึกอยู่แล้ว (busy) ให้ออกเลย กันคนกดปุ่มรัว ๆ แล้วได้รายการซ้ำ
89–93	แปลงข้อความในช่องเงินเป็นสตริง ถ้าแปลงไม่ได้ (เช่น พิมพ์ "abc" หรือ 0) จะได้ null แล้วขึ้นข้อความเตือนใต้ช่อง และหยุดไม่บันทึก
95	ถ้าวันที่หรือเวลาที่พิมพ์ (บนเว็บ) ผิดรูปแบบ ก็ไม่บันทึก
96–103	รวมค่าจากทุกช่องในฟอร์มเป็นก้อนเดียวชื่อ input: ประเภท จำนวนเงิน หมวด ชื่อ บันทึก และเวลา (แปลงวันและเวลาไทยเป็นรูปแบบสากลด้วย bkkToIso)
104	ตั้ง busy เป็นจริง ปุ่มจะหมุนรอและกดซ้ำไม่ได้
106–111	ถ้าเป็นการแก้ไขรายการเดิม เรียก updateTx แทน
113–121	ถ้าเป็นรายการใหม่ เรียก addTx พร้อมบอกว่ามาจากการกดเอง (manual) และยืนยันแล้ว (confirmed) จึงนับในยอดเงินทันที
122	ขึ้นข้อความสั้น ๆ ด้านล่าง (toast) ว่า "บันทึกรายการแล้ว"
124	ปิดหน้าฟอร์มกลับไปหน้าก่อน

ขั้น 2: AppProvider ส่งต่อให้ Repo แล้วอัปเดตรายการ

src/data/AppProvider.tsx

บรรทัด 385–392

```

385     async addTx(input) {
386         const r = need();
387         const tx = await r.insert(input);
388         if (repoRef.current !== r) return tx;
389         noteLocal(tx.id, tx);
390         setTxs((list) => [tx, ...list]);
391         return tx;
392     },

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:385

บรรทัด	ความหมาย
386	หยาบ Repo ตัวที่ใช้อยู่ (บัญชีจริงหรือโหมดทดลอง) ถ้ายังไม่ได้เข้าสู่ระบบ need () จะโยน error
387	สั่ง Repo บันทึก แล้วรอผล (await) ได้รายการที่มี id จริงกลับมา
388	ถ้าระหว่างรอ ผู้ใช้ลบบัญชีไปแล้ว ไม่ต้องเอารายการนี้ไปใส่ในบัญชีใหม่
389	จดไว้ว่ามีการเปลี่ยนแปลงในเครื่อง (กันรายการหายถ้ากำลังโหลดข้อมูลชุดใหม่พร้อมกัน)
390	เอารายการใหม่ไปไว้หน้าสุดของรายการทั้งหมด การเรียก setTxS ทำให้ React วาดทุกหน้าที่ใช้รายการนี้ใหม่

ขั้น 3: Repo ส่งคำสั่งไป Supabase

src/data/repo.ts	บรรทัด 217–222
<pre> 217 async insert(input) { 218 const { data, error } = await db.from('transactions').insert(toRow(input)).select().single(); 219 if (isUniqueViolation(error)) throw new DuplicateSlipError(); 220 if (error) throw error; 221 return fromRow(data as TxRow); 222 }, </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:217	

บรรทัด	ความหมาย
218	อ่านเหมือนประโยคภาษาอังกฤษ: "จากตาราง transactions, เพิ่ม (insert) แถวนี้, แล้วเลือก (select) แถวที่เพิ่ง เพิ่มกลับมา 1 แถว (single)" toRow แปลงชื่อช่องให้ตรงกับฐานข้อมูล
219	ถ้าฐานข้อมูลตอบว่า "ซ้ำ" (รหัส 23505) แปลว่าสลิปนี้เคยบันทึกแล้ว โยน error แบบพิเศษให้หน้าจอบอกผู้ใช้ว่า "มีรายการจากสลิปนี้อยู่แล้ว"
220	error อื่น ๆ (เช่น อินเทอร์เน็ตหลุด) โยนต่อให้หน้าจอขึ้น "บันทึกไม่สำเร็จ ลองอีกครั้ง"
221	แปลงแถวจากฐานข้อมูลกลับเป็นรูปแบบที่แอปใช้ (amount_satang → amountSatang) แล้วส่งคืน

ขั้น 5: ตัวเลขคำนวณใหม่เอง

src/data/AppProvider.tsx	บรรทัด 442–459
<pre> 442 <i>/** Derived money numbers used across screens (FR-2 balance, FR-6 runway). */</i> 443 export function useMoney() { 444 const { profile, txs, carrySatang, goals, today } = useApp(); 445 return useMemo(() => { 446 const opening = profile?.openingBalanceSatang ?? 0; 447 const floor = profile?.runwayFloorSatang ?? 50_000; 448 <i>// Money from before the loaded window (over ~13 months ago) still counts.</i> 449 const balance = computeBalance(opening + carrySatang, txs); 450 const average = averageDailyExpense(txs); 451 <i>// Money in savings goals is set aside: it does not count as money to spend.</i> 452 const reserved = reservedSatang(goals); 453 const runway = computeRunway({ balanceSatang: balance, floorSatang: floor + reserved, averageSatang: average.averageSatang }); 454 const drafts = txs.filter((t) => t.status === 'draft'); 455 return { balance, average, runway, drafts, reserved, floor }; </pre>	

```

456 // `today` makes the runway and averages start over at midnight.
457 // eslint-disable-next-line react-hooks/exhaustive-deps
458 }, [profile, txs, carrySatang, goals, today]);
459 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:442`

บรรทัด	ความหมาย
443	useMoney คือ "hook" ที่ทุกหน้าเรียกใช้เมื่ออยากได้ตัวเลขเงิน (บท A6 อธิบายว่า hook คืออะไร)
444	ดึงข้อมูลดิบจากสถานะกลาง: โปรไฟล์ รายการ ยอดยกมา กระปุกลูก และวันนี้
445	useMemo แปลว่า "คำนวณแล้วจำไว้" คำนวณใหม่เฉพาะเมื่อข้อมูลในบรรทัด 458 เปลี่ยน
446–447	ยอดเงินตอนเริ่มใช้แอป และเส้นเงินสำรอง (ถ้ายังไม่ตั้ง ใช้ 50,000 สตางค์ = ฿500)
449	คำนวณยอดคงเหลือ (FR-2) จากรายการทั้งหมด
450–453	คำนวณค่าเฉลี่ยรายจ่าย เงินในกระปุก และ Money Runway (FR-6)
454	หารายการแบบร่างที่รอตรวจ (FR-4)
455	ส่งตัวเลขทั้งหมดออกไปให้หน้าจอใช้
458	รายการ "สิ่งที่ต้องจับตา" ถ้า txs เปลี่ยน (เพิ่งเพิ่มข้าวมันไก่) จะคำนวณใหม่ทั้งหมด และ today ทำให้คำนวณใหม่ตอนเที่ยงคืน

★ จำให้ได้: ตัวเลขบนหน้าจอไม่ได้เก็บไว้ แต่คิดใหม่ทุกครั้ง

ยอดคงเหลือ ค่าเฉลี่ย และ Runway **ไม่ได้เก็บในฐานข้อมูล** ฐานข้อมูลเก็บแค่ "รายการ" แล้วแอปคำนวณตัวเลขทั้งหมดสด ๆ จากรายการทุกครั้งที่มีอะไรเปลี่ยน ข้อดีคือตัวเลขไม่มีทางขัดกันเอง (ไม่มีกรณียอดในฐานข้อมูลบอกอย่างหนึ่ง แต่รายการบอกอีกอย่าง) คอมเมนต์บนสุดของ [src/data/AppProvider.tsx](#) เขียนไว้ว่า "Numbers shown on screen are always derived from this state ... never stored twice"

ตามข้อมูลที่ใช้ AI: สแกนสลิป 1 ใบ

1. มือถือหารูปใหม่ในแคลเลอร์ [src/services/AutoScanProvider.tsx](#)

อ่าน QR บนรูปในเครื่องก่อน (ฟรี ไม่ใช้อินเทอร์เน็ต) รูปไหนไม่มี QR ของสลิปธนาคารจะไม่ถูกส่งไปไหนเลย

↓ เฉพาะรูปที่เป็นสลิป

2. ส่งรูปไป Edge Function [src/services/slips.ts](#) → [supabase/functions/parse-slip/index.ts](#)

เซิร์ฟเวอร์ตรวจว่าเข้าสู่ระบบจริง ตรวจโควตา (300 ใบต่อวัน) แล้วส่งรูปให้ AI อ่าน 2 ครั้งพร้อมกันด้วย 2 รุ่น

↓

3. AI ตอบเป็น JSON

ยอดเงิน วันที่ ชื่อผู้รับ ธนาคาร และ "ความมั่นใจ" ของแต่ละช่อง เซิร์ฟเวอร์รวมผลสองครั้ง ถ้าไม่ตรงกันจะให้ผู้ตรวจ

↓

4. แอปบันทึกรายการ

AI อ่านตรงกันทั้ง 2 ครั้งและมั่นใจทุกช่อง $\geq 80\%$ → เข้ายอดเงินเลย (ผู้ใช้ปิดตัวเลือกนี้ได้ในหน้าตั้งค่า) · ไม่มั่นใจหรืออ่านไม่ตรงกัน → เป็นแบบร่าง รอผู้ตรวจ แล้วตัวเลขทั้งแอปคำนวณใหม่เหมือนขึ้น 5 ด้านบน

รายละเอียดของ AI อยู่ในบท A10 และรายละเอียดเพิ่มเติมของการสแกนสลิปอยู่ในเล่ม FR-4

เครื่องมือ: เขียนโค้ดที่ไหน ใช้โปรแกรมอะไร และ VS Code ตั้งแต่ศูนย์

ทุกอย่างที่ต้องมีในเครื่อง วิธีเอาโค้ด MindPay มาเปิด รับแอป และใช้ VS Code อ่านโค้ดอย่างมืออาชีพ

กล่องเครื่องมือของทีมเรา

เครื่องมือ	คืออะไร	ใช้ทำอะไรใน MindPay
VS Code	โปรแกรมเขียนโค้ด (code editor) ฟรีจาก Microsoft	เปิด อ่าน แก้โค้ดทุกไฟล์ และรับคำสั่งผ่าน terminal ในตัว
Node.js (เวอร์ชัน 22 ขึ้นไป)	ตัวรับภาษา JavaScript นอกเบราว์เซอร์	รันเครื่องมือทั้งหมด: Expo, เทสต์, สคริปต์ build
npm	ตัวจัดการ "แพ็คเกจ" (โค้ดสำเร็จรูปที่คนอื่นเขียนไว้) มากับ Node.js	ติดตั้งไลบรารี 38 ตัวที่แอปใช้ (npm install) และรับคำสั่งใน package.json
Git	ระบบเก็บประวัติการแก้ไขโค้ด ย้อนเวอร์ชันได้	บันทึกการเปลี่ยนแปลงทุกครั้งเป็น "commit"
GitHub	เว็บที่เก็บโค้ดของ Git บนคลาวด์ ทำงานเป็นทีมได้	เก็บโค้ดที่ github.com/2550expo-tech/Socrates-and-Skeletons - และ build แอปให้อัตโนมัติ (un A13)
Expo	ชุดเครื่องมือทำแอปมือถือด้วย React Native	รันแอประหว่างพัฒนา (npx expo start), ทำเว็บ, build APK, ส่งอัปเดต
Expo Go	แอปบนมือถือสำหรับลองแอปที่กำลังเขียน	สแกน QR จากคอมแล้วเห็นแอปบนมือถือทันที
Supabase Dashboard	หน้าเว็บจัดการเซิร์ฟเวอร์ที่ supabase.com	ดูตาราง รับ SQL ใส่กุญแจ AI ดู log ของ Edge Functions
เบราว์เซอร์ Chrome	-	เปิดเว็บเวอร์ชัน และใช้ DevTools (F12) ดู error

ติดตั้งเครื่องมือ (ทำครั้งเดียว)

- VS Code:** ดาวน์โหลดจาก code.visualstudio.com ติดตั้งแบบค่าเริ่มต้น (Windows ตี๊ก "Add to PATH" และ "Open with Code" ไว้ด้วย)
- Node.js:** ดาวน์โหลดรุ่น LTS (22 ขึ้นไป) จาก nodejs.org แล้วเปิด terminal พิมพ์ node -v ต้องขึ้น v22 หรือสูงกว่า
- Git:** ดาวน์โหลดจาก git-scm.com (Mac มีมาให้แล้ว) ตรวจสอบด้วย git --version
- บัญชี GitHub:** สมัครที่ github.com และขอให้เจ้าของ repo เพิ่มเป็นสมาชิกถ้าต้องแก้ไข (อ่านอย่างเดียวไม่ต้องขอ เพราะ repo เป็นสาธารณะ)
- Expo Go** บนมือถือ (ไม่บังคับ): ดาวน์โหลดจาก Play Store หรือ App Store

เอาโค้ด MindPay มาไว้ในเครื่อง

เปิด terminal (Windows: PowerShell, Mac: Terminal หรือใช้ terminal ใน VS Code) แล้วพิมพ์ทีละบรรทัด

Terminalคำสั่งที่พิมพ์จริง

```
1 git clone https://github.com/2550expo-tech/Socrates-and-Skeletons-.git
2 cd Socrates-and-Skeletons-
3 npm install
4 cp .env.example .env
5 code .
```

บรรทัด	ความหมาย
1	clone = คัดลอกโค้ดทั้งหมดพร้อมประวัติจาก GitHub มาไว้ในโฟลเดอร์ใหม่ชื่อ Socrates-and-Skeletons-
2	cd (change directory) = เข้าไปในโฟลเดอร์นั้น
3	อ่านรายชื่อแพ็คเกจใน package.json แล้วดาวน์โหลดทั้งหมดลงโฟลเดอร์ node_modules (ประมาณ 1-3 นาที ใช้พื้นที่หลายร้อย MB)
4	คัดลอกไฟล์ตั้งค่าตัวอย่างเป็น .env ซึ่งบอกแอปว่าเซิร์ฟเวอร์ Supabase อยู่ที่ไหน (Windows PowerShell ใช้ copy .env.example .env)
5	เปิด VS Code ที่โฟลเดอร์นี้ (จุด . แปลว่า "โฟลเดอร์ปัจจุบัน")

ไฟล์ .env มีอะไร

.env.exampleบรรทัด 1-6

```
1 # Copy this file to .env and fill in the values from
2 # Supabase Dashboard -> Project Settings -> API (or the green "Connect" button).
3 # Only the PUBLISHABLE (anon) key goes here. Never put the secret/service_role key
4 # or any AI key (Gemini / Anthropic) in the app: those live in Supabase Edge Function secrets.
5 EXPO_PUBLIC_SUPABASE_URL=https://dufvcdswcaqsfbxjwat.supabase.co
6 EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_LKQP3TU8Fr3t8JD6bzg7TA_5Uy6zcm
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → .env.example:1

บรรทัด	ความหมาย
1-4	คอมเมนต์ (บรรทัดที่ขึ้นต้นด้วย # คอมพิวเตอร์ไม่อ่าน) บอกว่าให้ใส่เฉพาะกุญแจ "publishable" ห้ามใส่กุญแจลับหรือกุญแจ AI ในแอปเด็ดขาด
5	ที่อยู่ของเซิร์ฟเวอร์ Supabase ของทีม
6	กุญแจสาธารณะ (publishable key) ใช้บอกว่า "แอปนี้เป็นของโปรเจกต์ไหน" ปลอดภัยที่จะอยู่ในแอป เพราะฐานข้อมูลใช้ RLS กันไว้ (บท A9) ชื่อที่ขึ้นต้นด้วย EXPO_PUBLIC_ คือค่าที่ Expo ยอมใส่เข้าไปในแอป

ไฟล์ package.json: บัตรประชาชนของโปรเจกต์

ไฟล์นี้บอกว่าโปรเจกต์ชื่ออะไร ใช้แพ็คเกจอะไร และมีคำสั่งลัดอะไรบ้าง

package.jsonบรรทัด 1-59

```

1 {
2   "name": "mindpay",
3   "version": "1.0.0",
4   "main": "expo-router/entry",
5   "dependencies": {
6     "@expo-google-fonts/anuphan": "^0.4.1",
7     "@expo-google-fonts/noto-serif-thai": "^0.4.2",
8     "@expo/vector-icons": "^15.0.2",
9     "@react-native-community/datetimepicker": "9.1.0",
10    "@supabase/supabase-js": "^2.117.2",
11    "expo": "~57.0.25",
12    "expo-camera": "~57.0.5",
13    "expo-constants": "~57.0.19",
14    "expo-crypto": "~57.0.3",
15    "expo-file-system": "~57.0.7",
16    "expo-font": "~57.0.4",
17    "expo-haptics": "~57.0.3",
18    "expo-image-manipulator": "~57.0.20",
19    "expo-image-picker": "~57.0.20",
20    "expo-linear-gradient": "~57.0.2",
21    "expo-linking": "~57.0.11",
22    "expo-media-library": "~57.0.5",
23    "expo-router": "~57.0.23",
24    "expo-speech": "~57.0.3",
25    "expo-speech-recognition": "^57.1.0",
26    "expo-splash-screen": "~57.0.9",
27    "expo-sqlite": "~57.0.3",
28    "expo-status-bar": "~57.0.1",
29    "expo-system-ui": "~57.0.4",
30    "expo-updates": "~57.0.23",
31    "react": "19.2.3",
32    "react-dom": "19.2.3",
33    "react-native": "0.86.3",
34    "react-native-safe-area-context": "~5.7.0",
35    "react-native-screens": "~4.26.0",
36    "react-native-svg": "15.15.4",
37    "react-native-web": "~0.21.0"
38  },
39  "devDependencies": {
40    "@types/react": "~19.2.2",
41    "eslint": "^9.0.0",
42    "eslint-config-expo": "~57.0.2",
43    "playwright": "^1.56.0",
44    "typescript": "~6.0.3",
45    "vitest": "^5.0.2"
46  },
47  "scripts": {
48    "start": "expo start",
49    "android": "expo start --android",
50    "ios": "expo start --ios",
51    "web": "expo start --web",
52    "test": "vitest run",
53    "typecheck": "tsc --noEmit",
54    "lint": "expo lint",
55    "e2e:web": "EXPO_WEB_BASE_URL=/Socrates-and-Skeletons- expo export --platform web --output-dir dist && EXPO_WEB_BASE_URL=/Socrates-and-Skeletons- node scripts/web-home-screen.mjs dist && cp dist/index.html dist/404.html && node e2e/web-auth.mjs dist e2e-shots",
56    "demo:video": "node e2e/demo-video.mjs dist demo-video light android && node e2e/demo-video.mjs dist demo-video dark android && node e2e/demo-video.mjs dist demo-video light web"
57  },

```

```
58 "private": true
59 }
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → package.json:1

บรรทัด	ความหมาย
2–3	ชื่อโปรเจกต์ mindpay และเวอร์ชัน
4	จุดเริ่มต้นของแอป: ให้ Expo Router เป็นคนเริ่ม แล้วมันจะไปหาหน้าจอในโฟลเดอร์ src/app เอง
5–38	dependencies แพ็กเกจที่แอปต้องใช้ตอนทำงานจริง ตัวเลขหลังชื่อคือเวอร์ชัน (^ = ยอมรุ่นย่อยที่ใหม่กว่า, ~ = ยอมเฉพาะรุ่นแก้บั๊ก)
6–7	ฟอนต์ไทย Anuphan (ข้อความทั่วไป) และ Noto Serif Thai (ตัวเลขเงินและหัวข้อ)
8	ไอคอนสำเร็จรูป (Icons) เช่น รูปบ้าน รูปใบไม้ในแถบล่าง
9	ตัวเลือกวันที่และเวลาแบบของมือถือ (FR-1)
10	ตัวเชื่อมต่อ Supabase (ฐานข้อมูล, เข้าสู่ระบบ, Edge Functions)
11	Expo ตัวหลัก เวอร์ชัน 57
12	expo-camera ใช้ อ่าน QR จากรูป (ไม่ได้ใช้ถ่ายรูป แอปบล็อกสิทธิ์กล้องไว้)
14	expo-crypto ทำ "ลายนิ้วมือ" (hash) ของรูปสลิป กันสแกนซ้ำ (FR-4)
15, 18	อ่านไฟล์ และย่อ/ตัดรูปก่อนส่งให้ AI
17	expo-haptics สั่นเบา ๆ เวลากดปุ่ม
19, 22	เลือกรูปเอง และเข้าถึงแคเลอรีทั้งหมด (FR-4)
20	สีไล่ระดับ (gradient) ของการ์ดเขียว
21	เปิดลิงก์ เช่น ลิงก์ยืนยันอีเมลที่พาเข้าแอป
23	Expo Router ระบบเปลี่ยนหน้าจอ (บท A7)
24–25	expo-speech ให้น้องกล่าวพูดออกเสียง (FR-5), expo-speech-recognition ฟังเสียงพูดจด (FR-3)
26	หน้าจอแรกตอนเปิดแอป (splash)
27	expo-sqlite ฐานข้อมูลเล็ก ๆ ในเครื่อง ใช้เก็บการตั้งค่าและข้อมูลโหมดทดลอง
30	expo-updates รับอัปเดตโค้ดใหม่โดยไม่ต้องลงแอปใหม่ (EAS Update)
31–33	React (ตัววาดหน้าจอ), react-dom (สำหรับเว็บ), React Native (สำหรับมือถือ)
34–35	จัดการขอมจ (รอยบาก แถบสถานะ) และหน้าจอแบบ native
36	วาดภาพเวกเตอร์ (SVG) เช่น ต้นไม้เงิน น้องกล่าวกราฟ
37	react-native-web แปลงชิ้นส่วนของมือถือเป็น HTML ให้แสดงบนเว็บได้
39–46	devDependencies ใช้ตอนพัฒนาเท่านั้น ไม่ได้อยู่ในแอปที่ผู้ใช้งาน: ตัวตรวจภาษา TypeScript, ESLint (ตรวจรูปแบบโค้ด), Playwright (ควบคุมเบราว์เซอร์อัตโนมัติสำหรับ E2E), Vitest (รัน unit test)

บรรทัด	ความหมาย
47-57	scripts คำสั่งลัด พิมพ์ <code>npm run</code> ชื่อ (หรือ <code>npm test</code> , <code>npm start</code>) แทนคำสั่งยาว ๆ ทางขวา
58	<code>private</code> = ไม่เผยแพร่เป็นแพ็คเกจให้คนอื่นติดตั้ง

คำสั่งที่ใช้บ่อย

คำสั่ง	ทำอะไร	ผลที่ควรเห็น
<code>npm run web</code>	เปิดแอปเวอร์ชันเว็บในเบราว์เซอร์ แก้โค้ดแล้วหน้าเว็บเปลี่ยนเอง	เบราว์เซอร์เปิด localhost:8081
<code>npx expo start</code>	เปิด "dev server" มี QR ให้สแกนด้วย Expo Go	QR ใน terminal กด w = เปิดเว็บ, r = โหลดใหม่
<code>npm test</code>	รัน unit test ทั้งหมด	"Tests 157 passed (157)"
<code>npm run typecheck</code>	ให้ TypeScript ตรวจสอบข้อมูลทั้งโปรเจกต์	ไม่มีข้อความ = ไม่มีปัญหา
<code>npx expo lint</code>	ตรวจสอบรูปแบบโค้ดตามกฎ ESLint	ไม่มี error
<code>npm run e2e:web</code>	build เว็บแล้วรันเทสต์ในเบราว์เซอร์ 145 ข้อ ถ่ายภาพหน้าจอเก็บใน e2e-shots	"145/145 checks passed"

! ถ้าเจอ error ตอนติดตั้ง

"node: command not found" = ยังไม่ได้ติดตั้ง Node.js หรือต้องปิดแล้วเปิด terminal ใหม่ · **npm install** ซ้ำ หรือค้าง = รอได้ หรือลบโฟลเดอร์ `node_modules` แล้วลองใหม่ · **แอปขึ้นแต่เข้าสู่ระบบไม่ได้** = ลืมสร้างไฟล์ `.env` (บรรทัด 4) · ห้ามใช้ `npm install` ชื่อแพ็คเกจ ตรง ๆ ในโปรเจกต์ Expo ให้ใช้ `npx expo install` ชื่อแพ็คเกจ เพื่อให้ได้รุ่นที่เข้ากับ Expo 57 (กฎใน AGENTS.md)

รู้จักหน้าจอ VS Code

ส่วน	อยู่ตรงไหน	ใช้ทำอะไร
Activity Bar	แถบไอคอนซ้ายสุด	สลับระหว่าง Explorer, Search, Source Control, Run, Extensions
Explorer	ไอคอนรูปกระดาษ (Ctrl+Shift+E)	ต้นไม้โฟลเดอร์และไฟล์ทั้งโปรเจกต์ คลิกเพื่อเปิดไฟล์
Editor	พื้นที่ใหญ่ตรงกลาง	อ่านและแก้ไขโค้ด เปิดหลายไฟล์เป็นแท็บได้ ลากแท็บไปข้าง ๆ เพื่อดู 2 ไฟล์ คู่กัน
Search	ไอคอนแว่นขยาย (Ctrl+Shift+F)	ค้นคำในทุกไฟล์ เช่นค้น <code>computeRunway</code> จะเห็นทุกที่ที่ใช้
Source Control	ไอคอนกิ่งไม้ (Ctrl+Shift+G)	ดูไฟล์ที่แก้ไข, commit, push (un A13)

ส่วน	อยู่ตรงไหน	ใช้ทำอะไร
Terminal	ด้านล่าง (Ctrl+` ปุ่มตัวหนอนใต้ Esc)	พิมพ์คำสั่ง npm, git ได้โดยไม่ต้องออกจาก VS Code
Problems	แถบข้าง Terminal (Ctrl+Shift+M)	รวม error จาก TypeScript และ ESLint คลิกแล้วกระโดดไปบรรทัดที่ผิด
Status Bar	แถบด้านล่างสุด	บอก branch ของ Git, บรรทัด/คอลัมน์ที่เคอร์เซอร์อยู่, ภาษาของไฟล์

คีย์ลัดที่ทำให้อ่านโค้ดเร็วขึ้น 10 เท่า

Mac ใช้ Cmd แทน Ctrl และ Option แทน Alt

คีย์	ทำอะไร	ลองกับ MindPay
Ctrl+P	เปิดไฟล์จากชื่อ พิมพ์ : เลขบรรทัด ต่อท้ายเพื่อกระโดดไปบรรทัดนั้น	พิมพ์ runway.ts : 72 แล้ว Enter
Ctrl+G	ไปบรรทัดที่ต้องการในไฟล์ที่เปิดอยู่	Ctrl+G แล้วพิมพ์ 89
F12	Go to Definition ไปดูว่าฟังก์ชันหรือตัวแปรนี้ถูกสร้างไว้ที่ไหน (หรือ Ctrl+คลิก)	ใน AppProvider.tsx คลิกที่ computeRunway แล้วกด F12
Alt+F12	Peek Definition: ดูโค้ดปลายทางในกล่องเล็ก ๆ โดยไม่ต้องเปลี่ยนไฟล์	-
Shift+F12	Find All References หากทุกที่ที่เรียกใช้สิ่งนี้	ที่ computeRunway จะเห็นทั้งในแอปและในเทสต์
Alt+← / Alt+→	ย้อนกลับ/ไปข้างหน้า หลังกระโดดไปมาด้วย F12	-
เอาเมาส์ชี้ (hover)	เห็นชนิดข้อมูลและคอมเมนต์อธิบายของสิ่งนั้น	ชี้ที่ Transaction จะเห็นทุกช่องของรายการ
Ctrl+Shift+0	รายการฟังก์ชันทั้งหมดในไฟล์นี้ (outline)	ใน runway.ts จะเห็น 6 ฟังก์ชัน
Ctrl+T	ค้นหาชื่อฟังก์ชันทั้งโปรเจกต์	พิมพ์ parseSpokenEntry
Ctrl+Shift+F	ค้นข้อความในทุกไฟล์	ค้น "เงินพอใช้อีก" เพื่อหาว่าข้อความบนจอมาจากไฟล์ไหน
Ctrl+F	ค้นในไฟล์นี้	-
F2	เปลี่ยนชื่อตัวแปรทุกที่พร้อมกันอย่างปลอดภัย	-
Ctrl+/,	เปลี่ยนบรรทัดเป็นคอมเมนต์ (ปิดชั่วคราว) หรือกลับคืน	-
Ctrl+Shift+P	Command Palette เรียกคำสั่งทุกอย่างของ VS Code	พิมพ์ "Format Document"
Ctrl+B	ซ่อน/แสดงแถบด้านข้าง ให้มีที่อ่านโค้ดมากขึ้น	-

★ เทคนิคหาว่าข้อความบนจอมาจากโค้ดตรงไหน

เห็นคำว่า "เงินพอใช้อีก 16 วัน" บนหน้าจอหลัก แล้วอยากรู้ว่าโค้ดอยู่ไหน: กด **Ctrl+Shift+F** พิมพ์ "เงินพอใช้อีก" จะเจอไฟล์ที่มีข้อความนี้ทั้งหมด คลิกผลลัพธ์แล้วจะไปบรรทัดนั้นทันที วิธีนี้ใช้ได้กับทุกข้อความภาษาไทยในแอป เพราะข้อความทั้งหมดเขียนอยู่ในโค้ดตรง ๆ

ส่วนเสริม (Extensions) ที่แนะนำ

กด Ctrl+Shift+X แล้วค้นชื่อ กด Install

ชื่อ	ช่วยอะไร
ESLint (Microsoft)	ขีดเส้นใต้โค้ดที่ผิดกฎทันทีที่พิมพ์ ใช้กฎเดียวกับ <code>npm run lint</code>
Expo Tools	ช่วยเติมคำและตรวจไฟล์ <code>app.json</code> / <code>eas.json</code>
Vitest	มีปุ่ม ▶ ข้างเกสต์แต่ละข้อ กดรันทีละข้อได้
GitLens (ไม่บังคับ)	ขีดบรรทัดไหนก็เห็นว่าใครแก้ล่าสุด เมื่อไร ใน commit อะไร
Error Lens (ไม่บังคับ)	แสดงข้อความ error ถ้ายกบรรทัดเลย ไม่ต้องเอาเมาส์ชี้

ส่วน TypeScript ไม่ต้องลงเพิ่ม VS Code รองรับในตัวอยู่แล้ว และใช้ TypeScript รุ่นในโปรเจกต์ (`node_modules/typescript`) ตามไฟล์ `tsconfig.json`

ไม่อยากติดตั้งอะไร: ใช้ VS Code บนเว็บ

มี 2 ทางที่เปิดโค้ด MindPay ด้วย VS Code ได้ทันทีในเบราว์เซอร์

- **ลิงก์ในเล่มนี้** แตะชื่อไฟล์บนหัวโค้ดก้อนไหนก็ได้ จะเปิด **vscode.dev** ที่ไฟล์และบรรทัดนั้น
- **จากหน้า GitHub** เปิด repo บน github.com แล้วกดปุ่ม . (จุด) บนคีย์บอร์ด หน้าเว็บจะกลายเป็น VS Code (github.dev)

VS Code บนเว็บอ่านโค้ด ค้นหา (Ctrl+Shift+F) กด F12 ไปดูฟังก์ชัน และแก้ไขได้ แต่ **รันคำสั่งไม่ได้** (ไม่มี terminal จึงรัน `npm test` หรือเปิดแอปไม่ได้) ถ้าจะรันแอปต้องโคลนลงเครื่องตามขั้นตอนด้านบน

ฝึกใช้: ตามรอย Money Runway ใน 5 นาที

1. Ctrl+P พิมพ์ `index.tsx` เลือกไฟล์ใน `src/app/` (tabs) (หน้าจอหลัก)
2. Ctrl+F ค้น `useMoney` แล้วกด F12 ที่คำนั้น จะเดินไป `src/data/AppProvider.tsx` บรรทัด 443
3. ที่บรรทัด 453 คลิก `computeRunway` แล้วกด F12 จะไปที่ `src/domain/runway.ts` บรรทัด 72 ซึ่งเป็นสูตรของ FR-6
4. กด Shift+F12 ที่ชื่อฟังก์ชัน จะเห็นว่ามีเกสต์เรียกใช้ใน `src/domain/__tests__/domain.test.ts` ด้วย
5. กด Alt+← สองครั้ง ย้อนกลับไปหน้าจอหลัก

ทำแบบนี้ได้ แปลว่าพร้อมอ่านโค้ดส่วนไหนของแอปก็ได้แล้ว

พื้นฐานการเขียนโปรแกรม สำหรับคนที่ไม่เคยเขียนเลย

ทุกอย่างที่ต้องรู้เพื่ออ่านโค้ด MindPay ออก: ค่า ตัวแปร เงื่อนไข ฟังก์ชัน array และ object พร้อมตัวอย่างจากโค้ดจริง

โปรแกรมคืออะไร

โปรแกรมคือ รายการคำสั่งที่คอมพิวเตอร์ทำตามทีละบรรทัด จากบนลงล่าง เหมือนสูตรทำอาหาร ทีมเราเขียนด้วยภาษา **TypeScript** ซึ่งคือภาษา **JavaScript** (ภาษาของเว็บ) ที่เพิ่ม "ชนิดข้อมูล" เข้าไป (บท A5) ไฟล์ที่ลงท้ายด้วย `.ts` เป็น TypeScript ล้วน ส่วน `.tsx` คือ TypeScript ที่มีโค้ดหน้าจอบน React ปนอยู่ (บท A6)

1. ค่า (value) และชนิดของค่า

ชนิด	ตัวอย่าง	ใช้ใน MindPay
number ตัวเลข	8500, -3, 0.8	จำนวนเงินเป็นสตางค์ 8500 = ฿85, ความมั่นใจ 0.8 = 80%
string ข้อความ	'ข้าวมันไก่', "2026-10-09"	ชื่อรายการ วันที่ ข้อความบนจอ (ใส่ในเครื่องหมายคำพูด ' หรือ ")
boolean จริง/เท็จ	true, false	เข้าสู่ระบบแล้วหรือยัง, ตั้งค่าเสร็จหรือยัง (onboarded)
null	null	"ไม่มีค่า" โดยตั้งใจ เช่น รายการที่จองเองไม่มีเลขอ้างอิงสลิป (slipRef: null)
undefined	undefined	"ยังไม่ได้ใส่ค่า" เช่น ไม่ได้ส่งพารามิเตอร์นั้นมา

2. ตัวแปร: กล่องที่มีชื่อ

ตัวแปรคือ "กล่องที่ติดป้ายชื่อ" ไว้ใส่ค่า เพื่อเรียกใช้ทีหลัง

ตัวแปร

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const floor = 50_000;           // const = ค่าคงที่ ใส่แล้วเปลี่ยนไม่ได้
2 let total = 0;                  // let = เปลี่ยนค่าได้ภายหลัง
3 total = total + 8500;           // ตอนนี้ total = 8500
4 const title = 'ข้าวมันไก่';
```

- `const` ใช้บ่อยที่สุดในโค้ด MindPay เพราะค่าส่วนใหญ่ไม่ควรเปลี่ยน ใช้ `let` เฉพาะตอนต้องเปลี่ยนจริง ๆ
- `50_000` คือ 50000 เครื่องหมาย `_` ช่วยให้อ่านง่ายเหมือนคอมมา คอมพิวเตอร์ไม่สนใจมัน
- ข้อความหลัง `//` คือ **คอมเมนต์** คอมพิวเตอร์ข้าม มีไว้ให้คนอ่าน ส่วน `/* ... */` และ `/** ... */` เป็นคอมเมนต์หลายบรรทัด (แบบ `/**` VS Code จะเอาไปแสดงตอนเอาเมาส์ชี้)
- ตั้งชื่อแบบ **camelCase**: คำแรกตัวเล็ก คำต่อไปขึ้นต้นตัวใหญ่ เช่น `balanceSatang`, `averageDailyExpense` คำคงที่สำคัญใช้ตัวใหญ่ทั้งหมด เช่น `RUNWAY_CAP_DAYS`

★ กฎการตั้งชื่อของทีมเรา: ตัวแปรเงินลงท้ายด้วย Satang

ทุกตัวแปรที่เป็นจำนวนเงิน ลงท้ายด้วย Satang เสมอ เช่น amountSatang, floorSatang เห็นชื่อแล้วรู้ทันทีว่าต้องหาร 100 ก่อนแสดงเป็นบาท กันความผิดพลาดแบบเอาบาทไปบวกกับสตางค์

3. ตัวดำเนินการ: คำนวณและเปรียบเทียบ

เขียนว่า	ความหมาย	ตัวอย่าง → ผล
+ - * /	บวก ลบ คูณ หาร	300000 / 145 → 2068.96...
%	เศษจากการหาร	8550 % 100 → 50 (50 สตางค์)
=== !==	เท่ากับ / ไม่เท่ากับ (แบบเคร่ง ชนิดต้องตรงด้วย)	t.kind === 'income'
< <= > >=	น้อยกว่า น้อยกว่าหรือเท่ากับ ...	days < 7
&&	"และ" จริงเมื่อทั้งสองข้างจริง	d >= windowStart && d <= today
	"หรือ" จริงเมื่อข้างใดข้างหนึ่งจริง	event === 'SIGNED_IN' ...
!	"ไม่" กลับจริงเป็นเท็จ	!existing → ไม่มีรายการเดิม
ก ? ข : ค	ถ้า ก จริง ได้ ข ไม่จริงได้ ค (ternary)	kind === 'income' ? 'เพิ่มเงินเข้า' : 'ลดรายการ'
ก ?? ข	ใช้ ก ถ้า ก ไม่ใช่ null/undefined ไม่ขึ้นใช้ ข (ค่าสำรอง)	profile?.runwayFloorSatang ?? 50_000
ก?.ข	เข้าถึง ข ใน ก แต่ถ้า ก เป็น null ไม่ต้อง error ได้ undefined	profile?.openingBalanceSatang

ลองอ่านบรรทัดนี้จาก [src/data/AppProvider.tsx:447](#) ซึ่งใช้ทั้ง ?. และ ??

src/data/AppProvider.tsx

บรรทัด 447-447

```
447 const floor = profile?.runwayFloorSatang ?? 50_000;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:447

อ่านว่า: "เส้นเงินสำรอง = ค่าที่ผู้ตั้งไว้ในโปรไฟล์ แต่ถ้ายังไม่มีโปรไฟล์ (?.) หรือไม่มีค่า ให้ใช้ 50,000 สตางค์ (??)"

4. เงื่อนไข if: ถ้า... ให้ทำ...

if / else

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 if (spendable <= 0) {
2   // ทำส่วนนี้เมื่อเงื่อนไขจริง
3 } else if (averageSatang <= 0) {
4   // ไม่จริง ถ้าเงื่อนไขจริง ทำส่วนนี้
5 } else {
6   // ไม่ตรงอะไรเลย ทำส่วนนี้
```

ในโค้ด MindPay จะเห็นรูปแบบ "เช็กกรณีพิเศษก่อน แล้วออกเลย" (early return) บ่อยมาก เช่นในสูตร Money Runway

src/domain/runway.ts

บรรทัด 83-88

```
83  if (spendable <= 0) {
84    return { ...base, status: 'below_floor', days: 0, depletionDay: today, capped: false };
85  }
86  if (averageSatang <= 0) {
87    return { ...base, status: 'no_spending', days: null, depletionDay: null, capped: false };
88  }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:83

อ่านว่า: "ถ้าเงินที่ใช้ได้ ≤ 0 ให้ส่งผลกลับเลยว่าต่ำกว่าเงินสำรอง · ถ้าค่าเฉลี่ย ≤ 0 ส่งกลับว่ายังไม่มีรายจ่าย" พอผ่าน 2 ด่านนี้ไปได้ โค้ดด้านล่างมั่นใจได้ว่าหารได้แน่นอน ไม่หารด้วยศูนย์

5. ฟังก์ชัน: เครื่องจักรรับของเข้า ส่งของออก

ฟังก์ชันคือ "ชุดคำสั่งที่ตั้งชื่อไว้" เรียกใช้ซ้ำได้ รับ พารามิเตอร์ (ของที่ส่งเข้า) และ return ผลลัพธ์ออกมา

ฟังก์ชัน 2 แบบ

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1  // แบบที่ 1: function
2  function toBaht(satang: number): number {
3    return satang / 100;
4  }
5
6  // แบบที่ 2: arrow function (ลูกศร =>) ทำงานเหมือนกัน เขียนสั้นกว่า
7  const toBaht2 = (satang: number) => satang / 100;
8
9  toBaht(8500); // ได้ 85
```

- (satang: number) = รับ 1 พารามิเตอร์ชื่อ satang ต้องเป็นตัวเลข
- : number หลังวงเล็บ = ฟังก์ชันนี้จะคืนค่าเป็นตัวเลข
- arrow function ที่ไม่มี { } จะ return ค่าทางขวาของ => ให้อัตโนมัติ
- export หน้าฟังก์ชัน = ให้ไฟล์อื่น import ไปใช้ได้

★ Pure function: หัวใจของ src/domain

ฟังก์ชันใน src/domain ทุกตัวเป็น **pure function** คือ (1) ใส่ค่าเดิม ได้ผลเดิมทุกครั้ง และ (2) ไม่ไปทำอะไรข้างนอก ไม่เรียกอื่นเตอร์เน็ต ไม่แตะหน้าจอ เหมือนเครื่องคิดเลข จึงทดสอบง่ายมาก: ใส่ตัวเลขแล้วตรวจคำตอบ อาจารย์มักถามว่า "ทำไมแยก domain" ให้ตอบเรื่องนี้

6. Array: รายการของหลายค่า

Array คือรายการเรียงลำดับ เขียนใน [] ค้นด้วยคอมมา ตำแหน่งแรกคือ 0

Array พื้นฐาน

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```

1 const amounts = [12000, 0, 8500, 30000];
2 amounts[0];           // 12000 (ตัวแรก)
3 amounts.length;       // 4 (มีกี่ตัว)
4 amounts.includes(0);   // true

```

MindPay เก็บรายการเงินทั้งหมดเป็น array ชื่อ txs (ย่อจาก transactions) และใช้ "เมธอด" ของ array ต่อไปนี้ตลอด ต้องจำให้ได้

เมธอด	ทำอะไร	ตัวอย่าง
filter	คัดเฉพาะตัวที่ผ่านเงื่อนไข ได้ array ใหม่ที่สั้นลง	txs.filter((t) => t.status === 'confirmed')
map	แปลงทุกตัว ได้ array ยาวเท่าเดิม	txs.map((t) => t.amountSatang)
reduce	รวมทุกตัวให้เหลือค่าเดียว เช่น ผลรวม	list.reduce((s, t) => s + t.amountSatang, 0)
find	หาตัวแรกที่ตรงเงื่อนไข (ไม่เจอได้ undefined)	txs.find((t) => t.id === id)
some	มีอย่างน้อย 1 ตัวที่ตรงไหม (true/false)	list.some((t) => t.slipRef === ref)
sort	เรียงลำดับ	days.sort()
slice	ตัดเอาบางส่วน	list.slice(0, 5) = 5 ตัวแรก

ตัว (t) => ... ที่ส่งเข้าไปในเมธอดเหล่านี้คือ arrow function ที่จะถูกเรียกกับสมาชิกแต่ละตัว t คือ "รายการที่กำลังดูอยู่"

ตัวอย่างจริง: ยอดคงเหลือคำนวณยัง (FR-2)

src/domain/summary.ts

บรรทัด 9-17

```

9 export const confirmedOnly = (txs: Transaction[]) => txs.filter((t) => t.status === 'confirmed');
10
11 /** Balance now = money at start + all confirmed income - all confirmed expense. */
12 export function computeBalance(openingBalanceSatang: number, txs: Transaction[]): number {
13   return confirmedOnly(txs).reduce(
14     (sum, t) => sum + (t.kind === 'income' ? t.amountSatang : -t.amountSatang),
15     openingBalanceSatang,
16   );
17 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/summary.ts:9

บรรทัด	ความหมาย
9	สร้างฟังก์ชัน confirmedOnly: รับรายการทั้งหมด คัดเฉพาะรายการที่ยืนยันแล้ว (กรองแบบร่างทิ้ง) ด้วย filter
11	คอมเมนต์อธิบายสูตร: ยอดตอนนี้ = เงินตอนเริ่ม + รายรับที่ยืนยัน - รายจ่ายที่ยืนยัน
12	ฟังก์ชัน computeBalance รับ 2 อย่าง: เงินตอนเริ่ม (สตางค์) และรายการทั้งหมด คำนวณเป็นตัวเลข
13	เอาเฉพาะรายการที่ยืนยันแล้ว แล้ว reduce รวมให้เหลือตัวเลขเดียว

บรรทัด	ความหมาย
14	สำหรับแต่ละรายการ t: ถ้าเป็นรายรับ บวกเข้า (+t.amountSatang) ถ้าเป็นรายจ่าย ลบออก (-t.amountSatang) แล้วเก็บผลไว้ใน sum
15	ค่าเริ่มต้นของ sum คือเงินตอนเริ่มใช้แอป

ลองคิดตาม: เริ่ม ฿3,000 (300,000 สตางค์) มีรายรับ ฿10,000 ที่ยืนยันแล้ว และรายจ่าย ฿85 กับแบบร่าง ฿200

```
sum เริ่ม          = 300,000
+ รายรับ 1,000,000 = 1,300,000
- รายจ่าย 8,500    = 1,291,500
(แบบร่าง 20,000 ถูกกรองทิ้งตั้งแต่บรรทัด 13)
ผล = 1,291,500 สตางค์ = ฿12,915.00
```

7. Object: ของ 1 ชิ้นที่มีหลายช่อง

Object คือกลุ่มของ "ชื่อช่อง: ค่า" อยู่ใน { } รายการเงิน 1 รายการใน MindPay คือ object แบบนี้

รายการ 1 รายการ (Transaction)

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const tx = {
2   id: 'demo-12',
3   kind: 'expense',
4   amountSatang: 8500,
5   categoryKey: 'food',
6   title: 'ข้าวมันไก่',
7   status: 'confirmed',
8 };
9 tx.title;           // 'ข้าวมันไก่' (เข้าถึงช่องด้วยจุด)
10 tx.amountSatang;   // 8500
```

เทคนิค object ที่เจอบ่อยในโค้ด

เขียนว่า	ชื่อเรียก	ความหมาย
<code>const { a, b } = obj;</code>	destructuring (แกะของ)	ดึงช่อง a และ b ออกมาเป็นตัวแปรในบรรทัดเดียว เช่น <code>const { balance, runway } = useMoney();</code>
<code>{ ...obj, status: 'confirmed' }</code>	spread (กระจาย)	คัดลอกทุกช่องของ obj มาเป็น object ใหม่ แล้วเปลี่ยนเฉพาะช่อง status ของเดิมไม่ถูกแก้
<code>[tx, ...list]</code>	spread กับ array	array ใหม่ที่มี tx อยู่หน้าสุด ตามด้วยของเดิมทั้งหมด (ใช้ตอนเพิ่มรายการใหม่ในบท A2)
<code>{ kind, title }</code>	shorthand	ย่อจาก <code>{ kind: kind, title: title }</code>

! ทำไมโค้ดชอบสร้าง object ใหม่แทนการแก้ไขเดิม

React (un A6) รู้ว่าข้อมูลเปลี่ยนด้วยการดูว่า "เป็น object ตัวใหม่หรือเปล่า" ถ้าเราแก้ไขใน object เดิมตรง ๆ React อาจไม่รู้ และหน้าจอก็ไม่อัปเดต โค้ดจึงใช้ { ...t, status: 'confirmed' } สร้างของใหม่เสมอ (ดูตัวอย่างใน [src/data/AppProvider.tsx:418](#))

8. วงซ้ำ (loop)

for ... of

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 let total = 0;
2 for (const t of txs) {           // ทำซ้ำกับรายการทีละตัว
3   if (t.kind !== 'expense') continue; // continue = ข้ามตัวนี้ ไปตัวถัดไป
4   total += t.amountSatang;       // += แปลว่า total = total + ...
5 }
```

ส่วน for (let i = 0; i < 20; i++) คือวนแบบนับรอบ: เริ่ม i = 0 ทำซ้ำตราบที่ i < 20 และเพิ่ม i ทีละ 1 (i++)
ส่วน break คือหยุดวนทันที

9. ข้อความ (string) ที่ใช้บ่อย

เขียนว่า	ผล
`เงินพอใช้อีก \${days} วัน`	template string (ใช้ backtick) แทรกค่าตัวแปรใน \${ } ได้ เช่น "เงินพอใช้อีก 16 วัน"
'2026-10-09'.slice(0, 7)	'2026-10' (ตัดตัวอักษรตำแหน่ง 0 ถึงก่อน 7)
'a,b'.split(',')	['a', 'b'] (แยกเป็น array)
' ข้าว '.trim()	'ข้าว' (ตัดช่องว่างหัวท้าย)
'KFC'.toLowerCase()	'kfc'
String(5).padStart(2, '0')	'05' (เติม 0 ข้างหน้าให้ครบ 2 ตัว ใช้กับวันที่)
s.replace(/บาง/g, '')	ลบคำว่า "บาง" ทุกที่ (/.../g คือ regular expression ดู un A5)

10. คณิตศาสตร์ที่ใช้

คำสั่ง	ผล	ใช้ที่ไหน
Math.floor(20.69)	20 (ปัดลงเสมอ)	จำนวนวัน Money Runway
Math.round(680.2857)	680 (ปัดตามปกติ)	ค่าเฉลี่ยต่อวันเป็นสตางค์
Math.max(0, -500)	0 (ตัวที่มากกว่า)	กันเงินที่ใช้ได้ติดลบ
Math.min(7, 3)	3 (ตัวที่น้อยกว่า)	จำนวนวันที่ใช้หารค่าเฉลี่ย ไม่เกิน 7
Math.abs(-85)	85 (ค่าสัมบูรณ์)	แสดงยอดเงินโดยไม่สนเครื่องหมาย

แบบฝึกหัด A4

1. ถ้า `profile` เป็น null ค่าของ `profile?.runwayFloorSatang ?? 50_000` คืออะไร
2. `[100, 200, 300].filter((x) => x > 150).map((x) => x / 100)` ได้อะไร
3. เริ่ม ฿500 มีรายการยืนยันแล้ว: รายรับ ฿1,000 รายจ่าย ฿300 และแบบร่างรายจ่าย ฿200 `computeBalance` คืนกี่สตางค์
4. `const t2 = { ...t, title: 'ข้าวผัด' }` แล้ว `t.title` เปลี่ยนไหม

i เฉลย

- 1) 50000 · 2) `[2, 3]` (กรองเหลือ 200, 300 แล้วหาร 100) · 3) $50,000 + 100,000 - 30,000 = 120,000$ สตางค์ (แบบร่างไม่นับ) · 4) ไม่เปลี่ยน `t2` เป็น object ใหม่ `t` ยังเป็นค่าเดิม

TypeScript: ภาษาที่ทีมเราใช้ และชนิดข้อมูลของ MindPay

TypeScript คือ JavaScript ที่บังคับให้บอกว่าข้อมูลแต่ละตัวเป็นชนิดอะไร ทำให้จับผิดได้ตั้งแต่ตอนพิมพ์ ก่อนผู้ใช้จะเจอ

ทำไมต้อง TypeScript

ใน JavaScript ธรรมดา ถ้าพิมพ์ `tx.amountSatnag` (สะกดผิด) โปรแกรมจะรันได้ แต่ได้ค่า `undefined` แล้วยอดเงินเพี้ยนตอนผู้ใช้งานใช้ ส่วน TypeScript จะขีดเส้นแดงใน VS Code กันที่ว่า "ไม่มีช่องชื่อนี้" ทีมเราเปิดโหมดเข้มงวดที่สุด ("`strict`": `true` ในไฟล์ `tsconfig.json`) และรัน `npm run typecheck` ก่อน build APK ทุกครั้ง

≈ เปรียบเทียบ

TypeScript เหมือนแบบฟอร์มราชการที่มีช่องบังคับว่า "ช่องนี้ต้องเป็นตัวเลข 13 หลัก" กรอกผิดเจ้าหน้าที่ตักกลับทันทีที่โต๊ะ ตักว่าไปรู้ตอนเอกสารเข้าระบบแล้ว เมื่อ build แอป ชนิดข้อมูลทั้งหมดจะถูกลบออก เหลือ JavaScript ธรรมดา ชนิดข้อมูลจึงช่วยเราตอนเขียนโค้ดเท่านั้น ไม่ทำให้แอปช้าลง

การบอกชนิดข้อมูล (type annotation)

บอกชนิดด้วย :

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const days: number = 16;
2 const title: string = 'ข้าวมันไก่';
3 let slipRef: string | null = null; // เป็น string หรือ null ก็ได้
4 function toBaht(satang: number): number { return satang / 100; }
```

หลายครั้งไม่ต้องเขียนชนิดเอง TypeScript เดาเองได้ (type inference) เช่น `const days = 16` รู้เองว่าเป็น `number` เอาเมาส์ชี้ใน VS Code จะเห็นชนิดที่มันเดา

ไฟล์ชนิดข้อมูลกลาง: `src/domain/types.ts`

ไฟล์นี้สำคัญที่สุดไฟล์หนึ่ง เพราะกำหนดว่า "รายการเงิน" และ "โปรไฟล์ผู้ใช้" หน้าตาเป็นยังไง ทุก FR ใช้ร่วมกัน

`src/domain/types.ts`

บรรทัด 1-55

```
1 /**
2  * Core data types for MindPay.
3  *
4  * Money is always stored as an integer number of satang (1 baht = 100 satang)
5  * so that adding many amounts never produces floating-point errors
6  * (0.1 + 0.2 !== 0.3 in JavaScript).
7  */
8
9 export type TxKind = 'income' | 'expense';
10
```

```

11 /** draft = not counted in any total until the user confirms it (Charter: slip results stay drafts). */
12 export type TxStatus = 'draft' | 'confirmed';
13
14 export type TxSource = 'manual' | 'slip';
15
16 export interface Transaction {
17   id: string;
18   kind: TxKind;
19   amountSatang: number;
20   categoryKey: string;
21   title: string;
22   note: string | null;
23   /** ISO 8601 instant, e.g. "2026-09-27T01:15:00.000Z" */
24   occurredAt: string;
25   source: TxSource;
26   status: TxStatus;
27   /** Transaction reference read from the slip (QR or text). Used for duplicate detection. */
28   slipRef: string | null;
29   /** SHA-256 of the processed slip image. Second line of duplicate detection. */
30   slipImageHash: string | null;
31   /** Lowest field confidence (0..1) reported by the slip reader, null for manual entries. */
32   ocrConfidence: number | null;
33   /** Fields the slip reader was unsure about, shown to the user during review. */
34   reviewFlags: string[];
35   createdAt: string;
36 }
37
38 /** Everything the user can type or change in the form. */
39 export type TransactionInput = Omit<Transaction, 'id' | 'createdAt'>;
40
41 export type CoachTone = 'friend' | 'coach' | 'senior';
42
43 export interface Profile {
44   id: string;
45   displayName: string;
46   /** Money the user had when they started using MindPay. Balance = opening + income - expense. */
47   openingBalanceSatang: number;
48   /** FR-6: the "low" line. Runway counts the days until balance would reach this amount. */
49   runwayFloorSatang: number;
50   monthlyBudgetSatang: number | null;
51   coachTone: CoachTone;
52   onboarded: boolean;
53 }
54
55 export type RangeKey = 'today' | '7d' | '1m' | '6m' | '1y';

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/types.ts:1

บรรทัด	ความหมาย
1-7	คอมเมนต์อธิบายกฎสำคัญ: เงินเก็บเป็นจำนวนเต็มสตางค์เสมอ (1 บาท = 100 สตางค์) เพราะใน JavaScript 0.1 + 0.2 ไม่เท่ากับ 0.3
9	union type: ชนิด TxKind เป็นได้แค่ 2 ค่า คือข้อความ 'income' (รายรับ) หรือ 'expense' (รายจ่าย) เครื่องหมาย แปลว่า "หรือ" ถ้าพิมพ์ 'incom' จะขึ้นเส้นแดงทันที

บรรทัด	ความหมาย
11–12	สถานะของรายการ: 'draft' (แบบร่าง ยังไม่นับ) หรือ 'confirmed' (ยืนยันแล้ว นับในยอด) คอมเมนต์อ้างถึง Charter ของทีมว่าผลจากสลิปเป็นแบบร่างก่อน
14	รายการมาจากไหน: จดเอง ('manual') หรือจากสลิป ('slip') รายการที่พุดจด (FR-3) นับเป็น manual
16	interface = พิมพ์เขียวของ object บอกว่า "รายการ 1 รายการ ต้องมีช่องอะไรบ้าง และแต่ละช่องชนิดอะไร"
17	id สหสัมพันธ์ของรายการ (ฐานข้อมูลสร้างให้)
18	kind ใช้ชนิด TxKind จากบรรทัด 9
19	amountSatang จำนวนเงินเป็นสตางค์ (บวกเสมอ ส่วนเข้าหรือออกดูจาก kind)
20	categoryKey สหสัมพันธ์ เช่น 'food', 'transport' (รายชื่อหมวดอยู่ใน src/domain/categories.ts)
21–22	ชื่อรายการ และบันทึกเพิ่มเติม (string null = ไม่มีบันทึกก็ได้)
23–24	เวลาที่เกิดรายการ เก็บแบบสากล ISO 8601 เช่น "2026-09-27T01:15:00.000Z" (Z = เวลา UTC ซึ่งช้ากว่าไทย 7 ชั่วโมง ตัวอย่างนี้คือ 08:15 น. เวลาไทย)
25–26	ที่มาและสถานะ ใช้ชนิดจากบรรทัด 14 และ 12
27–30	เลขอ้างอิงจากสลิป และ "ลายนิ้วมือ" ของรูป (SHA-256) ใช้กันสลิปซ้ำ 2 ชั้น (FR-4)
31–32	ความมั่นใจต่ำสุดที่ AI อ่านได้ (0 ถึง 1) รายการที่จดเองเป็น null
33–34	รายชื่อช่องที่ AI ไม่แน่ใจ เช่น ['amount'] ให้นำมาตรวจสอบสลิปไฮไลต์ช่องนั้น string[] = array ของข้อความ
35	เวลาที่สร้างแถวในระบบ
38–39	TransactionInput = Transaction ที่ ไม่มี id และ createdAt (omit = ตัดช่องออก) ใช้ตอนสร้างรายการใหม่ เพราะสองช่องนี้ฐานข้อมูลเป็นคนสร้าง
41	โทนโคช 3 แบบเดิม (ไม่ได้ใช้แล้ว เพราะ 29 ก.ย. เปลี่ยนเป็นน้องกล่าเสียงเดียว แต่ยังเก็บชนิดไว้ให้ตรงกับคอลัมน์ในฐานข้อมูล)
43–53	Profile ข้อมูลผู้ใช้ 1 คน: ชื่อ, เงินตอนเริ่มใช้แอป (บรรทัด 47), เงินสำรองของ FR-6 (บรรทัด 49), จบต่อเดือน (ไม่ตั้งก็ได้ = null), โทนโคช และตั้งค่าครั้งแรกเสร็จหรือยัง
55	ช่วงเวลาที่เลือกดูได้ในหน้าหลัก: วันนี้, 7 วัน, 1 เดือน, 6 เดือน, 1 ปี (FR-2)

ชนิดข้อมูลแบบพิเศษที่เจอบ่อย

เขียนว่า	ความหมาย	ตัวอย่างใน MindPay
A B	เป็น A หรือ B (union)	'draft' 'confirmed'
ชื่อ?: ชนิด	ช่องนี้ไม่ใส่ก็ได้ (optional)	now?: Date
ชนิด[]	array ของชนิดนั้น	Transaction[]
Omit<T, 'a'>	T ที่ตัดช่อง a ออก	Omit<Transaction, 'id' 'createdAt'>

เขียนว่า	ความหมาย	ตัวอย่างใน MindPay
Partial<T>	T ที่ทุกช่องไม่บังคับ ใช้ตอน "แก้บางช่อง"	updateTx(id, patch: Partial<TransactionInput>)
Pick<T, 'a' 'b'>	T ที่เอาเฉพาะช่อง a และ b	Pick<TxRow, 'kind' 'amount_satang'>
Record<K, V>	object ที่ชื่อช่องเป็นชนิด K ค่าเป็นชนิด V	Record<string, unknown>
Promise<T>	"สัญญาว่าจะได้ T ในอนาคต" (งานที่ต้องรอ)	getProfile(): Promise<Profile>
type X = ...	ตั้งชื่อเล่นให้ชนิด	type TxKind = 'income' 'expense'
x as Y	บอก TypeScript ว่า "ชื่อฉัน x เป็นชนิด Y" (ใช้น้อย ๆ)	data as TxRow[]
'confirmed' as const	ให้ถือว่าเป็นค่าตายตัว ไม่ใช่ string ทั่วไป	status: 'confirmed' as const
unknown	ยังไม่รู้ชนิด ต้องตรวจสอบก่อนใช้ (ปลอดภัยกว่า any)	context: unknown

import และ export: แอร์โด้ระหว่างไฟล์

src/data/AppProvider.tsx

บรรทัด 6-17

```

6 import { Storage } from './storage';
7 import { createContext, useCallback, useContext, useEffect, useMemo, useRef, useState, type
  ReactNode } from 'react';
8 import { AppState } from 'react-native';
9 import { authErrorMessage, authLinkErrorMessage, parseAuthLink } from '../domain/auth';
10 import { bkkDayKey, msUntilNextBkkMidnight } from '../domain/dates';
11 import { applyDeposit, reservedSatang, type GoalInput, type SavingsGoal } from '../domain/goals';
12 import { averageDailyExpense, computeRunway } from '../domain/runway';
13 import { computeBalance } from '../domain/summary';
14 import type { Profile, Transaction, TransactionInput } from '../domain/types';
15 import { createCloudRepo, createDemoRepo, resetDemoData, type Repo } from './repo';
16 import { clearLinkFromAddressBar, onIncomingLink, openingLink } from './authLinks';
17 import { supabase } from './supabase';

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:6

บรรทัด	ความหมาย
6	นำ Storage มาจากไฟล์ ./storage (จุด = โฟลเดอร์เดียวกัน ไม่ต้องใส่نامสกุลไฟล์)
7	นำเครื่องมือของ React มาใช้ ชื่อที่ไม่มีจุดนำหน้า เช่น 'react' คือแพ็คเกจใน node_modules
9-13	../domain/... = ถอยออกไป 1 โฟลเดอร์ (จาก data ไป src) แล้วเข้า domain นำฟังก์ชันคำนวณมาใช้
14	import type = นำมาเฉพาะชนิดข้อมูล (หายไปตอน build ไม่เพิ่มขนาดแอป)

11 type GoalInput ในวงเล็บปีกกา = import ทั้งฟังก์ชันและชนิดจากไฟล์เดียวกันในบรรทัดเดียว

ฝั่งไฟล์ต้นทางต้องเขียน export หน้าสิ่งที่ยอมให้คนอื่นใช้ ส่วน export default ใช้กับ "ของหลัก" ของไฟล์ เช่นหน้าจอแต่ละหน้า (Expo Router หาหน้าจอกจาก default export)

งานที่ต้องรอ: async, await และ Promise

การคุยกับอินเทอร์เน็ตใช้เวลา (อาจครั้งวินาทีหรือหลายวินาที) ถ้าแอปหยุดรอเฉย ๆ หน้าจอจะค้าง JavaScript จึงใช้ **Promise** คือ "ใบรับของ" ที่บอกว่าเดี๋ยวผลจะมา

async / await

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 async function save() {           // async = ฟังก์ชันนี้มีงานที่ต้องรอ
2   const tx = await repo.insert(input); // await = รอจนจับได้ผล แต่หน้าจอไม่ค้าง
3   setTxs((list) => [tx, ...list]); // ทำต่อเมื่อได้ผลแล้ว
4 }
```

ถ้ามีหลายงานที่ไม่ต้องรอกัน ใช้ Promise.all ทำพร้อมกัน เช่น ตอนโหลดข้อมูลครั้งแรก แอปโหลดโปรไฟล์ รายการ และกระปุกออม พร้อมกันทั้ง 3 อย่าง

src/data/AppProvider.tsx

บรรทัด 131-133

```
131   try {
132     // Goals never block the app: if they cannot be loaded, the rest still works.
133     const [p, list, g] = await Promise.all([r.getProfile(), r.listTransactions(),
      r.listGoals().catch(() => [] as SavingsGoal[])]);
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:131

บรรทัด 133: เรียก 3 งานพร้อมกัน แล้วรอจนเสร็จทั้งหมด ผลออกมาเป็น array 3 ช่อง แะใส่ตัวแปร p, list, g ส่วน .catch(() => []) ทำงานที่ 3 แปลว่า "ถ้าโหลดกระปุกออมไม่ได้ ให้ถือว่าไม่มีกระปุก" แอปส่วนอื่นยังใช้ได้ (Supabase ฝั่งเซิร์ฟเวอร์ ใน Edge Function ก็ใช้ Promise.allSettled อ่านสลิป 2 ครั้งพร้อมกันแบบเดียวกัน un A10)

จัดการความผิดพลาด: try, catch, throw

try / catch / finally

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 try {
2   await addTx(input);           // ลองทำ
3   toast({ message: 'บันทึกรายการแล้ว' });
4 } catch (e) {                   // ถ้ามี error ระหว่างทาง กระโดดมาที่นี่
5   toast({ message: 'บันทึกไม่สำเร็จ ลองอีกครั้ง', tone: 'error' });
6 } finally {                    // ทำเสมอ ไม่ว่าสำเร็จหรือพลาด
7   setBusy(false);
8 }
```

throw คือ "โยน error" ออกไปให้ผู้เรียกจัดการ ทีมเราสร้าง error ชนิดของตัวเองเพื่อแยกเหตุผล เช่น DuplicateSlipError (สลิปซ้ำ) ใน [src/data/repo.ts:14](#)

```

14 export class DuplicateSlipError extends Error {
15   constructor() {
16     super('รายการนี้มีอยู่แล้ว');
17     this.name = 'DuplicateSlipError';
18   }
19 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:14

class ... extends Error = สร้างชนิด error ใหม่โดยต่อยอดจาก Error ธรรมดา super(...) ส่งข้อความภาษาไทยให้ Error ตัวแม่ และตั้ง name ไว้ให้หน้าจอลูกได้ว่าเป็น error แบบไหน (ดู [src/app/transaction.tsx:126](#))

ทำไมเก็บเงินเป็นสตางค์ (จำนวนเต็ม)

คอมพิวเตอร์เก็บทศนิยมเป็นเลขฐานสอง ซึ่งเก็บบางค่าได้ไม่เป๊ะ ทดลองรันใน Node.js จริง:

```

0.1 + 0.2      → 0.30000000000000004
19.99 * 100    → 1998.9999999999998

```

ถ้าบวกเงินเป็นบาททศนิยมหลายพันรายการ เศษเล็ก ๆ จะสะสมจนยอดเพี้ยน ทางแก้คือเก็บเป็น **สตางค์ (จำนวนเต็ม)** ทั้งหมด เช่น ฿85.50 เก็บเป็น 8550 จำนวนเต็มบวกหลบกันได้แม่นยำ 100% แล้วค่อยแปลงเป็นบาทตอนแสดงผลบนจอ ฐานข้อมูลก็เก็บเป็นจำนวนเต็ม (คอลัมน์ amount_satang bigint)

อ่านโค้ดจริง: แปลงข้อความที่ผู้ใช้พิมพ์เป็นสตางค์

ฟังก์ชัน parseBahtToSatang ใน [src/domain/money.ts](#) ใช้ทุกที่ที่ผู้ใช้กรอกจำนวนเงิน (FR-1 จดรายการ, ตั้งค่า, เช็กร่อนจ่ายของ FR-6, กระทบยอด) และใช้กับยอดเงินที่ AI อ่านจากสลิปด้วย

```

1 /**
2  * Money helpers. All amounts are integers in satang.
3  */
4
5 /** The largest amount the app takes: ฿1,000,000,000 (the database caps goals at the same value).
6  */
7 export const MAX_SATANG = 100_000_000_000;
8
9 /** "๑๒๓" -> "123": Thai digits typed on a Thai keyboard. */
10 export function toAsciiDigits(s: string): string {
11   return s.replace(/[\u0E50-\u0E59]/g, (d) => String(d.charCodeAt(0) - 0x0e50));
12 }
13
14 /**
15  * Parse what a user (or a slip) wrote into satang.
16  * Accepts "1,234.50", "1234.5", "฿85", "85 บาท", " 1 234.00 ", Thai digits "๘๕", and a decimal
17  * comma as typed on phones set to such regions ("85,50" = 85.50, "12,34" = 12.34; never 1,234).
18  * Other commas must group thousands ("1,234.50"); anything else ("1,23,4") is refused.
19  * Returns null for anything that is not a positive amount with at most 2 decimals, or above
20  * MAX_SATANG. With `allowZero`, "0" and "0.00" give 0 (an opening balance may be zero).

```

```

21 export function parseBahtToSatang(input: string | number | null | undefined, opts: { allowZero?:
    boolean } = {}): number | null {
22   if (input === null || input === undefined) return null;
23   const ok = (satang: number) => (satang > MAX_SATANG ? null : satang > 0 || (opts.allowZero &&
    satang === 0) ? satang : null);
24   if (typeof input === 'number') {
25     if (!Number.isFinite(input) || input < 0) return null;
26     return ok(Math.round(input * 100));
27   }
28   let cleaned = toAsciiDigits(input)
29     .replace(/฿|บาท|THB|baht/gi, '')
30     .replace(/,/g, '');
31   if (/^\d+,\d{1,2}$/.test(cleaned)) cleaned = cleaned.replace(',', '.');
32   else if (cleaned.includes(',')) {
33     if (!/^\d{1,3}(\,\d{3})+(\.\d{1,2})?$/.test(cleaned)) return null;
34     cleaned = cleaned.replace(/,/g, '');
35   }
36   if (!/^\d+(\.\d{0,2})?$/.test(cleaned)) return null;
37   const [whole, frac = ''] = cleaned.split('.');
38   if (whole.length > 12) return null;
39   return ok(Number(whole) * 100 + Number((frac + '00').slice(0, 2)));
40 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/money.ts:1`

บรรทัด	ความหมาย
5–6	ค่าคงที่ MAX_SATANG = 100,000,000,000 สตางค์ = ฿1,000,000,000 เป็นจำนวนเงินสูงสุดที่แอปรับ (ฐานข้อมูลตั้งเพดานเดียวกัน กับค่าที่อ่านผิดใหญ่เกินจริง)
8–11	toAsciiDigits แปลงเลขไทย "๑๒๓" เป็น "123" คนที่ใช้แป้นพิมพ์ไทยอาจพิมพ์เลขไทยมา ทำงานโดยหาตัวอักษรในช่วงเลขไทย (รหัส U+0E50 ถึง U+0E59) แล้วลบด้วยรหัสของ "๐" (0x0e50) จะได้ 0–9
13–20	คอมเมนต์บอกว่ารับรูปแบบไหนได้บ้าง: "1,234.50", "1234.5", "฿85", "85 บาท", เลขไทย และทศนิยมแบบคอมมาที่มีมือถือบางประเทศใช้ ("85,50" = 85.50) แต่ "1,23,4" ไม่รับ
21	รับค่าเป็นข้อความหรือตัวเลขก็ได้ และมีตัวเลือก allowZero (ปกติ 0 ไม่ผ่าน เพราะรายการเงิน 0 บาทไม่มีความหมาย แต่ยอดเงินตั้งต้นเป็น 0 ได้) คืนค่าเป็นสตางค์ หรือ null ถ้าไม่ใช่จำนวนเงินที่ถูกต้อง
22	ไม่มีค่าเลย คืน null
23	ฟังก์ชันย่อย ok ตรวจสอบสุดท้าย: เกินเพดานไม่ผ่าน, มากกว่า 0 ผ่าน, เท่ากับ 0 ผ่านเฉพาะเมื่อ allowZero
24–27	ถ้าส่งเข้ามาเป็นตัวเลขอยู่แล้ว ต้องเป็นตัวเลขจริงและไม่ติดลบ แล้วคูณ 100 ปัดเป็นจำนวนเต็ม (ปัดเพื่อแก้ปัญหา $19.99 \times 100 = 1998.999\dots$)
28–30	ถ้าเป็นข้อความ: แปลงเลขไทย ลบสัญลักษณ์ ฿, คำว่า "บาท", THB, baht (ไม่สนตัวพิมพ์เล็กใหญ่) และลบช่องว่างทั้งหมด
31	ถ้ารูปแบบคือ "ตัวเลข,ตัวเลข 1–2 หลัก" เช่น "85,50" ถือว่าคอมมาคือจุดทศนิยม
32–35	ถ้ามีคอมมาแบบอื่น ต้องเป็นคอมมาคั่นหลักพันที่ถูกต้อง (เช่น 1,234 หรือ 1,234.50) ถึงจะลบคอมมาออก ไม่จิ้นคืน null
36	สุดท้ายต้องเหลือแค่ตัวเลข กับทศนิยมไม่เกิน 2 ตำแหน่ง ไม่จิ้นคืน null (เช่น "85.555" หรือ "abc")
37	แยกส่วนจำนวนเต็มกับทศนิยมที่จุด ถ้าไม่มีทศนิยมให้ frac เป็นข้อความว่าง

บรรทัด	ความหมาย
38	เกิน 12 หลักไม่รับ (กันตัวเลขยาวผิดปกติ)
39	คำนวณสตางค์ด้วยจำนวนเต็มล้วน: บาท × 100 + สตางค์ (เติม "00" แล้วตัด 2 ตัว ทำให้ ".5" กลายเป็น 50 สตางค์) ไม่มีการคูณทศนิยมเลย จึงไม่เพี้ยน

ผลจากการรับฟังค์จริง

ผู้ใช้พิมพ์	ได้สตางค์	เหตุผล
85	8500	-
85.5	8550	".5" → 50 สตางค์
1,234.50	123450	คอมมาคั่นหลักพันถูกต้อง
฿85 / 85 บาท / ๘๕	8500	ลบสัญลักษณ์และแปลงเลขไทย
85,50	8550	คอมมาเป็นทศนิยม
1,23,4	null	คอมมาผิดรูปแบบ
0	null	ต้องมากกว่า 0 (เว้นแต่ allowZero)
85.555	null	ทศนิยมเกิน 2 ตำแหน่ง
-5	null	มีเครื่องหมายลบ ไม่ใช่ตัวเลขล้วน
1000000000.01	null	เกินเพดาน ฿1,000,000,000

Regular expression (regex) อ่านยังงี้

บรรทัด 10 และ 29–36 มีของแปลก ๆ ใน / / เรียกว่า **regular expression** คือ "แพตเทิร์นสำหรับค้นหาข้อความ" ไม่ต้องจำทั้งหมด แค่อ่านออก

สัญลักษณ์	แปลว่า
\d	ตัวเลข 0–9 หนึ่งตัว
\s	ช่องว่าง (space, tab, ขึ้นบรรทัด)
+	ตัวข้างหน้ามี 1 ตัวขึ้นไป (\d+ = ตัวเลขที่หลักก็ได้)
{1,3}	ตัวข้างหน้ามี 1 ถึง 3 ตัว
?	ตัวข้างหน้ามีหรือไม่มีก็ได้
^ และ \$	ต้นข้อความ และท้ายข้อความ (ต้องตรงทั้งข้อความ ไม่ใช่แค่บางส่วน)
()	จับกลุ่ม
a b	a หรือ b
[๐-๙]	ตัวอักษร 1 ตัวในช่วงเลขไทย ๐–๙

สัญลักษณ์	แปลว่า
g, i (ท้าย /)	g = หากจุด, i = ไม่สนตัวพิมพ์เล็กใหญ่

ตัวอย่าง `/^\d{1,3}(\,\d{3})+(\.\d{1,2})?$/` อ่านว่า: "ขึ้นต้นด้วยตัวเลข 1-3 หลัก ตามด้วย (คอมมา + ตัวเลข 3 หลัก) อย่างน้อย 1 ชุด แล้วอาจมี (จุด + ตัวเลข 1-2 หลัก) และจบข้อความ" ซึ่งก็คือรูปแบบ 1,234 หรือ 12,345,678.50

แปลงสตริงกลับเป็นข้อความบาท

src/domain/money.ts

บรรทัด 42-63

```

42 /** 123450 -> "1,234.50" */
43 export function formatSatang(satang: number, opts: { decimals?: boolean } = {}): string {
44   const decimals = opts.decimals ?? true;
45   const negative = satang < 0;
46   const abs = Math.abs(Math.round(satang));
47   // Whole baht only: round to the nearest baht (#99.99 reads #100, not #99).
48   const whole = decimals ? Math.floor(abs / 100) : Math.round(abs / 100);
49   const frac = abs % 100;
50   const grouped = String(whole).replace(/\B(?=(\d{3})+(?!\d))/g, ',');
51   const body = decimals ? `${grouped}.${String(frac).padStart(2, '0')}` : grouped;
52   return negative ? `-${body}` : body;
53 }
54
55 /** 123450 -> "฿1,234.50"; with sign: "+฿1,234.50" / "-฿1,234.50" */
56 export function formatBaht(
57   satang: number,
58   opts: { decimals?: boolean; sign?: boolean } = {},
59 ): string {
60   const body = formatSatang(Math.abs(satang), opts);
61   if (opts.sign) return `${satang < 0 ? '-' : '+'}฿${body}`;
62   return `${satang < 0 ? '-' : ''}฿${body}`;
63 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/money.ts:42

บรรทัด	ความหมาย
42-43	formatSatang แปลง 123450 เป็น "1,234.50"
44	ตัวเลือก decimals ถ้าไม่ส่งมาใช้ค่าเริ่มต้น true (แสดงสตางค์)
45-46	จำไว้ว่าติดลบไหม แล้วทำงานกับค่าบวกที่ปิดเป็นจำนวนเต็มแล้ว
47-48	ส่วนบาท: ถ้าแสดงสตางค์ด้วย ใช้ปัดลง (สตางค์จะแสดงแยก) ถ้าไม่แสดงสตางค์ ปิดตามปกติ #99.99 จึงแสดงเป็น #100 ไม่ใช่ #99
49	สตางค์คือเศษจากการหาร 100
50	ใส่คอมมาค้นหลักพันด้วย regex (หาตำแหน่งที่ด้านหลัมีตัวเลขเป็นชุดละ 3 ตัวพอดี)
51-52	ต่อสตางค์ 2 หลัก (5 สตางค์ → "05") และเติมเครื่องหมายลบถ้าติดลบ
55-63	formatBaht เพิ่ม "฿" ข้างหน้า และถ้าขอเครื่องหมาย (sign) จะใส่ + หรือ - เช่น formatBaht(-8500, {sign: true}) → "-฿85.00"

i ฅลจรง

```
formatBaht(123450) → "฿1,234.50" · formatBaht(-8500, { sign: true }) → "-฿85.00" ·  
formatBaht(9999, { decimals: false }) → "฿100"
```

React: วาดหน้าจจากข้อมูล

React คือวิธีคิดที่ว่า "หน้าจ = ฟังก์ชันของข้อมูล" ข้อมูลเปลี่ยนเมื่อไร หน้าจวาดใหม่เอง ไม่ต้องสั่งแก้ทีละจุด

แนวคิดหลัก: หน้าจเป็นผลลัพธ์ของข้อมูล

สมัยก่อน ถ้ายอดเงินเปลี่ยน โปรแกรมเมอร์ต้องเขียนเองว่า "ไปหาข้อความยอดเงิน แก่ตัวเลข ไปหาต้นไม้ เปลี่ยนจำนวนใบ ไปหาป้ายสถานะ เปลี่ยนสี..." พลาดจุดเดียวหน้าจก็ขัดกันเอง React เปลี่ยนวิธีคิด: เราเขียนแค่ว่า "ถ้าข้อมูลเป็นแบบนี้ หน้าจหน้าตาแบบนี้" แล้ว React จะวาดใหม่ให้เองทุกครั้งที่ข้อมูลเปลี่ยน

ข้อมูล (state)

เช่น รายการทั้งหมด txs, ยอดคงเหลือ

↓ React เรียก component ใหม่

Component (ฟังก์ชันวาดหน้าจ)

อ่านข้อมูล แล้วคืน "หน้าตา" เป็น JSX

↓ React เทียบกับของเดิม แก้เฉพาะจุดที่ต่าง

หน้าจที่ผู้ใช้เห็น

Component: ชิ้นส่วนหน้าจที่เป็นฟังก์ชัน

Component คือฟังก์ชันที่ชื่อขึ้นต้นด้วยตัวใหญ่ และคืนค่าเป็น **JSX** (หน้าตาที่เขียนคล้าย HTML อยู่ในโค้ด) หน้าจทั้งหน้าก็เป็น component และปุ่มเล็ก ๆ ก็เป็น component ซ้อนกันเป็นชั้น ๆ

Component เล็กที่สุด

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 function Hello({ name }: { name: string }) {
2   return <Text>สวัสดี {name}</Text>;
3 }
4
5 // เรียกใช้ที่อื่น:
6 <Hello name="มินท์" />    // แสดง "สวัสดี มินท์"
```

ส่วน	ความหมาย
<Text>...</Text>	JSX แท็กเปิดและปิด เหมือน HTML (Text คือชิ้นส่วนแสดงข้อความของ React Native)
{name}	วงเล็บปีกกาใน JSX = "ใส่ค่าจากโค้ด JavaScript ตรงนี้"
{ name }: { name: string }	props คือค่าที่ผู้เรียกส่งเข้ามา (เหมือนพารามิเตอร์) ในที่นี้แกะเอาช่อง name ออกมา
<Hello name="มินท์" />	ใช้ component เหมือนแท็ก ส่ง props เป็น attribute แท็กที่ไม่มีของข้างในปิดด้วย />

ตัวอย่างจริง: T ชิ้นส่วนตัวหนังสือที่ทุกหน้าใช้

ทุกข้อความในแอปใช้ <T> แทนการตั้งขนาดและสีเอง เพื่อให้ทั้งแอปหน้าตาเหมือนกัน (กฎใน Design System un A11)

```

36 type Variant = 'display' | 'h1' | 'h2' | 'h3' | 'body' | 'small' | 'micro' | 'label';
37
38 const VARIANT_FONT: Record<Variant, string> = {
39   display: fonts.serif,
40   h1: fonts.serif,
41   h2: fonts.sansBold,
42   h3: fonts.sansSemi,
43   body: fonts.sans,
44   small: fonts.sans,
45   micro: fonts.sansMedium,
46   label: fonts.sansSemi,
47 };
48
49 export function T({
50   v = 'body',
51   color,
52   style,
53   children,
54   numberOfLines,
55   center,
56   selectable,
57   accessibilityRole,
58 }: {
59   v?: Variant;
60   color?: string;
61   style?: StyleProp<TextStyle>;
62   children: ReactNode;
63   numberOfLines?: number;
64   center?: boolean;
65   selectable?: boolean;
66   accessibilityRole?: 'header' | 'text' | 'alert';
67 }) {
68   const theme = useTheme();
69   const size = v === 'label' ? type.micro : type[v];
70   return (
71     <Text
72       accessibilityRole={accessibilityRole}
73       numberOfLines={numberOfLines}
74       selectable={selectable}
75       style={[
76         size,
77         {
78           fontFamily: VARIANT_FONT[v],
79           color: color ?? (v === 'small' || v === 'micro' ? theme.inkSoft : theme.ink),
80           textAlign: center ? 'center' : undefined,
81         },
82       ]} // Labels are text: inkSoft (inkFaint is for placeholders, rules and quiet icons, below
4.5:1).
83       v === 'label' && { letterSpacing: 0.8, textTransform: 'uppercase', color: color ??
theme.inkSoft },
84       style,
85     ]}
86   >
87     {children}
88   </Text>
89 );

```

บรรทัด	ความหมาย
36	ระดับตัวหนังสือมี 8 แบบ ตั้งแต่ใหญ่สุด display (ยอดเงินบนการ์ด) ไปจนเล็กสุด micro
38–47	ตารางจับคู่ว่าแต่ละระดับใช้ฟอนต์อะไร: หัวข้อใหญ่ใช้ Noto Serif Thai (มีหัว) ที่เหลือใช้ Anuphan น้ำหนักต่างกัน Record<Variant, string> บังคับให้ต้องใส่ครบทั้ง 8 ระดับ
49–57	ประกาศ component ชื่อ T พร้อมแกะ props ออกมา v = 'body' แปลว่าถ้าไม่ระบุระดับ ใช้ body
58–67	ชนิดของ props แต่ละตัว (เครื่องหมาย ? = ไม่บังคับ) children คือสิ่งที่อยู่ระหว่าง <T> กับ </T> ส่วน accessibilityRole บอกโปรแกรมอ่านหน้าจอสำหรับคนตาบอดว่าเป็นหัวข้อหรือข้อความแจ้งเตือน
68	เรียก useTheme () เอาชุดสีของธีมที่ผู้ใช้เลือก (สว่าง/มืด, ป่าทอง/ม่วง ฯลฯ)
69	เลือกขนาดและความสูงบรรทัดจากตาราง type (label ใช้ขนาดเท่า micro)
70–86	คืน JSX: ชิ้นส่วน Text ของ React Native พร้อม style ที่ซ้อนกันเป็นชั้น (array ของ style ตัวหลังทับตัวหน้า)
78	ฟอนต์ตามระดับ
79	สี: ถ้าผู้เรียกส่งสีมาใช้สีนั้น ไม่จันข้อความเล็กใช้สีรอง (inkSoft) ข้อความอื่นใช้สีหลัก (ink)
80	จัดกึ่งกลางถ้าส่ง center
82–83	ระดับ label (หัวข้อเล็กตัวพิมพ์ใหญ่ เช่น "MONEY RUNWAY") ห่างตัวอักษรกว้างขึ้น และใช้สีที่ผ่านเกณฑ์อ่านง่าย
84	style ที่ผู้เรียกส่งมาเองใส่ท้ายสุด จึงทับค่าทั้งหมดได้
87	แสดงสิ่งที่อยู่ข้างใน (ข้อความ)

ใช้งานแบบนี้: <T v="h3">จดรายการ</T> หรือ <T v="small" color={theme.watch}>อ่านวันที่ไม่ชัด</T>

State: ข้อมูลที่เปลี่ยนได้ และทำให้หน้าจอลoadใหม่

ตัวแปรธรรมดาในฟังก์ชันจะหายไปทุกครั้งที React วาดใหม่ ถ้าอยากให้ component "จำ" ค่าได้ ต้องใช้ **state** ผ่าน hook ชื่อ useState

useState

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const [amount, setAmount] = useState(''); // ค่าเริ่มต้น = ข้อความว่าง
2 // amount    = ค่าปัจจุบัน
3 // setAmount = ฟังก์ชันเปลี่ยนค่า เรียกแล้ว React วาดหน้าจอใหม่
4
5 <AmountField value={amount} onChangeText={setAmount} />
6 // ผู้ใช้พิมพ์ "8" → setAmount('8') → วาดใหม่ ช่องแสดง 8
```

หน้าจอดรายการ (FR-1) ใช้ state จำค่าทุกช่องในฟอร์ม ดูจากโค้ดจริง:

src/app/transaction.tsx

บรรทัด 35–46

```
35 const startKind: TxKind = existing?.kind ?? (kindParam === 'income' ? 'income' : 'expense');
```

```

36 const [kind, setKind] = useState<TxKind>(startKind);
37 const [amount, setAmount] = useState(existing ? satangToInput(existing.amountSatang) : '');
38 const [title, setTitle] = useState(existing?.title ?? '');
39 const [category, setCategory] = useState(existing?.categoryKey ?? defaultCategory(startKind));
40 const [categoryTouched, setCategoryTouched] = useState(!existing);
41 const [day, setDay] = useState(existing ? bkkDayKey(existing.occurredAt) : bkkDayKey(new
Date()));
42 const [time, setTime] = useState(existing ? bkkTime(existing.occurredAt) : bkkTime(new
Date()));
43 const [note, setNote] = useState(existing?.note ?? '');
44 const [busy, setBusy] = useState(false);
45 const [askDelete, setAskDelete] = useState(false);
46 const [amountError, setAmountError] = useState<string | null>(null);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:35

บรรทัด	ความหมาย
35	ประเภทเริ่มต้น: ถ้าแก้ไขรายการเดิม ใช้ประเภทเดิม ไม่จําเป็นดูว่าเปิดมาจากปุ่ม "เพิ่มเงินเข้า" ใหม่ (เป็นรายรับ) ไม่จําเป็นรายจ่าย
36	state ประเภท (รายรับ/รายจ่าย) useState<TxKind> บอกว่าเก็บได้แค่ 'income' หรือ 'expense'
37	state ช่องจํานวนเงิน เก็บเป็น ข้อความ ตามที่ผู้ใช้พิมพ์ (แปลงเป็นสตริงตอนกดบันทึก) ถ้าแก้ไขรายการเดิม เติมค่าเดิมให้
38–39	ชื่อรายการ และหมวด
40	จํานวนที่ผู้ใช้เลือกหมวดเองแล้วหรือยัง (ถ้ายัง แอปจะเดาหมวดจากชื่อที่พิมพ์ให้)
41–42	วันและเวลา (ค่าเริ่มต้นคือตอนนี้ ตามเวลาไทย)
43	บันทึกเพิ่มเติม
44–46	สถานะของหน้าจอเอง: กำลังบันทึกอยู่ไหม, กำลังถามยืนยันการลบไหม, ข้อความ error ใต้ช่องเงิน

! กฎของ Hook

ฟังก์ชันที่ขึ้นต้นด้วย use เรียกว่า **hook** มีกฎ 2 ข้อ: (1) เรียกได้เฉพาะใน component หรือใน hook อื่น (2) ต้องเรียกที่ระดับบนสุดของ component ห้ามอยู่ใน if หรือ loop เพราะ React จํา state ตามลำดับการเรียก ESLint ของโปรเจกต์ตรวจกฎนี้ให้อัตโนมัติ

Hook ที่ใช้ในโปรเจกต์

Hook	ทำอะไร	ตัวอย่างใน MindPay
useState	จําค่าที่เปลี่ยนได้ เปลี่ยนแล้ววาดใหม่	ทุกช่องในฟอร์ม, รายการทั้งหมดใน AppProvider
useEffect	ทำงาน "ข้างเคียง" หลังวาดเสร็จ เช่น ตั้งเวลา ฟัง event โหลดข้อมูล และเก็บกวาดเมื่อเลิกใช้	ตั้งนาฬิกาให้วันเปลี่ยนตอนเที่ยงคืน
useMemo	คำนวณแล้วจําผลไว้ คำนวณใหม่เฉพาะเมื่อข้อมูลที่ระบุเปลี่ยน	useMoney () คำนวณยอดเงินและ Runway

Hook	ทำอะไร	ตัวอย่างใน MindPay
useCallback	จำ "ฟังก์ชัน" ไว้ไม่ให้สร้างใหม่ทุกครั้ง	loadAll, activateRepo
useRef	กล่องจำค่าที่เปลี่ยนแล้ว ไม่ ต้องวาดใหม่	จำว่ากำลังใช้บัญชีไหนอยู่ (activeUserRef)
useContext	หยิบข้อมูลที่ "แชร์ทั้งแอป" มาใช้	useApp()
hook ของเรา	รวม hook หลายตัวเป็นชิ้นเดียว ตั้งชื่อขึ้นต้นด้วย use	useMoney, useTheme, useToast

ตัวอย่าง useEffect: วันใหม่เริ่มตอนเที่ยงคืนตามเวลาไทย

src/data/AppProvider.tsx

บรรทัด 182–212

```

182 // A new day starts at 00:00 Bangkok time: while the app is open (timer) or
183 // when it comes back to the screen the next morning (AppState / tab focus).
184 useEffect(() => {
185   let timer: ReturnType<typeof setTimeout>;
186   const check = () => {
187     const now = bkkDayKey(Date.now());
188     if (now !== todayRef.current) {
189       todayRef.current = now;
190       setToday(now);
191       setNewDay(true);
192       Storage.setItem(LAST_DAY_KEY, now).catch(() => {});
193     }
194     clearTimeout(timer);
195     timer = setTimeout(check, msUntilNextBkkMidnight() + 1000);
196   };
197   check();
198   // First open on a different day from the last one.
199   Storage.getItem(LAST_DAY_KEY)
200     .then((last) => {
201       if (last && last !== todayRef.current) setNewDay(true);
202       return Storage.setItem(LAST_DAY_KEY, todayRef.current);
203     })
204     .catch(() => {});
205   const sub = AppState.addEventListener('change', (s) => {
206     if (s === 'active') check();
207   });
208   return () => {
209     clearTimeout(timer);
210     sub.remove();
211   };
212 }, []);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:182

บรรทัด	ความหมาย
182–183	คอมเมนต์: วันใหม่เริ่ม 00:00 เวลากรุงเทพ ทั้งตอนเปิดแอปค้างไว้ (ใช้นาฬิกาจับเวลา) และตอนกลับมาเปิดแอปตอนเช้า
184	useEffect(() => {...}, []) array วางท้ายสุดแปลว่า "ทำครั้งเดียวตอน component เริ่มทำงาน"
185	ตัวแปรเก็บนาฬิกาจับเวลา

บรรทัด	ความหมาย
186–196	ฟังก์ชัน check: ดูวันที่ตอนนี้ตามเวลาไทย ถ้าไม่ตรงกับ "วันนี้" ที่จำไว้ แปลว่าข้ามวันแล้ว ให้อัปเดตวันนี้ (setToday) ซึ่งทำให้ตัวเลขทุกอย่างที่ขึ้นกับวันนี้คำนวณใหม่ (เช่น ค่าเฉลี่ย 7 วันของ FR-6)
191	ตั้งธง "วันใหม่" ให้น้องกล่าวกักตายนและสรุปเมื่อวาน
192	จดวันที่ล่าสุดลงเครื่อง
194–195	ยกเลิกนาฬิกาเดิม แล้วตั้งใหม่ให้ปลุกตอนเที่ยงคืนครั้งหน้า (+1 วินาทีเพื่อ) msUntilNextBkkMidnight คำนวณว่าเหลืออีกกี่มิลิวินาที
197	เช็กกันทีหนึ่งครั้งตอนเริ่ม
198–204	อ่านวันที่เปิดแอปครั้งล่าสุดจากเครื่อง ถ้าไม่ใช่วันนี้ แปลว่าเป็นการเปิดครั้งแรกของวัน
205–207	ฟังเหตุการณ์ "แอปกลับมาอยู่หน้าจอ" (เช่น เปิดแอปค้างไว้ข้ามคืน แล้วเข้ามาสลับกลับมา) แล้วเช็กวันใหม่อีกครั้ง
208–211	ฟังก์ชันเก็บกวาด (cleanup): ถ้า component นี้ถูกปิด ให้ยกเลิกนาฬิกาและเลิกฟังเหตุการณ์ ไม่ให้ทำงานค้างกินแบตเตอรี่

Context: แשרข้อมูลให้ทุกหน้าโดยไม่ต้องส่งต่อเป็นทอด ๆ

ถ้าหน้าหลัก หน้ารายการ และหน้าเงินพอถึง ต้องใช้รายการเงินชุดเดียวกัน จะส่ง props ต่อกันเป็นทอด ๆ ก็ยุ่ง React มี **Context** เหมือน "ประกาศเสียงตามสาย" ตัวให้ข้อมูล (Provider) อยู่บนสุด ใครอยู่ข้างในจะหยิบไปใช้ได้เลย MindPay มี Provider หลายชั้นครอบคลุมทั้งแอปไว้ใน `src/app/_layout.tsx` (un A7) ตัวสำคัญที่สุดคือ **AppProvider**

`src/data/AppProvider.tsx`

บรรทัด 87–121

```

87 const AppContext = createContext<AppContextValue | null>(null);
88
89 export function useApp() {
90   const ctx = useContext(AppContext);
91   if (!ctx) throw new Error('useApp must be used inside AppProvider');
92   return ctx;
93 }
94
95 /** Same as useApp, but null outside the provider (for small shared pieces such as the
96     companion). */
97 export function useAppMaybe() {
98   return useContext(AppContext);
99 }
100 export function AppProvider({ children }: { children: ReactNode }) {
101   const [status, setStatus] = useState<AuthStatus>('loading');
102   const [repo, setRepo] = useState<Repo | null>(null);
103   const [userId, setUserId] = useState<string | null>(null);
104   const [profile, setProfile] = useState<Profile | null>(null);
105   const [txs, setTxs] = useState<Transaction[]>([]);
106   const [carrySatang, setCarry] = useState(0);
107   const [goals, setGoals] = useState<SavingsGoal[]>([]);
108   const [loadError, setLoadError] = useState<string | null>(null);
109   const [refreshing, setRefreshing] = useState(false);
110   const [authNotice, setAuthNotice] = useState<AuthNotice | null>(null);
111   const [today, setToday] = useState(() => bkkDayKey(Date.now()));
112   const [newDay, setNewDay] = useState(false);

```

```

113 const todayRef = useRef(today);
114 const repoRef = useRef<Repo | null>(null);
115 /** The account whose data is loaded (or being loaded): prevents loading the same account
    twice. */
116 const activeUserRef = useRef<string | null>(null);
117 const handledLinks = useRef(new Set<string>());
118 /** Each load gets a number; an older load that finishes late is ignored (it may be another
    account's). */
119 const loadGen = useRef(0);
120 /** Changes made on this phone while a load is running (null = deleted), put back over what it
    returns. */
121 const localChanges = useRef(new Map<string, Transaction | null>());

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:87

บรรทัด	ความหมาย
87	สร้าง Context เปล่า ๆ ไว้ (เริ่มต้นเป็น null)
89–93	useApp(): hook ที่ทุกหน้าใช้หับข้อมูลกลาง ถ้าเรียกนอก Provider จะ error กันที (กันลืมหวน)
95–98	แบบที่ไม่ error แต่คืน null สำหรับชิ้นส่วนเล็กที่อาจอยู่นอก Provider
100	ประกาศ component AppProvider ซึ่งครอบหน้าจอทั้งหมด (children)
101	สถานะการเข้าสู่ระบบ: กำลังโหลด / ยังไม่เข้า / พร้อม
102–103	Repo ที่ใช้อยู่ (คลาวด์หรือเครื่อง) และ id ของผู้ใช้
104–107	ข้อมูลหลักของแอป: โปรไฟล์, รายการทั้งหมด, ยอดยกมาจากรายการเก่ามาก, กระปุกลูก
108–110	ข้อความ error ตอนโหลด, คำสั่งรีเฟรชใหม่, ข้อความแจ้งเรื่องบัญชี (เช่น "สมัครสำเร็จ")
111	วันนี้ตามเวลาไทย (ฟังก์ชันใน useState ทำงานครั้งเดียวตอนเริ่ม)
112	รวันใหม่
113–121	คำที่ใช้ useRef เพราะไม่ต้องวาดหน้าจอใหม่เมื่อเปลี่ยน: วันนี้ (ไว้ใช้ในนาฬิกา), Repo ปัจจุบัน, บัญชีที่กำลังโหลด, ลิงก์อีเมลที่จัดการแล้ว, เลขรอบการโหลด และการเปลี่ยนแปลงในเครื่องระหว่างโหลด

ส่วนท้ายของ AppProvider (บรรทัด 325–439) รวมข้อมูลและฟังก์ชันทั้งหมด (เช่น addTx, removeTx, saveProfile) เป็น object เดียว แล้วส่งให้ทุกหน้าผ่าน <AppContext.Provider value={value}> ที่บรรทัด 439

ทุกอย่างประกอบกัน: หน้าจอใช้ข้อมูลกลางยังไง

รูปแบบที่ทุกหน้าใช้

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```

1 export default function SomeScreen() {
2   const theme = useTheme();           // สีของธีม
3   const { txs, addTx } = useApp();     // ข้อมูลและคำสั่งกลาง
4   const { balance, runway } = useMoney(); // ตัวเลขที่คำนวณแล้ว
5   const [open, setOpen] = useState(false); // state ของหน้านี้เอง
6
7   return (
8     <Screen>

```

```

9      <T v="h1">ยอด {formatBaht(balance)}</T>
10      {runway.days !== null ? <T>เงินพอใช้ซัก {runway.days} วัน</T> : null}
11    </Screen>
12  );
13 }

```

บรรทัด `{runway.days !== null ? ... : null}` คือ **การแสดงผลแบบมีเงื่อนไข** ใน JSX: ถ้ามีจำนวนวัน แสดงข้อความ ถ้าไม่มี (null) ไม่แสดงอะไร จะเจอรูปแบบนี้ทุกหน้า อีกแบบที่เจอบ่อยคือ `{list.map((x) => <Row key={x.id} ... />)}` แสดงรายการหลายแถวจาก array (ต้องใส่ key ที่ไม่ซ้ำ ให้ React รู้ว่าแถวไหนเป็นแถวไหน)

★ สรุปบท A6 ใน 4 ประโยค

(1) Component คือฟังก์ชันที่คืนหน้าจอ (JSX) (2) props คือของที่ส่งเข้า component, state คือความจำของ component (3) เปลี่ยน state ด้วย `setXxx` แล้ว React วาดใหม่เอง (4) ข้อมูลที่ทั้งแอปใช้ร่วมกันอยู่ใน `AppProvider` หยิบใช้ด้วย `useApp()` และตัวเลขเงินหยิบด้วย `useMoney()`

React Native และ Expo: โค้ดชุดเดียว ได้ทั้งแอปและเว็บ

ทำไมทีมเราเขียนโค้ดครั้งเดียว แล้วได้ทั้งแอป Android, iPhone และเว็บ และการเปลี่ยนหน้าจอด้วย Expo Router ทำงานยังไ้

React Native คืออะไร

React (un A6) เดิมใช้ทำเว็บ ซึ่งวาดหน้าจอด้วย HTML แต่แอปมือถือไม่มี HTML **React Native** จึงเปลี่ยน "ชิ้นส่วนพื้นฐาน" ให้เป็นของจริงบนมือถือ เราเขียน `<View>` แล้วบน Android กลายเป็นกล่องของ Android บน iPhone กลายเป็นกล่องของ iOS แอปจึงสั้นเหมือนแอปที่เขียนด้วยภาษาของมือถือโดยตรง

สำหรับเว็บ มีแพ็คเกจ **react-native-web** (bussกั 37 ใน package.json) แปลชิ้นส่วนเดียวกันเป็น HTML เช่น `<View>` → `<div>` `<Text>` → `<span>` โค้ดชุดเดียวจึงได้ 3 แพลตฟอร์ม

ชิ้นส่วน React Native	คล้ายกับ (เว็บ)	ใช้ทำอะไร
View	div	กล่องสำหรับจัดวาง (การ์ด แถว คอลัมน์)
Text	span / p	ข้อความ (ข้อความทุกตัวต้องอยู่ใน Text)
Pressable	button	พื้นที่ที่แตะได้ มี onPress, onPressIn, onPressOut
ScrollView / FlatList / SectionList	หน้าที่เลื่อนได้	เลื่อนเนื้อหายาว ๆ (FlatList/SectionList เหมาะกับรายการยาวมาก เพราะวาดเฉพาะแถวที่เห็น)
TextInput	input	ช่องพิมพ์ (ห่ออยู่ใน Field / AmountField ของเรา)
Image	img	รูปภาพ
SafeAreaView	-	ไม่ให้เนื้อหาไปซ้อนกับรอยบาก แถบสถานะ หรือขอบล่างของมือถือ
Animated.View	-	กล่องที่ขยับได้ (แอนิเมชัน)

การจัดวางด้วย style

React Native ไม่มีไฟล์ CSS แต่ใส่ `style={{ ... }}` เป็น object ตรง ๆ ชื่อเหมือน CSS แต่เป็น camelCase เช่น `backgroundColor` หน่วยเป็นตัวเลขเฉย ๆ (หมายถึง "จุด" ที่ปรับตามความละเอียดจอเอง) การจัดวางใช้ **Flexbox**:

style	ความหมาย
<code>flexDirection: 'row'</code>	เรียงลูกในแนวนอน (ค่าเริ่มต้นคือแนวตั้ง 'column')
<code>flex: 1</code>	ขยายเต็มพื้นที่ที่เหลือ
<code>alignItems: 'center'</code>	จัดลูกให้อยู่กลาง (แกนขวาง)
<code>justifyContent: 'space-between'</code>	กระจายลูกให้ชิดหัวท้าย (แกนหลัก)

style	ความหมาย
gap: 8	ระยะห่างระหว่างลูกแต่ละตัว
padding / margin	ระยะห่างด้านใน / ด้านนอก
borderRadius	มุมโค้ง

ตัวอย่างจากชิ้นส่วน Row ที่ใช้ทั้งแอป: บรรทัดเดียวจบ

src/ui/components.tsx

บรรทัด 189–203

```

189 export function Row({
190   children,
191   gap = space.sm,
192   style,
193   align = 'center',
194   justify,
195 }: {
196   children: ReactNode;
197   gap?: number;
198   style?: StyleProp<ViewStyle>;
199   align?: ViewStyle['alignItems'];
200   justify?: ViewStyle['justifyContent'];
201 }) {
202   return <View style={[{ flexDirection: 'row', alignItems: align, justifyContent: justify, gap },
203     style] >{children}</View>;
204 }

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/ui/components.tsx:189

บรรทัด 202: สร้าง View ที่เรียงลูกแนวนอน (row) จัดกลางแนวตั้ง และเว้นระยะ gap ค่าเริ่มต้น 8 แล้วต่อด้วย style ที่ผู้เรียกส่งมา

Expo: ชุดเครื่องมือที่ทำให้ React Native ง่ายขึ้น

React Native เปล่า ๆ ต้องติดตั้ง Android Studio, Xcode และตั้งค่ามากมาย **Expo** ห่อทุกอย่างไว้ และมีโมดูลสำเร็จรูป (expo-camera, expo-speech ฯลฯ) ที่ใช้ได้เลย สิ่งที่ Expo ให้กับเรา:

- **Expo Go + dev server:** แก้โค้ดแล้วเห็นผลบนมือถือภายในไม่กี่วินาที
- **Expo Router:** ระบบเปลี่ยนหน้าจอจากโครงสร้างไฟล์ (หัวข้อถัดไป)
- **expo export:** ทำเว็บเวอร์ชันเป็นไฟล์นิ่ง ๆ เอาไปวางบนเว็บไซต์ได้
- **prebuild + EAS Build:** สร้างโปรเจกต์ Android/iOS และไฟล์ APK
- **EAS Update:** ส่งโค้ดใหม่ไปถึงแอปที่ลงแล้วโดยไม่ต้องลง APK ใหม่

! Expo ที่เราใช้เวอร์ชัน 57

Expo เปลี่ยนเร็วมาก ทุกเวอร์ชันมีคำสั่งที่เปลี่ยนชื่อหรือย้ายที่ ไฟล์ AGENTS.md ในโปรเจกต์จึงเตือนว่าก่อนใช้ API ของ Expo ให้เปิดเอกสารของเวอร์ชัน 57 (docs.expo.dev/versions/v57.0.0/) ไม่ใช่จำจากที่เคยเห็น

Expo Router: ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า

ใน `src/app/` ทุกไฟล์คือหน้าจอ ชื่อไฟล์คือที่อยู่ (URL) ของหน้านั้น บนเว็บพิมพ์ที่อยู่ได้ตรง ๆ เลย

ไฟล์	ที่อยู่	หน้าจอ	FR
<code>_layout.tsx</code>	-	โครงทั้งแอป: ฟอนต์, Provider, กฎว่าเข้าหน้าไหนได้ (ไม่ใช่หน้าจอ)	ทุก FR
<code>welcome.tsx</code>	<code>/welcome</code>	สมัคร เข้าสู่ระบบ ลืมรหัสผ่าน ลองใช้ข้อมูลตัวอย่าง	บัญชี
<code>onboarding.tsx</code>	<code>/onboarding</code>	ตั้งค่าเงินครั้งแรก (ยอดเงินตอนนี้, เงินสำรอง, ...)	FR-2, FR-6
<code>(tabs)/_layout.tsx</code>	-	แถบเมนูล่าง 4 แท็บ + ปุ่มสแกน	-
<code>(tabs)/index.tsx</code>	<code>/</code>	หน้าหลัก	FR-2 (+FR-6)
<code>(tabs)/transactions.tsx</code>	<code>/transactions</code>	แท็บรายการ ปฏิทินการใช้จ่าย	FR-1, FR-2
<code>(tabs)/runway.tsx</code>	<code>/runway</code>	แท็บเงินพอถึง	FR-6
<code>(tabs)/coach.tsx</code>	<code>/coach</code>	แท็บโค้ชน้องกล้า	FR-5
<code>transaction.tsx</code>	<code>/transaction</code>	จด/แก้/ลบ/ตรวจสอบสลิป 1 รายการ	FR-1, FR-4
<code>voice.tsx</code>	<code>/voice</code>	พูดจด	FR-3
<code>scan.tsx</code>	<code>/scan</code>	สแกนสลิป	FR-4
<code>drafts.tsx</code>	<code>/drafts</code>	สลิปรอยืนยัน	FR-4
<code>settings.tsx</code>	<code>/settings</code>	ตั้งค่า (เงินสำรอง, ธีม, เสียงน้องกล้า ฯลฯ)	หลาย FR
<code>goals.tsx, goal.tsx</code>	<code>/goals, /goal</code>	กระปุกออม	FR-6
<code>recap.tsx</code>	<code>/recap</code>	สรุปเดือนแบบเรื่องเล่า	FR-2
<code>achievements.tsx, skins.tsx</code>	<code>/achievements, /skins</code>	เหรียญ การกิจ ตูสกินน้องกล้า	FR-5 (เสริม)
<code>whatsnew.tsx</code>	<code>/whatsnew</code>	มีอะไรใหม่	-

- โฟลเดอร์ในวงเล็บ เช่น `(tabs)` เป็น "กลุ่ม" โมไฟล์ในที่อยู่ ใช้จัดหน้าที่อยู่ใต้แถบเมนูเดียวกัน
- `index.tsx` คือหน้าแรกของโฟลเดอร์นั้น (ที่อยู่ `/`)
- ไฟล์ `_layout.tsx` ไม่ใช่หน้าจอ แต่เป็น "กรอบ" ที่ครอบหน้าจอในโฟลเดอร์เดียวกัน

เปลี่ยนหน้าในโค้ด

โค้ด	ผล
<code>router.push('/scan')</code>	เปิดหน้าจอสแกนซ้อนขึ้นมา (กดกลับได้)
<code>router.push({ pathname: '/transaction', params: { kind: 'income' } })</code>	เปิดหน้าจอรายการ พร้อมส่งค่า kind=income (ปุ่ม "เพิ่มเงินเข้า")
<code>useLocalSearchParams()</code>	หน้าปลายทางอ่านค่าที่ส่งมา (src/app/transaction.tsx:28)
<code>router.navigate('/runway')</code>	ไปที่เก็บเงินพอถึง
<code>goBack()</code>	กลับหน้าก่อน ถ้าไม่มี (เช่น เปิดลิงก์ตรงบนเว็บ) กลับหน้าหลัก (src/ui/nav.ts)

อ่านโค้ด: `_layout.tsx` โครงของทั้งแอป

ไฟล์นี้ทำงานเป็นไฟล์แรกเมื่อเปิดแอป ทุกหน้าจออยู่ข้างในมัน

`src/app/_layout.tsx`

บรรทัด 1-51

```
1 import { useFonts } from 'expo-font';
2 import { Stack } from 'expo-router';
3 import * as SplashScreen from 'expo-splash-screen';
4 import { StatusBar } from 'expo-status-bar';
5 import { useCallback, useState, type ReactNode } from 'react';
6 import { Platform, View } from 'react-native';
7 import { SafeAreaProvider } from 'react-native-safe-area-context';
8 import {
9   Anuphan_400Regular,
10  Anuphan_500Medium,
11  Anuphan_600SemiBold,
12  Anuphan_700Bold,
13 } from '@expo-google-fonts/anuphan';
14 import { NotoSerifThai_600SemiBold, NotoSerifThai_700Bold } from '@expo-google-fonts/noto-serif-thai';
15 import { AppProvider, useApp } from '../data/AppProvider';
16 import { AutoScanProvider } from '../services/AutoScanProvider';
17 import { AuthNoticeHost } from '../ui/AuthNotice';
18 import { CelebrationProvider } from '../ui/effects';
19 import { ToastProvider } from '../ui/feedback';
20 import { LaunchIntro } from '../ui/LaunchIntro';
21 import { UpdateBanner } from '../ui/UpdateBanner';
22 import { mix, useTheme } from '../ui/theme';
23
24 // The native splash stays until the opening animation (LaunchIntro) takes over.
25 SplashScreen.preventAutoHideAsync().catch(() => {});
26
27 export default function RootLayout() {
28   const [fontsLoaded, fontError] = useFonts({
29     Anuphan_400Regular,
30     Anuphan_500Medium,
31     Anuphan_600SemiBold,
32     Anuphan_700Bold,
33     NotoSerifThai_600SemiBold,
34     NotoSerifThai_700Bold,
```

```

35   });
36   return (
37     <SafeAreaProvider>
38       <WebFrame>
39         <CelebrationProvider>
40           <AppProvider>
41             <ToastProvider>
42               <AutoScanProvider>
43                 <Navigator ready={fontsLoaded || !!fontError} />
44               </AutoScanProvider>
45             </ToastProvider>
46           </AppProvider>
47         </CelebrationProvider>
48       </WebFrame>
49     </SafeAreaProvider>
50   );
51 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/_layout.tsx:1

บรรทัด	ความหมาย
1–22	นำเครื่องมือมาใช้: โหลดฟอนต์, Stack (การซ้อนหน้า), หน้าจอเปิดแอป, แถบสถานะ, ฟอนต์ Anuphan 4 น้ำหนัก และ Noto Serif Thai 2 น้ำหนัก, Provider ต่าง ๆ ของเรา
24–25	สั่งให้หน้าจอเปิดแอป (splash สีเขียวรูปน้องกล้า) ค้างไว้ก่อน จนกว่าแอนิเมชันเปิดแอปจะมารับช่วง .catch(() => {}) = ถ้าทำไม่ได้ก็ไม่เป็นไร
27	export default = นี่คือ component หลักของไฟล์ Expo Router จะเรียกตัวนี้
28–35	โหลดฟอนต์ทั้ง 6 ตัว ได้ผลว่าโหลดเสร็จหรือยัง (fontsLoaded) หรือมี error ไหม
36–50	ซ้อน Provider เป็นชั้น ๆ เหมือนตุ๊กตาแม่ลูกดก ชั้นนอกสุดถึงชั้นในสุด: ขอบจอปลอดภัย → กรอบเว็บ → ระบบพลูลอง → ข้อมูลกลาง (AppProvider) → ข้อความแจ้งเตือน (toast) → สแกนสลิปอัตโนมัติ (FR-4) → ตัวเปลี่ยนหน้า ชั้นในใช้ของชั้นนอกได้ เช่น AutoScanProvider ใช้ข้อมูลจาก AppProvider ได้เพราะอยู่ข้างใน
43	ส่ง ready = ฟอนต์โหลดเสร็จ หรือโหลดพังกี่ให้ไปต่อ (ไม่ค้างหน้าขาวตลอดไป)

src/app/_layout.tsx

บรรทัด 53–73

```

53  /** On a computer browser, show the app at phone width in the middle of the screen. */
54  function WebFrame({ children }: { children: ReactNode }) {
55    const theme = useTheme();
56    if (Platform.OS !== 'web') return <>{children}</>;
57    return (
58      <View style={{ flex: 1, alignItems: 'center', backgroundColor: theme.dark ? mix(theme.bg,
59        '#000000', 0.55) : theme.surfaceAlt }}>
60        <View
61          style={{
62            flex: 1,
63            width: '100%',
64            maxWidth: 440,
65            backgroundColor: theme.bg,
66            overflow: 'hidden',
67            boxShadow: '0 0 40px rgba(0, 0, 0, 0.18)',
68          }}
69        >
70          {children}
71        </View>
72      </View>
73    );
74  }

```

```

70     </View>
71   </View>
72 );
73 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/_layout.tsx:53

บรรทัด ความหมาย

53–54 WebFrame: บนคอม แสดงแอปกว้างเท่ามือถือไว้ตรงกลางจอ

56 ถ้าไม่ใช่เว็บ (Platform.OS !== 'web') ไม่ต้องทำอะไร ค้นของข้างในตรง ๆ (<>...</> คือกล่องที่มองไม่เห็น)

58 พื้นหลังเต็มจอ จัดกลาง สีเข้มขึ้นเล็กน้อยในโหมดมืด

59–68 กล่องแอป กว้างสูงสุด 440 จุด มีเงารอบ ๆ ให้ดูเหมือนมือถือวางบนโต๊ะ

src/app/_layout.tsx

บรรทัด 75–126

```

75 function Navigator({ ready }: { ready: boolean }) {
76   const theme = useTheme();
77   const { status, repo, profile } = useApp();
78   const loading = !ready || status === 'loading';
79   const [introDone, setIntroDone] = useState(false);
80   const finishIntro = useCallback(() => setIntroDone(true), []);
81
82   const signedIn = status === 'ready' && !!repo;
83   const needsOnboarding = signedIn && !!profile && !profile.onboarded;
84   const inApp = signedIn && !needsOnboarding;
85
86   return (
87     <>
88       <StatusBar style={!introDone || theme.dark ? 'light' : 'dark'} />
89       {loading ? null : <Screens signedIn={signedIn} needsOnboarding={needsOnboarding} inApp=
{inApp} />}
90       {loading || !introDone ? null : <AuthNoticeHost />}
91       {loading || !introDone ? null : <UpdateBanner />}
92       <LaunchIntro ready={!loading} fontsReady={ready} onDone={finishIntro} />
93     </>
94   );
95 }
96
97 function Screens({ signedIn, needsOnboarding, inApp }: { signedIn: boolean; needsOnboarding:
boolean; inApp: boolean }) {
98   const theme = useTheme();
99   return (
100     <>
101       /* Screens slide in from the side; scanning and adding a transaction rise from the bottom.
*/
102       <Stack screenOptions={{ headerShown: false, contentStyle: { backgroundColor: theme.bg },
animation: 'slide_from_right' }}>
103         <Stack.Protected guard={!signedIn}>
104           <Stack.Screen name="welcome" options={{ animation: 'fade' }} />
105         </Stack.Protected>
106         <Stack.Protected guard={needsOnboarding}>
107           <Stack.Screen name="onboarding" options={{ animation: 'fade' }} />
108         </Stack.Protected>
109         <Stack.Protected guard={inApp}>
110           <Stack.Screen name="(tabs)" options={{ animation: 'fade' }} />

```

```

111     <Stack.Screen name="transaction" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
112     <Stack.Screen name="scan" options={{ presentation: 'fullScreenModal', animation:
'slide_from_bottom' }} />
113     <Stack.Screen name="voice" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
114     <Stack.Screen name="achievements" />
115     <Stack.Screen name="skins" />
116     <Stack.Screen name="recap" options={{ presentation: 'fullScreenModal', animation:
'fade' }} />
117     <Stack.Screen name="goals" />
118     <Stack.Screen name="whatsnew" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
119     <Stack.Screen name="goal" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
120     <Stack.Screen name="drafts" />
121     <Stack.Screen name="settings" />
122   </Stack.Protected>
123 </Stack>
124 </>
125 );
126 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/_layout.tsx:75`

บรรทัด	ความหมาย
75–78	Navigator ดูสถานะจากข้อมูลกลาง: เข้าสู่ระบบหรือยัง มี Repo ใหม่ โปรไฟล์ตั้งค่าเสร็จใหม่ และยังโหลดอยู่ไหม
79–80	จำว่าแอนิเมชันเปิดแอปจบหรือยัง
82–84	คำนวณ 3 สถานะ: signedIn (เข้าสู่ระบบแล้ว), needsOnboarding (เข้าแล้วแต่ยังไม่ตั้งค่าเงิน), inApp (เข้าแล้วและตั้งค่าแล้ว ใช้แอปได้เต็มที่)
88	สไลด์คอนแทกสถานะด้านบน (สว้างบนพื้นเข้ม)
89	ถ้ายังโหลดอยู่ ยังไม่แสดงหน้าจอ
90–91	หลังแอนิเมชันจบ แสดงหน้าต่างแจ้งเรื่องบัญชี และแถบ "มีอัปเดตใหม่"
92	แอนิเมชันเปิดแอป (น้องกล้าไฟล์) ช้อนอยู่บนสุดจนจบ
97–102	Stack = หน้าจอซ้อนกันเป็นกอง หน้าใหม่กับหน้าเก่า กดกลับก็หิบบนหน้าออก ตั้งค่าทุกหน้า: ไม่มีหัวหน้าสำเร็จรูป, พื้นตามืด, เลื่อนเข้ามาจากขวา
103–105	Stack.Protected = ประตูกุญแจ: หน้า welcome เข้าได้เฉพาะตอน ยังไม่ เข้าสู่ระบบ
106–108	หน้า onboarding เข้าได้เฉพาะตอนเข้าแล้วแต่ยังไม่ตั้งค่าเงิน
109–122	หน้าในแอปทั้งหมดเข้าได้เฉพาะตอนพร้อมใช้งาน บางหน้าแสดงแบบ modal (เลื่อนขึ้นจากล่าง) เช่น จดรายการ พุดจด, บางหน้าเต็มจอ เช่น สแกนสลิป สรุปเดือน

★ ผลของ Stack.Protected

ผู้ใช้ไม่ต้องทำอะไร พอสมัครเสร็จ สถานะเปลี่ยนเป็น needsOnboarding ยามจะปิดประตู welcome และเปิดประตู onboarding แอปจึงพาไปหน้าตั้งค่าเงินเอง พอตั้งค่าเสร็จก็ไปหน้าหลักเอง และกดย้อนกลับก็ไม่เจอหน้าสมัครอีก (ตรงกับ acceptance criteria ในเอกสาร Traceability)

แถบเมนูล่าง: (tabs)/_layout.tsx

src/app/(tabs)/_layout.tsx

บรรทัด 23–28

```
23 const TABS: Record<string, { label: string; icon: IconName; iconActive: IconName }> = {
24   index: { label: 'หน้าหลัก', icon: 'home-outline', iconActive: 'home' },
25   transactions: { label: 'รายการ', icon: 'list-outline', iconActive: 'list' },
26   runway: { label: 'เงินพอถึง', icon: 'leaf-outline', iconActive: 'leaf' },
27   coach: { label: 'โค้ช', icon: 'chatbubble-ellipses-outline', iconActive: 'chatbubble-ellipses' },
28 };
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/_layout.tsx:23

บรรทัด 23–28: ตารางของ 4 แท็บ แต่ละแท็บมีชื่อไทย ไอคอนตอนไม่ได้เลือก และไอคอนตอนเลือก (แบบทึบ) ชื่อช่อง (index, transactions, runway, coach) ตรงกับชื่อไฟล์ในโฟลเดอร์ (tabs)

src/app/(tabs)/_layout.tsx

บรรทัด 134–186

```
134   {routes.slice(0, half).map((r, i) => tab(r, i))}
135   <View style={{ width: 76, alignItems: 'center' }}>
136     <View style={{ width: 60, height: 60, marginTop: -26, alignItems: 'center',
137       justifyContent: 'center' }}>
138       <PulseRing size={60} active times={auto.state.phase === 'scanning' ? 'always' : 3}
139         color={theme.accent} />
140       <Animated.View style={{ transform: [{ scale: press.scale }] }}>
141         <Pressable
142           accessibilityRole="button"
143           accessibilityLabel="สแกนสลิป"
144           onPressIn={press.onPressIn}
145           onPressOut={press.onPressOut}
146           onPress={() => {
147             Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Medium).catch(() => {});
148             router.push('/scan');
149           }}
150           style={{
151             width: 60,
152             height: 60,
153             borderRadius: 30,
154             backgroundColor: theme.hero,
155             borderWidth: 3,
156             borderColor: theme.accent,
157             alignItems: 'center',
158             justifyContent: 'center',
159             shadowColor: theme.accent,
160             shadowOpacity: 0.35,
161             shadowRadius: 10,
162             shadowOffset: { width: 0, height: 4 },
163             elevation: 8,
164           }}
165         />
166       />
167     />
168   />
```

```

164         <Ionicons name="scan" size={26} color={theme.heroAccent} />
165     </Pressable>
166 </Animated.View>
167 </View>
168     <Text style={[type.micro, { fontFamily: fonts.sansSemi, color: theme.ink, marginTop: 4
    ]}]>สแกนสลิป</Text>
169 </View>
170     {routes.slice(half).map((r, i) => tab(r, i + half))}
171 </View>
172 );
173 }
174
175 export default function TabsLayout() {
176 // Records today's visit and hands out น้องกล้า's skins for finished missions and events.
177 useKlaSync();
178 return (
179     <Tabs tabBar={props} => <TabBar {...props} /> screenOptions={{ headerShown: false }}>
180         <Tabs.Screen name="index" />
181         <Tabs.Screen name="transactions" />
182         <Tabs.Screen name="runway" />
183         <Tabs.Screen name="coach" />
184     </Tabs>
185 );
186 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/_layout.tsx:134

บรรทัด	ความหมาย
134	วาดแท็บครั้งแรก (หน้าหลัก, รายการ) ทางซ้าย
135–169	ปุ่มทองตรงกลาง: วงกลม 60 จุด ลอยขึ้นมา 26 จุด (marginTop: -26) มีวงแหวนกระเพื่อม (PulseRing) 3 รอบ ตอนเปิดแอป และกระเพื่อมตลอดระหว่างสแกนอัตโนมัติ
144–147	แตะแล้วมือถือสั่นแรงปานกลาง และเปิดหน้าสแกน
170	วาดแท็บครั้งหลัง (เงินพอถึง, โค้ช) ทางขวา
175–186	component หลัก: ใช้แถบเมนูที่ทีมออกแบบเอง (tabBar=) แทนของสำเร็จรูป และประกาศ 4 แท็บ useKlaSync() บันทึกว่าวันนี้เปิดแอป (ใช้บันทึกการกิจ "เปิดแอป 30 วันติด")

โค้ดที่ต่างกันระหว่างมือถือกับเว็บ

บางอย่างทำได้ต่างกัน เช่น มือถือเก็บข้อมูลด้วย SQLite แต่เว็บใช้ localStorage ของเบราว์เซอร์ ทีมเราแก้ด้วย **ไฟล์คู่**: ตั้งชื่อ storage.ts สำหรับมือถือ และ storage.web.ts สำหรับเว็บ ตอน build ระบบ (Metro bundler) เลือกไฟล์ให้เองตามแพลตฟอร์ม ที่เหลือของแอป import แค่ './storage'

src/data/storage.ts

บรรทัด 1–12

```

1 /**
2  * Small key-value store used for app
  settings and demo data.
3  * Phone: SQLite-backed store (survives
  app restarts).
4  * Web: see storage.web.ts (browser
  localStorage).

```

src/data/storage.web.ts

บรรทัด 1–24

```

1 /** Web version of storage.ts: the
  browser's localStorage, wrapped so it
  never throws. */
2 export const Storage = {
3   async getItem(key: string):
    Promise<string | null> {
4     try {

```

```

5  */
6  import KV from 'expo-sqlite/kv-store';
7
8  export const Storage = {
9    getItem: (key: string) =>
10     KV.getItem(key),
11    setItem: (key: string, value: string) =>
12     KV.setItem(key, value),
13    removeItem: (key: string) =>
14     KV.removeItem(key),
15  };

```

► เปิดใน VS
Code (เว็บ)

GitHub

VS Code ในเครื่อง: Ctrl+P →
src/data/storage.ts:1

```

5      return
6      window.localStorage.getItem(key);
7    } catch {
8      return null;
9    },
10   async setItem(key: string, value:
11     string): Promise<void> {
12     try {
13       window.localStorage.setItem(key,
14         value);
15     } catch {
16       // Private mode or storage full:
17       // keep working in memory.
18     }
19   },
20   async removeItem(key: string):
21     Promise<void> {
22     try {
23       window.localStorage.removeItem(key);
24     } catch {
25       // ignore
26     }
27   },
28 };

```

► เปิดใน VS
Code (เว็บ)

GitHub

VS Code ในเครื่อง: Ctrl+P →
src/data/storage.web.ts:1

ทั้งสองไฟล์มี getItem, setItem, removeItem หน้าตาเดียวกัน ฉบับเว็บห่อด้วย try/catch เพราะเบราว์เซอร์โหมดส่วนตัวอาจห้ามเก็บข้อมูล คู่มือฉบับนี้มีทั้งหมด 5 คู่มือ:

ไฟล์	มือถือ	เว็บ
data/storage	SQLite (expo-sqlite)	localStorage
data/sessionStorageSetup	ที่เก็บการเข้าสู่ระบบของ Supabase บนมือถือ	ของเบราว์เซอร์
services/gallery	อ่านแกลเลอรีทั้งหมด (expo-media-library)	ให้ผู้ใช้เลือกรูปเอง (เบราว์เซอร์ไม่อนุญาตให้เว็บเปิดแกลเลอรี)
services/speech	ตัวฟังเสียงของมือถือ (Google/Apple)	Web Speech API ของเบราว์เซอร์
services/tts	เสียงอ่านของมือถือ (expo-speech)	speechSynthesis ของเบราว์เซอร์

ส่วนที่ต่างเล็กน้อยใช้ Platform.OS ในไฟล์เดียวกัน เช่น หน้าจอรายการบนเว็บให้พิมพ์วันที่ แต่บน Android เปิดปฏิทินของเครื่อง ([src/app/transaction.tsx:288](#))

Native module แบบไม่บังคับ

src/services/speech.ts

บรรทัด 1–24

```

1  /**
2   * Speech to text on the phone ("พูดจด"), in Thai, with the phone's own speech

```

```

3  * recognizer (Google on Android, Apple on iOS) through expo-speech-recognition.
4  *
5  * The native module is looked up optionally: app builds made before it was
6  * added do not have it, and an over-the-air update must not crash them. There
7  * the voice screen simply offers typing instead (the keyboard's own mic works).
8  * The web version is speech.web.ts.
9  */
10 import { requireOptionalNativeModule } from 'expo';
11
12 interface Subscription {
13   remove(): void;
14 }
15
16 interface NativeSpeech {
17   start(options: Record<string, unknown>): void;
18   stop(): void;
19   abort(): void;
20   requestPermissionsAsync(): Promise<{ granted: boolean; canAskAgain?: boolean }>;
21   isRecognitionAvailable(): boolean;
22   addListener(event: string, listener: (e: never) => void): Subscription;
23 }
24
25 const Speech = requireOptionalNativeModule<NativeSpeech>('ExpoSpeechRecognition');

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/speech.ts:1

บรรทัด 24 ใช้ requireOptionalNativeModule = "ขอโมดูลฟังเสียง ถ้าไม่มีก็ให้ null ไม่ต้อง crash" เพราะแอปที่ลงไว้ก่อนเพิ่มฟีเจอร์พูดจดไม่มีโมดูลนี้ แต่ยังได้รับโค้ดใหม่ผ่าน EAS Update (หัวข้อถัดไป) ถ้าเรียกแบบบังคับ แอปเก่าจะเดิ๊งปิดแบบนี้แอปเก่าแค่แสดงช่องพิมพ์แทนไมค์

จากโค้ด สู่เว็บและแอปที่ผู้ใช้ลง

โค้ดใน src/ (TypeScript + React Native)



เว็บ

expo export --platform web รวมโค้ดเป็นไฟล์ JavaScript + HTML ในโฟลเดอร์ dist แล้ววางบน GitHub Pages

APK (Android)

expo prebuild สร้างโปรเจกต์ Android จาก app.json แล้ว Gradle build เป็นไฟล์ .apk

EAS Update

ส่งเฉพาะโค้ด JavaScript ใหม่ แอปที่ลงแล้วโหลดเองตอนเปิดแอป ใช้ตอนเปิดครั้งถัดไป

ทั้ง 3 ทางทำอัตโนมัติทุกครั้งที่เรา push โค้ดขึ้น branch main บน GitHub (บน A13) ไฟล์ตั้งค่าของแอปคือ app.json ส่วนสำคัญ:

ค่าใน app.json	ความหมาย
"name": "MindPay", "slug": "mindpay"	ชื่อแอปที่เห็นใต้ไอคอน
"version": "1.0.0" + "runtimeVersion": {"policy": "appVersion"}	EAS Update ส่งโค้ดใหม่ให้เฉพาะแอปที่เวอร์ชันตรงกัน ถ้าเพิ่ม native module ใหม่ ต้องเพิ่มเวอร์ชัน แล้วลง APK ใหม่
"android.package"	รหัสแอป com.socratesandskeletons.mindpay

ค่าใน app.json	ความหมาย
"blockedPermissions"	บล็อกสิทธิ์กล้อง วิทยุโอ เสียงในเครื่อง ที่ไม่ได้ใช้ (ความเป็นส่วนตัว)
"plugins"	ตั้งค่า native ของโมดูล เช่น ข้อความขอสิทธิ์แคลเลอร์/ไมโครโฟนเป็นภาษาไทย
"scheme": "mindpay"	ลิงก์ที่ขึ้นต้นด้วย mindpay:// เปิดแอปได้ (ใช้กับลิงก์ยืนยันอีเมล)
"updates"	ที่อยู่ของเซิร์ฟเวอร์ EAS Update และให้เช็กอัปเดตทุกครั้งที่เปิดแอป
"web": {"output": "single"}	เว็บเป็นหน้าเดียว (single page app) ให้ JavaScript เปลี่ยนหน้าเอง

ไฟล์ `app.config.js` เพิ่มค่าเฉพาะตอน build เว็บ: เว็บอยู่ที่ `/Socrates-and-Skeletons - /` บน GitHub Pages จึงต้องตั้ง `baseUrl` แต่ห้ามใส่ค่านี้ในแอปมือถือ (แอปจะหาฟอนต์และรูปผิดที่)

วิธีอ่านโค้ด MindPay ด้วยตัวเอง

โค้ดทั้งแอปมีเกือบ 19,000 บรรทัด ไม่มีใครจำได้หมด แต่ทุกไฟล์ใช้รูปแบบซ้ำ ๆ กัน รู้รูปแบบแล้วจะอ่านไฟล์ไหนก็ออก

ขั้นตอนอ่านไฟล์ที่ไม่เคยเห็น

1. **อ่านคอมเมนต์บนสุดของไฟล์ก่อน** ทุกไฟล์ในโปรเจกต์เริ่มด้วยคอมเมนต์ `/** ... */` บอกว่าไฟล์นี้ทำอะไร เป็นของ FR ไหน และมีกฎอะไรสำคัญ (ภาษาอังกฤษง่าย ๆ ใช้ Google Translate ช่วยได้)
2. **ดูบรรทัด import** บอกว่าไฟล์นี้พึ่งใครบ้าง ถ้า import จาก `../domain` แปลว่ามีการคำนวณ ถ้าจาก `../data/AppProvider` แปลว่าใช้ข้อมูลกลาง
3. **กด Ctrl+Shift+O** ดูรายชื่อฟังก์ชันทั้งหมดในไฟล์ เหมือนดูสารบัญ
4. **หาฟังก์ชันที่มี export** นั่นคือ "ประตูหน้าบ้าน" ที่ไฟล์อื่นเรียก ฟังก์ชันที่ไม่มี export คือผู้ช่วยภายใน
5. **อ่านชนิดข้อมูลก่อนอ่านตัวโค้ด** ดูว่ารับอะไรเข้า คืนอะไรออก ก็เข้าใจไปครึ่งหนึ่งแล้ว
6. **หาเทสต์ของมัน** กด Shift+F12 ที่ชื่อฟังก์ชัน ถ้ามีเทสต์ใน `__tests__` จะเห็นตัวอย่างการใช้งานจริงพร้อมคำตอบที่ถูกต้อง อ่านง่ายกว่าตัวโค้ดเสียอีก

รูปแบบที่เจอซ้ำทั้งแอป (ตารางโยง)

เห็นอะไร	แปลว่าอะไร
ชื่อลงท้าย Satang	จำนวนเงินเป็นสตางค์ (จำนวนเต็ม) หาร 100 ได้บาท
ชื่อขึ้นต้น bkk เช่น <code>bkkDayKey</code> , <code>bkkToIso</code>	ฟังก์ชันจัดการวันเวลาแบบเวลาไทย (กรุงเทพ UTC+7) อยู่ใน <code>src/domain/dates.ts</code>
ข้อความวันที่ <code>'2026-10-09'</code>	"day key" วันที่ไทยรูปแบบ ปี-เดือน-วัน (ค.ศ.) เอามาเทียบมากกว่า/น้อยกว่า กันได้ตรง ๆ เพราะเรียงตามตัวอักษรก็ตรงกับเรียงตามเวลา
<code>useApp()</code>	หยิบข้อมูลกลาง (ไปรไฟล์ รายการ คำสั่ง addTx ฯลฯ)
<code>useMoney()</code>	หยิบตัวเลขที่คำนวณแล้ว: ยอดคงเหลือ ค่าเฉลี่ย Runway แบบร่าง
<code>useTheme()</code> แล้วตามด้วย <code>theme.ink</code> , <code>theme.primary</code>	สีจากธีมที่ผู้ใช้เลือก (ห้ามใส่รหัสสีเองในหน้าจอ)
<code><T v="h3"></code> , <code><Money satang={...}></code> , <code><Card></code> , <code><Button></code>	ชิ้นส่วนหน้าตาจาก <code>src/ui/components.tsx</code>
<code>space.lg</code> , <code>radius.md</code>	ระยะห่างและมุมโค้งมาตรฐาน (16 จุด, 14 จุด) จาก <code>src/ui/theme.ts</code>
<code>toast({ message })</code>	ข้อความแจ้งเตือนสั้น ๆ ด้านล่างจอ
<code>router.push('/xxx')</code>	เปิดหน้า <code>src/app/xxx.tsx</code>
<code>accessibilityLabel=""</code>	ข้อความที่โปรแกรมอ่านหน้าจออ่านให้คนตาบอดฟัง และเทสต์ E2E ใช้หาปุ่ม

เห็นอะไร	แปลว่าอะไร
status === 'confirmed' หรือ confirmedOnly(tx)	นับเฉพาะรายการที่ยืนยันแล้ว
Reveal, usePressSpring, useReduceMotion	แอนิเมชัน (ค่อย ๆ โฟกัส, ปุ่มยุบตอนกด, เคารพ "ลดการเคลื่อนไหว")
คอมเมนต์มีรหัส TC-12	รหัสเทสต์ ตรงกับในเอกสาร docs/TRACEABILITY.md
ไฟล์ชื่อ xxx.web.ts	ฉบับเว็บของไฟล์ xxx.ts
ชื่อแบบ snake_case เช่น amount_satang	ชื่อคอลัมน์ในฐานข้อมูล (ฝั่งแอปใช้ camelCase amountSatang) แปลงกันใน src/data/repo.ts

ฟังก์ชันวันเวลาที่ต้องรู้ (ทุก FR ใช้)

เวลาในฐานข้อมูลเก็บเป็นเวลาสากล (UTC) แต่ผู้ใช้ชาวไทย (เร็วกว่า 7 ชั่วโมง) ถ้าไม่แปลง รายการตอน 06:00 น. ของวันที่ 8 จะถูกนับเป็นวันที่ 7 (เพราะ UTC ยังเป็น 23:00 ของวันที่ 7) ทุก FR จึงใช้ฟังก์ชันจาก [src/domain/dates.ts](#)

src/domain/dates.ts

บรรทัด 1-51

```

1  /**
2   * Date helpers fixed to Thailand time (Asia/Bangkok, UTC+7, no daylight saving).
3   *
4   * Everything the user sees ("today", "this week", the date printed on a slip)
5   * is a Bangkok calendar day, so all grouping and range logic goes through
6   * `bkkDayKey`. Because Thailand has no DST, a fixed +7h offset is exact.
7   */
8  import type { RangeKey } from './types';
9
10 export const BKK_OFFSET_MS = 7 * 60 * 60 * 1000;
11 export const DAY_MS = 24 * 60 * 60 * 1000;
12
13 /** Instant -> "YYYY-MM-DD" in Bangkok. */
14 export function bkkDayKey(at: Date | string | number): string {
15   const ms = new Date(at).getTime() + BKK_OFFSET_MS;
16   return new Date(ms).toISOString().slice(0, 10);
17 }
18
19 /** "YYYY-MM-DD" (Bangkok day) -> the UTC instant of 00:00 that day in Bangkok. */
20 /** Milliseconds until the next 00:00 in Bangkok (when "วันนี้" starts over). */
21 export function msUntilNextBkkMidnight(nowMs: number = Date.now()): number {
22   const next = bkkDayStart(addDays(bkkDayKey(nowMs), 1)).getTime();
23   return Math.max(0, next - nowMs);
24 }
25
26 export function bkkDayStart(dayKey: string): Date {
27   return new Date(Date.parse(`${dayKey}T00:00:00.000Z`) - BKK_OFFSET_MS);
28 }
29
30 /** Add whole days to a "YYYY-MM-DD" key. */
31 export function addDays(dayKey: string, days: number): string {
32   return new Date(Date.parse(`${dayKey}T00:00:00.000Z`) + days * DAY_MS)
33     .toISOString()
34     .slice(0, 10);
35 }

```

```

36
37 /** Whole days from a to b (both "YYYY-MM-DD"). */
38 export function daysBetween(a: string, b: string): number {
39   return Math.round((Date.parse(`${b}T00:00:00Z`) - Date.parse(`${a}T00:00:00Z`)) / DAY_MS);
40 }
41
42 /** Combine a Bangkok day and "HH:MM" into an ISO instant. */
43 export function bkkToIso(dayKey: string, time: string | null = '12:00'): string {
44   const t = time && /\d{1,2}:\d{2}$/.test(time) ? time.padStart(5, '0') : '12:00';
45   return new Date(Date.parse(`${dayKey}T${t}:00.000Z`) - BKK_OFFSET_MS).toISOString();
46 }
47
48 /** "HH:MM" of an instant in Bangkok. */
49 export function bkkTime(at: Date | string): string {
50   return new Date(new Date(at).getTime() + BKK_OFFSET_MS).toISOString().slice(11, 16);
51 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/dates.ts:1

บรรทัด	ความหมาย
1–7	คอมเมนต์: ทุกอย่างที่ใช้ให้เห็น (วันนี้, สัปดาห์นี้, วันที่บนสลิป) เป็นวันตามปฏิทินกรุงเทพ และเพราะไทยไม่มีการปรับเวลา ฤดูร้อน (daylight saving) การบวก 7 ชั่วโมงตายตัวจึงถูกต้องเสมอ
10	7 ชั่วโมงในหน่วยมิลลิวินาที ($7 \times 60 \times 60 \times 1000 = 25,200,000$)
11	1 วันในหน่วยมิลลิวินาที ($86,400,000$)
13–17	bkkDayKey : รับเวลาแบบไหนก็ได้ (Date, ข้อความ, ตัวเลข) บวก 7 ชั่วโมง แล้วตัดเอา 10 ตัวอักษรแรกของรูปแบบ สากล ได้วันที่ไทย เช่น <code>bkkDayKey('2026-10-07T23:00:00Z')</code> → <code>'2026-10-08'</code> (23:00 UTC = 06:00 น. ของวันที่ 8 เวลาไทย)
20–24	msUntilNextBkkMidnight : เหลืออีกกี่มิลลิวินาทีถึงเที่ยงคืนไทยครั้งหน้า (เอาวันพรุ่งนี้ 00:00 ลบเวลาตอนนี้) ใช้ตั้ง นาฬิกาใน AppProvider (un A6)
26–28	bkkDayStart : วันที่ไทย → เวลาสากลของ 00:00 น. วันนั้น (ลบ 7 ชั่วโมงกลับ)
30–35	addDays : บวกหรือลบจำนวนวันจากวันที่ เช่น <code>addDays('2026-10-08', 20)</code> → <code>'2026-10-28'</code> ข้ามเดือน ข้ามปีให้เอง
37–40	daysBetween : นับจำนวนวันจาก a ถึง b เช่น <code>daysBetween('2026-10-08', '2026-10-31')</code> → 23
42–46	bkkToIso : เอาวันไทย + เวลา "HH:MM" มารวมเป็นเวลาสากลสำหรับเก็บในฐานข้อมูล ถ้าเวลาผิดรูปแบบใช้ 12:00 แทน เช่น <code>bkkToIso('2026-10-08', '06:00')</code> → <code>'2026-10-07T23:00:00.000Z'</code>
48–51	bkkTime : เวลาสากล → "HH:MM" ตามเวลาไทย ใช้แสดงเวลาของรายการ

แผนที่: อยากรู้อะไรไปเปิดไฟล์ไหน

อยากรู้	เปิดไฟล์
หน้าจอนี้หน้าตาแบบนี้เพราะ อะไร	src/app/ชื่อหน้า.tsx (ตารางใน un A7)

อยากรู้	เปิดไฟล์
ตัวเลขนี้คิดยังไง	src/domain/: src/domain/summary.ts (ยอด), src/domain/runway.ts (Runway), src/domain/insights.ts (การ์ดโค้ช)
ข้อมูลโหลด/บันทึกยังไง	src/data/repo.ts , src/data/AppProvider.tsx
ตารางในฐานข้อมูลมีคอลัมน์อะไร	supabase/migrations/20260927000000_init.sql
AI ได้รับคำสั่งอะไร	supabase/functions/coach/index.ts , supabase/functions/parse-slip/index.ts
สี ขนาด ฟอนต์	src/ui/theme.ts , docs/DESIGN_SYSTEM.md
ฟีเจอร์นี้ตอบ requirement ข้อไหน มีทดสอบอะไร	docs/TRACEABILITY.md
ทีมใช้ AI ช่วยเขียนตรงไหน ตรวจอะไรไปแล้ว	docs/AI_USAGE_LOG.md
build / deploy อัตโนมัติ	.github/workflows/*.yaml

★ ถ้าอ่านแล้วงง

ใส่ `console.log('ค่า', ตัวแปร)` ไว้ในโค้ดแล้วเปิดแอปด้วย `npm run web` กด F12 ในเบราว์เซอร์ แถบ Console จะเห็นค่าที่พิมพ์ออกมา (อย่าลืมลบก่อน commit) หรือเขียนเทสต์เล็ก ๆ ลองส่งค่าเข้าฟังก์ชันใน `src/domain` แล้วดูผล

เซิร์ฟเวอร์และฐานข้อมูล: Supabase

ข้อมูลของผู้ใช้เก็บที่ไหน เก็บยังไง แอปต่อเซิร์ฟเวอร์ยังไง และทำไมคนอื่นแอบดูข้อมูลเราไม่ได้ ทั้งที่กุญแจอยู่ในแอป

ฐานข้อมูลคืออะไร

ฐานข้อมูล (database) คือที่เก็บข้อมูลแบบมีระเบียบ คิดว่าเป็น **สมุด Excel บนคลาวด์** ที่มีหลายชีต แต่ละชีตเรียกว่า **ตาราง (table)** แต่ละแถวคือข้อมูล 1 ชัน (row) แต่ละคอลัมน์คือช่องข้อมูล (column) เช่น ตาราง transactions มี 1 แถวต่อรายการเงิน 1 รายการ

id	user_id	kind	amount_satang	title	status	occurred_at
a1b2...	มินท์	expense	8500	ข้าวมันไก่	confirmed	2026-10-08T05:10Z
c3d4...	มินท์	income	1000000	ค่าขนมจากที่บ้าน	confirmed	2026-10-01T02:00Z
e5f6...	ต้น	expense	125000	โอนให้ นาย ส***	draft	2026-10-07T11:42Z

ตัวอย่างสมมติ คอลัมน์ user_id จริงเป็นรหัสยาว (UUID) ไม่ใช่ชื่อ

ภาษาที่ใช้สั่งฐานข้อมูลคือ **SQL** คำสั่งพื้นฐานมี 4 คำ

SQL	ความหมาย	ในแอปเขียนว่า (supabase-js)
SELECT * FROM transactions	อ่านข้อมูล	db.from('transactions').select('*')
INSERT INTO transactions ...	เพิ่มแถว	db.from('transactions').insert(row)
UPDATE transactions SET ... WHERE id = ...	แก้แถว	db.from('transactions').update(row).eq('id', id)
DELETE FROM transactions WHERE id = ...	ลบแถว	db.from('transactions').delete().eq('id', id)

แอปไม่ได้เขียน SQL ตรง ๆ แต่ใช้ไลบรารี supabase-js ซึ่งแปลงคำสั่งเป็นคำขอผ่านอินเทอร์เน็ต แล้วเซิร์ฟเวอร์แปลงเป็น SQL ให้อีกที

Supabase ให้อะไรกับเราบ้าง

ส่วน	คืออะไร	ใช้ทำอะไรใน MindPay
PostgreSQL	ฐานข้อมูลแบบตาราง ที่นิยมที่สุดตัวหนึ่งของโลก	เก็บ 4 ตาราง: profiles, transactions, savings_goals, ai_usage
Auth	ระบบบัญชีผู้ใช้สำเร็จรูป	สมัคร เข้าสู่ระบบ ส่งรหัส 6 หลักทางอีเมล ลืมรหัสผ่าน

ส่วน	คืออะไร	ใช้ทำอะไรใน MindPay
REST API อัตโนมัติ	สร้าง "ประตู" ให้เข้าถึงตารางผ่านอินเทอร์เน็ทให้เอง	แอปอ่าน/เขียนตารางได้โดยไม่ต้องเขียนเซิร์ฟเวอร์เอง
Row Level Security	กฎความปลอดภัยรายแถว	แต่ละคนเห็นเฉพาะแถวของตัวเอง
Edge Functions	ที่รันโค้ด TypeScript ผังเซิร์ฟเวอร์ (Deno)	parse-slip และ coach ซึ่งถือกุญแจ AI (un A10)
Dashboard	หน้าเว็บจัดการทั้งหมด	ดูตาราง รัน SQL ตั้ง Secrets ดู log

โปรเจกต์ของทีมอยู่ภูมิภาค Singapore (ใกล้ไทย ตอบเร็ว) และใช้แพ็คเกจฟรี

Migration: ไฟล์สร้างฐานข้อมูล

แทนที่จะคลิกสร้างตารางในหน้าเว็บ ทีมเราเขียนคำสั่ง SQL ไว้ในไฟล์ เรียกว่า **migration** เรียงตามวันที่ในชื่อไฟล์ ใครจะสร้างฐานข้อมูลใหม่ก็แค่รันไฟล์ตามลำดับ ได้ฐานข้อมูลเหมือนกันทุกตัว และเก็บประวัติใน Git ได้

ไฟล์	ทำอะไร
<code>supabase/migrations/20260927000000_init.sql</code>	สร้างตารางหลัก 3 ตาราง กฎ RLS และ trigger (27 ก.ย. 2569)
<code>supabase/migrations/20260927000100_harden.sql</code>	แก้ตามคำแนะนำของตัวตรวจความปลอดภัย Supabase Advisor
<code>supabase/migrations/20260929000000_savings_goals.sql</code>	เพิ่มตารางกระปุกออม (FR-6)
<code>supabase/migrations/20261001000000_atomic_ai_quota.sql</code>	ฟังก์ชันนับโควตา AI แบบกันแย่งกัน และเพดานจำนวนเงิน

ตาราง profiles: ผู้ใช้ 1 คน = 1 แถว

supabase/migrations/20260927000000_init.sqlบรรทัด 1-22

```
1 -- MindPay database schema (Supabase / PostgreSQL)
2 -- Run once: Supabase Dashboard -> SQL Editor -> paste -> Run
3 -- or with the CLI: supabase db push
4 --
5 -- Security model: every row belongs to one user and Row Level Security (RLS)
6 -- makes sure a user can only ever read or change their own rows, even though
7 -- the app talks to the database directly with the public (publishable) key.
8
9 -- -----
10 -- profiles: one row per user (created automatically at sign-up)
11 -- -----
12 create table if not exists public.profiles (
13   id uuid primary key references auth.users (id) on delete cascade,
14   display_name text not null default '',
15   opening_balance_satang bigint not null default 0,
16   runway_floor_satang bigint not null default 50000 check (runway_floor_satang >= 0), -- FR-6 low
17   line, default #500
```

```

17 monthly_budget_satang bigint check (monthly_budget_satang is null or monthly_budget_satang >
0),
18 coach_tone text not null default 'friend' check (coach_tone in ('friend', 'coach', 'senior')),
-- FR-5
19 onboarded boolean not null default false,
20 created_at timestamptz not null default now(),
21 updated_at timestamptz not null default now()
22 );

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:1

บรรทัด	ความหมาย
1-7	คอมเมนต์ SQL (ขึ้นต้นด้วย --) อธิบายวิธีรัน และหลักความปลอดภัย: ทุกแถวเป็นของผู้ใช้คนเดียว RLS ทำให้อ่านแก้ไขได้เฉพาะของตัวเอง แม้แอปจะคุยกับฐานข้อมูลตรง ๆ ด้วยคุณดูแลสาธารณะ
12	สร้างตาราง profiles ในส่วน public (ถ้ายังไม่มี)
13	id ชนิด uuid (รหัสยาวไม่ซ้ำ) เป็น primary key (ตัวระบุแถว) และ อ้างอิง ผู้ใช้ในระบบ Auth ถ้าลบผู้ใช้ โพรไฟล์ถูกลบตาม (on delete cascade)
14	ชื่อเล่น ห้ามว่าง (not null) ค่าเริ่มต้นเป็นข้อความว่าง
15	เงินตอนเริ่มใช้แอป เป็นสตางค์ ชนิด bigint (จำนวนเต็มขนาดใหญ่)
16	เส้นเงินสำรองของ FR-6 ค่าเริ่มต้น 50,000 สตางค์ (฿500) และมี check บังคับว่าต้อง ≥ 0 ถ้าแอปส่งค่าติดลบมาฐานข้อมูลปฏิเสธเอง
17	งบท่อเดือน ไม่ตั้งก็ได้ (null) ถ้าตั้งต้องมากกว่า 0
18	โทนโค้ช (ของเดิม FR-5) ต้องเป็น 1 ใน 3 ค่าเท่านั้น
19	ตั้งค่าเงินครั้งแรกเสร็จหรือยัง (ใช้กับ Stack.Protected ใน un A7)
20-21	เวลาสร้างและแก้ไขล่าสุด ใส่เวลาปัจจุบันให้เอง (now())

ตาราง transactions: รายการเงิน (FR-1)

supabase/migrations/20260927000000_init.sql

บรรทัด 24-53

```

24 -- -----
25 -- transactions (FR-1). Money is stored in satang (integer) to avoid rounding errors.
26 -- -----
27 create table if not exists public.transactions (
28   id uuid primary key default gen_random_uuid(),
29   user_id uuid not null default auth.uid() references auth.users (id) on delete cascade,
30   kind text not null check (kind in ('income', 'expense')),
31   amount_satang bigint not null check (amount_satang > 0),
32   category_key text not null,
33   title text not null default '' check (char_length(title) <= 120),
34   note text check (note is null or char_length(note) <= 500),
35   occurred_at timestamptz not null,
36   source text not null default 'manual' check (source in ('manual', 'slip')),
37   status text not null default 'confirmed' check (status in ('draft', 'confirmed')),
38   slip_ref text, -- FR-4 duplicate key #1 (QR / printed reference)
39   slip_image_hash text, -- FR-4 duplicate key #2 (SHA-256 of the image)
40   ocr_confidence numeric(3, 2) check (ocr_confidence is null or (ocr_confidence >= 0 and
ocr_confidence <= 1)),

```

```

41 review_flags text[] not null default '{}',
42 created_at timestamptz not null default now(),
43 updated_at timestamptz not null default now()
44 );
45
46 create index if not exists transactions_user_time_idx on public.transactions (user_id,
    occurred_at desc);
47
48 -- The database itself refuses a second copy of the same slip, even if two
49 -- phones scan at the same moment (FR-4 "ตรวจสอบรายการซ้ำ").
50 create unique index if not exists transactions_user_slip_ref_uidx
51 on public.transactions (user_id, slip_ref) where slip_ref is not null;
52 create unique index if not exists transactions_user_slip_hash_uidx
53 on public.transactions (user_id, slip_image_hash) where slip_image_hash is not null;

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:24

บรรทัด	ความหมาย
28	id สร้างให้อัตโนมัติด้วยฟังก์ชันสุ่ม (gen_random_uuid())
29	user_id เจ้าของรายการ ค่าเริ่มต้นคือ auth.uid() = id ของคนที่ส่งคำขอมา แอปจึงไม่ต้องส่ง user_id เอง (และปลอมเป็นคนอื่นไม่ได้)
30	ประเภทต้องเป็น income หรือ expense เท่านั้น (กฎเดียวกับ TxKind ในแอป)
31	จำนวนเงินเป็นสตางค์ ต้องมากกว่า 0
32	รหัสหมวด
33–34	ชื่อยาวไม่เกิน 120 ตัวอักษร บันทึกไม่เกิน 500 (ตรงกับ maxLength ในหน้าจอดรายการ)
35	เวลาที่เกิดรายการ ชนิด timestamptz (เวลาพร้อมเขตเวลา)
36–37	ที่มา (manual/slip) และสถานะ (draft/confirmed) ค่าเริ่มต้นคือยืนยันแล้ว
38–39	เลขอ้างอิงสลิป และ hash ของรูป สำหรับกันซ้ำ (FR-4)
40	ความมั่นใจของ AI ตัวเลข 0.00 ถึง 1.00
41	ช่องที่ต้องตรวจ เป็น array ของข้อความ (text[])
46	index = สารบัญของตาราง ช่วยให้ค้น "รายการของผู้ใช้คนนี้ เรียงตามเวลาล่าสุด" ได้เร็ว ไม่ต้องไล่ทุกแถว
48–53	unique index = ห้ามซ้ำ: ผู้ใช้คนเดียวกันจะมีเลขอ้างอิงสลิปซ้ำกันไม่ได้ และ hash รูปซ้ำไม่ได้ ถ้ามือถือ 2 เครื่องสแกนสลิปใบเดียวกันพร้อมกัน ฐานข้อมูลปฏิเสธใบที่ 2 เอง (ตอบรหัส 23505 ที่ repo.ts แปลงเป็น DuplicateSlipError)

ตาราง ai_usage: นับการใช้ AI

supabase/migrations/20260927000000_init.sql

บรรทัด 55–65

```

55 -- -----
56 -- ai_usage: counts calls to the AI so one account cannot run up the bill.
57 -- Written only by the Edge Functions (service role), readable by the owner.
58 -- -----
59 create table if not exists public.ai_usage (
60   id bigserial primary key,
61   user_id uuid not null references auth.users (id) on delete cascade,

```

```

62 kind text not null check (kind in ('slip', 'coach')),
63 created_at timestamptz not null default now()
64 );
65 create index if not exists ai_usage_user_day_idx on public.ai_usage (user_id, kind, created_at
desc);

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:55

ทุกครั้งที่ AI ถูกเรียก (อ่านสลิป 1 ใบ หรือถามโค้ช 1 ครั้ง) Edge Function จะเพิ่ม 1 แถว เพื่อบันทึกว่าวันนี้ใช้ไปกี่ครั้งแล้ว กันบัญชีเดียวใช้ AI จนค่าใช้จ่ายบานปลาย (ผู้ใช้อ่านของตัวเองได้ แต่เขียนได้เฉพาะเซิร์ฟเวอร์)

Row Level Security (RLS): หัวใจความปลอดภัย

≈ เปรียบเทียบ: ตู้ล็อกเกอร์ในห้องสมุด

กุญแจสาธารณะ (publishable key) ในแอปเหมือน "บัตรเข้าห้องสมุด" ใครก็มีเดินเข้าห้องได้ แต่ทุกคนมีล็อกเกอร์ของตัวเอง RLS คือ "ยามประจำล็อกเกอร์" ที่ดูบัตรประชาชน (การเข้าสู่ระบบ) ทุกครั้ง และเปิดให้เฉพาะล็อกเกอร์ที่มีชื่อเรา แม้แขกเกอร์จะโยมบัตรเข้าห้องสมุด (กุญแจในแอป) ไปได้ ก็เปิดล็อกเกอร์คนอื่นไม่ได้

supabase/migrations/20260927000000_init.sql

บรรทัด 67-87

```

67 -- -----
68 -- Row Level Security
69 -- -----
70 alter table public.profiles enable row level security;
71 alter table public.transactions enable row level security;
72 alter table public.ai_usage enable row level security;
73
74 drop policy if exists "own profile read" on public.profiles;
75 create policy "own profile read" on public.profiles
76   for select using (auth.uid() = id);
77 drop policy if exists "own profile update" on public.profiles;
78 create policy "own profile update" on public.profiles
79   for update using (auth.uid() = id) with check (auth.uid() = id);
80
81 drop policy if exists "own transactions" on public.transactions;
82 create policy "own transactions" on public.transactions
83   for all using (auth.uid() = user_id) with check (auth.uid() = user_id);
84
85 drop policy if exists "own ai usage read" on public.ai_usage;
86 create policy "own ai usage read" on public.ai_usage
87   for select using (auth.uid() = user_id);

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:67

บรรทัด	ความหมาย
70-72	เปิด RLS ทั้ง 3 ตาราง พอเปิดแล้ว ค่าเริ่มต้นคือ ห้ามทุกอย่าง ต้องมี policy อนุญาตทีละเรื่อง
74-76	policy "own profile read": อ่านโปรไฟล์ได้ เฉพาะแถวที่ id ตรงกับคนที่เข้าสู่ระบบ (auth.uid() = id) drop policy if exists ด้านบนคือลบของเก่าก่อน ทำให้รันไฟล์ซ้ำได้
77-79	แก้โปรไฟล์ได้เฉพาะของตัวเอง using = ตรวจสอบแถวเดิม, with check = ตรวจสอบแถวหลังแก้ (กันแก้ id ให้เป็นของคนอื่น) ไม่มี policy insert/delete เพราะโปรไฟล์สร้างโดย trigger เท่านั้น

บรรทัด	ความหมาย
81–83	รายการเงิน: ทำได้ทุกอย่าง (for all = อ่าน เพิ่ม แก้ ลบ) เฉพาะแถวที่ user_id เป็นของตัวเอง
85–87	ai_usage อ่านได้อย่างเดียวและเฉพาะของตัวเอง

ทีมทดสอบเรื่องนี้ด้วยแผน MT-10 "บัญชี A ไม่เห็นรายการของบัญชี B" และรับตัวตรวจ Security Advisor ของ Supabase แล้วแก้ไขไม่เหลือคำเตือนเรื่องตาราง ไฟล์ harden.sql เปลี่ยน auth.uid() เป็น (select auth.uid()) เพื่อให้ฐานข้อมูลคำนวณครั้งเดียวต่อคำสั่ง แทนที่จะคำนวณทุกแถว (เร็วขึ้นเมื่อมีรายการเยอะ)

supabase/migrations/20260927000100_harden.sql
บรรทัด 1–25

```

1 -- Hardening after running the Supabase security/performance advisors on the first migration.
2
3 -- 1. Fixed search_path on trigger functions (advisor 0011).
4 create or replace function public.touch_updated_at() returns trigger
5 language plpgsql set search_path = '' as $$
6 begin
7   new.updated_at := now();
8   return new;
9 end $$;
10
11 -- 2. The sign-up trigger must not be callable through the REST API (advisors 0028/0029).
12 revoke execute on function public.handle_new_user() from public, anon, authenticated;
13
14 -- 3. Evaluate auth.uid() once per query instead of once per row (advisor 0003).
15 drop policy if exists "own profile read" on public.profiles;
16 create policy "own profile read" on public.profiles
17   for select using ((select auth.uid()) = id);
18
19 drop policy if exists "own profile update" on public.profiles;
20 create policy "own profile update" on public.profiles
21   for update using ((select auth.uid()) = id) with check ((select auth.uid()) = id);
22
23 drop policy if exists "own transactions" on public.transactions;
24 create policy "own transactions" on public.transactions
25   for all using ((select auth.uid()) = user_id) with check ((select auth.uid()) = user_id);

```

▶ เปิดใน VS Code (เร็ว)
GitHub
VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000100_harden.sql:1

Trigger: งานที่ฐานข้อมูลทำเองอัตโนมัติ

supabase/migrations/20260927000000_init.sql
บรรทัด 89–118

```

89 -- -----
90 -- Triggers
91 -- -----
92 create or replace function public.touch_updated_at() returns trigger
93 language plpgsql as $$
94 begin
95   new.updated_at := now();
96   return new;
97 end $$;
98
99 drop trigger if exists profiles_touch on public.profiles;
100 create trigger profiles_touch before update on public.profiles
101   for each row execute function public.touch_updated_at();

```

```

102 drop trigger if exists transactions_touch on public.transactions;
103 create trigger transactions_touch before update on public.transactions
104   for each row execute function public.touch_updated_at();
105
106 -- Create the profile row when a user signs up.
107 create or replace function public.handle_new_user() returns trigger
108 language plpgsql security definer set search_path = public as $$
109 begin
110   insert into public.profiles (id, display_name)
111   values (new.id, coalesce(new.raw_user_meta_data ->> 'display_name', ''))
112   on conflict (id) do nothing;
113   return new;
114 end $$;
115
116 drop trigger if exists on_auth_user_created on auth.users;
117 create trigger on_auth_user_created after insert on auth.users
118   for each row execute function public.handle_new_user();

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:89

บรรทัด	ความหมาย
92–97	ฟังก์ชัน touch_updated_at (เขียนด้วยภาษา plpgsql ของ PostgreSQL) ตั้งเวลาแก้ไขล่าสุดเป็นตอนนี้
99–104	trigger = "ทุกครั้งก่อนแก้ไขในตาราง profiles หรือ transactions ให้เรียกฟังก์ชันนั้น" updated_at จึงถูกต้องเสมอ ไม่ต้องพึ่งแอป
107–114	ฟังก์ชัน handle_new_user: สร้างแถวโปรไฟล์ใหม่ โดยเอาชื่อเล่นที่ผู้ใช้กรอกตอนสมัคร (เก็บใน raw_user_meta_data) ถ้าไม่มีใช้ข้อความว่าง และถ้ามีแถวนี้อยู่แล้วไม่ต้องทำ (on conflict do nothing) security definer = รันด้วยสิทธิ์ของเจ้าของฟังก์ชัน (เพราะผู้ใช้ไม่มีสิทธิ์ insert โปรไฟล์เอง)
116–118	trigger บนตารางผู้ใช้ของระบบ Auth: หลังจากมีคนสมัครใหม่ ให้สร้างโปรไฟล์ทันที แอปจึงไม่ต้องสร้างเอง

กฎแฉ 2 แบบ: อันไหนอยู่ในแอปได้ อันไหนห้าม

	Publishable key (anon)	Service role key (secret)
อยู่ที่ไหน	ในแอป (.env, eas.json) เผยแพร่ได้	บนเซิร์ฟเวอร์ใน Edge Functions เท่านั้น (Supabase ใส่ให้เอง)
ทำอะไรได้	ทุกคำขอต้องผ่าน RLS เห็นเฉพาะข้อมูลของคน que เข้าสู่ระบบ	ข้าม RLS ได้ทั้งหมด อ่านเขียนทุกแถว
ใช้ทำอะไร	แอปอ่าน/เขียนรายการของผู้ใช้	ตรวจตัวตนผู้ใช้ และนับโควตา AI ใน Edge Function
ถ้ารั่ว	ไม่เป็นไร (ออกแบบมาให้เปิดเผยได้)	อันตรายมาก ต้องเปลี่ยนกฎแฉทันที

กฎแฉ AI (Claude หรือ Gemini) ก็เป็นความลับระดับเดียวกับ service role เก็บใน Supabase → Edge Functions → Secrets เท่านั้น

แอปต่อ Supabase ยังไง: src/data/supabase.ts

src/data/supabase.ts

บรรทัด 1-38

```
1 /**
2  * Supabase client. Keys come from environment variables (see .env.example).
3  * The publishable key is safe to ship in the app: Row Level Security in the
4  * database decides what each signed-in user can read and write.
5  * AI keys are NOT here; they live only in the Edge Functions.
6  *
7  * detectSessionInUrl is off: email links are handled by AppProvider (via
8  * authLinks.ts) the same way on the web and in the phone app.
9  */
10 import './sessionStorageSetup';
11 import './authLinks'; // records the address the web page was opened with
12 import { createClient, type SupabaseClient } from '@supabase/supabase-js';
13 import { AppState } from 'react-native';
14
15 const url = process.env.EXPO_PUBLIC_SUPABASE_URL;
16 const key = process.env.EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY;
17
18 /** null when the app was built without Supabase settings: only demo mode is available. */
19 export const supabase: SupabaseClient | null =
20   url && key
21     ? createClient(url, key, {
22       auth: {
23         storage: localStorage,
24         autoRefreshToken: true,
25         persistSession: true,
26         detectSessionInUrl: false,
27       },
28     })
29     : null;
30
31 if (supabase) {
32   AppState.addEventListener('change', (state) => {
33     if (state === 'active') supabase.auth.startAutoRefresh();
34     else supabase.auth.stopAutoRefresh();
35   });
36 }
37
38 export const cloudAvailable = supabase !== null;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/supabase.ts:1

บรรทัด	ความหมาย
1-9	คอมเมนต์: คุญแจมาจากตัวแปรสภาพแวดล้อม (.env) คุญแจ publishable ปลอดภัยเพราะ RLS และคุญแจ AI ไม่อยู่ที่นี่
10	ตั้งที่เก็บการเข้าสู่ระบบ (บนมือถือใช้ SQLite เก็บไว้ ปิดแอปแล้วยังเข้าสู่ระบบอยู่)
11	จดที่อยู่ของหน้าเว็บตอนเปิด (ไว้จัดการลิงก์ยืนยันอีเมล)
15-16	อ่านที่อยู่เซิร์ฟเวอร์และคุญแจจาก .env (process.env.EXPO_PUBLIC_...)
18-29	ถ้ามีทั้งสองค่า สร้าง "client" สำหรับคุยกับ Supabase ถ้าไม่มี (build แบบไม่มีการตั้งค่า) ได้ null แล้วแอปใช้ได้เฉพาะโหมดทดลอง

บรรทัด	ความหมาย
23–26	ตั้งค่าการเข้าสู่ระบบ: เก็บ session ไว้ (persistSession), ต่ออายุ token อัตโนมัติ (token ของ Supabase มีอายุสั้น ค่าปกติคือ 1 ชั่วโมง), และไม่ให้ไลบรารีอ่านลิงก์ในแถบที่อยู่เอง (AuthProvider จัดการเองเพื่อให้เว็บกับแอปทำงานเหมือนกัน)
31–36	ต่ออายุ token เฉพาะตอนแอปเปิดอยู่บนหน้าจอ พอแอปถูกพับไว้หยุดต่ออายุ (ประหยัดแบตเตอรี่)
38	บอกส่วนอื่นของแอปว่าใช้บัญชีจริงได้ไหม

การสมัครและเข้าสู่ระบบ

1. กรอกชื่อเล่น อีเมล รหัสผ่าน ≥ 8 ตัว [src/app/welcome.tsx](#)

↓ `supabase.auth.signUp`

2. Supabase สร้างผู้ใช้

trigger `handle_new_user` สร้างแถวโปรไฟล์ให้ทันที แล้วส่งอีเมลที่มีรหัส 6 หลักและลิงก์ยืนยัน

↓ `supabase.auth.verifyOtp` (กรอกรหัส) หรือแตะลิงก์ในอีเมล

3. ได้ session

แอปเก็บ token ไว้ในเครื่อง AppProvider เห็นเหตุการณ์ `SIGNED_IN` แล้วสร้าง CloudRepo โหลดข้อมูล

↓

4. โปรไฟล์ยังไม่ onboarded

Stack.Protected พาไปหน้าตั้งค่าเงิน กรอกรายละเอียดตอนนี้และเงินสำรอง แล้วเข้าหน้าหลัก

[src/app/welcome.tsx](#)

บรรทัด 148–172

```

148     const displayName = name.trim();
149     const { data, error } = await supabase!.auth.signUp({
150       email: address,
151       password,
152       options: { data: { display_name: displayName }, emailRedirectTo: authRedirectUrl() },
153     });
154     if (error) return setNotice({ text: authErrorMessage(error), tone: 'error' });
155     if (isExistingAccountSignUp(data.user)) {
156       setMode('signin');
157       setPassword('');
158       setNotice({
159         text: 'อีเมลนี้มีบัญชีอยู่แล้ว เข้าสู่ระบบด้วยรหัสผ่านเดิม หรือกด "ลืมรหัสผ่าน" เพื่อตั้งใหม่',
160         tone: 'info',
161         action: 'forgot',
162       });
163       return;
164     }
165     if (data.session) {
166       showAuthNotice({ kind: 'signed_up', name: displayName });
167       resetForm();
168       return;
169     }
170     setSent({ kind: 'confirm', email: address, name: displayName });
171     setResendIn(RESEND_WAIT_S);

```

บรรทัด	ความหมาย
148	ตัดช่องว่างหัวท้ายของชื่อเล่น
149–153	ส่งสมัคร: ส่งอีเมล รหัสผ่าน ชื่อเล่น (เก็บใน metadata ของผู้ใช้ ซึ่ง trigger จะคัดลอกไปใส่โปรไฟล์) และที่อยู่ลิงก์ในอีเมลจะพาคลับมา
154	ถ้า error (เช่น รหัสผ่านสั้นเกิน) แสดงข้อความภาษาไทยที่บอกวิธีแก้
155–164	กรณีพิเศษ: อีเมลนี้เคยสมัครแล้ว Supabase จะตอบเหมือนสำเร็จ (เพื่อไม่ให้คนนอกรู้ว่าอีเมลไหนมีบัญชี) แอปตรวจจบได้แล้วบอกให้ไปเข้าสู่ระบบหรือกดลิ้งรหัสผ่าน แทนที่จะขึ้น "สมัครสำเร็จ" หลอก ๆ
165–169	ถ้าโปรเจกต์ปิดการยืนยันอีเมลไว้ จะได้ session ทันที ขึ้นหน้าต่าง "สมัครบัญชีสำเร็จ"
170–171	ไม่ขึ้นไปหน้ากรอกรหัส 6 หลัก และเริ่มนับถอยหลังก่อนให้กดส่งรหัสใหม่ได้

Repo: ชั้นที่ซ่อนว่าข้อมูลอยู่ที่ไหน

หน้าจอไม่เคยคุยกับ Supabase หรือที่เก็บในเครื่องโดยตรง แต่คุยผ่าน **Repo** (ย่อจาก repository แปลว่าคลัง) ซึ่งมี 2 แบบที่หน้าตาเหมือนกันทุกประการ

src/data/repo.ts

บรรทัด 1–38

```

1  /**
2   * Data access. The screens never talk to Supabase or storage directly; they use
3   * a Repo. Two implementations:
4   *   - CloudRepo: Supabase (real accounts, data in PostgreSQL with RLS)
5   *   - DemoRepo: on-device only, starts with sample data (for trying the app
6   *                 without an account and as a fallback during a presentation)
7   */
8  import { Storage } from '../storage';
9  import { buildSampleTransactions, DEMO_OPENING_BALANCE_SATANG } from '../domain/sample';
10 import type { Profile, Transaction, TransactionInput } from '../domain/types';
11 import type { GoalInput, SavingsGoal } from '../domain/goals';
12 import { supabase } from '../supabase';
13
14 export class DuplicateSlipError extends Error {
15   constructor() {
16     super('รายการนี้มีอยู่แล้ว');
17     this.name = 'DuplicateSlipError';
18   }
19 }
20
21 export interface Repo {
22   mode: 'cloud' | 'demo';
23   getProfile(): Promise<Profile>;
24   updateProfile(patch: Partial<Omit<Profile, 'id'>>): Promise<Profile>;
25   /**
26    * Transactions of about the last 13 months, plus the net of confirmed money before them
27    * (it still counts in the balance, but is not needed as a list).
28    */
29   listTransactions(): Promise<{ txs: Transaction[]; carrySatang: number }>;
30   insert(input: TransactionInput): Promise<Transaction>;

```

```

31 update(id: string, patch: Partial<TransactionInput>): Promise<Transaction>;
32 remove(id: string): Promise<void>;
33 confirmMany(ids: string[]): Promise<void>;
34 listGoals(): Promise<SavingsGoal[]>;
35 insertGoal(input: GoalInput): Promise<SavingsGoal>;
36 updateGoal(id: string, patch: Partial<GoalInput>): Promise<SavingsGoal>;
37 removeGoal(id: string): Promise<void>;
38 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:1

บรรทัด	ความหมาย
1-7	คอมเมนต์: หน้าจอใช้ Repo เสมอ มี 2 ตัว คือ CloudRepo (Supabase, บัญชีจริง) และ DemoRepo (เก็บในเครื่อง มีข้อมูลตัวอย่าง ใช้ลองแอปหรือสำรองตอนพรีเซนต์)
14-19	error เฉพาะสำหรับสลิปซ้ำ (อธิบายในบท A5)
21	interface Repo = สัญญาว่า Repo ทุกตัวต้องทำอะไรเหล่านี้ได้
22	บอกว่าเป็นแบบไหน
23-24	อ่านและแก้ไขไฟล์
25-29	โหลดรายการประมาณ 13 เดือนล่าสุด และ "ยอดยกมา" ของรายการที่เก่ากว่านั้น (ยังต้องนับในยอดคงเหลือ แต่ไม่ต้องโหลดมาเป็นรายการ)
30-33	เพิ่ม แก้ ลบ รายการ และยืนยันหลายรายการพร้อมกัน (สลิปรอยืนยัน)
34-37	คำสั่งของกระปุกออม (FR-6)

≈ ทำไมต้องมี 2 แบบ

เหมือนปลั๊กไฟ: หน้าจอคือเครื่องใช้ไฟฟ้า Repo คือเต้าเสียบ จะต่อกับไฟบ้าน (Supabase) หรือแบตเตอรี่สำรอง (เครื่อง) หน้าจอไม่ต้องรู้ เสียบแล้วใช้ได้เหมือนกัน ทีมเราจึงพรีเซนต์ได้แม้อินเทอร์เน็ตในห้องเรียนล่ม และ E2E ทดสอบได้โดยไม่แตะบัญชีจริง

แปลงชื่อระหว่างแอปกับฐานข้อมูล

src/data/repo.ts

บรรทัด 44-93

```

44 type TxRow = {
45   id: string;
46   kind: 'income' | 'expense';
47   amount_satang: number;
48   category_key: string;
49   title: string;
50   note: string | null;
51   occurred_at: string;
52   source: 'manual' | 'slip';
53   status: 'draft' | 'confirmed';
54   slip_ref: string | null;
55   slip_image_hash: string | null;
56   ocr_confidence: number | null;
57   review_flags: string[] | null;

```

```

58   created_at: string;
59 };
60
61 const fromRow = (r: TxRow): Transaction => ({
62   id: r.id,
63   kind: r.kind,
64   amountSatang: Number(r.amount_satang),
65   categoryKey: r.category_key,
66   title: r.title,
67   note: r.note,
68   occurredAt: r.occurred_at,
69   source: r.source,
70   status: r.status,
71   slipRef: r.slip_ref,
72   slipImageHash: r.slip_image_hash,
73   ocrConfidence: r.ocr_confidence === null ? null : Number(r.ocr_confidence),
74   reviewFlags: r.review_flags ?? [],
75   createdAt: r.created_at,
76 });
77
78 function toRow(p: Partial<TransactionInput>) {
79   const row: Record<string, unknown> = {};
80   if (p.kind !== undefined) row.kind = p.kind;
81   if (p.amountSatang !== undefined) row.amount_satang = p.amountSatang;
82   if (p.categoryKey !== undefined) row.category_key = p.categoryKey;
83   if (p.title !== undefined) row.title = p.title;
84   if (p.note !== undefined) row.note = p.note;
85   if (p.occurredAt !== undefined) row.occurred_at = p.occurredAt;
86   if (p.source !== undefined) row.source = p.source;
87   if (p.status !== undefined) row.status = p.status;
88   if (p.slipRef !== undefined) row.slip_ref = p.slipRef;
89   if (p.slipImageHash !== undefined) row.slip_image_hash = p.slipImageHash;
90   if (p.ocrConfidence !== undefined) row.ocr_confidence = p.ocrConfidence;
91   if (p.reviewFlags !== undefined) row.review_flags = p.reviewFlags;
92   return row;
93 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:44

บรรทัด	ความหมาย
44–59	TxRow = หน้าตาของแถวในฐานข้อมูล ใช้ชื่อแบบ snake_case (คั่นด้วยขีดล่าง) ตามธรรมเนียมของ SQL
61–76	fromRow: แถวฐานข้อมูล → Transaction ของแอป ใช้ Number(. . .) ครอบตัวเลข เพราะ bigint บางครั้งส่งมาเป็นข้อความ และ review_flags ที่เป็น null ให้เป็น array ว่าง
78–93	toRow: กลับทาง ใส่เฉพาะช่องที่มีค่า (ไม่ใช่ undefined) เพราะตอนแก้รายการ เราส่งแค่บางช่อง ถ้าใส่ทุกช่อง ช่องที่ไม่ได้ส่งจะถูกเขียนทับเป็นค่าว่าง

โหลดรายการจากคลาวด์

src/data/repo.ts

บรรทัด 166–216

```

166 export function createCloudRepo(userId: string): Repo {
167   const db = supabase!;
168   return {
169     mode: 'cloud',
170     async getProfile() {
171       const { data, error } = await db.from('profiles').select('*').eq('id', userId).single();

```

```

172     if (error) throw error;
173     return profileFromRow(data as ProfileRow);
174 },
175 async updateProfile(patch) {
176     const { data, error } = await db.from('profiles').update(profileToRow(patch)).eq('id',
userId).select().single();
177     if (error) throw error;
178     return profileFromRow(data as ProfileRow);
179 },
180 async listTransactions() {
181     // Up to ~13 months: enough for the 1-year range. Paged to stay under the API row limit.
182     const since = new Date(Date.now() - 400 * 24 * 3600 * 1000).toISOString();
183     const all: Transaction[] = [];
184     for (let from = 0; ; from += 1000) {
185         const { data, error } = await db
186             .from('transactions')
187             .select('*')
188             .gte('occurred_at', since)
189             .order('occurred_at', { ascending: false })
190             // A unique tiebreaker, so rows with the same time are never skipped or read twice
across pages.
191             .order('id', { ascending: true })
192             .range(from, from + 999);
193         if (error) throw error;
194         all.push...(data as TxRow[]).map(fromRow);
195         if (!data || data.length < 1000) break;
196     }
197     // Older confirmed money still counts in the balance: add it up (two columns only).
198     let carrySatang = 0;
199     for (let from = 0; ; from += 1000) {
200         const { data, error } = await db
201             .from('transactions')
202             .select('kind,amount_satang')
203             .eq('status', 'confirmed')
204             .lt('occurred_at', since)
205             .order('occurred_at', { ascending: false })
206             // A unique tiebreaker, so rows with the same time are never skipped or read twice
across pages.
207             .order('id', { ascending: true })
208             .range(from, from + 999);
209         if (error) throw error;
210         for (const r of (data ?? []) as Pick<TxRow, 'kind' | 'amount_satang'>[]) {
211             carrySatang += r.kind === 'income' ? Number(r.amount_satang) : -
Number(r.amount_satang);
212         }
213         if (!data || data.length < 1000) break;
214     }
215     return { txs: all, carrySatang };
216 },

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:166

บรรทัด	ความหมาย
166–169	สร้าง CloudRepo ของผู้ใช้คนนี้ supabase! (เครื่องหมายตกใจ) บอก TypeScript ว่า "มั่นใจว่าไม่ใช่ null"
170–174	อ่านโปรไฟล์: จากตาราง profiles เลือกทุกคอลัมน์ ที่ id เท่ากับผู้ใช้ เอาแถวเดียว (RLS ก็กันไว้อีกชั้นอยู่แล้ว)
175–179	แก้โปรไฟล์แล้วขอแถวใหม่กลับมา

บรรทัด	ความหมาย
182	คำนวณวันที่ย้อนหลัง 400 วัน (ประมาณ 13 เดือน พอสำหรับช่วง "1 ปี" ในหน้าหลัก)
184–196	โหลดทีละหน้า หน้าละ 1,000 แถว (API จำกัดจำนวนแถวต่อครั้ง) เรียงจากใหม่ไปเก่า ถ้าได้น้อยกว่า 1,000 แปลว่าหมดแล้ว หยุด
190–191	เรียงด้วย id เป็นตัวตัดสินรอง ถ้ามีหลายรายการเวลาเดียวกัน จะไม่หลุดหรือซ้ำข้ามหน้า (บ๊ีกที่ทีมเจอและแก้ตอนตรวจทั้ง repo)
197–214	ยอดยกมา: รายการที่ ยืนยันแล้ว และเก่ากว่า 400 วัน โหลดมาแค่ 2 คอลัมน์ (ประเภทและจำนวน) แล้วรวมเป็นตัวเลขเดียว รายรับบวก รายจ่ายลบ ยอดคงเหลือจึงยังถูกต้องแม้ผู้ใช้ใช้แอปมาหลายปี

โมดกทดลอง: DemoRepo

src/data/repo.ts

บรรทัด 264–305

```

264 // v2: rebalanced sample data (v1 could show a negative balance). Old v1 data is simply ignored.
265 const DEMO_KEY = 'mindpay.demo.v2';
266
267 interface DemoState {
268   profile: Profile;
269   txs: Transaction[];
270   goals?: SavingsGoal[];
271 }
272
273 const demoProfile: Profile = {
274   id: 'demo',
275   displayName: 'ผู้ทดลองใช้',
276   openingBalanceSatang: DEMO_OPENING_BALANCE_SATANG,
277   runwayFloorSatang: 50_000,
278   monthlyBudgetSatang: 1_000_000,
279   coachTone: 'friend',
280   onboarded: true,
281 };
282
283 const newId = () => `local-${Date.now().toString(36)}-${Math.random().toString(36).slice(2, 8)}`;
284
285 export async function resetDemoData() {
286   await Storage.removeItem(DEMO_KEY);
287 }
288
289 export function createDemoRepo(): Repo {
290   let cache: DemoState | null = null;
291
292   async function load(): Promise<DemoState> {
293     if (cache) return cache;
294     try {
295       const raw = await Storage.getItem(DEMO_KEY);
296       if (raw) cache = JSON.parse(raw) as DemoState;
297     } catch {
298       cache = null;
299     }
300     if (!cache) {
301       cache = { profile: demoProfile, txs: buildSampleTransactions() };
302       await save();
303     }
304     return cache;

```

บรรทัด 265: ข้อมูลโหมดทดลองเก็บในเครื่องด้วยชื่อ `mindpay.demo.v2` (v2 เพราะข้อมูลชุดแรกเคยทำให้ยอดติดลบ จึงเปลี่ยนชุดใหม่) บรรทัด 273–281 คือโปรไฟล์ของผู้ทดลองใช้: เงินตั้งต้น ฿3,000 เลื่อนสำรอง ฿500 จบ ฿10,000 บรรทัด 292–305: ครั้งแรกที่เปิดยังไม่มีข้อมูล จะสร้างรายการตัวอย่าง (`buildSampleTransactions` ใน [src/domain/sample.ts](#)) ประมาณ 75 วันย้อนหลังจากวันนี้ แล้วเก็บไว้ ข้อมูลตัวอย่างจึงสดใหม่เสมอไม่ว่าเปิดวันไหน

เปิดแอปแล้วเกิดอะไรขึ้น: AppProvider เลือกโหมด

src/data/AppProvider.tsx

บรรทัด 271–318

```

271 // Decide the starting mode once, then follow Supabase auth changes.
272 useEffect(() => {
273   let cancelled = false;
274   (async () => {
275     // An email link that opened the app wins over the saved mode.
276     if (await handleAuthLink(await openingLink())) return;
277     const mode = await Storage.getItem(MODE_KEY).catch(() => null);
278     if (mode === 'demo') {
279       if (!cancelled) await activateRepo(createDemoRepo(), 'demo');
280       return;
281     }
282     if (!supabase) {
283       if (!cancelled) setStatus('signedOut');
284       return;
285     }
286     const { data } = await supabase.auth.getSession();
287     if (cancelled) return;
288     const id = data.session?.user.id;
289     if (id) {
290       if (activeUserRef.current !== id) await activateRepo(createCloudRepo(id), id);
291     } else if (!activeUserRef.current) setStatus('signedOut');
292   })();
293
294   const sub = supabase?.auth.onAuthStateChange((event, session) => {
295     // Supabase warns against calling other supabase methods inside this callback
296     // (it can deadlock the auth lock), so the actual work is deferred.
297     setTimeout(() => {
298       // A password-reset code signs in with the PASSWORD_RECOVERY event instead of SIGNED_IN.
299       // Opened offline with an expired token, the session only comes back later as
300       // TOKEN_REFRESHED:
301       // that signs in too, but only while nothing is open (never over the demo or another
302       // account).
303       const signedIn =
304         event === 'SIGNED_IN' || event === 'PASSWORD_RECOVERY' || (event === 'TOKEN_REFRESHED'
305         && !activeUserRef.current);
306       if (signedIn && session && activeUserRef.current !== session.user.id) {
307         Storage.setItem(MODE_KEY, 'cloud').catch(() => {});
308         activateRepo(createCloudRepo(session.user.id), session.user.id);
309       }
310       if (event === 'SIGNED_OUT' && repoRef.current?.mode === 'cloud') clear();
311     }, 0);
312   });
313
314   const stopLinks = onIncomingLink((url) => {
315     handleAuthLink(url);
316   });

```

```

313     return () => {
314         cancelled = true;
315         sub?.data.subscription.unsubscribe();
316         stopLinks();
317     };
318 }, [activateRepo, clear, handleAuthLink]);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:271`

บรรทัด	ความหมาย
272	ทำครั้งเดียวตอนแอปเริ่ม
275–276	ถ้าแอปถูกเปิดจากลิงก์ในอีเมล (ยืนยันสมัคร/ตั้งรหัสใหม่) จัดการลิงก์ก่อน ถ้าเข้าสู่ระบบสำเร็จก็จบ
277–281	อ่านโหมดที่จำไว้ในเครื่อง ถ้าเป็นโหมดทดลอง ใช้ DemoRepo
282–285	build ที่ไม่มี Supabase: ไปหน้าต้อนรับ
286–291	ถามว่ามี session ค้างอยู่ไหม (เคยเข้าสู่ระบบไว้) ถ้ามี ใช้ CloudRepo ของคนนั้น ไม่มีก็ไปหน้าต้อนรับ
294–309	ฟังเหตุการณ์ของระบบ Auth: เข้าสู่ระบบ (SIGNED_IN), กู้รหัสผ่าน (PASSWORD_RECOVERY) หรือ token กลับมา หลังออฟไลน์ → เปิดบัญชีนั้น · ออกจากระบบ (SIGNED_OUT) → ล้างข้อมูล ทั้งหมดห่อด้วย <code>setTimeout(..., 0)</code> เพราะ Supabase เตือนว่าเรียกคำสั่งอื่นในนี้ตรง ๆ อาจค้าง
310–312	ฟังลิงก์ที่เข้ามาระหว่างแอปเปิดอยู่
313–317	เก็บกวาดเมื่อปิด: เลิกฟังทุกอย่าง

`src/data/AppProvider.tsx`

บรรทัด 127–167

```

127     const loadAll = useCallback(async (r: Repo) => {
128         const gen = ++loadGen.current;
129         localChanges.current = new Map();
130         setLoadError(null);
131         try {
132             // Goals never block the app: if they cannot be loaded, the rest still works.
133             const [p, list, g] = await Promise.all([r.getProfile(), r.listTransactions(),
134             r.listGoals().catch(() => [] as SavingsGoal[])]);
135             if (gen !== loadGen.current || repoRef.current !== r) return;
136             // A slip saved by the automatic scan during the load is not lost when the list arrives.
137             const changes = localChanges.current;
138             const fresh = list.txs.filter((t) => !changes.has(t.id));
139             const kept = [...changes.values()].filter((t): t is Transaction => t !== null);
140             setProfile(p);
141             setTxs([...kept, ...fresh]);
142             setCarry(list.carrySatang);
143             setGoals(g);
144             setStatus('ready');
145         } catch (e) {
146             if (gen !== loadGen.current || repoRef.current !== r) return;
147             setLoadError(e instanceof Error ? e.message : 'โหลดข้อมูลไม่สำเร็จ');
148             setStatus('ready');
149         }
150     }, []);
151
152     const activateRepo = useCallback(
153         async (r: Repo, id: string) => {

```

```

153     if (activeUserRef.current !== id) {
154         // Another account (or leaving the demo): never show or use the previous one's data
           meanwhile.
155         setProfile(null);
156         setTxs([]);
157         setCarry(0);
158         setGoals([]);
159     }
160     activeUserRef.current = id;
161     repoRef.current = r;
162     setRepo(r);
163     setUserId(id);
164     await loadAll(r);
165 },
166 [loadAll],
167 );

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:127

บรรทัด	ความหมาย
127–128	loadAll: โหลดข้อมูลทั้งหมด แต่ละครั้งได้เลขรอบใหม่ (gen)
129–130	เริ่มจัดการเปลี่ยนแปลงในเครื่องใหม่ และล้าง error เก่า
133	โหลด 3 อย่างพร้อมกัน (un A5)
134	ถ้ามีการโหลดรอบใหม่กว่าเริ่มไปแล้ว หรือสลับบัญชีไปแล้ว ทั้งผลรอบนี้ (กันข้อมูลบัญชีเก่าโผล่ในบัญชีใหม่)
135–140	ถ้าระหว่างโหลด สแกนสลิปอัตโนมัติบันทึกรายการใหม่ไป เอารายการนั้นใส่กลับไว้ด้านบน ไม่ให้หายเพราะผลโหลดมากับ
139–143	ใส่ข้อมูลลง state ทั้งหมด แล้วบอกว่าพร้อม (หน้าจอวาดใหม่)
144–147	ถ้าโหลดพลาด เก็บข้อความ error ให้หน้าหลักแสดงปุ่ม "ลองอีกครั้ง"
151–167	activateRepo: เปลี่ยนไปใช้ Repo ของบัญชีใหม่ ถ้าเป็นคนละบัญชีกับเดิม ล้างข้อมูลเก่าออกก่อนทันที แล้วจํา Repo และโหลดข้อมูล

★ สรุป un A9

ฐานข้อมูลมี 4 ตาราง ทุกตารางเปิด RLS ให้แต่ละคนเห็นเฉพาะของตัวเอง เงินเก็บเป็นสตางค์และมี check กันค่าผิด สลิปซ้ำถูกกันด้วย unique index ระดับฐานข้อมูล แอปคุยกับฐานข้อมูลผ่าน Repo ซึ่งมีแบบคลาวด์และแบบในเครื่อง และ AppProvider เป็นคนเลือกว่าจะใช้แบบไหน

การเชื่อม AI: ทำไมต้องผ่านเซิร์ฟเวอร์ และทำงานยังไง

MindPay ใช้ AI 2 งาน คืออ่านสลิป (FR-4) และโค้ชน้องกล้า (FR-5) บทนี้อธิบายเส้นทางตั้งแต่แอปถามจนได้คำตอบ ทีละบรรทัด

AI ในแอปคืออะไร

AI ที่ใช้คือ โมเดลภาษาขนาดใหญ่ (Large Language Model, LLM) เช่น Claude ของบริษัท Anthropic และ Gemini ของ Google เราส่ง "ข้อความ" (และรูป) ไปทางอินเทอร์เน็ต มันตอบกลับเป็นข้อความ บริษัทเหล่านี้คิดเงินตามจำนวนที่ใช้ จึงต้องมี API key (กุญแจ) ไว้ระบุว่าใครเป็นคนจ่าย

คำ	ความหมาย
API	"ประตู" ให้โปรแกรมคุยกับโปรแกรม ส่งคำขอเป็น JSON ทาง HTTPS แล้วได้คำตอบเป็น JSON
API key	รหัสลับยาว ๆ ที่แนบไปกับคำขอ ใครได้กุญแจไป ก็ใช้ AI โดยให้เราจ่ายเงินได้
System prompt	"คำสั่งประจำตัว" ที่บอก AI ว่าต้องเป็นใคร ตอบยังไง ห้ามอะไร (ผู้ใช้ไม่เห็น)
Token	หน่วยนับข้อความของ AI (ประมาณคำหรือพยางค์) ใช้คิดราคาและจำกัดความยาวคำตอบ
JSON Schema	แบบฟอร์มที่บังคับให้ AI ตอบเป็นโครงสร้างตายตัว เช่น ต้องมีช่อง amount, dateIso
JSON	รูปแบบข้อมูลที่เขียนเหมือน object ของ JavaScript เช่น {"text": "สวัสดี"}

ทำไมแอปไม่เรียก AI เอง

! ห้ามใส่กุญแจ AI ไว้ในแอปเด็ดขาด

ไฟล์ APK หรือเว็บที่ผู้ใช้โหลดไป ใครก็แกะดูข้างในได้ ถ้ากุญแจ AI อยู่ในแอป คนแกะจะเอาไปใช้ฟรีโดยไม่ให้เราจ่าย (หรือทำให้กุญแจโดนแบน) ทีมเราจึงเก็บกุญแจไว้บนเซิร์ฟเวอร์ใน **Edge Functions** เท่านั้น แอปเรียก Edge Function แล้ว Edge Function ไปเรียก AI แทน

การมีเซิร์ฟเวอร์คั่นกลางให้ประโยชน์อีกหลายอย่าง

1. **ตรวจตัวตน:** เฉพาะผู้ใช้ที่เข้าสู่ระบบแล้วเท่านั้นที่ใช้ AI ได้
2. **จำกัดโควตา:** อ่านสลิปได้ 300 ใบ/วัน/คน ถามโค้ชได้ 60 ครั้ง/วัน/คน กันค่าใช้จ่ายบานปลาย
3. **เปลี่ยน AI ได้โดยไม่ต้องอัปเดตแอป:** สลับ Claude กับ Gemini แค่เปลี่ยน Secret บนเซิร์ฟเวอร์
4. **ทำความเข้าใจคำตอบ:** ตรวจสอบและแก้คำตอบของ AI ก่อนส่งถึงแอป (เช่น ปี พ.ศ. ที่ AI สัมแปลง)
5. **ความเป็นส่วนตัว:** ควบคุมได้ว่าส่งอะไรให้ AI บ้าง

เส้นทางของคำถาม 1 ข้อยิ่งน้องกล้า

1. แอป `src/app/(tabs)/coach.tsx`

ผู้ใช้พิมพ์ "ชื่อรองเท้า 1,290 ได้ไหม" แอปคำนวณ "ข้อเท็จจริง" เอง (ยอดเงิน ค่าเฉลี่ย Runway ก่อนและหลังชื่อ) ด้วยโค้ดใน `src/domain`

↓ `askCoach()` ใน `src/services/coach.ts`

2. `supabase.functions.invoke('coach')`

ส่ง `{context, question}` พร้อม token การเข้าสู่ระบบของผู้ใช้ ทาง HTTPS



3. Edge Function `coach` `supabase/functions/coach/index.ts`

ตรวจสอบ token → ตรวจสอบขนาดคำขอ → ตรวจสอบโควตา → สร้างข้อความให้ AI (system prompt + DATA + QUESTION)

↓ คุกกี้แวลู (Secret)

4. Claude หรือ Gemini

เขียนคำตอบภาษาไทย ไม่เกิน 90 คำ ใช้เฉพาะตัวเลขที่ได้รับ

↑ คำตอบย้อนกลับทางเดิม

5. แอปแสดงและอ่านออกเสียง

น้องกล้าขยับปากพร้อมซับไต้เตล (FR-5)

ฝั่งแอป: `src/services/coach.ts`

`src/services/coach.ts`

บรรทัด 1-25

```
1 /** FR-5: ask น้องกล้า, the AI coach (Edge Function "coach"). */
2 import { FunctionsHttpError } from '@supabase/supabase-js';
3 import { supabase } from '../data/supabase';
4
5 export class CoachError extends Error {}
6
7 export async function askCoach(params: { context: unknown; question: string }): Promise<string> {
8   if (!supabase) throw new CoachError('น้องกล้าตอบคำถามได้เมื่อเข้าสู่ระบบด้วยบัญชีจริง ระหว่างนี้แตะการ์ด "สิ่งที่ควรรู้ตอนนี้" ให้กล้าเล่าให้ฟังได้');
9   const { data, error } = await supabase.functions.invoke('coach', { body: params });
10  if (error) {
11    if (error instanceof FunctionsHttpError) {
12      const body = await (error.context as Response).json().catch(() => null);
13      if (body?.error?.code === 'quota') throw new CoachError('วันนี้คุยกับน้องกล้าครบโควตาแล้ว พรุ่งนี้มาคุยกันใหม่');
14      if (body?.error?.code === 'not_configured') {
15        throw new CoachError('โค้ช AI ยังไม่พร้อม: ผู้ดูแลต้องใส่ GEMINI_API_KEY ใน Supabase → Edge Functions → Secrets (หรือ key ที่ใส่ไว้ใช้ไม่ได้) ระหว่างนี้ดูคำแนะนำด้านบนได้เลย');
16      }
17      if (body?.error?.code === 'busy') throw new CoachError('ตอนนี้มีคนคุยกับน้องกล้าเยอะ รอสักครู่แล้วถามใหม่');
18      if (body?.error?.code === 'unauthorized') throw new CoachError('เซสชันหมดอายุ กรุณาเข้าสู่ระบบอีกครั้ง');
19    }
20    throw new CoachError('ติดต่อน้องกล้าไม่ได้ตอนนี้ ลองใหม่อีกครั้ง');
21  }
22  const text = (data as { text?: string })?.text;
```

```

23 if (!text) throw new CoachError('น้องกล้าไม่ได้ตอบกลับ ลองถามใหม่อีกครั้งนะ');
24 return text;
25 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/coach.ts:1

บรรทัด	ความหมาย
2	นำชนิด error ของ Supabase ที่บอกว่า "เซิร์ฟเวอร์ตอบกลับมาว่าผิดพลาด" มาใช้
5	สร้าง error ชนิดของตัวเอง ข้อความของมันคือสิ่งที่จะแสดงให้ผู้ใช้เห็น
7	ฟังก์ชัน askCoach รับข้อมูลสรุป (context) และคำถาม คืนข้อความคำตอบ
8	ไม่มี Supabase (เช่น build ที่ไม่ได้ตั้งค่า) → บอกผู้ใช้ว่าต้องใช้บัญชีจริง
9	เรียก Edge Function ชื่อ coach ส่ง context และ question ไป ไสวราวีแบบ token การเข้าสู่ระบบให้เอง
10–21	ถ้าผิดพลาด อ่านรหัส error ที่เซิร์ฟเวอร์ส่งมา แล้วแปลงเป็นข้อความภาษาไทยที่เป็นมิตร: ครบโควตา (quota), ยังไม่ใส่กุญแจ AI (not_configured), AI ไม่ว่าง (busy), หมดอายุการเข้าสู่ระบบ (unauthorized) อย่างอื่นบอกให้ลองใหม่
22–24	ดึงข้อความคำตอบ ถ้าไม่มีก็แจ้ง ถ้ามีส่งกลับไปให้น้องกล้า

ฝั่งเซิร์ฟเวอร์: Edge Function coach ที่ละบรรทัด

Edge Function เขียนด้วย TypeScript เหมือนแอป แต่รันบนเซิร์ฟเวอร์ของ Supabase ด้วยโปรแกรมชื่อ **Deno** (คล้าย Node.js) ฟังก์ชันหนึ่งตัวคือไฟล์เดอร์หนึ่งใน supabase/functions/ มีไฟล์ index.ts และไฟล์เดอร์ _shared เก็บโค้ดที่ใช้ร่วมกัน

supabase/functions/coach/index.ts

บรรทัด 1–26

```

1 // FR-5: the AI coach is น้องกล้า, the MindPay mascot. It explains the user's
2 // CONFIRMED numbers in one calm, friendly voice; the app reads the answer
3 // aloud, so it is written to be spoken. The app sends only totals by category
4 // (no slip images, no names of people, no account numbers).
5 import { aiProvider, BusyError, callAI, corsHeaders, envLimit, fail, json, NotConfiguredError,
6   requireUser, takeQuota } from '../_shared/common.ts';
7 import { plainText } from '../_shared/helpers.ts';
8
9 const DAILY_LIMIT = envLimit('COACH_DAILY_LIMIT', 60);
10 /** A question and a summary of totals: far below this. */
11 const MAX_BODY_BYTES = 64_000;
12
13 const SYSTEM = `You are น้องกล้า (Nong Kla), the mascot of MindPay, a Thai personal finance app for
14   students and first-jobbers.
15   You are a small golden money-tree sapling and the user's personal money coach.
16   Voice: calm, composed and warm (สุขุม เป็นมิตร), like a thoughtful friend who is good with money.
17   - Speak natural, everyday Thai. Call yourself "กล้า". Do not use ครับ orค่ะ; soft endings such as
18     "นะ" are fine.
19   - Your answer is read aloud by a text-to-speech voice and shown as subtitles, so write short
20     spoken sentences separated by spaces,
21     no emoji, no bullet points, no markdown, no lists, no headings.
22   Rules:
23   - Reply in Thai, at most 90 words.

```

- 22 - Use ONLY the numbers in the DATA block. Never invent amounts, dates, shops or trends. If the data cannot answer, say what is missing.
- 23 - Structure: what the numbers show -> why it matters -> one or two small suggestions -> leave the decision to the user.
- 24 - Never shame or scold ("ใช้เยอะเกินไป!" is not allowed). Be kind about mistakes.
- 25 - When the user asks "can I buy X", use the "purchaseCheck" numbers if present and state the runway before and after.
- 26 - You are not a licensed financial advisor: no investment, loan or crypto recommendations.`;

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/coach/index.ts:1`

บรรทัด	ความหมาย
1-4	คอมเมนต์: โค้ชคือน้องกล้า อธิบายตัวเลขที่ยืนยันแล้ว คำตอบจะถูกอ่านออกเสียงจึงเขียนแบบภาษาพูด แอปส่งเฉพาะยอดรวมตามหมวด (ไม่มีรูปสลิป ชื่อคน หรือเลขบัญชี)
5-6	นำตัวช่วยจาก <code>_shared</code> มาใช้ (นามสกุล <code>.ts</code> ต้องเขียนเต็มใน Deno)
8	โควตาต่อวัน อ่านจาก <code>Secret COACH_DAILY_LIMIT</code> ถ้าไม่ได้ตั้งใช้ 60
10	ขนาดคำขอสองสูงสุด 64,000 ไบต์ (คำถามกับตัวเลขสรุปเล็กกว่านี้มาก กันคนส่งข้อมูลขยะก้อนใหญ่)
12-26	System prompt (เขียนเป็นภาษาอังกฤษเพราะ AI ทำตามคำสั่งภาษาอังกฤษได้แม่นยำกว่า แต่สั่งให้ตอบเป็นไทย) บอกว่า: เป็นน้องกล้า มาสคอตต้นกล้าสีทอง · น้ำเสียงสุ่ม เป็นมิตร เรียกตัวเองว่า "กล้า" ไม่ใช่ครับ/ค่ะ · คำตอบถูกอ่านออกเสียง จึงห้ามอีโมจิ ห้ามรายการ ห้าม markdown · ตอบไม่เกิน 90 คำ · ใช้เฉพาะตัวเลขในส่วน DATA ห้ามแต่งตัวเลขขึ้นมาเอง · โครงสร้างคำตอบ: ตัวเลขบอกอะไร → สำคัญยัง → คำแนะนำเล็ก ๆ 1-2 ข้อ → ให้ผู้ใช้ตัดสินใจเอง · ห้ามตำหนิ · ถ้าถาม "ซื้อได้ไหม" ให้ใช้ตัวเลข <code>purchaseCheck</code> บอก Runway ก่อนและหลัง · ไม่ใช่ที่ปรึกษาการเงินที่มีใบอนุญาต ห้ามแนะนำลงทุน กู้ยืม ทรัพย์สิน

`supabase/functions/coach/index.ts`

บรรทัด 28-76

```

28 Deno.serve(async (req) => {
29   if (req.method === 'OPTIONS') return new Response('ok', { headers: corsHeaders });
30   if (req.method !== 'POST') return fail('method_not_allowed', 'Use POST', 405);
31
32   const userId = await requireUser(req);
33   if (!userId) return fail('unauthorized', 'Sign in again', 401);
34
35   if (Number(req.headers.get('content-length') ?? 0) > MAX_BODY_BYTES) return fail('too_large',
'Request is too large', 413);
36   let body: { context?: unknown; question?: string };
37   try {
38     body = await req.json();
39   } catch {
40     return fail('bad_request', 'Body must be JSON', 400);
41   }
42   if (!body || typeof body !== 'object') return fail('bad_request', 'Body must be a JSON object',
400);
43   const question = (body.question ?? '').toString().slice(0, 400).trim();
44   const data = JSON.stringify(body.context ?? {}).slice(0, 6000);
45
46   if (!aiProvider()) return fail('not_configured', 'AI key is not set on the server', 503);
47
48   try {
49     if (!(await takeQuota(userId, 'coach', DAILY_LIMIT))) {
50       return fail('quota', 'Daily coach limit reached, try again tomorrow', 429);
51     }
52     const text = await callAI({

```

```

53     task: 'coach',
54     system: SYSTEM,
55     maxTokens: 500,
56     content: [
57       {
58         type: 'text',
59         text: `DATA:\n${data}\n\nQUESTION:\n${question} || 'สรุปสถานะการเงินของเงินช่วงนี้ และบอกสิ่งที่คว
60       }`,
61     ],
62   });
63   return json({ text: plainText(text) });
64 } catch (e) {
65   if (e instanceof NotConfiguredError) {
66     console.error('coach: AI key missing or refused', e.message);
67     return fail('not_configured', 'AI key is not set on the server or was refused', 503);
68   }
69   if (e instanceof BusyError) {
70     console.warn('coach: AI busy', e.message);
71     return fail('busy', 'The AI service is busy, try again shortly', 503);
72   }
73   console.error('coach failed', e);
74   return fail('coach_error', 'The coach is unavailable right now', 502);
75 }
76 });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/coach/index.ts:28

บรรทัด	ความหมาย
28	Deno.serve = "เปิดร้าน" รอรับคำขอ ทุกคำขอที่เข้ามาจะเรียกฟังก์ชันนี้ 1 ครั้ง
29	คำขอแบบ OPTIONS คือเบราว์เซอร์ "ถามทาง" ก่อนส่งจริง (กฎ CORS ของเว็บ) ตอบ ok พร้อม header ที่อนุญาต
30	รับเฉพาะ POST อย่างอื่นตอบ 405
32–33	ตรวจตัวตน: อ่าน token จากคำขอแล้วถาม Supabase ว่าเป็นใคร ไม่ผ่าน → 401 (แอปจะบอก "เซสชันหมดอายุ")
35	คำขอใหญ่เกิน → 413 (ตรวจจากหัวคำขอ ก่อนอ่านเนื้อหา จึงไม่เปลืองหน่วยความจำ)
36–42	อ่านเนื้อหาเป็น JSON ถ้าอ่านไม่ได้หรือไม่ใช่ object → 400
43	คำถามตัดเหลือไม่เกิน 400 ตัวอักษร และตัดช่องว่างหัวท้าย
44	ข้อมูลสรุปแปลงเป็นข้อความ JSON ตัดเหลือไม่เกิน 6,000 ตัวอักษร (คุมค่าใช้จ่าย)
46	ไม่มีกุญแจ AI ใน Secrets → 503 not_configured
49–51	ตรวจโควตา และนับเพิ่ม 1 ในคำสั่งเดียว เกิน 60 ครั้งใน 24 ชั่วโมง → 429
52–62	เรียก AI ส่ง system prompt, จำกัดคำตอบ 500 token, และข้อความที่มี 2 ส่วน: DATA: ตามด้วยตัวเลข และ QUESTION: ตามด้วยคำถาม (ถ้าผู้ใช้ไม่ได้ถาม ใช้คำถามตั้งต้น "สรุปสถานะการเงินของเงินช่วงนี้...")
63	ล้าง markdown ที่ AI อาจใส่มา (plainText) แล้วตอบแอปเป็น {"text": "..."}
64–75	จัดการ error ตามชนิด: กุญแจผิด/ไม่มี → 503 not_configured, AI ไม่ว่าง → 503 busy, อื่น ๆ → 502 และบันทึก log ไว้ดูใน Dashboard (บันทึกเฉพาะชนิด error ไม่บันทึกคำถามหรือคำตอบของผู้ใช้)

ตัวช่วยกลาง: supabase/functions/_shared/common.ts

บรรทัด 17-83

```
17 /** Read one environment variable; a name the runtime refuses counts as "not set". */
18 const env = (name: string) => {
19   try {
20     return Deno.env.get(name);
21   } catch {
22     return undefined;
23   }
24 };
25 const geminiKey = () => readKey(env, GEMINI_KEY_NAMES);
26 const claudeKey = () => readKey(env, CLAUDE_KEY_NAMES);
27
28 export const corsHeaders = {
29   'Access-Control-Allow-Origin': '*',
30   'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
31   'Access-Control-Allow-Methods': 'POST, OPTIONS',
32 };
33
34 export function json(body: unknown, status = 200) {
35   return new Response(JSON.stringify(body), {
36     status,
37     headers: { ...corsHeaders, 'Content-Type': 'application/json' },
38   });
39 }
40
41 /** Error body the app understands: { error: { code, message } } */
42 export function fail(code: string, message: string, status: number) {
43   return json({ error: { code, message } }, status);
44 }
45
46 /** Returns the signed-in user's id, or null when the request has no valid session. */
47 export async function requireUser(req: Request): Promise<string | null> {
48   const token = req.headers.get('Authorization')?.replace(/^Bearer\s+/i, '');
49   if (!token) return null;
50   const { data, error } = await admin().auth.getUser(token);
51   if (error || !data.user) return null;
52   return data.user.id;
53 }
54
55 function admin() {
56   return createClient(Deno.env.get('SUPABASE_URL')!, Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!,
57     {
58       auth: { persistSession: false },
59     });
60 }
61
62 /** Thrown when no AI key is set in the function secrets, or the key was refused. */
63 export class NotConfiguredError extends Error {}
64
65 /** The AI service is rate-limiting us (e.g. Gemini free tier) or overloaded: try again shortly. */
66 export class BusyError extends Error {}
67
68 /** A daily limit from the function settings; a missing or non-number setting uses the default. */
69 export function envLimit(name: string, fallback: number): number {
70   const n = Number(Deno.env.get(name));
```

```

70 return Number.isFinite(n) && n > 0 ? Math.floor(n) : fallback;
71 }
72
73 /**
74  * Daily cap per user so a leaked account or a bug cannot run up the AI bill.
75  * Returns false when the user is over the limit; otherwise records one use.
76  * Counting and recording happen in one database call under a per-user lock
77  * (take_ai_quota), so many requests at once cannot all slip under the limit.
78  */
79 export async function takeQuota(userId: string, kind: 'slip' | 'coach', dailyLimit: number) {
80   const { data, error } = await admin().rpc('take_ai_quota', { p_user: userId, p_kind: kind,
81     p_limit: dailyLimit });
82   if (error) throw error;
83   return data === true;
84 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/_shared/common.ts:17`

บรรทัด	ความหมาย
17–24	อ่านค่า Secret จากเชิร์ฟเวอร์ (<code>Deno.env.get()</code>) ถ้าชื่อแปลจนอ่านไม่ได้ ถือว่าไม่ได้ตั้ง
25–26	หากุญแจ Gemini และ Claude จากรายชื่อที่ยอมรับ (ดู <code>readKey</code> ด้านล่าง)
28–32	header CORS: ยอมให้เว็บจากทุกที่เรียกได้ (ความปลอดภัยมาจาก token ไม่ใช่จากที่อยู่เว็บ)
34–39	<code>json()</code> สร้างคำตอบ HTTP ที่เป็น JSON พร้อม header
41–44	<code>fail()</code> สร้างคำตอบ error รูปแบบเดียวกันทุกครั้ง <code>{ "error": { "code", "message" } }</code> ที่แอปอ่าน code ได้ (บรรทัด 13–18 ของ <code>coach.ts</code> ผังแอป)
46–53	<code>requireUser</code> : ตัดคำว่า "Bearer " ออกจาก header Authorization ได้ token แล้วถาม Auth ว่า token นี้เป็นของใคร คืน id ผู้ใช้ หรือ null
55–59	<code>admin()</code> : client ที่ใช้ service role key (ข้าม RLS ได้) มีเฉพาะบนเชิร์ฟเวอร์
61–65	error 2 ชนิด: ไม่ได้ตั้งกุญแจ/กุญแจใช้ไม่ได้ และ AI ไม่ว่าง (ติดลิมิตหรือโหลดเกิน)
67–71	อ่านโควตาจาก Secret ถ้าไม่ใช่ตัวเลขบวกใช้ค่าตั้งต้น
73–83	<code>takeQuota</code> : เรียกฟังก์ชัน SQL <code>take_ai_quota</code> ในฐานข้อมูล (rpc = เรียกฟังก์ชันระยะไกล) ซึ่งนับและบันทึกในชั้นตอนเดียว

ทำไมนับโควตาต้อง "ทำในชั้นตอนเดียว"

`supabase/migrations/20261001000000_atomic_ai_quota.sql`

บรรทัด 1–44

```

1 -- Atomic daily AI quota (audit 1 Oct 2569).
2 -- Before: the Edge Functions counted today's uses and inserted a new one in two
3 -- separate calls, so many requests sent at once could all pass the limit.
4 -- Now counting and recording happen in one call, under a lock per user and kind.
5 -- Called only by the Edge Functions with the service role.
6
7 create or replace function public.take_ai_quota(p_user uuid, p_kind text, p_limit integer)
8 returns boolean
9 language plpgsql
10 security definer
11 set search_path = ''

```

```

12 as $$
13 declare
14     used integer;
15 begin
16     if p_kind not in ('slip', 'coach') or p_limit is null or p_limit < 1 then
17         return false;
18     end if;
19     perform pg_advisory_xact_lock(hashtextextended(p_user::text || ':' || p_kind, 0));
20     select count(*) into used
21         from public.ai_usage
22         where user_id = p_user and kind = p_kind and created_at >= now() - interval '24 hours';
23     if used >= p_limit then
24         return false;
25     end if;
26     insert into public.ai_usage (user_id, kind) values (p_user, p_kind);
27     return true;
28 end;
29 $$;
30
31 revoke execute on function public.take_ai_quota(uuid, text, integer) from public, anon,
    authenticated;
32 grant execute on function public.take_ai_quota(uuid, text, integer) to service_role;
33
34 -- Amounts have the same upper bound as savings goals ($1,000,000,000), so a misread
35 -- amount can never overflow what the app handles.
36 alter table public.transactions
37     drop constraint if exists transactions_amount_max,
38     add constraint transactions_amount_max check (amount_satang <= 1000000000000);
39 alter table public.profiles
40     drop constraint if exists profiles_amounts_max,
41     add constraint profiles_amounts_max check (
42         abs(opening_balance_satang) <= 1000000000000
43         and (monthly_budget_satang is null or monthly_budget_satang <= 1000000000000)
44     );

```

► เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20261001000000_atomic_ai_quota.sql:1

บรรทัด	ความหมาย
1-5	คอมเมนต์เล่าบ๊วกเดิม: เดิมนับจำนวนครั้งวันนี้นับกับบันทึกครั้งใหม่เป็น 2 คำสั่งแยกกัน ถ้าส่งคำขอพร้อมกันหลายอัน ทุกอันนับได้ "ยังไม่เกิน" แล้วผ่านหมด (race condition) แก้ให้นับและบันทึกในคำสั่งเดียวภายใต้กุญแจล็อก
7-14	สร้างฟังก์ชัน SQL รับ id ผู้ใช้ ชนิตงาน (slip/coach) และเพดาน คำนวณจริง/เท็จ รับด้วยสิทธิ์เจ้าของ (security definer)
16-18	ค่าที่ส่งมาผิด → ไม่อนุญาต
19	ล็อก ต่อผู้ใช้ต่อชนิตงาน: คำขอที่ 2 ของคนเดียวกันต้องรอคำขอแรกเสร็จก่อน (ล็อกปลดเองเมื่อจบคำสั่ง)
20-22	นับจำนวนการใช้ใน 24 ชั่วโมงล่าสุด
23-25	ถึงเพดานแล้ว → ไม่อนุญาต
26-27	ยังไม่ถึง → บันทึกการใช้ 1 ครั้ง แล้วอนุญาต
31-32	ห้ามผู้ใช้ทั่วไปเรียกฟังก์ชันนี้ตรง ๆ ให้เฉพาะเซิร์ฟเวอร์ (service_role)
34-44	(ไฟล์เดียวกัน) เพิ่มเพดานจำนวนเงิน \$1,000,000,000 ให้ตารางรายการและโปรไฟล์ ตรงกับ MAX_SATANG ในแอป

เลือก AI ตัวไหน: Claude หรือ Gemini

supabase/functions/_shared/helpers.ts

บรรทัด 1-45

```
1 // Pure helpers for the Edge Functions. No Deno APIs here, so the same file is
2 // unit-tested with vitest (see __tests__/helpers.test.ts).
3
4 export type AiProvider = 'claude' | 'gemini';
5
6 /**
7  * Secret names accepted for each service. The first is the documented one;
8  * the rest are Google's other standard name and common typing slips seen in
9  * the Supabase dashboard (a secret saved as "GEMINI API KEY", 28 ก.ย. 2569).
10 */
11 export const GEMINI_KEY_NAMES = [
12   'GEMINI_API_KEY',
13   'GOOGLE_API_KEY',
14   'GOOGLE_GENERATIVE_AI_API_KEY',
15   'GEMINI API KEY',
16   'GEMINI-API-KEY',
17   'GEMINI_KEY',
18 ];
19 export const CLAUDE_KEY_NAMES = ['ANTHROPIC_API_KEY', 'CLAUDE_API_KEY', 'ANTHROPIC API KEY'];
20
21 /**
22  * The first secret that is set, with spaces, line breaks or quotes from
23  * copy-paste removed. Returns null when none is set.
24 */
25 export function readKey(get: (name: string) => string | undefined | null, names: string[]):
  string | null {
26   for (const name of names) {
27     const raw = get(name);
28     const clean = raw?.trim().replace(/^[\"']|['\"]$/g, '').trim();
29     if (clean) return clean;
30   }
31   return null;
32 }
33
34 /**
35  * Which AI service answers: Claude when its key is set, otherwise Gemini.
36  * AI_PROVIDER=claude|gemini forces one. Returns null when no usable key is set.
37 */
38 export function pickProvider(env: { forced?: string | null; hasClaude: boolean; hasGemini:
  boolean }): AiProvider | null {
39   const forced = env.forced?.trim().toLowerCase();
40   if (forced === 'claude') return env.hasClaude ? 'claude' : null;
41   if (forced === 'gemini') return env.hasGemini ? 'gemini' : null;
42   if (env.hasClaude) return 'claude';
43   if (env.hasGemini) return 'gemini';
44   return null;
45 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/helpers.ts:1

บรรทัด ความหมาย

1-2 โฟลน์นี้เป็นฟังก์ชันล้วน ไม่ใช้คำสั่งของ Deno จึงเอาไปเทสต์ด้วย vitest ได้เหมือนโค้ดในแอป

บรรทัด	ความหมาย
6–19	รายชื่อ Secret ที่ยอมรับ ตัวแรกคือชื่อที่ถูกต้อง ตัวอื่นเป็นชื่อสำรองและ ชื่อที่พิมพ์ผิดบ่อย คอมเมนต์เล่าว่าเคยมีคนตั้งชื่อ Secret เป็น "GEMINI API KEY" (มีเว้นวรรค) เมื่อ 28 ก.ย. 2569 ทำให้ AI ไม่ทำงาน ทีมจึงให้เซิร์ฟเวอร์รับชื่อเหล่านี้ด้วย
25–32	readKey: ใส่ชื่อที่ละตัว เจอตัวแรกที่มีค่า ตัดช่องว่าง การขึ้นบรรทัด และเครื่องหมายคำพูดที่ติดมาตอนถือปวงออก
38–45	pickProvider: ถ้าตั้ง AI_PROVIDER บังคับไว้ ใช้ตัวนั้น (ต้องมีคีย์แจด้วย) ไม่จันมีคีย์แจ Claude ใช้ Claude ก่อน ไม่มีก็ใช้ Gemini ไม่มีเลยคืน null

	Claude (Anthropic)	Gemini (Google)
Secret	ANTHROPIC_API_KEY	GEMINI_API_KEY
รุ่นที่ใช้ (ค่าเริ่มต้น)	claude-haiku-4-5 (เปลี่ยนได้ด้วย SLIP_MODEL, COACH_MODEL)	gemini-3.5-flash-lite แล้วลองรุ่นถัดไปเองถ้าใช้ไม่ได้
ค่าใช้จ่าย	ต้องเติมเครดิต อ่านสลิปใบละประมาณ 10 สตางค์	มีแบบฟรี (มีเพดานต่อนาที/ต่อวัน)
ข้อมูลของผู้ใช้	ไม่นำไปฝึกโมเดล	แบบฟรี Google อาจนำข้อมูลไปปรับปรุงโมเดล (README บอกไว้ตรง ๆ)

ส่งคำขอไปที่ Claude: callClaude

supabase/functions/_shared/common.ts

บรรทัด 114–200

```
114 /** Ask the configured AI and return its text reply. */
115 export async function callAI(req: AiRequest): Promise<string> {
116   return (await callAIWithModel(req)).text;
117 }
118
119 /**
120  * Longest wait for one call to the AI service, and for all tries of one request together
121  * (the platform stops a function after 150 s; both slip reads run at the same time).
122  */
123 const FETCH_TIMEOUT_MS = 45_000;
124 const REQUEST_DEADLINE_MS = 110_000;
125
126 /** Ask the configured AI; also says which model answered (the two slip reads compare it). */
127 export async function callAIWithModel(req: AiRequest): Promise<{ text: string; model: string }> {
128   const provider = aiProvider();
129   if (!provider) throw new NotConfiguredError('No AI key is set');
130   const started = Date.now();
131   const deadline = started + REQUEST_DEADLINE_MS;
132   const result = provider === 'claude' ? await callClaude(req, deadline) : await callGemini(req,
133     deadline);
134   // Metadata only: never log images, questions or replies.
135   console.log(JSON.stringify({ ai: provider, model: result.model, task: req.task, variant:
136     req.variant ?? 'primary', ms: Date.now() - started }));
137   return result;
138 }
139
140 /** fetch with a time limit; running out of time counts as the AI being busy (the app waits and
141   retries). */
142 async function timedFetch(url: string, init: RequestInit, deadline: number): Promise<Response> {
```

```

140   const left = deadline - Date.now();
141   if (left <= 1000) throw new BusyError('AI request deadline reached');
142   try {
143     return await fetch(url, { ...init, signal: AbortSignal.timeout(Math.min(FETCH_TIMEOUT_MS,
144   left)) });
145   } catch (e) {
146     const name = (e as Error)?.name;
147     if (name === 'TimeoutError' || name === 'AbortError') throw new BusyError(`AI request timed
148   out (${url.split('/')[2]})`);
149     throw e;
150   }
151 }
152
153 const sleep = (ms: number) => new Promise((resolve) => setTimeout(resolve, ms));
154
155 async function callClaude(req: AiRequest, deadline: number): Promise<{ text: string; model:
156   string }> {
157   const key = claudeKey();
158   const main = Deno.env.get(req.task === 'slip' ? 'SLIP_MODEL' : 'COACH_MODEL') ??
159   CLAUDE_DEFAULT_MODEL;
160   const model = req.variant === 'verify' ? (Deno.env.get('VERIFY_MODEL') ?? main) : main;
161   const body: Record<string, unknown> = {
162     model,
163     max_tokens: req.maxTokens,
164     system: req.system,
165     messages: [
166       {
167         role: 'user',
168         content: req.content.map((c) =>
169           c.type === 'text' ? c : { type: 'image', source: { type: 'base64', media_type:
170   c.mediaType, data: c.data } },
171       ),
172     ],
173   };
174   if (req.schema) {
175     body.output_config = { format: { type: 'json_schema', schema: req.schema } };
176   }
177   const res = await timedFetch(
178     'https://api.anthropic.com/v1/messages',
179     {
180       method: 'POST',
181       headers: {
182         'x-api-key': key,
183         'anthropic-version': '2023-06-01',
184         'content-type': 'application/json',
185       },
186       body: JSON.stringify(body),
187     },
188     deadline,
189   );
190   if (!res.ok) {
191     const detail = await res.text();
192     const msg = `Claude API ${res.status}: ${detail.slice(0, 300)}`;
193     if (isInvalidKey(res.status, detail)) throw new NotConfiguredError(msg);
194     if (res.status === 429 || res.status === 529) throw new BusyError(msg);
195     throw new Error(msg);
196   }
197   const data = await res.json();
198   const text = (data.content ?? []).
199     .filter((b: { type: string }) => b.type === 'text')

```

```

196     .map((b: { text: string }) => b.text)
197     .join('');
198     if (!text) throw new Error('Claude API returned no text');
199     return { text, model };
200 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/common.ts:114

บรรทัด	ความหมาย
114–117	callAI เรียกตัวที่บอกชื่อรุ่นด้วย แล้วคืนแค่ข้อความ
119–124	เวลาสูงสุด: รอ AI ครั้งละไม่เกิน 45 วินาที และทั้งคำขอไม่เกิน 110 วินาที (Supabase จะตัดฟังก์ชันที่ 150 วินาที)
127–136	เลือก AI แล้วเรียก จับเวลา และเขียน log เฉพาะข้อมูลคำกับ (ตัวไหน รุ่นไหน งานอะไร ใช้กี่มิลลิวินาที) ไม่เคยบันทึกรูปคำถาม หรือคำตอบ
139–149	timedFetch: ส่งคำขอทางอินเทอร์เน็ตแบบมีเวลาจำกัด ถ้าหมดเวลาถือว่า AI ไม่ว่าง (แอปจะรอแล้วลองใหม่)
153–156	เลือกรุ่นของ Claude ตามงาน ส่วนการอ่านสลิปครั้งที่ 2 (verify) ใช้รุ่น VERIFY_MODEL ถ้าตั้งไว้
157–169	สร้างเนื้อหาคำขอตามรูปแบบของ Anthropic Messages API: รุ่น, จำนวน token สูงสุด, system prompt และข้อความของผู้ใช้ (รูปแปลงเป็นรูปแบบ base64 ที่ Claude รับ)
170–172	ถ้างานนี้ต้องการ JSON ตามแบบฟอร์ม (อ่านสลิป) ใส่ schema ไปด้วย Claude จะตอบตามแบบฟอร์มเป็น
173–185	ส่งไปที่ https://api.anthropic.com/v1/messages แบบถูกใจใน header x-api-key
186–192	ตอบกลับไม่สำเร็จ: คุญแจผิด → NotConfigured, 429/529 (ส่งถี่เกิน/ระบบเต็ม) → Busy, อื่น ๆ → error ทั่วไป
193–199	สำเร็จ: เอาเฉพาะส่วนที่เป็นข้อความมาต่อกัน คืนพร้อมชื่อรุ่น

ฝั่ง Gemini (callGemini บรรทัด 202–268) หลักการเดียวกันแต่ซับซ้อนกว่า เพราะแบบฟรีติดลิมิตบ่อย: ไล่ลองหลายรุ่นตามลำดับ ถ้ารุ่นไหนใช้ไม่ได้กับคุญแจนี้ข้ามไปรุ่นถัดไป ถ้าทุกรุ่น "ไม่ว่าง" รอตามเวลาที่ Google บอก (ไม่เกิน 12 วินาที) แล้วลองทั้งหมดอีก 1 รอบ ก่อนจะบอกแอปว่า busy การตัดสินใจหลังแต่ละ error อยู่ในฟังก์ชัน geminiStep

supabase/functions/_shared/helpers.ts

บรรทัด 75–93

```

75 /**
76  * What to do after a failed Gemini call:
77  * - invalid_key: stop, the key is wrong (tell the app "not configured")
78  * - plain_schema: retry the same model with the schema described in the prompt
79  * - next_model: this model is not available to this key; try the next one
80  * - busy: rate limit or overload on this model. Each model has its own free
81  *   quota, so try the next model first and wait only when all are busy.
82  * - fail: anything else
83  */
84 export type GeminiStep = 'invalid_key' | 'plain_schema' | 'next_model' | 'busy' | 'fail';
85
86 export function geminiStep(status: number, body: string, usingJsonSchema: boolean): GeminiStep {
87   if (isInvalidKey(status, body)) return 'invalid_key';
88   if (status === 404) return 'next_model';
89   if (status === 429 && isZeroQuota(body)) return 'next_model';
90   if (status === 400 && usingJsonSchema) return 'plain_schema';
91   if (status === 429 || status === 500 || status === 503) return 'busy';
92   return 'fail';

```

บรรทัด	ความหมาย
87	กุญแจผิด (401/403 หรือข้อความ API key not valid) → หยุด บอกว่ายังไม่ได้ตั้งค่า
88	404 = ไม่มีรุ่นนี้ → ลองรุ่นถัดไป
89	429 ที่บอกว่าโควตาเป็น 0 (กุญแจนี้ไม่มีสิทธิ์ใช้รุ่นนี้เลย) → รุ่นถัดไป
90	400 ตอนใช้ JSON schema (รุ่นเก่าบางรุ่นไม่รองรับ) → ลองใหม่โดยเขียนแบบฟอร์มลงใน prompt แทน
91	429/500/503 → ไม่ว่าง ลองรุ่นอื่นก่อน แล้วค่อยรอ
92	อย่างอื่น → ล้มเหลว

ตั้งค่า AI (ผู้ดูแลทำครั้งเดียว)

1. สร้างกุญแจ: Gemini ที่ aistudio.google.com/apikey (ฟรี) หรือ Claude ที่ console.anthropic.com
2. เปิด Supabase Dashboard → **Edge Functions** → **Secrets** (ไม่ใช่ Vault และไม่ใช่หน้า API keys)
3. เพิ่ม Secret ชื่อ GEMINI_API_KEY หรือ ANTHROPIC_API_KEY ใส่ค่ากุญแจ
4. ไม่ต้อง deploy ใหม่ ฟังก์ชันอ่าน Secret ทุกครั้งที่มีการขอ
5. ถ้าแก้ไขโค้ดใน supabase/functions ต้อง deploy ด้วย `npx supabase functions deploy coach` (หรือ `parse-slip`)

หลักการใช้ AI อย่างรับผิดชอบที่ทีมเราใช้

หลักการ	ในโค้ด
AI ไม่คิดเลข โค้ดที่ทดสอบแล้วคิด AI แค่อธิบาย	แอปคำนวณ Runway และ purchaseCheck เองก่อนส่ง (FR-5, FR-6) system prompt ห้ามแต่งตัวเลข
AI ผิดได้ ต้องมีคนตรวจ	สลิปที่ไม่มั่นใจเป็นแบบร่าง อ่าน 2 ครั้งด้วย 2 รุ่น ไม่ตรงกันให้ผู้ใช้เช็ค (FR-4)
ส่งข้อมูลให้ AI น้อยที่สุด	โค้ชได้เฉพาะยอดรวม รูปที่ส่งอ่านคือรูปที่มี QR สลิปเท่านั้น และไม่เก็บรูปบนเซิร์ฟเวอร์
คุมค่าใช้จ่าย	โควตาต่อวันแบบ atomic, ตัดความยาวคำถาม, จำกัด token คำตอบ
บันทึกการใช้ AI ในการพัฒนา	docs/AI_USAGE_LOG.md บันทึกตาม SRS ว่าใช้ AI ช่วยอะไร แก้อะไร และสมาชิกตรวจอะไร

UX/UI: การออกแบบหน้าจอและ Design System

UI คือหน้าตา (สี ตัวอักษร ปุ่ม) UX คือประสบการณ์ (ใช้งานง่ายไหม รู้สึกยังไง) บทนี้อธิบายหลักการออกแบบของ MindPay และโค้ดที่บังคับให้ทุกหน้าทำตาม

หลักการออกแบบของ MindPay

หลักการ	ทำอย่างไรในแอป
ตัวเลขเดียว เข้าใจทันที	หน้าหลักมีการ์ดเขียวที่บอกยอดเงินและ "เงินพอใช้อีก 16 วัน" ตัวใหญ่ที่สุด ผู้ใช้ไม่ต้องคิดเลขเอง
ทำสิ่งที่ทำบ่อยให้ง่ายที่สุด	ปุ่มสแกนสลิปใหญ่สุดกลางแถบล่าง, สแกนอัตโนมัติตอนเปิดแอป, พูดจได้, ปุ่มลัด + ฿100/500/1,000/5,000 ตอนเพิ่มเงินเข้า
ไม่ดู ไม่ทำให้รู้สึกผิด	บอกล่าพุดว่า "ลองพักของที่ไม่จำเป็นสักหน่อยนะ ค่อย ๆ ไปด้วยกัน" แทน "ใช้เงินเยอะเกินไป!" (system prompt ของโค้ชห้ามตำหนิ)
บอกตรง ๆ ว่ามันใจแค่นี้	สลิปที่ไม่ชัดเป็น "แบบร่าง" พร้อมบอกว่าช่องไหนต้องตรวจ, Runway ใช้คำว่า "ถึงประมาณ"
อธิบายตัวเองได้	การ์ด "คำนวณยังงี้" ในหน้าเงินพอถึงแสดงสูตรที่ละบรรทัดเหมือนใบเสร็จ
ย้อนกลับได้	ลบรายการแล้วกด "เลิกทำ" ได้ (FR-1), สลิปที่เข้ายอดเองกด "ยังไม่รวมยอด" ได้ (FR-4)
ข้อความผิดพลาดต้องบอกวิธีแก้	"ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50" แทน "Invalid input"
ทุกคนใช้ได้ (accessibility)	สีผ่านเกณฑ์อ่านง่าย, ปุ่มใหญ่พอแตะ, มีข้อความสำหรับโปรแกรมอ่านหน้าจอ, ปิดแอนิเมชันได้

Design System คืออะไร

Design System คือ "กฎกลาง + ชิ้นส่วนสำเร็จรูป" ที่ทุกหน้าใช้ร่วมกัน เหมือนชุดตัวต่อเลโก้ที่มีขนาดและสีมาตรฐาน หน้าจอทุกหน้าจึงดูเป็นแอปเดียวกัน และเปลี่ยนธีมสีได้ทั้งแอปในจุดเดียว ของ MindPay อยู่ใน

- `src/ui/theme.ts` ค่ากลาง (token): สี ฟอนต์ ขนาดตัวอักษร ระยะห่าง มุมโค้ง
- `src/ui/components.tsx`, `src/ui/inputs.tsx`, `src/ui/feedback.tsx` ชิ้นส่วน: ตัวหนังสือ ปุ่ม การ์ด ช่องกรอก ข้อความแจ้งเตือน
- `docs/DESIGN_SYSTEM.md` เอกสารกฎ กฎข้อแรก: ห้ามหน้าจอไหนคิดสีหรือขนาดเอง

สี: Green + Gold

ทิศทางแบรนด์คือ สีเขียว (เงิน การเติบโต ความมั่นคง) ใช้กับโครงสร้าง และ สีทอง ใช้ "น้อย ๆ" กับสิ่งที่สำคัญที่สุดบนหน้า เช่น ยอดเงิน ปุ่มหลัก ใบของต้นไม้เงิน คอมเมนต์บนสุดของ theme.ts เขียนว่า "Never paint a whole screen green"

`src/ui/theme.ts`

บรรทัด 1-21

1 /**

```

2  * MindPay design system tokens.
3  *
4  * Brand: Green (money, growth, stability) + Gold (accent for what matters most).
5  * Rule from the brand direction: green is for structure, gold is used sparingly
6  * for the single most important thing on a screen (the balance, the main action).
7  * Never paint a whole screen green.
8  */
9  import { useColorThemeKey, useSchemeChoice } from './themeMode';
10
11 export const palette = {
12   forest: '#0E3B2C', // hero surfaces, brand
13   forestDeep: '#082A1F',
14   leaf: '#1F7A52', // primary actions, positive
15   mint: '#DDEEE3',
16   gold: '#C8992A', // accent: balance, main CTA ring, tree leaves
17   goldBright: '#E2B64A',
18   goldSoft: '#F4E7C2',
19   clay: '#B4442F', // critical / expense emphasis
20   amber: '#B7791F', // watch
21 };

```

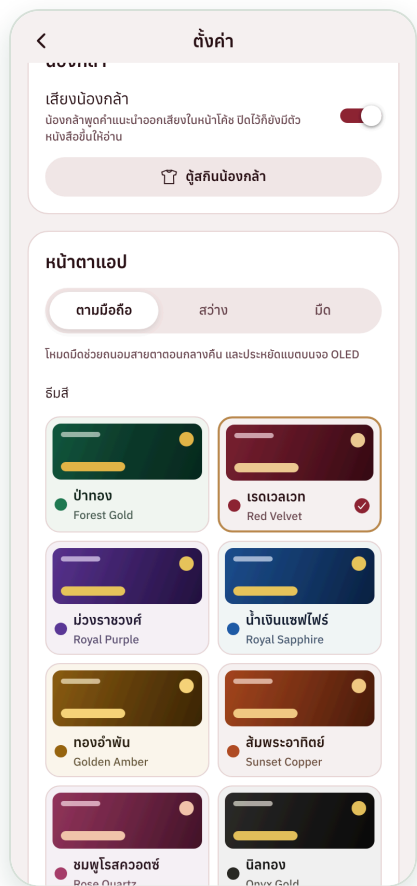
▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:1

สีทั้งหมดถูกตั้งชื่อตาม **หน้าที่** (token) ไม่ใช่ตามสี เช่น ink = สีตัวหนังสือหลัก, primary = สีปุ่มหลัก หน้าจอเรียก theme.ink โดยไม่สนว่าตอนนี้เป็นธีมไหน

Token	ใช้กับ	ป่าทอง สว่าง	ป่าทอง มืด
bg	พื้นหลังหน้าจอ	#F2F5F1	#0A1410
surface	การ์ด ช่องกรอก	#FFFFFF	#111E18
ink	ตัวหนังสือหลัก	#11231B	#E7EFE9
inkSoft	ตัวหนังสือรอง คำอธิบาย	#4B5F55	#A2B6AA
inkFaint	placeholder ไอคอนเจียม ๆ (ห้ามใช้กับข้อความที่ต้องอ่าน)	#8A9A91	#6B8076
primary	ปุ่มหลัก สิ่ง que เลือกอยู่	#1F7A52	#3FA774
accent	ปุ่มทอง ขอบการ์ดที่ต้องสนใจ	#C8992A	#E2B64A
income / good / watch / critical	สีที่มีความหมาย: เงินเข้า, ดี, ระวัง, วิกฤต เหมือนกันทุกธีม ผู้ใช้จึงไม่สับสน	เขียว / เขียว / ส้มอำพัน / แดงอิฐ	

9 ธีม × สว่าง/มืด



ตั้งค่า → ธีมสี: เลือก 1 ใน 9 ธีม และโหมด ตามมือถือ / สว่าง / มืด

ผู้ใช้เลือกธีมได้ 9 แบบ: ป่าทอง (ค่าเริ่มต้น), เรดเวลเวท, ม่วงราชวงศ์, น้ำเงินแซฟไฟร์, ทองอำพัน, ส้มพระอาทิตย์, ชมพูโรสควอตซ์, นิลทอง และธีมเทศกาลฮาโลวีน แต่ละธีมมีชุดสีสว่างและมืด (COLOR_THEMES ใน theme.ts บรรทัด 93–204) เลือกแล้วจำไว้ในเครื่อง

ฟังก์ชัน buildTheme สร้างชุดสีเต็ม (ประมาณ 30 token) จากค่าพื้นฐานของแต่ละธีม และคำนวณสีบางตัวให้อ่านง่ายอัตโนมัติ เช่น สีตัวหนังสือบนปุ่มทอง (onAccent) เลือกระหว่างสีเข้มกับสีขาว ตัวที่อ่านออกบนพื้นทงนั้น

หน้าจอเรียกแค่ const theme = useTheme() ได้ชุดสีของธีมและโหมดที่ผู้ใช้เลือกตอนนี้ เปลี่ยนธีมแล้วทุกหน้าเปลี่ยนตามทันที

ความอ่านง่ายของสี (WCAG) และเทสต์ที่บังคับ

มาตรฐานสากล WCAG AA กำหนดว่าตัวหนังสือกับพื้นหลังต้องต่างกันอย่างน้อย 4.5 : 1 (1 = สีเดียวกัน, 21 = ดำบนขาว) คนสายตาไม่ดีหรือใช้มือถือกลางแดดจึงอ่านออก

คู่สี (ป่าทอง สว่าง)	อัตราส่วน	ผ่านไหม
ink uu bg	14.93	ผ่าน
inkSoft uu surface	6.85	ผ่าน
ตัวหนังสือขาว uu primary	5.30	ผ่าน
inkFaint uu surface	2.95	ไม่ผ่าน → จึงห้ามใช้กับข้อความ ใช้กับ placeholder และไอคอนตกแต่งเท่านั้น
ทอง accent เป็นตัวหนังสือบนพื้นขาว	2.61	ไม่ผ่าน → จึงมี token accentInk ที่เข้มกว่าไว้ใช้แทน

คำนวณด้วยสูตรเดียวกับฟังก์ชัน contrast() ในแอป

src/ui/theme.tsบรรทัด 261–279

```
261 function luminance(hex: string): number {
262   const n = parseInt(hex.replace('#', '').slice(0, 6), 16);
263   const ch = (v: number) => {
```

```

264     const c = v / 255;
265     return c <= 0.03928 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4;
266 };
267 return 0.2126 * ch((n >> 16) & 255) + 0.7152 * ch((n >> 8) & 255) + 0.0722 * ch(n & 255);
268 }
269
270 /** WCAG contrast ratio between two hex colours (1 to 21). */
271 export function contrast(a: string, b: string): number {
272     const [hi, lo] = [luminance(a), luminance(b)].sort((x, y) => y - x);
273     return (hi + 0.05) / (lo + 0.05);
274 }
275
276 /** The first colour that reads on `bg` at 4.5:1 or better (the last one otherwise). */
277 export function readableOn(bg: string, ...colors: string[]): string {
278     return colors.find((c) => contrast(c, bg) >= 4.5) ?? colors[colors.length - 1];
279 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:261

บรรทัด ความหมาย

- 261–268** luminance: ความสว่างที่ตาคนรับรู้ของสีหนึ่ง (0 = ดำ ถึง 1 = ขาว) แยกสีแดง เขียว น้ำเงิน แปลงค่า แล้วถ่วงน้ำหนัก เขียวหนักสุด (0.7152) เพราะตาคนไวต่อสีเขียวที่สุด
- 270–274** contrast: เอาสีที่สว่างกว่าเป็นตัวตั้ง (บวก 0.05) หาสีที่มืดกว่า (บวก 0.05) ตามสูตรของ WCAG
- 276–279** readableOn: ให้รายการสีมา ค้นสีแรกที่อ่านออกบนพื้นนั้น (≥ 4.5)

src/ui/__tests__/theme.test.ts

บรรทัด 34–58

```

34 for (const spec of COLOR_THEMES) {
35     for (const dark of [false, true]) {
36         it(`${spec.key} ${dark ? 'dark' : 'light'}: every text colour reads on the background it is
used on`, () => {
37             const t = buildTheme(spec.key, dark);
38             const pairs: [string, string, string][] = [
39                 ['inkSoft on bg', t.inkSoft, t.bg],
40                 ['inkSoft on surface', t.inkSoft, t.surface],
41                 ['onPrimary on primary', t.onPrimary, t.primary],
42                 ['onAccent on accent', t.onAccent, t.accent],
43                 ['accentInk on surface', t.accentInk, t.surface],
44                 ['accentInk on accentSoft', t.accentInk, t.accentSoft],
45                 ['good on goodSoft', t.good, t.goodSoft],
46                 ['watch on watchSoft', t.watch, t.watchSoft],
47                 ['critical on criticalSoft', t.critical, t.criticalSoft],
48                 ['good on bg', t.good, t.bg],
49                 ['watch on bg', t.watch, t.bg],
50                 ['critical on bg', t.critical, t.bg],
51                 ['heroInkSoft on hero', t.heroInkSoft, t.hero],
52                 ['heroInk on heroTop', t.heroInk, t.heroTop],
53             ];
54             const low = pairs.filter(([a, b]) => contrast(a, b) < 4.5).map(([n, a, b]) => `${n}
${contrast(a, b).toFixed(2)}`);
55             expect(low).toEqual([]);
56         });
57     }
58 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/__tests__/theme.test.ts:34

เทสต์ TC-78 ตรวจสอบทุกธีม (9) × ทุกโหมด (สว่าง/มืด) = 18 ชุด แต่ละชุดตรวจ 14 คู่มือ (บรรทัด 38–53) ถ้าคู่มือต่ำกว่า 4.5 เทสต์ล้ม และบอกชื่อคู่มือพร้อมค่าที่ได้ (บรรทัด 54) ใครเพิ่มธีมใหม่แล้วเลือกสีอ่านยาก npm test จะไม่ผ่าน และ GitHub จะไม่ build แอปให้

ตัวอักษร

src/ui/theme.ts

บรรทัด 301–328

```
301 /**
302  * Two typefaces:
303  * - Noto Serif Thai for money and headings: the "international banking" feel
304  *   from the logo direction, and clear numerals.
305  * - Anuphan for everything else: a modern Thai sans that reads well small.
306  */
307 export const fonts = {
308   serif: 'NotoSerifThai_600SemiBold',
309   serifBold: 'NotoSerifThai_700Bold',
310   sans: 'Anuphan_400Regular',
311   sansMedium: 'Anuphan_500Medium',
312   sansSemi: 'Anuphan_600SemiBold',
313   sansBold: 'Anuphan_700Bold',
314 };
315
316 export const space = { xs: 4, sm: 8, md: 12, lg: 16, xl: 20, xxl: 28, xxxl: 40 };
317 export const radius = { sm: 8, md: 14, lg: 20, xl: 28, pill: 999 };
318
319 /** Type scale (size / line height). Thai needs generous line height for vowels above and below.
320  */
321 export const type = {
322   display: { fontSize: 38, lineHeight: 52 },
323   h1: { fontSize: 26, lineHeight: 38 },
324   h2: { fontSize: 20, lineHeight: 30 },
325   h3: { fontSize: 17, lineHeight: 26 },
326   body: { fontSize: 15, lineHeight: 24 },
327   small: { fontSize: 13, lineHeight: 20 },
328   micro: { fontSize: 11, lineHeight: 16 },
329 };
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:301

- **Noto Serif Thai** (มีหัว) ใช้กับตัวเลขเงินและหัวข้อใหญ่ ให้ความรู้สึก "ธนาคารสากล" และตัวเลขอ่านชัด
- **Anuphan** (ไม่มีหัว กันสมัย) ใช้กับข้อความอื่นทั้งหมด อ่านง่ายแม้ตัวเล็ก
- บรรทัด 316–317: ระยะห่างมาตรฐาน 4/8/12/16/20/28/40 และมุมโค้ง 8/14/20/28/999 (999 = ปลายมนเป็นเม็ดยา)
- บรรทัด 319–328: ขนาดตัวอักษรคู่กับความสูงบรรทัดเสมอ ภาษาไทยมีสระบนล่างและวรรณยุกต์ ต้องมีความสูงบรรทัดประมาณ 1.4–1.6 เท่าของขนาดตัวอักษร ไม่ขึ้นสระจะชนบรรทัดข้างบน

ชิ้นส่วนสำเร็จรูป: ปุ่ม (Button)

src/ui/components.tsx

บรรทัด 261–338

```
261 export function Button({
262   label,
```

```

263   onPress,
264   kind = 'primary',
265   icon,
266   loading,
267   disabled,
268   style,
269   small,
270   shine,
271 }): {
272   label: string;
273   onPress: () => void;
274   /** onDark: outline button on the dark green hero surfaces */
275   kind?: 'primary' | 'gold' | 'ghost' | 'danger' | 'soft' | 'onDark';
276   icon?: IconName;
277   loading?: boolean;
278   disabled?: boolean;
279   style?: StyleProp<ViewStyle>;
280   small?: boolean;
281   /** A band of light sweeps across (for the one main action on a screen). */
282   shine?: boolean;
283 }) {
284   const theme = useTheme();
285   const press = usePressSpring(0.95);
286   const colors = {
287     primary: { bg: theme.primary, fg: theme.onPrimary, border: theme.primary },
288     gold: { bg: theme.accent, fg: theme.onAccent, border: theme.accent },
289     ghost: { bg: 'transparent', fg: theme.ink, border: theme.line },
290     danger: { bg: 'transparent', fg: theme.critical, border: theme.critical },
291     soft: { bg: theme.surfaceAlt, fg: theme.ink, border: theme.surfaceAlt },
292     onDark: { bg: 'rgba(244,241,230,0.10)', fg: '#F4F1E6', border: 'rgba(244,241,230,0.45)' },
293   }[kind];
294   const off = disabled || loading;
295   return (
296     <AnimatedPressable
297       onPress={() => {
298         Haptics.selectionAsync().catch(() => {});
299         onPress();
300       }}
301       onPressIn={off ? undefined : press.onPressIn}
302       onPressOut={press.onPressOut}
303       disabled={off}
304       // Small buttons are about 40pt tall: the touch area reaches 44pt and more (up and down
305       only,
306       // so buttons side by side never share a touch area).
307       hitSlop={small ? { top: 4, bottom: 4 } : undefined}
308       accessibilityRole="button"
309       aria-disabled={!off}
310       aria-busy={!loading}
311       style={[
312         {
313           backgroundColor: colors.bg,
314           borderColor: colors.border,
315           borderWidth: 1.5,
316           borderRadius: radius.pill,
317           paddingVertical: small ? 8 : 12,
318           paddingHorizontal: small ? 14 : 20,
319           flexDirection: 'row',
320           alignItems: 'center',
321           justifyContent: 'center',
322           gap: 8,
323           opacity: off ? 0.5 : 1,

```

```

323         overflow: shine ? 'hidden' : undefined,
324     },
325     style,
326     { transform: [{ scale: press.scale }] },
327   ]}
328   >
329   {shine && !off ? <Shine times={1} delay={900} color={kind === 'gold' ?
'rgba(255,255,255,0.45)' : 'rgba(255,255,255,0.28)'} /> : null}
330   {loading ? (
331     <ActivityIndicator color={colors.fg} />
332   ) : icon ? (
333     <Ionicons name={icon} size={small ? 16 : 19} color={colors.fg} />
334   ) : null}
335     <Text style={[small ? type.small : type.body, { fontFamily: fonts.sansSemi, color:
colors.fg }]}>{label}</Text>
336   </AnimatedPressable>
337 );
338 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/components.tsx:261`

บรรทัด	ความหมาย
261–283	props ของปุ่ม: ข้อความ, สิ่งที่ทำเมื่อกด, ชนิด 6 แบบ (ค่าเริ่มต้น primary), ไอคอน, คำสั่งโหลด, ปิดใช้งาน, ขนาดเล็ก, แสงวิ่งผ่าน
284–285	สีของธีม และแอนิเมชันยุบตัวตอนกด (เหลือ 95%)
286–293	ตารางสีของปุ่มแต่ละชนิด: primary = พื้นสีขาวตัวขาว, gold = พื้นทอง, ghost = โปร่งมีขอบ, danger = ขอบแดง (ปุ่มลบ), soft = พื้นเทาอ่อน, onDark = โปร่งบนการ์ดเขียว ทุกสีมาจาก theme
294	ปุ่ม "ปิด" ถ้าถูกปิดใช้งานหรือคำสั่งโหลด (กันกดซ้ำ)
297–300	กดแล้วมือถือสั่นเบา ๆ (haptic) แล้วทำสิ่งที่ผู้เรียกกำหนด
304–306	ปุ่มเล็กสูงประมาณ 40 จุด ขยายพื้นที่แตะขึ้นลงอีก 4 จุด ให้ถึง 44 จุดตามแนวทางของ Apple/Google (นิ้วคนกว้างกว่าที่คิด) ขยายเฉพาะแนวตั้ง ปุ่มที่วางติดกันจึงไม่แย่งพื้นที่แตะ
307–309	บอกโปรแกรมอ่านหน้าจอว่าเป็นปุ่ม และสถานะปิด/คำสั่งโหลด
310–327	รูปร่าง: ขอบ 1.5, มุมโค้งแบบเบียด, ระยะในตามขนาด, ไอคอนกับข้อความเรียงแนวนอนตรงกลาง, จางลง 50% เมื่อปิดใช้งาน
329	แสงวิ่งผ่าน 1 ครั้ง (ใช้กับปุ่มหลักปุ่มเดียวของหน้า)
330–334	คำสั่งโหลดแสดงวงหมุนแทนไอคอน
335	ข้อความบนปุ่ม ขนาดตามปุ่ม ฟอนต์หนาถึงหนา

ชิ้นส่วน	ใช้เมื่อ
Screen	ทุกหน้า: พื้นหลัง ขอบ เลื่อนช่องที่คำสั่งพิมพ์ให้พื้นที่บอร์ด
Card	กลุ่มเนื้อหา มี 3 โท่น: ปกติ, พิเศษ (alt), ท่องอ่อน (accent)
Button / IconButton	การกระทำ 1 หน้ามีปุ่ม primary แค่ 1 ปุ่ม / ปุ่มไอคอนต้องมี label เสมอ

ชิ้นส่วน	ใช้เมื่อ
Segmented	สลับมุมมอง 2–4 แบบ เช่น วันนี้ / 7 วัน / 1 เดือน, รายจ่าย / รายรับ
Chip	ตัวเลือกหลายอัน เช่น หมวดหมู่, จำนวนเงินลด
Field / AmountField	ช่องกรอก ขอบเปลี่ยนสีตอนพิมพ์และตอนผิด บนเว็บตัวหนังสือ 16 เพื่อไม่ให้ iPhone ชุมหน้า
Badge	ป้ายสถานะสั้น ๆ เช่น "สบาย ๆ"
useToast	ยืนยันผล เช่น "บันทึกรายการแล้ว" มีปุ่ม "เลิกทำ" ได้
ConfirmSheet	ถามยืนยันก่อนทำสิ่งที่ย้อนไม่ได้ เช่น ลบ
EmptyState	ยังไม่มีข้อมูล: น้องกล้า + คำอธิบาย + ปุ่มให้เริ่ม

Accessibility: ทุกคนใช้ได้

เรื่อง	ทำอย่างไร
คนตาบอด/สายตาเลือนราง	ปุ่มทุกปุ่มมี accessibilityLabel ให้ TalkBack (Android) / VoiceOver (iPhone) อ่าน เช่นการ์ดยอดเงินอ่านว่า "ยอดคงเหลือ ฿11,359.00 เงินพอใช้อีก 16 วัน"
แยกสีไม่ออก (ตาบอดสี)	สถานะไม่ใช่สีอย่างเดียว มีข้อความกำกับเสมอ เช่น "ใกล้เส้นสำรอง"
เมารถจากอาการเคลื่อนไหว	เปิด "ลดการเคลื่อนไหว" (Reduce motion) ในมือถือ ทุกแอนิเมชันหยุดและแสดงผลทันที (useReduceMotion())
นิ้วใหญ่ มือสั่น	พื้นที่แตะอย่างน้อย 44 จุด
อ่านยาก	สีผ่าน WCAG AA ทุกธีม (TC-78)

แอนิเมชัน: สวยแต่ไม่กินแบตเตอรี่

แอปมีเอฟเฟกต์เยอะ (ส่วนต่าง ๆ ค่อย ๆ โผล่, ปุ่มแดง, ตันไม้เงินโยก, พลุตอนสำเร็จ) แต่มีกฎ 4 ข้อจาก README:

1. ขยับเฉพาะ transform (ย้าย หมุน ย่อขยาย) กับ opacity (ความจาง) และใช้ **native driver** ให้การขยับทำงานส่วน native ของมือถือ ไม่ผ่าน JavaScript จึงสิ้นเปลืองแบตเตอรี่ราคาประหยัด
2. ทุกเอฟเฟกต์หยุดเอง ไม่มีอะไรยับยั้งค้างตอนผู้ใช้ไม่ได้ทำอะไร (มี E2E ตรวจสอบว่าหน้าหลักนิ่งภายใน 9 วินาที)
3. เคารพ "ลดการเคลื่อนไหว"
4. ไม่ใช้ native module ใหม่ แอปที่ลงแล้วจึงได้เอฟเฟกต์ใหม่ทาง OTA โดยไม่ต้องลง APK ใหม่

เช็กลิสต์ก่อนเพิ่มหน้าจอใหม่ (จาก DESIGN_SYSTEM.md)

1. ไม่มีรหัสสี #... หรือ rgba (...) ในหน้าจอ ใช้ token หรือ alpha()
2. ตัวหนังสือใช้ <T> และเงินใช้ <Money>
3. ปุ่ม ชิป ช่องกรอก การ์ด มาจาก component ไม่สร้างใหม่เอง
4. ลองเปิดหน้านั้น 2–3 ธีม ทั้งสว่างและมืด

5. รัน `npm test, npx tsc --noEmit, npx expo lint, npm run e2e:web`

i ทบทวนศึกษา UX จากที่ไหน

บันทึกใน [docs/AI_USAGE_LOG.md](#) แถว 10 และ 19: ศึกษาแอปเหมียวจวด (ผู้ช่วยที่พูดกับผู้ใช้, "แค่เปิดแอปก็จดให้") และแนวการออกแบบของ Hugging Face (มาสคอตตัวเดียวหลายอารมณ์, สีอุ้งบนพื้นเรียบ, การ์ดโค้ง เส้นบาง) และหลัก Laws of UX เช่น aesthetic-usability effect (ของสวยทำให้รู้สึกว่ายาง่าย) และ peak-end rule (คนจำช่วงพีคและตอนจบ จึงมีพลุดอนทำสำเร็จ) แล้วออกแบบตัวละครน้องกล้ายของ MindPay เอง ไม่ลอกของใคร

การทดสอบ: พิสูจน์ว่าระบบทำงานถูก

ทีมเราทำการทดสอบ 3 ระดับ: unit test 157 ข้อ, E2E ในเบราว์เซอร์ 145 ข้อ และแผนทดสอบบนมือถือจริง 12 ข้อ บทนี้สอนอ่านและเขียนเทส

ทำไมต้องเขียนเทส

เวลาแก้ไขจุดหนึ่ง อาจทำให้จุดอื่นพังโดยไม่รู้ตัว (เรียกว่า **regression**) ถ้าต้องกดทดสอบทุกหน้าเองทุกครั้งก็แย่ จะใช้เวลาหลายชั่วโมงและลืมบางจุดแน่นอน เทสอัตโนมัติคือ "โปรแกรมที่ตรวจโปรแกรม" รันเสร็จในไม่กี่วินาที และทีมตั้งไว้ว่าถ้าเทสไม่ผ่าน GitHub จะไม่ปล่อยเว็บและ APK ให้ผู้ใช้

157

Unit test (Vitest)

ตรวจฟังก์ชันทีละตัว · รันไม่ถึง 2 วินาที

145

E2E (Playwright)

เปิดเว็บจริงแล้วกดเหมือนคน · ประมาณ 3 นาที

12

Manual test (MT-01-12)

คนทดสอบบนมือถือจริง

Unit test: ตรวจฟังก์ชันทีละตัว

ไฟล์เทสอยู่ในโฟลเดอร์ `__tests__` ชื่อลงท้าย `.test.ts` รันด้วยคำสั่ง `npm test` ใช้เครื่องมือ **Vitest** ตัวอย่างจากเทสของ FR-6:

src/domain/__tests__/domain.test.ts

บรรทัด 35-58

```
35 // Sunday 27 Sep 2026, 14:00 in Bangkok
36 const NOW = new Date('2026-09-27T07:00:00.000Z');
37
38 let seq = 0;
39 function tx(p: Partial<Transaction> & { day: string; amount: number }): Transaction {
40   const { day, amount, ...rest } = p;
41   return {
42     id: `t${++seq}`,
43     kind: 'expense',
44     amountSatang: amount * 100,
45     categoryKey: 'food',
46     title: 'test',
47     note: null,
48     occurredAt: bkkToIso(day, '12:00'),
49     source: 'manual',
50     status: 'confirmed',
51     slipRef: null,
52     slipImageHash: null,
53     ocrConfidence: null,
54     reviewFlags: [],
55     createdAt: NOW.toISOString(),
56     ...rest,
57   };
58 }
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:35

บรรทัด	ความหมาย
35–36	ล็อก "วันนี้" ให้ตายตัว เป็นวันอาทิตย์ที่ 27 ก.ย. 2569 เวลา 14:00 น. ไม่ว่ารันเทสต์วันไหน ผลก็เหมือนเดิม (ถ้าใช้วันจริง เทสต์จะผ่านบ้างไม่ผ่านบ้างตามวันที่)
38–58	ฟังก์ชันช่วยสร้างรายการปลอม tx({ day, amount }) ใส่แค่วันกับจำนวนบาท ที่เหลือใช้ค่าตั้งต้น (รายจ่าย หมวดอาหาร ยืนยันแล้ว) ...rest ท้ายสุดให้เปลี่ยนช่องอื่นได้ เช่น tx({ day, amount, status: 'draft' })

src/domain/__tests__/domain.test.ts

บรรทัด 184–210

```

184 it('TC-14 computes days and depletion date', () => {
185     const r = computeRunway({ balanceSatang: 350_000, floorSatang: 50_000, averageSatang: 20_000,
now: NOW });
186     expect(r.days).toBe(15);
187     expect(r.depletionDay).toBe('2026-10-12');
188     expect(r.status).toBe('healthy');
189 });
190
191 it('TC-15 handles zero average without dividing by zero', () => {
192     const r = computeRunway({ balanceSatang: 350_000, floorSatang: 50_000, averageSatang: 0, now:
NOW });
193     expect(r.status).toBe('no_spending');
194     expect(r.days).toBeNull();
195     expect(r.depletionDay).toBeNull();
196 });
197
198 it('TC-16 flags balance at or below the low line', () => {
199     const r = computeRunway({ balanceSatang: 50_000, floorSatang: 50_000, averageSatang: 10_000,
now: NOW });
200     expect(r.status).toBe('below_floor');
201     expect(r.days).toBe(0);
202 });
203
204 it('TC-17 status thresholds', () => {
205     const at = (days: number) =>
206         computeRunway({ balanceSatang: 50_000 + days * 10_000, floorSatang: 50_000, averageSatang:
10_000, now: NOW }).status;
207     expect(at(6)).toBe('critical');
208     expect(at(7)).toBe('watch');
209     expect(at(14)).toBe('healthy');
210 });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:184

บรรทัด	ความหมาย
184	it('ชื่อเทสต์', () => {...}) = เทสต์ 1 ข้อ ชื่อขึ้นต้นด้วยรหัส TC ที่ตรงกับเอกสาร Traceability
185	Arrange + Act: เตรียมข้อมูล (ยอด ฿3,500, เส้นสำรอง ฿500, ใช้วันละ ฿200) แล้วเรียกฟังก์ชันจริง
186–188	Assert: expect(ค่าจริง).toBe(ค่าที่ต้องได้) ตรวจสอบว่าได้ 15 วัน, วันที่ 12 ต.ค. 2569 และสถานะ healthy ถ้าไม่ตรงข้อใดข้อหนึ่ง เทสต์ล้มพร้อมบอกว่าได้อะไรแทน
191–196	TC-15 ทดสอบ กรณีพิเศษ: ค่าเฉลี่ยเป็น 0 ต้องไม่หารด้วยศูนย์ ต้องได้ no_spending และวันเป็น null
198–202	TC-16 ทดสอบ ขอบ: ยอดเท่ากับเส้นสำรองพอดี ต้องนับว่า "ต่ำกว่าเงินสำรอง"

บรรทัด	ความหมาย
204–210	TC-17 ทดสอบ ขอบของเกณฑ์ : 6 วัน critical, 7 วัน watch, 14 วัน healthy ถ้ามีคนเผลอแก้ < เป็น <= เทสต์นี้จะจับได้ทันที

★ เขียนเทสต์ที่ดี: ทดสอบขอบ ไม่ใช่แค่ค่ากลาง ๆ

บ๊ีกชอบซ่อนอยู่ตรงขอบ เช่น 0, ค่าเท่ากันพอดี, ค่าว่าง, วันแรกของเดือน, เกียงคืน, ข้อมูลภาษาไทยแปลก ๆ เทสต์ของทิมจึงมีทั้งกรณีปกติ กรณีพิเศษ และกรณีขอบ อาจารย์มักถามว่า "ทดสอบกรณีไหนบ้าง" ให้ตอบเป็น 3 กลุ่มนี้

ไฟล์ unit test หลัก (มีทั้งหมด 18 ไฟล์)

ไฟล์	ทดสอบ
domain.test.ts	เงิน, วันที่ไทย, ยอดคงเหลือ (FR-2), Runway (FR-6), สลิป (FR-4), คำแนะนำโค้ช (FR-5), หมวด
voice.test.ts	แยกประโยคพูดเป็นรายการ ตัวเลขคำไทย (FR-3)
slipAccuracy.test.ts, autoScan.test.ts	ความแม่นยำการอ่านสลิป และสแกนอัตโนมัติ (FR-4)
recurring.test.ts, recap.test.ts	รายการประจำเดือน (FR-1) และสรุปเดือนปฏิทิน (FR-2)
goals.test.ts	กระปุกออม (FR-6)
buddy.test.ts, klaTalk.test.ts	คำพูดน้องกล้า และการตัดซับไตเติลอ่านออกเสียง (FR-5)
auth.test.ts	ลิงก์อีเมลและข้อความ error ภาษาไทย
audit.test.ts	บ๊ีกที่เจอตอนตรวจทั้ง repo (1–3 ต.ค.) กันไม่ให้กลับมาอีก
ui/__tests__/theme.test.ts	สีทุกธีมอ่านง่าย (TC-78)
supabase/functions/_shared/__tests__/helpers.test.ts	ตัวช่วยของเซิร์ฟเวอร์: เลือก AI, ทำความสะอาดผลอ่านสลิป, รวมผลอ่าน 2 ครั้ง

E2E: ทดสอบทั้งแอปเหมือนคนใช้จริง

End-to-End (E2E) test เปิดเว็บเวอร์ชันจริงในเบราว์เซอร์อัตโนมัติ (เครื่องมือ **Playwright**) แล้วกด พิมพ์ เลื่อน ตามขั้นตอนเหมือนคน และตรวจว่าเห็นสิ่งที่ควรเห็น ไฟล์หลักคือ [e2e/web-auth.mjs](#) (ยาวกว่า 1,000 บรรทัด)

≈ ไม่แตะบัญชีจริง: เซิร์ฟเวอร์ปลอม

ถ้า E2E ใช้ Supabase จริง จะสร้างบัญชีขยะ ส่งอีเมลจริง และเปลืองโควตา AI ทีมเราจึงเขียน

`e2e/fake-backend.mjs` เป็น "Supabase ปลอม" ที่ตอบเหมือนของจริงแต่อยู่ในหน่วยความจำ ทุกคำขอที่ไม่ได้อยู่ของ Supabase จะถูกดักมาตอบด้วยตัวปลอมนี้ (`page.route`) เหมือนซ้อมละครกับนักแสดงแทน

`e2e/web-auth.mjs`

บรรทัด 29–60

```
29 // ----- helpers
30 const results = [];
31 function check(name, ok, detail = '') {
32   results.push({ name, ok, detail });
33   console.log(`${ok ? 'PASS' : 'FAIL'} ${name}${detail ? ` - ${detail}` : ''}`);
34   // In GitHub Actions a failed check also becomes an annotation on the run.
35   if (!ok && IN_CI) console.log(`::error title=${ENGINE} browser test::${name}${detail ? `
36   (${detail.replace(/\n/g, ' ').slice(0, 300)}):` : ''}`);
37 }
38 const browser = await (ENGINE === 'webkit' ? webkit : chromium).launch();
39 const phone = {
40   ...(ENGINE === 'webkit' ? { isMobile: true, hasTouch: true } : {}),
41   // E2E_SCHEME=dark runs everything with the phone in dark mode.
42   colorScheme: process.env.E2E_SCHEME === 'dark' ? 'dark' : 'light',
43 };
44 async function freshPage(tag) {
45   const ctx = await browser.newContext({ viewport: { width: 390, height: 844 },
46     deviceScaleFactor: 2, locale: 'th-TH', ...phone });
47   const page = await ctx.newPage();
48   page.on('pageerror', (e) => console.log(`[${tag}] pageerror`, e.message));
49   page.on('console', (m) => { if (m.type() === 'error') console.log(`[${tag}] console.error`,
50     m.text().slice(0, 200)); });
51   page.on('requestfailed', (r) => { if (!r.url().startsWith(SUPA)) console.log(`[${tag}] request
52     failed`, r.url().slice(0, 160)); });
53   await page.route(`${SUPA}/**`, supabase);
54   // The web QR reader downloads its decoder from a CDN: serve the same file from node_modules.
55   await routeQrDecoder(page);
56   return { ctx, page };
57 }
58 const shot = (page, name) => page.screenshot({ path: join(SHOTS, `${name}.png`) });
59 const visible = (page, text, timeout = 6000) => page.getByText(text, { exact: false
60   }).first().waitFor({ state: 'visible', timeout }).then(() => true, () => false);
61 const gone = (page, text, timeout = 6000) => page.getByText(text, { exact: false
62   }).first().waitFor({ state: 'hidden', timeout }).then(() => true, () => false);
63 const introGone = (page) => page.getByLabel('กำลังเปิด MindPay').waitFor({ state: 'detached',
64   timeout: 8000 }).catch(() => {});
65 const button = (page, name) => page.getByRole('button', { name, exact: false }).first();
66 /** Waits (up to `timeout`) until the element is visible. */
67 const shown = (locator, timeout = 6000) => locator.waitFor({ state: 'visible', timeout }).then(()
68   => true, () => false);
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `e2e/web-auth.mjs:29`

บรรทัด ความหมาย

30–36 `check` (ชื่อ, ผ่านไหม, รายละเอียด) บันทึกผลแต่ละข้อ พิมพ์ PASS หรือ FAIL ถ้ารันบน GitHub และไม่ผ่าน จะขึ้นเป็นป้าย error บนหน้า Actions

บรรทัด	ความหมาย
37–42	เปิดเบราว์เซอร์ Chromium (หรือ WebKit แบบ Safari) โหมดสว่างหรือมืด
43–53	freshPage: เปิดแท็บใหม่ขนาดมือถือ (390×844) ภาษาไทย ดักคำขอไป Supabase ให้ตัวปลอมตอบ และพิมพ์ error ของหน้าเว็บถ้ามี
54	shot: ถ่ายภาพหน้าจอเก็บไว้ (ภาพหน้าจอหลายภาพในเล่มนี้มาจากตรงนี้)
55–56	visible / gone: รอจนเห็นข้อความ (หรือข้อความหายไป) ภายใน 6 วินาที
58	button: หาปุ่มจากชื่อที่โปรแกรมอ่านหน้าจออ่าน (accessibilityLabel) แอปที่ทำ accessibility ดีจึงเขียนทดสอบง่าย

ตัวอย่าง: ทดสอบแถบ "ลองปรับ" ของ FR-6

e2e/web-auth.mjs

บรรทัด 572–627

```

572 // 14. Runway: "can I buy this?" shows the days before/after and hands the question to the
    coach
573 {
574   const { ctx, page } = await freshPage('runway');
575   await page.goto(APP);
576   await visible(page, 'ลองใช้ด้วยข้อมูลตัวอย่าง', 8000);
577   await introGone(page);
578   await page.getByText('ลองใช้ด้วยข้อมูลตัวอย่าง').click();
579   await visible(page, 'ยอดคงเหลือ', 8000);
580   await page.goto(`${APP}runway`);
581   await introGone(page);
582   // What-if slider: drag the gold thumb to spend 30% less a day.
583   const slider = page.getByRole('slider', { name: 'ใช้น้อยลงวันละกี่เปอร์เซ็นต์' }).first();
584   await slider.scrollIntoViewIfNeeded();
585   // A raw mouse drag does not wait like click() does: make sure the opening animation has
    fully gone.
586   const introWait = Date.now();
587   await page.getByLabel('กำลังเปิด MindPay').waitFor({ state: 'detached', timeout: 20000
    }).catch(() => {});
588   const introMs = Date.now() - introWait;
589   // Wait until the page stops moving (cards slide in), so the drag starts on the thumb.
590   let box = await slider.boundingBox();
591   for (let i = 0; i < 12; i++) {
592     await page.waitForTimeout(250);
593     const again = await slider.boundingBox();
594     const still = box && again && Math.abs(again.y - box.y) < 0.5 && Math.abs(again.x - box.x)
    < 0.5;
595     box = again;
596     if (still) break;
597   }
598   const under = box
599     ? await page.evaluate(({ x, y }) => {
600       const el = document.elementFromPoint(x, y);
601       return el ? `${el.tagName} role=${el.getAttribute('role')}`
        label=${el.getAttribute('aria-label')}` : 'nothing';
602     }, { x: box.x + 15, y: box.y + box.height / 2 })
603     : 'no box';
604   if (box) {
605     // hover() waits until the slider itself receives the pointer (nothing on top of it).
606     await slider.hover({ position: { x: 15, y: box.height / 2 } }).catch(() => {});
607     await page.mouse.down();

```

```

608     await page.mouse.move(box.x + 15 + (box.width - 30) * 0.3, box.y + box.height / 2, { steps:
8 });
609     await page.mouse.move(box.x + 15 + (box.width - 30) * 0.6, box.y + box.height / 2, { steps:
8 });
610     await page.mouse.up();
611 }
612 const slid = (await slider.getAttribute('aria-valuenow').catch(() => null)) ?? '';
613 await page.waitForTimeout(900);
614 await shot(page, '28-runway-slider');
615 check('Runway: dragging the slider to 30% less a day shows the extra days', slid === '30' &&
(await visible(page, 'เพิ่มขึ้น'), `value=${slid} box=${JSON.stringify(box)} under=${under}
introWaitMs=${introMs}`);
616 // The arrow keys move it too (keyboard users on the web).
617 await slider.focus();
618 await page.keyboard.press('ArrowRight');
619 const keyed = (await slider.getAttribute('aria-valuenow').catch(() => null)) ?? '';
620 check('Runway: the slider also moves with the arrow keys', keyed === String(Number(slid || 0)
+ 5), `value=${keyed}`);
621 await page.keyboard.press('Home');
622 await page.fill('#runway-price', '2000');
623 check('Runway: buying ฿2,000 shows the days before and after', (await visible(page, 'ถ้าซื้อ
฿2,000')) && (await visible(page, 'ตอนนี้')));
624 await button(page, 'ถามโค้ชเรื่องนี้').click();
625 const asked = await page.locator('#coach-input').inputValue().catch(() => '');
626 check('Runway: "ask the coach" opens the coach with the question ready',
asked.includes('฿2,000'), asked);
627 await ctx.close();

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → e2e/web-auth.mjs:572

บรรทัด	ความหมาย
574-579	เปิดแท็บใหม่ ไปหน้าต้อนรับ รอแอนิเมชันเปิดแอปหาย และ "ลองใช้ด้วยข้อมูลตัวอย่าง" แล้วรอเห็นคำว่า "ยอดคงเหลือ"
580	ไปที่หน้าเงินพอถึง
583	หาแถบเลื่อนจากชื่อ "ใช้น้อยลงวันละกี่เปอร์เซ็นต์"
589-597	รอนหน้าจอหยุดขยับ (การ์ดค่อย ๆ เลื่อนเข้ามา) ไม่เงินจะลากพลาด
604-611	จำลองเมาส์: กดที่ปุ่มกลมของแถบ ลากไป 30% แล้ว 60% ของความกว้าง แล้วปล่อย
612-615	อ่านค่าปัจจุบันของแถบ ถ่ายภาพ และตรวจว่าได้ 30 และมีคำว่า "เพิ่มขึ้น"
617-620	ทดสอบคีย์บอร์ด: กดลูกศรขวา ค่าต้องเพิ่ม 5
622-623	พิมพ์ราคา 2000 ในเช็ก่อนจ่าย ต้องเห็น "ถ้าซื้อ ฿2,000" และ "ตอนนี้"
624-626	กด "ถามโค้ชเรื่องนี้" ช่องพิมพ์ในหน้าโค้ชต้องมีคำถามที่มี "฿2,000" รออยู่ (เชื่อม FR-6 กับ FR-5)

ผลล่าสุด: **145/145 ขั้วผ่าน** ทั้งโหมดสว่างและมืด (รันซ้ำตอนทำเล่มนี้ วันที่ 9 ต.ค. 2569 ก็ได้ 145/145 และ unit test 157/157) และมี workflow รันชุดเดียวกันบน WebKit (เอนจินของ Safari) แบบ iPhone โหมดมืดด้วย

Manual test: ทดสอบบนมือถือจริง

บางอย่างทดสอบอัตโนมัติไม่ได้ เช่น ไมโครโฟนจริง แกลเลอรีจริง สลิปจริงจากธนาคาร ทีมจึงมีแผน MT-01 ถึง MT-12 ใน [docs/TRACEABILITY.md](#) ให้สมาชิกทดสอบบนมือถือแล้วกรอกผล

ID	ขั้นตอน	ผลที่คาดหวัง
MT-02	จดรายจ่าย ฿85 หมวดอาหาร	ยอดคงเหลือลด ฿85, หมวดอาหารเพิ่ม, Runway เปลี่ยน
MT-03	ลบรายการ แล้วกด "เลิกทำ"	รายการกลับมา ยอดเท่าเดิม
MT-08	ถามโค้ช "ชื่อของ 500 บาทวันนี้ได้ไหม"	ตอบไทย อ้าง Runway ก่อน/หลัง ไม่ตำหนิ
MT-10	บัญชี A ไม่เห็นรายการของบัญชี B	RLS กันไว้
MT-11	บันทึกสลิปจริงลงแกลเลอรี แล้วเปิดแอป	ภายในไม่กี่วินาทีขึ้น "เพิ่ม 1 รายการจากสลิปใหม่"
MT-12	พูด "ข้าว 50 บาท น้ำ 15"	เห็น 2 รายการ บันทึกแล้วยอดลด ฿65

! ตรวจสอบนำเสนอ

ในเอกสาร Traceability ช่อง "ผล" ของ MT ทุกข้อยังเป็น ☐ (ยังไม่ได้กรอก) และในบันทึกการใช้ AI ช่อง "Human verification" ยังเป็น ☐ หลายข้อ ทีมควรทดสอบจริงและกรอกผลก่อนส่งงาน เพราะอาจารย์อาจถามว่า "ทดสอบบนมือถือจริงหรือยัง"

Traceability: ผูก Requirement → โค้ด → เทสต์

วิชา Software Engineering เน้น "Golden Thread" คือทุก requirement ต้องโยงไปหาโค้ดที่ทำ และเทสต์ที่พิสูจน์ได้ ทีมเราเก็บไว้ใน [docs/TRACEABILITY.md](#) เป็นตาราง 3 คอลัมน์: Acceptance criteria | Code | Test เช่น

Acceptance criteria (FR-6)	Code	Test
ค่าเฉลี่ยนับวันที่ไม่ได้ใช้เงินด้วย	averageDailyExpense	TC-12
ยอด ≤ เส้นสำรอง → สถานะ below_floor	computeRunway	TC-16

แต่ละเล่มในภาค ข มีตารางแบบนี้ครบทุกข้อของ FR นั้น

ตรวจคุณภาพอื่น ๆ

คำสั่ง	ตรวจอะไร
npm run typecheck	TypeScript ตรวจชนิดข้อมูลทั้งโปรเจกต์ (0 error)
npx expo lint	ESLint ตรวจรูปแบบโค้ดและกฎของ React Hooks
Supabase Advisors	ตัวตรวจความปลอดภัยและประสิทธิภาพของฐานข้อมูล (แก้จนเหลือ 0 ปัญหาเรื่องตาราง)

คำสั่ง	ตรวจอะไร
ผู้ตรวจอิสระ	ตอนตรวจทั้ง repo (1–3 ต.ค.) ให้ผู้ตรวจที่ไม่ได้เห็นงานเทียบพฤติกรรมก่อน/หลังแก้ (เช่น ประโยคพูด 260 ประโยค ชื่อผู้รับ 120 ชื่อ) เจอ regression 10 ข้อ แก่ครบพร้อมทดสอบ

Git และ GitHub: เก็บโค้ด ทำงานเป็นทีม และ build อัตโนมัติ

ทุกการแก้ไขโค้ดถูกบันทึกเป็นประวัติ และทุกครั้งที่เราส่งโค้ดขึ้น GitHub จะมีหุ่นยนต์ 4 ตัวช่วยทดสอบ สร้างเว็บ สร้าง APK และส่งอัปเดตให้ผู้ใช้

Git: ระบบบันทึกประวัติ

≈ เปรียบเทียบ

Git เหมือนปุ่ม "Save game" ของเกม ทุกครั้งที่ทำสำเร็จหนึ่งด่าน (แก้ฟิเจอร์เสร็จ) เราเซฟไว้พร้อมโน้ตว่าทำอะไร ถ้าพลาดภายหลังก็โหลดเซฟเก่ากลับมาได้ และดูย้อนได้ว่าใครเปลี่ยนอะไรเมื่อไร

คำ	ความหมาย
repository (repo)	โฟลเดอร์โปรเจกต์ที่ Git ดูแล รวมประวัติทั้งหมด (ซ่อนอยู่ในโฟลเดอร์ .git)
commit	"จุดเซฟ" 1 ครั้ง มีรหัส (เช่น 646b485) ข้อความอธิบาย ผู้ทำ และเวลา
branch	"เส้นทางคู่ขนาน" ของโค้ด แยกไปทำฟิเจอร์โดยไม่กระทบเส้นหลัก เส้นหลักของทีมชื่อ main
remote / origin	สำเนาของ repo บนเซิร์ฟเวอร์ (GitHub)
push	ส่ง commit ในเครื่องขึ้น GitHub
pull	ดึง commit ใหม่จาก GitHub ลงเครื่อง
clone	คัดลอก repo ทั้งหมดจาก GitHub ลงเครื่องครั้งแรก
pull request (PR)	คำขอรวม branch หนึ่งเข้า main ให้เพื่อนรีวิวก่อน
merge conflict	สองคนแก้บรรทัดเดียวกัน Git ไม่รู้จะเลือกของใคร ต้องตัดสินใจเอง
.gitignore	รายชื่อไฟล์ที่ไม่ต้องเก็บใน Git เช่น node_modules (ใหญ่และติดตั้งใหม่ได้), .env (อาจมีความลับ), dist (สร้างใหม่ได้)

ใช้ Git ประจำวัน

Terminal

วงจรทำงาน 1 รอบ

```

1 git pull                # 1. ดึงของล่าสุดก่อนเริ่ม
2 git checkout -b fix-runway-text  # 2. แยก branch ใหม่ (ไม่บังคับ)
3 # ... แก้โค้ดใน VS Code ...
4 npm test                # 3. รันเทสต์ให้ผ่านก่อน
5 git status              # 4. ดูว่าแค่ไฟล์ไหนไปบ้าง
6 git add src/domain/runway.ts  # 5. เลือกไฟล์ที่จะเซฟ
  
```

```
7 git commit -m "Runway: say 'ประมาณ' in the date" # 6. เซฟพร้อมข้อความ
8 git push -u origin fix-runway-text # 7. ส่งขึ้น GitHub แล้วเปิด Pull Request
```

ใน VS Code ทำแบบเดียวกันได้ด้วยเมาส์ที่เก็บ **Source Control** (Ctrl+Shift+G): ไฟล์ที่แก้ไขขึ้นในรายการ กด + เพื่อเลือก
พิมพ์ข้อความ กด Commit แล้วกด Sync Changes

★ ข้อความ commit ที่ดี

บอกว่า "ทำอะไร และทำไม" สั้น ๆ เช่นของทีมเรา: *"Runway: arrow keys on the web; steadier drag test"*, *"Audit follow-up: fix regressions found by independent review"* อ่านประวัติแล้วรู้เรื่องโดยไม่ต้องเปิดโค้ด ดูประวัติทั้งหมดด้วย `git log --oneline`

ประวัติของ MindPay บน GitHub

repo อยู่ที่ github.com/2550expo-tech/Socrates-and-Skeletons - (สาธารณะ ใครก็อ่านได้) branch main มี **66 commit** ตั้งแต่ 27 ก.ย. ถึง 3 ต.ค. 2569

วันที่	commit	งานหลักของวันนั้น (จากข้อความ commit)
27 ก.ย.	20	MVP แรก FR-1, FR-2, FR-4, FR-5, FR-6 · build APK อัตโนมัติ · เว็บบน GitHub Pages · EAS Update · AI ใช้ Gemini หรือ Claude · อ่านสลิปใหม่อัตโนมัติตอนเปิดแอป
28 ก.ย.	22	ระบบสมัครแบบรหัสอีเมล, นื่องกล้า, สแกนเองแยกตามช่วงเวลา, แก้ปัญหาคุกกี้ AI, อ่าน QR มุมขวาล่าง, เทสต์ในเบราว์เซอร์ (รวม Safari)
29 ก.ย.	19	พุดจต (FR-3), นื่องกล้าพุดได้, ธีมสี 8 แบบ, เพิ่มเงินเข้า, รายการประจำ, กระปุกออม
30 ก.ย.	2	ธีมฮาโลวีน, เทสต์ความอ่านง่ายของสี
1-3 ต.ค.	3	ตรวจทั้ง repository แก่บ๊ิก จัด Design System ให้ตรงกัน

GitHub Actions: หุ่นยนต์ที่ทำงานทุกครั้ง push

CI/CD (Continuous Integration / Continuous Delivery) คือให้คอมพิวเตอร์ทดสอบและส่งของให้อัตโนมัติทุกครั้งโค้ดเปลี่ยน ของทีมเราเขียนเป็นไฟล์ YAML ใน `.github/workflows/` มี 4 ตัว

ไฟล์	ทำอะไร	ผลลัพธ์
<code>web.yml</code>	build เว็บ → unit test → E2E 145 ข้อ → ถ้าผ่านทั้งหมดจึงเผยแพร่	เว็บที่ 2550expo-tech.github.io/Socrates-and-Skeletons/
<code>android-apk.yml</code>	unit test → typecheck → สร้างโปรเจกต์ Android → build APK (15-25 นาที)	ไฟล์ <code>mind-pay.apk</code> ในหน้า Releases "latest-apk"
<code>eas-update.yml</code>	unit test → ส่งโค้ด JavaScript ใหม่ขึ้น EAS Update	แอปที่ลงแล้วได้โค้ดใหม่ตอนเปิดครั้งถัดไป

ไฟล์	ทำอะไร	ผลลัพธ์
e2e-webkit.yml	E2E ชัดเดียวกันบนบนแอนดรอยด์ Safari แบบ iPhone โหมดมืด	รายงานผล (ไม่บล็อกการเผยแพร่)

git push ขึ้น main

↓ GitHub เปิดเครื่องเสมือน Ubuntu ให้ฟรีตัวละ 1 เครื่อง ทำพร้อมกัน

web.yml

เทสไม่ผ่าน = เว็บไม่เปลี่ยน ผู้ใช้ยังได้เวอร์ชันเดิมที่ต่ออยู่

android-apk.yml

เทสหรือ typecheck ไม่ผ่าน = ไม่มี APK ใหม่

eas-update.yml

ต้องมี secret EXPO_TOKEN และเทสผ่าน

e2e-webkit.yml

ตรวจสอบ Safari เพิ่ม

อ่าน web.yml ที่ละบรรทัด

.github/workflows/web.yml

บรรทัด 1-76

```

1 # Publishes the web version of MindPay to GitHub Pages on every push to main:
2 # https://2550expo-tech.github.io/Socrates-and-Skeletons-/
3 # The web version shows the app at phone width, works with real accounts and
4 # demo data, and reads slips from photos the user uploads. Scanning the whole
5 # gallery is only in the phone app. On iPhone: open in Safari -> Share ->
6 # Add to Home Screen, and it opens full screen with the MindPay icon.
7 name: Deploy web app
8
9 on:
10   push:
11     branches: [main]
12   workflow_dispatch:
13
14 permissions:
15   contents: write
16
17 concurrency:
18   group: web-pages
19   cancel-in-progress: true
20
21 jobs:
22   web:
23     runs-on: ubuntu-latest
24     timeout-minutes: 20
25     steps:
26       - uses: actions/checkout@v4
27
28       - uses: actions/setup-node@v4
29         with:
30           node-version: 22
31           cache: npm
32
33       - run: npm ci
34
35       - name: App settings (public Supabase URL and publishable key)
36         run: cp .env.example .env

```

```

37
38   - name: Export web build
39     env:
40       EXPO_WEB_BASE_URL: /Socrates-and-Skeletons-
41     run: npx expo export --platform web --output-dir dist
42
43   - name: Home-screen icon and full-screen mode (iPhone "Add to Home Screen")
44     env:
45       EXPO_WEB_BASE_URL: /Socrates-and-Skeletons-
46     run: node scripts/web-home-screen.mjs dist
47
48   - name: Single-page app fallback for deep links
49     run: cp dist/index.html dist/404.html
50
51   - name: Unit tests
52     run: npm test
53
54   # Browser test of every sign-in path (sign up, email code, links, reset
55   # password, sign out) against a local stand-in for Supabase. If anything
56   # fails, the website is NOT published and the screenshots are attached.
57   - name: Install the test browser
58     run: npx playwright install --with-deps chromium
59
60   - name: Browser tests (sign-in flows)
61     run: node e2e/web-auth.mjs dist e2e-shots
62
63   - name: Keep screenshots when the browser tests fail
64     if: failure()
65     uses: actions/upload-artifact@v4
66     with:
67       name: e2e-screenshots
68       path: e2e-shots
69
70   - name: Publish to the gh-pages branch
71     uses: peaceiris/actions-gh-pages@v4
72     with:
73       github_token: ${ secrets.GITHUB_TOKEN }
74       publish_dir: ./dist
75       force_orphan: true
76       commit_message: 'Deploy web app'

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `.github/workflows/web.yml:1`

บรรทัด	ความหมาย
1-6	คอมเมนต์ (YAML ใช้ #) บอกว่าเผยแพร่เว็บไปที่ไหน และวิธีเพิ่มลงหน้าจอโฮมบน iPhone
7	ชื่อที่แสดงในแท็บ Actions ของ GitHub
9-12	เมื่อไรจะทำงาน: ทุกครั้งที่ push ขึ้น branch main และกดสั่งเองได้ (workflow_dispatch)
14-15	สิทธิ์เขียนลง repo (ต้องใช้ตอนเผยแพร่ไป branch gh-pages)
17-19	ถ้า push ใหม่มาระหว่างที่ตัวเก่ายังทำงาน ยกเลิกตัวเก่า ทำแต่ตัวล่าสุด
21-24	งานชื่อ web รับบนเครื่อง Ubuntu ล่าสุด ไม่เกิน 20 นาที
26	ดีงโค้ดของ commit นี้ลงเครื่อง (uses = ใช้ action สำเร็จรูปที่คนอื่นเขียนไว้)

บรรทัด	ความหมาย
28–31	ติดตั้ง Node.js 22 และจำ cache ของ npm ให้ครั้งหน้าเร็วขึ้น
33	npm ci ติดตั้งแพ็คเกจตาม package-lock.json แบบเป๊ะ (ต่างจาก npm install ตรงที่ไม่เปลี่ยนเวอร์ชันเลย)
35–36	สร้าง .env จากตัวอย่าง (มีแค่กุญแจสาธารณะ)
38–41	build เว็บลงโพลเดอร์ dist พร้อมบอกว่าเว็บอยู่ใต้ /Socrates-and-Skeletons-
43–46	เพิ่มไอคอนและตั้งค่าให้ "เพิ่มไปยังหน้าจ่อโฮม" แล้วเปิดเต็มจอเหมือนแอป
48–49	คัดลอกหน้าแรกเป็น 404.html เพราะ GitHub Pages ไม่รู้จักที่อยู่อย่าง /runway ถ้ามีคนเปิดตรง ๆ จะได้หน้า 404 ซึ่งก็คือแอปเราที่พาไปหน้านั้นให้
51–52	รัน unit test 157 ข้อ ไม่ผ่าน = หยุดทั้งงาน
57–61	ติดตั้งเบราร์วเซอร์ทดสอบ แล้วรัน E2E 145 ข้อ
63–68	if: failure() = เฉพาะตอนพัง: เก็บภาพหน้าจอจาก E2E ไว้ให้ดาวนโหลดดูว่าพังตรงไหน
70–76	ผ่านทุกด่านแล้วจึงเผยแพร่โพลเดอร์ dist ขึ้น branch gh-pages ซึ่ง GitHub Pages ใช้เป็นเว็บไซต์

อ่าน android-apk.yml (ส่วนสำคัญ)

.github/workflows/android-apk.yml

บรรทัด 12–90

```

12 on:
13   push:
14     branches: [main]
15     # Editing documents only does not need a new APK.
16     paths-ignore: ['**/*.md', 'docs/**']
17   workflow_dispatch:
18
19 permissions:
20   contents: write
21
22 # A build that already started runs to the end; newer pushes wait in line
23 # (only the newest waiting push is kept).
24 concurrency:
25   group: android-apk
26   cancel-in-progress: false
27
28 jobs:
29   apk:
30     runs-on: ubuntu-latest
31     timeout-minutes: 45
32     steps:
33       - uses: actions/checkout@v4
34
35       - uses: actions/setup-node@v4
36         with:
37           node-version: 22
38           cache: npm
39
40       - uses: actions/setup-java@v4
41         with:
42           distribution: temurin

```

```

43     java-version: 17
44
45     - name: Install dependencies
46       run: npm ci
47
48     - name: Unit tests
49       run: npm test
50
51     - name: Typecheck
52       run: npx tsc --noEmit
53
54     - name: App settings (public Supabase URL and publishable key)
55       run: cp .env.example .env
56
57     - name: Generate the native Android project
58       run: npx expo prebuild --platform android --no-install
59
60     - name: Give Gradle enough memory
61       # The default Metaspace ran out during Android lint and the build hung until the job
timeout.
62       run: |
63         sed -i '/^org.gradle.jvmargs=/d' android/gradle.properties
64         # The generated file has no trailing newline: start with one so the setting is not
glued onto the last line.
65         printf '\norg.gradle.jvmargs=-Xmx4g -XX:MaxMetaspaceSize=1g -Dfile.encoding=UTF-8\n' >>
        android/gradle.properties
66         tail -n 3 android/gradle.properties
67
68     - name: Build release APK
69       working-directory: android
70       # arm64 only: covers practically every Android phone from the last 7 years and builds
much faster.
71       # lintVitalAnalyzeRelease is skipped: it is a code-style check that is not needed to
produce the APK.
72       run: ./gradlew assembleRelease --no-daemon -PreactNativeArchitectures=arm64-v8a -x
        lintVitalAnalyzeRelease --stacktrace
73
74     - name: Rename APK
75       run: cp android/app/build/outputs/apk/release/app-release.apk mind-pay.apk
76
77     - uses: actions/upload-artifact@v4
78       with:
79         name: mind-pay-apk
80         path: mind-pay.apk
81
82     - name: Publish to the "latest-apk" release
83       env:
84         GH_TOKEN: ${GITHUB_TOKEN}
85       run: |
86         gh release delete latest-apk --yes --cleanup-tag || true
87         gh release create latest-apk mind-pay.apk \
88           --title "MindPay (latest APK)" \
89           --notes "Built from ${GITHUB_SHA::7} on $(date -u +%Y-%m-%d %H:%M UTC'). Open this
        page on an Android phone, tap mind-pay.apk, and install. Debug-signed build for testing." \
90           --prerelease

```

บรรทัด	ความหมาย
12–17	ทำงานเมื่อ push ขึ้น main ยกเว้นแค่เอกสาร (.md, docs/) ซึ่งไม่ต้อง build APK ใหม่
24–26	ตัวที่เริ่ม build แล้วให้ทำงานจบ ตัวใหม่ต่อคิว (build ใช้เวลานาน ไม่อยากทิ้งกลางทาง)
31	เวลาสูงสุด 45 นาที
40–43	ติดตั้ง Java 17 เพราะเครื่องมือ build ของ Android (Gradle) ต้องใช้
48–52	ด้านตรวจ: unit test และ typecheck ต้องผ่านก่อน
57–58	expo prebuild สร้างไฟล์เดอร์ android ทั้งโปรเจกต์จาก app.json (ทีมไม่เก็บไฟล์เดอร์นี้ใน Git สร้างใหม่ทุกครั้ง)
60–66	เพิ่มหน่วยความจำให้ Gradle (คอมเมนต์เล่าว่าเคยค้างจนหมดเวลาเพราะหน่วยความจำไม่พอ)
68–72	build APK แบบ release เฉพาะสถาปัตยกรรม arm64 (ครอบคลุมมือถือแทบทุกเครื่องใน 7 ปีหลัง และ build เร็วกว่า)
74–80	เปลี่ยนชื่อเป็น mind-pay.apk และเก็บไว้
82–90	ลบ release "latest-apk" เดิม แล้วสร้างใหม่พร้อมไฟล์ APK ใหม่ (ลิงก์ดาวน์โหลดจึงเหมือนเดิมตลอด) บอกว่า build จาก commit ไหน เมื่อไร

📌 APK แบบ debug-signed

APK นี้ "เซ็น" ด้วยกุญแจ debug ของ Android ใช้ทดสอบและพีเชนต์ได้ แต่ส่งขึ้น Play Store ไม่ได้ ถ้าจะลง Play Store ต้อง build ด้วย EAS Build แบบ production และมีบัญชีนักพัฒนา Google (README อธิบายไว้)

Secrets บน GitHub

ค่าลับที่ workflow ต้องใช้ เก็บใน Settings → Secrets and variables → Actions ของ repo ไม่ได้อยู่ในโค้ด

Secret	ใช้ที่	ได้มาจาก
GITHUB_TOKEN	web.yml, android-apk.yml	GitHub สร้างให้อัตโนมัติทุกครั้ง
EXPO_TOKEN	eas-update.yml	expo.dev → Account settings → Access tokens (ถ้าไม่ตั้ง workflow นี้ ข้ามไปเลย ๆ)

ส่วนกุญแจ AI ไม่ได้อยู่บน GitHub เลย อยู่ใน Supabase Secrets (un A10)

ไฟล์เอกสารใน repo

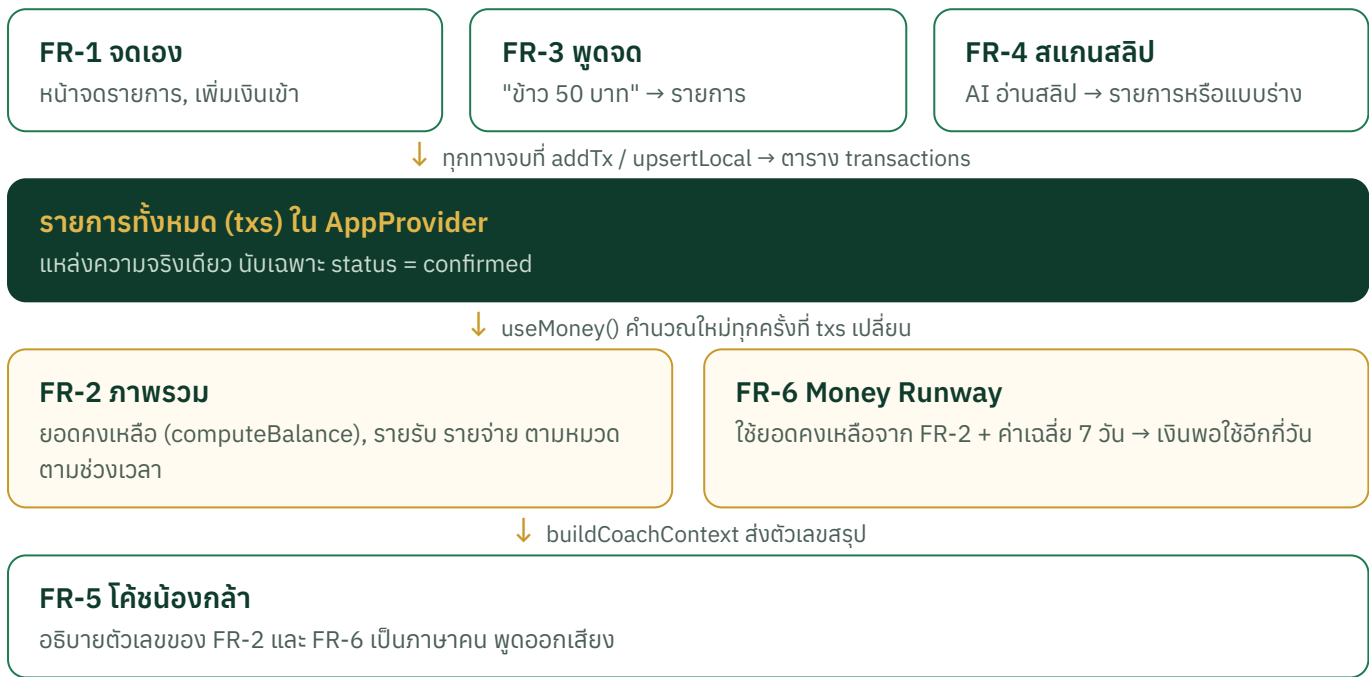
ไฟล์	มีอะไร
README.md	หน้าแรกของ repo: ลิงก์ลองใช้, FR, สถาปัตยกรรม, วิธีติดตั้ง, ตั้งค่า AI, ความปลอดภัย, ข้อจำกัด
docs/TRACEABILITY.md	FR → acceptance criteria → โค้ด → เทสต์ และแผนทดสอบมือ (MT)

ไฟล์	มีอะไร
docs/AI_USAGE_LOG.md	บันทึกการใช้ AI ตามที่ SRS กำหนด 20 แถว
docs/DESIGN_SYSTEM.md	กฎการออกแบบ (บท A11)
docs/SUPABASE_AUTH_SETUP.md	ตั้งค่าอีเมลยืนยันตัวตนของ Supabase
AGENTS.md / CLAUDE.md	คำแนะนำสำหรับผู้ช่วย AI ที่ช่วยเขียนโค้ด (ใช้ Expo 57, คำสั่งที่ควรรับ)

6 FR ทำงานร่วมกันยังไง และคำถามระดับระบบ

ไม่ว่ารับผิดชอบ FR ไหน อาจารย์อาจถามว่า "FR ของเธอเชื่อมกับของเพื่อนยังไง" หรือ "ทำไมเลือกเทคโนโลยีนี้" บทนี้เตรียมคำตอบไว้ให้

สายข้อมูลของทั้งระบบ



ใครใช้ของใคร

FR	ใช้ข้อมูล/ฟังก์ชันจาก	ส่งผลต่อ
FR-1	money.ts (แปลงจำนวนเงิน), categories.ts (หมวด), dates.ts	ทุกรายการที่จด/แก้/ลบ ทำให้ FR-2, FR-6, FR-5 คำนวณใหม่; หน้าจดยรายการยังเป็นหน้าตรวจสอบสลิปของ FR-4
FR-2	รายการจาก FR-1/3/4, ยอดตั้งต้นจากโปรไฟล์	ยอดคงเหลือเป็นตัวตั้งของ FR-6, ยอดตามหมวดส่งให้โค้ช FR-5
FR-3	categories.ts (เดาหมวด), money/dates, addTx ของ FR-1	สร้างรายการแบบยืนยันแล้ว เข้ายอดทันที
FR-4	Edge Function parse-slip + AI, หน้าจดยรายการของ FR-1 (ตรวจแบบร่าง), categories.ts	รายการจากสลิป: ชัด → เข้ายอด, ไม่ชัด → แบบร่าง (ไม่นับใน FR-2/FR-6 จนยืนยัน)
FR-5	ตัวเลขจาก FR-2 และ FR-6, Edge Function coach + AI	ไม่เปลี่ยนข้อมูล (อ่านอย่างเดียว) แต่ชวนผู้ใช้ไปหน้า Runway/สแกน
FR-6	computeBalance (FR-2), รายจ่ายที่ยืนยันแล้ว (FR-1/3/4), เส้นสำรองจากโปรไฟล์, กระปุกออม	ตัวเลข Runway บนหน้าหลัก, การ์ดเตือนของโค้ช, "เช็กก่อนจ่าย" ส่งคำถามให้ FR-5

ไฟล์ที่หลาย FR ใช้ร่วมกัน (แก้แล้วกระทบหลายคน)

ไฟล์	ใครใช้	ถ้าแก้ผิดจะเกิดอะไร
<code>src/domain/types.ts</code>	ทุก FR	TypeScript จะแจ้ง error ทั่วทั้งแอป
<code>src/domain/money.ts</code>	FR-1, 3, 4, 6	จำนวนเงินเพี้ยนทุกที่
<code>src/domain/dates.ts</code>	ทุก FR	รายการไปอยู่ผิดวัน ค่าเฉลี่ย 7 วันผิด
<code>src/domain/summary.ts</code>	FR-2, 5, 6	ยอดคงเหลือผิด → Runway และโค้ชผิดตาม
<code>src/domain/categories.ts</code>	FR-1, 2, 3, 4	หมวดผิด กราฟตามหมวดผิด
<code>src/data/AppProvider.tsx</code>	ทุกหน้า	ข้อมูลไม่โหลด/ไม่อัปเดต
<code>src/app/transaction.tsx</code>	FR-1 และ FR-4 (ตรวจสอบ)	จดยรายการหรือยืนยันสลิปไม่ได้

นี่คือเหตุผลที่ทีมมี unit test ครอบคลุมไฟล์เหล่านี้ทั้งหมด และให้ GitHub รับเทสต์ก่อนปล่อยทุกครั้ง

คำถามระดับระบบ ที่อาจารย์ถามใครก็ได้

- Q1** ทำไมเลือก Expo / React Native?

โค้ดชุดเดียวได้ทั้ง Android, iPhone และเว็บ ทีมเล็กเวลาน้อยจึงไม่ต้องเขียน 3 รอบ ภาษาเดียว (TypeScript) ทั้งแอปและเซิร์ฟเวอร์ มีเครื่องมือ build APK และส่งอัปเดตอัตโนมัติ
- Q2** ทำไมเลือก Supabase?

ได้ฐานข้อมูล PostgreSQL, ระบบสมัคร/เข้าสู่ระบบ, และที่รับโค้ดฝั่งเซิร์ฟเวอร์ในที่เดียว มีแพ็คเกจฟรี และ Row Level Security ทำให้แอปคุยกับฐานข้อมูลได้ตรง ๆ อย่างปลอดภัยโดยไม่ต้องเขียนเซิร์ฟเวอร์ API เอง
- Q3** ข้อมูลผู้ใช้ปลอดภัยยังไง?

ทุกตารางเปิด RLS แต่ละคนอ่านเขียนได้เฉพาะแถวของตัวเอง (ทดสอบ MT-10) คุกกี้ในแอปเป็นคุกกี้สาธารณะ คุกกี้แวลบและคุกกี้ AI อยู่บนเซิร์ฟเวอร์เท่านั้น รูปสลิปไม่ถูกเก็บบนเซิร์ฟเวอร์ โค้ชได้แค่ยอดรวม
- Q4** ทำไมเก็บเงินเป็นสตางค์?

ทศนิยมในคอมพิวเตอร์ไม่เป๊ะ ($0.1 + 0.2 = 0.30000000000000004$) เก็บเป็นจำนวนเต็มสตางค์จึงบวกลบได้แม่นยำ 100% แปลงเป็นบาทตอนแสดงผลเท่านั้น
- Q5** ทำไมต้องจัดวันตามเวลาไทย?

ฐานข้อมูลเก็บเวลา UTC ซึ่งช้ากว่าไทย 7 ชั่วโมง ถ้าไม่แปลง รายการตอนเข้ามีดจะไปอยู่วันก่อนหน้า ยอด "วันนี้" และค่าเฉลี่ย 7 วันจะผิด ทุกจุดจึงใช้ bkkDayKey
- Q6** ยอดคงเหลือเก็บไว้ในฐานข้อมูลไหม?

ไม่ เก็บเฉพาะรายการ ตัวเลขทั้งหมด (ยอด ค่าเฉลี่ย Runway) คำนวณสดในเครื่องจากรายการทุกครั้งที่มีข้อมูลเปลี่ยน ตัวเลขจึงไม่มีทางขัดกันเอง
- Q7** แบบร่าง (draft) คืออะไร ทำไมต้องมี?

รายการจากสลิปที่ AI อ่านไม่มั่นใจ รอผู้ใช้ตรวจสอบก่อน ยังไม่นับในยอดใด ๆ เพราะถ้า AI อ่านผิดแล้วนับทันที ทุกตัวเลขในแอปจะผิดตาม
- Q8** ใช้ AI ตรงไหน และป้องกัน AI ผิดยังไง?

2 ที่: อ่านสลิป (FR-4) และโค้ช (FR-5) ป้องกันด้วย: อ่าน 2 ครั้งด้วย 2 รุ่น, เหนือความมั่นใจ 80%, ตรวจสอบ QR บนสลิป, แบบร่างรอตรวจ, โค้ชห้ามแต่งตัวเลข ตัวเลขทั้งหมดคำนวณโดยโค้ดที่ทดสอบแล้ว

Q9	ทำไมไม่เรียก AI จากแอปตรง ๆ? กฎแฉ AI จะหลุดไปกับแอป Edge Function ถือกฎแฉไว้ ตรวจสอบว่าเข้าสู่ระบบจริง และจำกัดโควตาต่อวันต่อคน
Q10	ทดสอบยังไง? 3 ระดับ: unit test 157 ข้อ (ฟังก์ชันคำนวณ), E2E 145 ข้อ (เปิดเว็บจริงเหมือนคน ด้วยเซิร์ฟเวอร์ปลอม), manual test MT-01 ถึง MT-12 บนมือถือจริง ทุกครั้งที่ push GitHub รันเทสต์ก่อนเผยแพร่
Q11	ถ้าอินเทอร์เน็ทหลุดระหว่างฟรีเซนต์? ใช้โหมดทดลอง ข้อมูลอยู่ในเครื่อง ทำงานได้ทุกอย่างยกเว้น AI
Q12	Requirement โยงกับโค้ดยังไง? เอกสาร docs/TRACEABILITY.md ผูกทุก acceptance criteria ของทุก FR กับไฟล์/ฟังก์ชันที่ทำ และรหัสเทสต์ (TC/MT) ที่พิสูจน์
Q13	ใช้ AI ช่วยเขียนโค้ดไหม? ใช้ บันทึกไว้ใน docs/AI_USAGE_LOG.md ตามที่ SRS กำหนด ทุกแถวบอกจุดประสงค์ ผลลัพธ์ สิ่งที่ทีมแก้หรือตัดสินใจ และสิ่งที่สมาชิกต้องตรวจสอบ (ควรตอนตามจริงว่าตัวเองตรวจอะไรไปแล้วบ้าง)
Q14	ถ้ามีผู้ใช้เยอะขึ้น ระบบรับไหวไหม? การคำนวณทำในเครื่องของแต่ละคน เซิร์ฟเวอร์แค่เก็บข้อมูล มี index ช่วยค้นเร็ว และโหลดเป็นหน้า หน้าละ 1,000 แถว ส่วนที่ต้องระวังคือค่าใช้จ่าย AI ซึ่งมีโควตาต่อวันคุมไว้ และควรเปลี่ยนจาก Gemini แบบฟรีเป็นแบบเสียเงินเมื่อมีผู้ใช้จริง
Q15	อะไรที่ยังไม่ได้ทำ (นอก MVP)? Accountability Mode (UC-7), ออฟไลน์เต็มระบบ (UC-8), เชื่อม Gmail/LINE, รวบรวมรับในอนาคตเข้า Money Runway และยังไม่ได้ทดสอบบนมือถือหลายรุ่น

คำศัพท์รวม

คำ	ความหมายแบบง่าย
Acceptance criteria	เงื่อนไขที่ต้องผ่าน จึงจะถือว่า requirement นั้นเสร็จ
API / API key	ประตูให้โปรแกรมคุยกัน / รหัสลับที่ใช้ผ่านประตู
APK	ไฟล์ติดตั้งแอป Android
async / await	เขียนงานที่ต้องรอ (เช่นอินเทอร์เน็ท) โดยหน้าจอไม่ค้าง
CI/CD	ให้คอมพิวเตอร์ทดสอบและส่งของให้อัตโนมัติทุกครั้งที่เกิดเปลี่ยน (GitHub Actions)
Commit / Push / Branch	จุดเซฟ / ส่งขึ้น GitHub / เส้นทางคู่ขนานของโค้ด
Component	ชิ้นส่วนหน้าจอใน React เป็นฟังก์ชันที่คืน JSX
Context / Provider	วิธีแชร์ข้อมูลให้ทุกหน้าใน React (AuthProvider)
Design System / Token	กฎและชิ้นส่วนกลางของการออกแบบ / ค่ากลางที่ตั้งชื่อตามหน้าที่ เช่น theme.ink
E2E test	ทดสอบทั้งแอปเหมือนคนกดจริงในเบราว์เซอร์
Edge Function	โค้ดฝั่งเซิร์ฟเวอร์บน Supabase (parse-slip, coach)
EAS Update / OTA	ส่งโค้ดใหม่ไปถึงแอปที่ลงแล้ว โดยไม่ต้องลง APK ใหม่

คำ	ความหมายแบบง่าย
Expo Router	ระบบเปลี่ยนหน้าที่ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า
Hook	ฟังก์ชันพิเศษของ React ขึ้นต้นด้วย use เช่น useState
JSON	รูปแบบข้อมูลข้อความที่โปรแกรมใช้รับส่งกัน
JSX	หน้าที่ที่เขียนคล้าย HTML ในโค้ด React
Migration	ไฟล์ SQL ที่สร้าง/แก้ไขโครงสร้างฐานข้อมูล เรียงตามวันที่
Native module	ส่วนที่เขียนด้วยภาษาของมือถือ (เช่น ไมค์ แกลเลอรี) ต้องลง APK ใหม่ถึงจะมี
props / state	ค่าที่ส่งเข้า component / ความจำของ component
Pure function	ฟังก์ชันที่ใช้ค่าเดิมได้ผลเดิม ไม่ยุ่งกับข้างนอก ทดสอบง่าย
Regression	ของที่เคยทำงาน กลับพังหลังแก้ไขโค้ดส่วนอื่น
Repo (Repository)	1) โพลเดอร์โปรเจกต์ใน Git 2) ชั้น Repo ในแอปที่ซ่อนว่าข้อมูลเก็บที่ไหน
RLS	Row Level Security กฎในฐานข้อมูลให้แต่ละคนเห็นเฉพาะแถวของตัวเอง
Satang	หน่วยเงินในระบบ 100 สตางค์ = 1 บาท
Schema (JSON)	แบบฟอร์มบังคับโครงสร้างคำตอบของ AI
Session / Token	สถานะการเข้าสู่ระบบ / บัตรผ่านดิจิทัลที่แนบไปกับทุกคำขอ
SQL	ภาษาสั่งฐานข้อมูล
System prompt	คำสั่งประจำตัวที่บอก AI ว่าต้องเป็นใคร ตอบยังไ้
Traceability	การโยง requirement → การออกแบบ → โค้ด → เทสต์
TypeScript	JavaScript ที่มีชนิดข้อมูล ช่วยจับผิดตั้งแต่ตอนเขียน
Unit test	ทดสอบฟังก์ชันทีละตัว
WCAG	มาตรฐานความเข้าถึงได้ของเว็บ เช่น ความต่างสีตัวหนังสือ $\geq 4.5:1$

ภาค ข

FR-3 Voice Entry

พูดจบ "ข้าว 50 บาท" แล้วบันทึกให้ ภาคนี้เขียนในมุมมองของคนสร้าง FR-3 เอง ตั้งแต่ ปัญหาที่ยากแก่ หน้าจอ หลักการคำนวณ ได้จริงที่ละบรรทัด การทดสอบ ไปจนถึง คำถามที่อาจารย์น่าจะถาม

FR-3 ในหน้าเดียว

ถ้ามีคนถามว่า "FR-3 ของเธอทำอะไร" ให้ตอบจากหน้านี้

FR-3 **Voice Entry (พูดจุด)** ให้ผู้ใช้ **แตะไมค์แล้วพูด** เช่น "ข้าว 50 บาท" แอปเปลี่ยนเสียงเป็นข้อความ แล้ว **แยกเป็นรายการ** ให้เอง: ชื่อ จำนวนเงิน รายรับหรือรายจ่าย หมวด และวัน แสดงให้ดูก่อน แล้ว **กดบันทึกครั้งเดียว** ได้หลายรายการพร้อมกัน ถ้าเครื่องฟังเสียงไม่ได้ ก็ **พิมพ์แบบที่พูด** ได้ผลเหมือนกัน

เสียง → ข้อความ (ระบบของมือถือ/เบราร์เซอร์) → แยกเป็นรายการ (voice.ts)
→ ตรวจสอบ → บันทึก (addTx ของ FR-1)

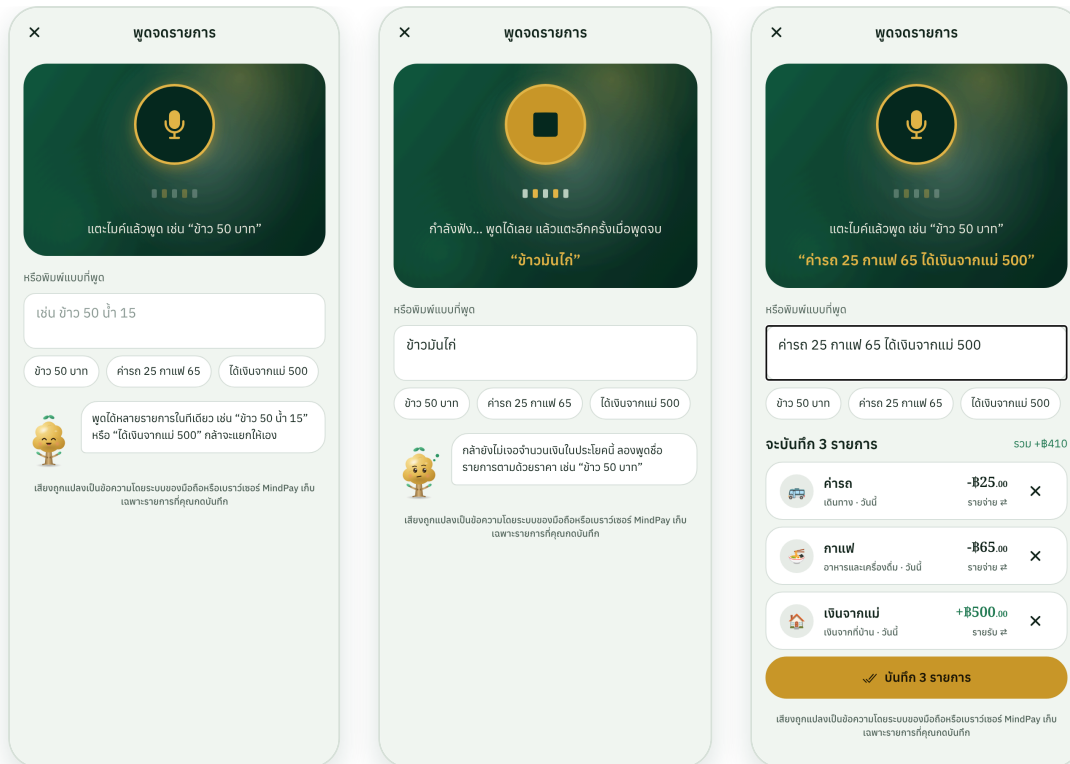
"ค่ารถ 25 กาแฟ 65 ได้เงินจากแม่ 500" → ค่ารถ -฿25 (เดินทาง) · กาแฟ -฿65 (อาหาร) · เงินจากแม่ +฿500 (เงินจากที่บ้าน) ·
 ทั้ง 3 รายการเป็นของวันนี้

เข้าใจได้แค่ไหน (ตัวอย่างที่รับกับโค้ดจริงแล้ว)

พูดว่า	แอปเข้าใจเป็น
ข้าว 50 บาท	ข้าว ฿50 · อาหาร · วันนี้
ข้าวมันไก่ห้าสิบบาทครับ	ข้าวมันไก่ ฿50 (เลขคำไทย + ตัดคำว่าครับ)
ค่ารถ 25 กาแฟ 65	2 รายการ: ค่ารถ ฿25 (เดินทาง), กาแฟ ฿65 (อาหาร)
ได้เงินจากแม่ 500	เงินจากแม่ +฿500 รายรับ · เงินจากที่บ้าน
เมื่อวาน หมูกระทะ 299	หมูกระทะ ฿299 · เมื่อวาน
ข้าว 2 จาน 100 บาท	"ข้าว 2 จาน" ฿100 (เลข 2 คือจำนวนจาน ไม่ใช่เงิน)
รองเท้าน่องพันห้า	รองเท้า ฿2,500 (ภาษาพูด "สองพันห้า" = 2,500)
ค่าไฟ 711 บาท	ค่าไฟ ฿711 (ไม่กลายเป็นเซเวน)
สวัสดีครับ	ไม่มีรายการ (ไม่มีจำนวนเงิน) น้องกล้าแนะนำให้พูดราคา

3 ข้อที่ต้องจำ

- แยก 3 ชั้น:** ฟังเสียง ([src/services/speech.ts](#) และ [.web.ts](#)) · เข้าใจประโยค ([src/domain/voice.ts](#) ฟังก์ชัน `loun` มีเทสต์) · หน้าจอ ([src/app/voice.tsx](#)) — เปลี่ยนตัวฟังเสียงได้โดยไม่แตะตัวเข้าใจประโยค
- ผู้ใช้เห็นก่อนบันทึกเสมอ** สลับรายรับ/รายจ่าย หรือเอารายการที่ฟังผิดออกได้ แล้วจึงกดบันทึก รายการที่บันทึกเป็น **"ยืนยันแล้ว"** กันที (ต่างจากสลิปที่เป็นแบบร่าง) เพราะผู้ใช้เห็นและกดเองแล้ว
- ไม่เก็บเสียง** เสียงถูกแปลงเป็นข้อความโดยระบบของมือถือหรือเบราร์เซอร์ MindPay เก็บเฉพาะรายการที่กดบันทึก และขอสิทธิ์ไมโครโฟนเฉพาะตอนแตะไมค์ครั้งแรก



หน้าพุดจดยรายการ: ไมค์ทอง ช่อง
พิมพ์ และตัวอย่าง

กำลังฟัง: ปุ่มกลายเป็นปุ่มหยุด แท่ง
เสียงเต็ม

3 รายการจากประโยคเดียว รวม +
฿410

1 ภาพหน้าจอในเล่มนี้

ถ่ายจากเว็บเวอร์ชันจริง โหมดทดลอง นาฬิกาตั้งเป็น 19:00 น. วันศุกร์ที่ 9 ต.ค. 2569 เบราว์เซอร์ทดสอบไม่มีตัวฟังภาษาไทย จึงใช้ "ตัวฟังเสียงจำลอง" แบบเดียวกับเทสต์ E2E ที่ส่งข้อความ "ข้าวมันไก่" (ระหว่างพุด) แล้ว "ข้าวมันไก่ 50 บาท" (พุดจบ) ส่วนภาพอื่นใช้การพิมพ์ ซึ่งผ่านตัวเข้าใจประโยคเดียวกันทุกประการ

ปัญหาที่ฉันอยากแก้

การจดเงินสดคือส่วนที่คนเลิกทำก่อน เพราะต้องกดหลายขั้น

FR-4 ช่วยเรื่องเงินโอน (มีสลิป) แต่ **เงินสดไม่มีสลิป**: ข้าวกลางวัน ค่าวัน ขนมนหน้าโรงเรียน ถ้าต้องเปิดฟอร์ม พิมพ์เงิน พิมพ์ชื่อ เลือกหมวด กดบันทึก ทีละรายการ หลายคนจะเลื่อนไปจดตอนเย็นแล้วก็ลืม FR-3 จึงทำให้ **"พูดประโยคเดียว จดได้หลายรายการ"**

จดเงินสด 3 รายการ	ด้วยฟอร์ม (FR-1)	ด้วยเสียง (FR-3)
ขั้นตอน	ทำซ้ำ 3 รอบ: เปิดฟอร์ม → พิมพ์เงิน → พิมพ์ชื่อ → (เลือกหมวด) → บันทึก	แตะไมค์ → พูด "ข้าว 50 น้ำ 15 ค่ารถ 25" → แตะบันทึก
จำนวนแตะโดยประมาณ	15 ครั้งขึ้นไป	3 ครั้ง
หมวด	เดาให้จากชื่อ	เดาให้จากคำพูด (ข้าว น้ำ ค่ารถ) และคำของ FR-1

เป้าหมายที่ฉันตั้งไว้ตอนออกแบบ

เป้าหมาย	ทำยังไงในแอป
เข้าใจภาษาพูดแบบคนไทยจริงๆ	เลขคำไทย ("ห้าสิบบ", "ร้อยห้า" = 150, "สองพันห้า" = 2,500, "ร้อยครึ่ง"), เลขไทย (๖๕), "2k", ตัดคำลงท้าย "ครับ ค่ะ นะ", คำเชื่อม "และ กับ แล้วก็"
หลายรายการในประโยคเดียว	หาจำนวนเงินทุกตัวในประโยค แล้วแบ่งชื่อให้แต่ละจำนวน
ไม่เข้าใจผิดแบบน่าอาย	"ข้าว 2 จาน" ไม่นับ 2 เป็นเงิน, "ห้า" ไม่ใช่ 5, "ค่าไฟ 711" ไม่ใช่เซเว่น, "จ่ายค่าจ้างช่าง" เป็นรายจ่าย
ผู้ใช้คุมได้เสมอ	เห็นข้อความที่แอปได้ยิน เห็นทุกรายการก่อนบันทึก สลับรายรับ/รายจ่าย และเอาออกได้
ใช้ได้ทุกเครื่อง	มือถือใช้ตัวฟังของระบบ (Google บน Android), เว็บใช้ Web Speech API, เครื่องที่ไม่มีก็พิมพ์ได้ (หรือใช้ไมค์บนคีย์บอร์ด) แอปไม่ฟัง
เป็นส่วนตัว	ไม่อัดเสียง ไม่ส่งเสียงไปเซิร์ฟเวอร์ของเรา เก็บเฉพาะรายการที่กดบันทึก

Requirement และเงื่อนไขผ่าน

จาก docs/TRACEABILITY.md: Acceptance criteria → โค้ด → เทสต์

1 Requirement FR-3 (เพิ่มเข้า MVP เมื่อ 29 ก.ย. 2569)

"ผู้ใช้พูดรายการ เช่น 'ข้าว 50 บาท' แล้วแอปบันทึกเป็นรายการให้"

Acceptance criteria	โค้ด	เทสต์
พูด "ข้าว 50 บาท" ได้ ข้าว ฿50 หมวดอาหาร; ตัวเลขเป็นเลข หรือคำไทย ("ห้าสิบบ", "ร้อยห้า" = 150, "สองพันห้า" = 2,500, "ร้อยครึ่ง")	voice.ts parseSpokenEntry, thaiNumberWords	TC-59, TC-61
หลายรายการในประโยคเดียว, "ได้เงินจากแม่ 500" เป็นรายรับ, "เมื่อวาน" ลงวันที่เมื่อวาน, "ข้าว 2 จาน 100" ไม่นับ 2 เป็นเงิน	parseSpokenEntry, normalizeSpoken	TC-60
เห็นรายการก่อนบันทึก สลับรายรับ/รายจ่ายหรือเอาออกได้ แล้ว กดบันทึกครั้งเดียว ยอดเงินอัปเดตทันที	app/voice.tsx	E2E voice
ฟังภาษาไทย (th-TH) ด้วยตัวแปลงเสียงของมือถือ (Android: Google) หรือเบราว์เซอร์; เครื่องที่ฟังไม่ได้หรือแอปป่วนก็อาจพิมพ์แบบที่พูดได้ ไม่ crash	services/speech.ts (โหลด native module แบบไม่บังคับ), speech.web.ts	E2E voice (ตัวฟังเสียงจำลอง), MT-12
ขอสิทธิ์ไมโครโฟนเฉพาะตอนแตะไมค์ครั้งแรก; เก็บเฉพาะรายการที่กดบันทึก ไม่เก็บเสียง	voice.tsx, app.json (expo-speech-recognition)	MT-12
(ตรวจ repo 1–3 ต.ค.) "ค่าไฟ 711" ไม่กลายเป็นเซเวน, "2 พันห้า" = 2,500, "ร้อยหนึ่ง" = 101, พูดจำนวนก่อนซื้อ, เงินเข้าที่จะเอาไปซื้อของยังเป็นรายรับ, แต่ละรายการได้วันของตัวเอง, บันทึกซ้ำหลังพลาดบางรายการไม่ซ้ำ	voice.ts, app/voice.tsx	TC-80

ประวัติสั้น ๆ ใน Git

วันเวลา	commit	เกิดอะไรขึ้น
29 ก.ย. 2569 03:10	c0359d8	"Voice entry (พูดจด): say 'ข้าว 50 บาท' and it is recorded" — FR-3 เกิดขึ้น
29 ก.ย. 03:57	e70e18a	เตรียม "พูดแล้วจด" สำหรับการจดด้วยเสียงครั้งแรก
1–3 ต.ค.	c7bd4b7, 646b485	ตรวจทั้ง repository: แก้ 711, เลขผสมคำ, คำที่ดูเหมือนคำลงท้าย, วันของแต่ละรายการ, บันทึกซ้ำ (TC-80)

เส้นทางของ 1 ประโยค

จากเสียงจนถึงยอดเงินบนหน้าหลัก ผ่าน 3 ชั้น 6 ขั้นตอน

1. แตะไ้มค์ `src/app/voice.tsx`

`toggleListening()`: ขอสิทธิ์ไมโครโฟน (ครั้งแรก) แล้วเรียก `listen()`



2. ฟังเสียง `src/services/speech.ts` / `speech.web.ts`

ตัวแปลงเสียงของ Android/iOS หรือ Web Speech API ฟังภาษาไทย (th-TH) ส่งข้อความกลับมาเรื่อย ๆ ระหว่างพูด (`onText`)

↓ ข้อความ "ข้าวมันไก่ 50 บาท"

3. ใส่ข้อความลงช่อง

`changeText()`: ข้อความเดียวกันที่ผู้ใช้พิมพ์เองได้ (ช่อง "หรือพิมพ์แบบที่พูด")



4. เข้าใจประโยค `src/domain/voice.ts`

`parseSpokenEntry()`: ทำให้เป็นมาตรฐาน → หาจำนวนเงิน → แบ่งชื่อ → รายรับ/รายจ่าย → หมวด → วัน

↓ รายการ [ข้าวมันไก่ 50 บาท อาหาร วันนี้]

5. ผู้ใช้ตรวจ

การ์ดรายการ: สลับ ⇌ รายรับ/รายจ่าย หรือ ✕ เอาออก แล้วกด "บันทึก N รายการ"



6. บันทึก

`saveAll()`: addTx ทีละรายการ (FR-1) note = "จดด้วยเสียง" สถานะยืนยันแล้ว → ยอดคงเหลือ (FR-2) และ Runway (FR-6) อัปเดตทันที → พลั + toast "บันทึก N รายการแล้ว"

แผนที่โค้ดของ FR-3

ไฟล์	หน้าที่
<code>src/domain/voice.ts</code>	สมอง: เลขคำไทย, คำรายรับ/จ่าย, หมวดจากคำพูด, ทั้งประโยค (268 บรรทัด)
<code>src/app/voice.tsx</code>	หน้าจอ: ไ้มค์ แ่่งเสียง ช่องพิมพ์ การ์ดรายการ ปุ่ม บันทึก (346 บรรทัด)
<code>src/services/speech.ts</code>	ฟังเสียงบนมือถือผ่าน expo-speech-recognition (96 บรรทัด)
<code>src/services/speech.web.ts</code>	ฟังเสียงบนเว็บผ่าน Web Speech API (70 บรรทัด)
<code>src/domain/categories.ts</code>	<code>suggestCategory</code> ของ FR-1 (สำรองเมื่อคำพูดไม่ตรงคำของ FR-3)

ไฟล์	หน้าที่
<code>src/domain/achievements.ts</code>	VOICE_NOTE = "จดด้วยเสียง" ใช้แยกรายการจากเสียง (เหรียญ + สรุปเดือน)
<code>app.json</code>	ตั้งค่าปพลิเคชัน expo-speech-recognition และข้อความขอสิทธิ์ไมโครโฟน
<code>src/app/(tabs)/index.tsx</code>	ปุ่มลัด "จดด้วยเสียง" สักองมีชีพจบบนหน้าหลัก
<code>src/domain/__tests__/voice.test.ts</code> , <code>src/domain/__tests__/audit.test.ts</code>	TC-59, TC-60, TC-61, TC-80
<code>e2e/web-auth.mjs</code>	E2E ข้อ 19 Voice entry (บรรทัด 823–906)

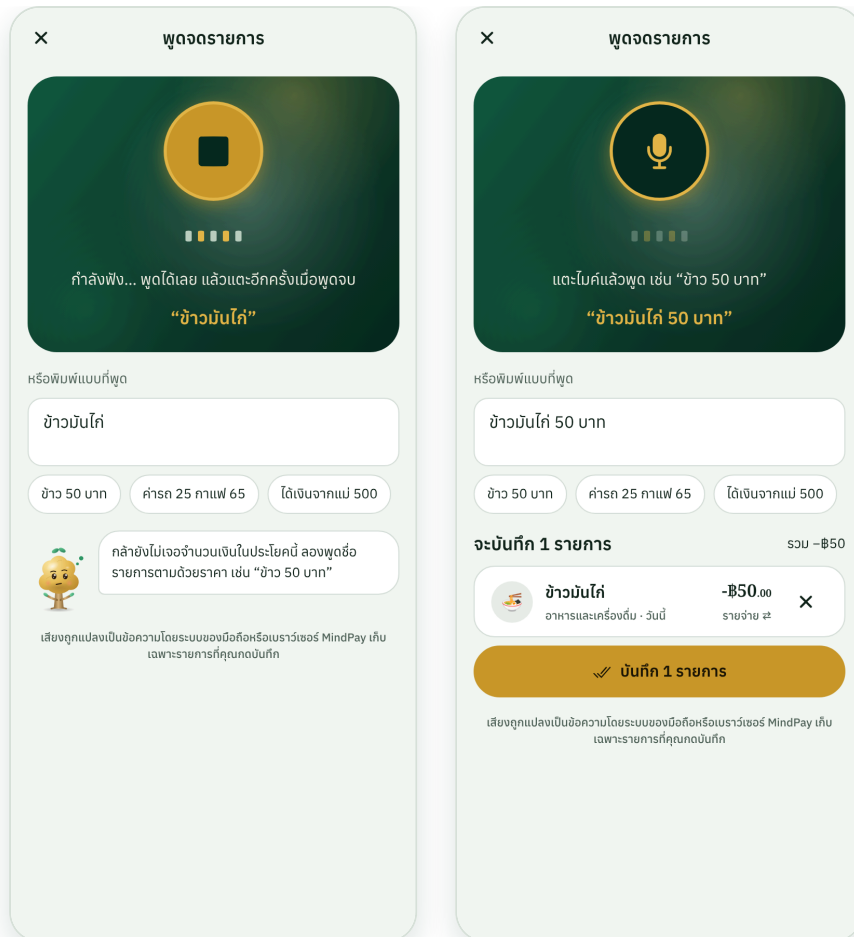
ทัวร์หน้าจอ: เปิดหน้าพูดจุด และฟังเสียง

ปุ่มลัดทอง 'จดด้วยเสียง' บนหน้าหลัก เปิดหน้านี้เลื่อนขึ้นจากด้านล่าง



หน้าพูดจดยรายการเมื่อเปิดครั้งแรก

- 1 ห้วนหน้าจอ × ปิด และ "พูดจดยรายการ"
- 2 การ์ดไมค์ พื้นเขียวไล่สีเหมือนการ์ดยอดเงิน ปุ่มกลม 96 จุด ขอบทอง ไอคอนไมค์
- 3 แท่งเสียง 5 แท่ง จางและนิ่งเมื่อยังไม่ฟัง
- 4 คำแนะนำ "แตะไมค์แล้วพูด เช่น "ข้าว 50 บาท""
- 5 "หรือพิมพ์แบบที่พูด" ช่องพิมพ์หลายบรรทัด ตัวอย่างจาง "เช่น ข้าว 50 น้ำ 15"
- 6 ซิปตัวอย่าง เลื่อนข้างได้: ข้าว 50 บาท · ค่ารถ 25 คาเฟ่ 65 · ได้เงินจากแม่ 500 · เมื่อวาน หมู่กระตะ 299 (แตะแล้วเติมลงช่องทันที)
- 7 นื่องกล้าแนะนำ "พูดได้หลายรายการในที่เดียว..."
- 8 ข้อความความเป็นส่วนตัว "เสียงถูกแปลงเป็นข้อความโดยระบบของมือถือหรือเบราว์เซอร์ MindPay เก็บเฉพาะรายการที่คุณกดบันทึก"



แต่ไม้ค้: ปุ่มเป็นสีทองไอคอนหยุด ■
ข้อความ 'กำลังฟัง... พูดได้เลย แล้วแตะอีกครั้งเมื่อพูดจบ' ระหว่างพูดเห็นคำที่ได้ยินแล้ว 'ข้าวมันไก่' (ยังไม่มีราคา จึงยังไม่มีรายการ)

พูดจบ: ข้อความ "ข้าวมันไก่ 50 บาท" ขึ้นในการ์ดและในช่องพิมพ์ ได้ 1 รายการ – ฿50.00 อาหารและเครื่องดื่ม · วันนี้

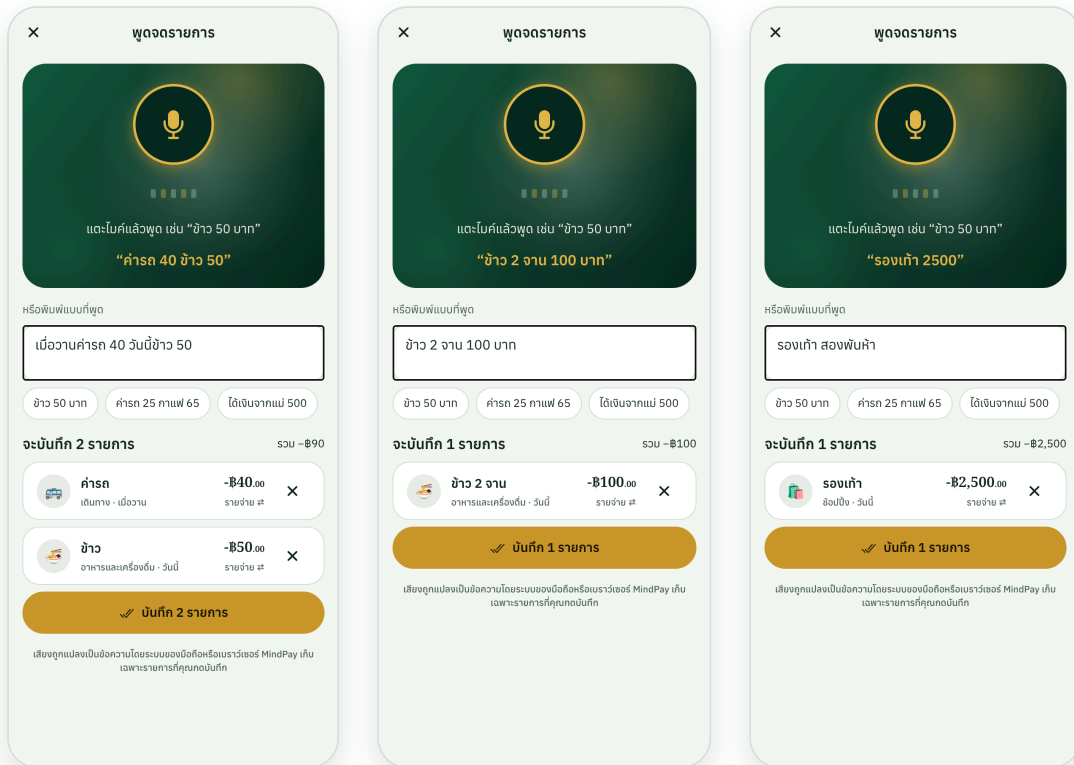
1 ข้อความในเครื่องหมายคำพูดคือ 'สิ่งที่แอปเข้าใจ'

บรรทัดสีทอง “...” ในการ์ดไม้ค้ไม่ใช่สิ่งที่ผู้ใช้พิมพ์ตรง ๆ แต่เป็นข้อความหลังทำให้เป็นมาตรฐานแล้ว (ตัวเลขเป็นเลขอารบิก ตัดคำลงท้าย) เช่น พิมพ์ "รองเท้านี สองพันห้า" บรรทัดทองจะเป็น “รองเท้า 2500” ผู้ใช้จึงเห็นทันทีว่าแอปตีความตัวเลขว่าจะไร

บทที่ 6

ทิวรหน้าจอ: แอปเข้าใจประโยคยังง

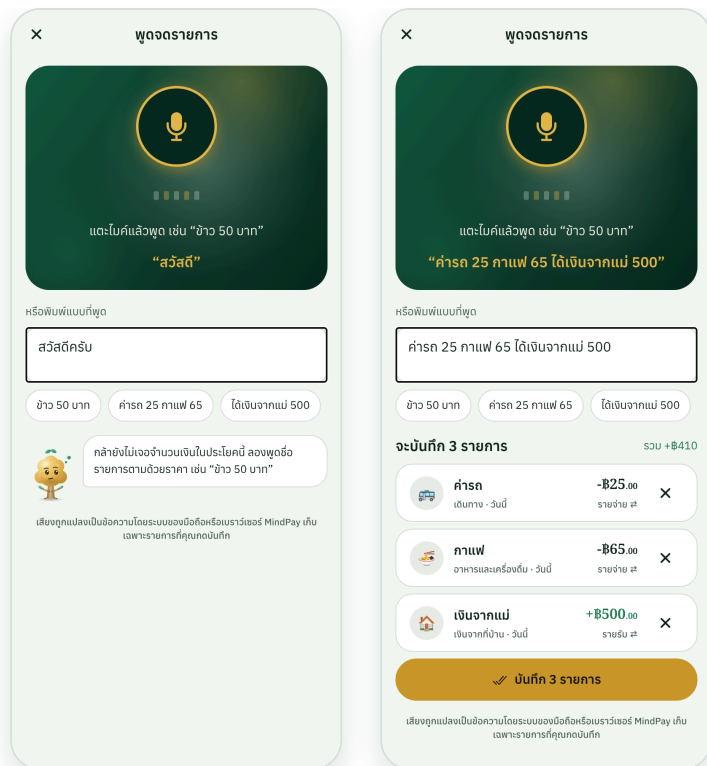
ตัวอย่างจริง 5 แบบ ทุกภาพคือผลของฟังก์ชัน parseSpokenEntry ตัวเดียวกัน



'เมื่อวานค่ารถ 40 วันนี้ข้าว 50' →
ค่ารถ ฿40 · เมื่อวาน และ ข้าว ฿50
· วันนี้ (แต่ละรายการได้วันของตัวเอง)

'ข้าว 2 จาน 100 บาท' → 'ข้าว 2
จาน' ฿100 (2 เป็นจำนวนจาน)

'รองเท้านองเท้า 2500' → แอปเข้าใจว่า
"รองเท้า 2500" → ฿2,500 ซ้อป
ปิ้ง



'สวัสดีครับ' → ไม่มีรายการ น้องกล้าบอก 'ยังไม่เจอจำนวนเงินในประโยค' นี้ ลองพูดจดยรายการตามด้วยราคา'

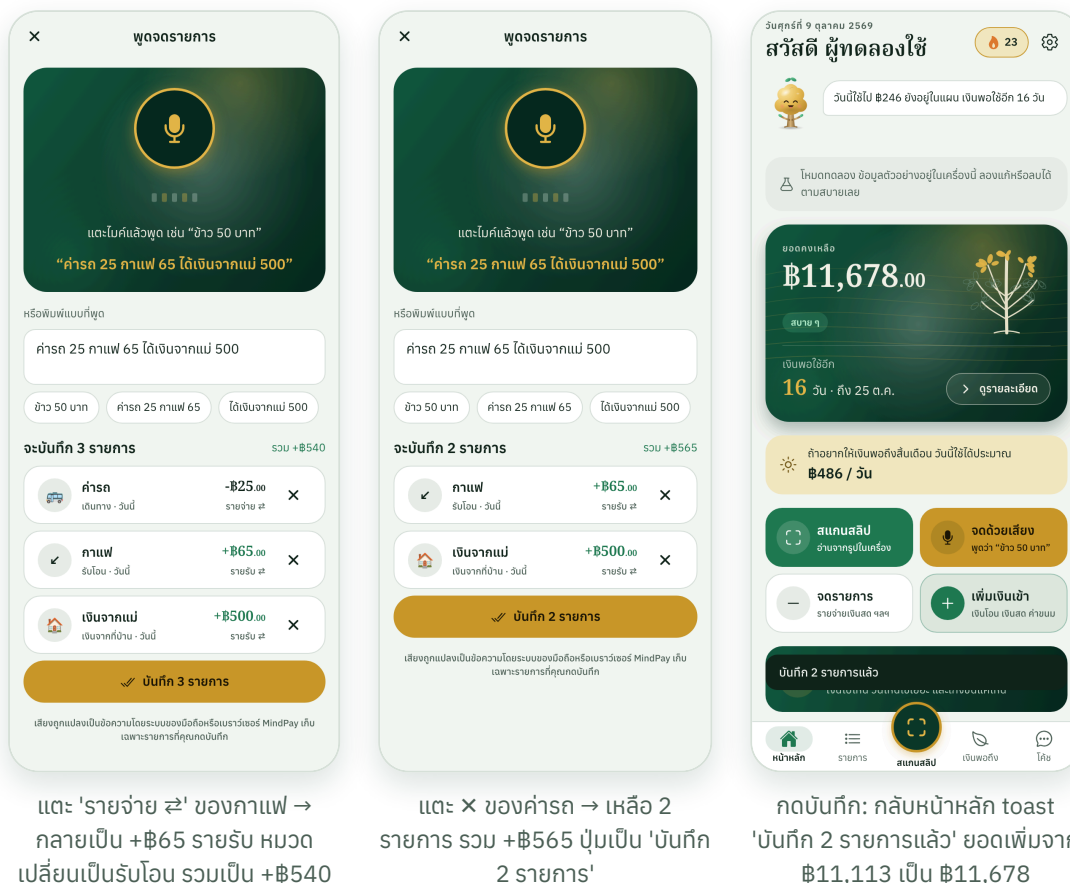
'ค่ารถ 25 บาทกาแฟ 65 บาทได้เงินจากแม่ 500' → 3 รายการ รวม +฿410

ภาพ	รายการที่ได้	สิ่งที่น่าสังเกต
สองวัน	ค่ารถ -฿40 (เดินทาง · เมื่อวาน), ข้าว -฿50 (อาหาร · วันนี้)	คำว่า "เมื่อวาน" และ "วันนี้" หายไปจากชื่อ แต่ไปอยู่ในวันของรายการ
จำนวนจาน	ข้าว 2 จาน -฿100 (อาหาร)	เลขที่ตามด้วยหน่วย (จาน แก้ว ช้อน ...) ถือเป็นส่วนของชื่อ
เลขคำไทย	สองพันห้า -฿2,500 (ช้อปปิ้ง)	"สองพันห้า" ภาษาพูด = 2,500 ไม่ใช่ 2,005
ไม่มีเงิน	(ไม่มี)	ไม่เอาเงินขึ้นมาเอง และ "ครับ" ถูกตัดออกจากบรรทัดทอง ("สวัสดี")
สามรายการ	ค่ารถ -฿25, กาแฟ -฿65, เงินจากแม่ +฿500	"ได้เงินจาก..." = รายรับ ชื่อถูกตัดเหลือ "เงินจากแม่" หมวด "เงินจากที่บ้าน" ยอดรวมหัวรายการเป็นสี่รายรับเพราะเป็นบวก

บทที่ 7

ทัวร์หน้าจอ: แก่ก่อนบันทึก แล้วบันทึก

ทุกรายการแก้ไขได้ 2 อย่างก่อนบันทึก: สลับรายรับ/รายจ่าย และเอาออก



1. การ์ดรายการ: ไอคอนหมวด, ชื่อ (ตัวหนา), "หมวด · วัน", จำนวนเงินมีเครื่องหมาย (รายรับสีเขียว รายจ่ายสีหมึก) ใต้เงิน มีคำว่า "รายจ่าย ๕" และทั้งก้อนเงินเพื่อสลับ
2. สลับแล้วหมวดเปลี่ยนตาม: หมวดอาหารใช้กับรายรับไม่ได้ แอปจึงเดาหมวดใหม่ของประเภทใหม่ (กาแฟแบบรายรับ → รับโอน)
3. ✕ "ไม่บันทึก ค่ารถ" เอารายการออกจากรายการที่จะบันทึก (ข้อความในช่องยังเหมือนเดิม)
4. หัวรายการ "จะบันทึก N รายการ" และ "รวม +฿..." (บวกเมื่อรายรับมากกว่า)
5. ปุ่มทอง "บันทึก N รายการ" มีแสงวิ่ง กดแล้วบันทึกทีละรายการ จุดพลุ แฉ่ง toast แล้วปิดหน้า

ตัวเลขหลังบันทึก (ตรวจด้วยการคำนวณ)

	ก่อน	บันทึก	หลัง
ยอดคงเหลือ	฿11,113	+฿65 (กาแฟที่สลับเป็นรายรับ) +฿500 (เงินจากแม่)	฿11,678
เงินพอใช้อีก (FR-6)	15 วัน	ค่าเฉลี่ยรายจ่ายไม่เปลี่ยน (ไม่มีรายจ่ายใหม่) แต่เงินมากขึ้น	16 วัน $(11,178 \div 695.14 = 16.08)$

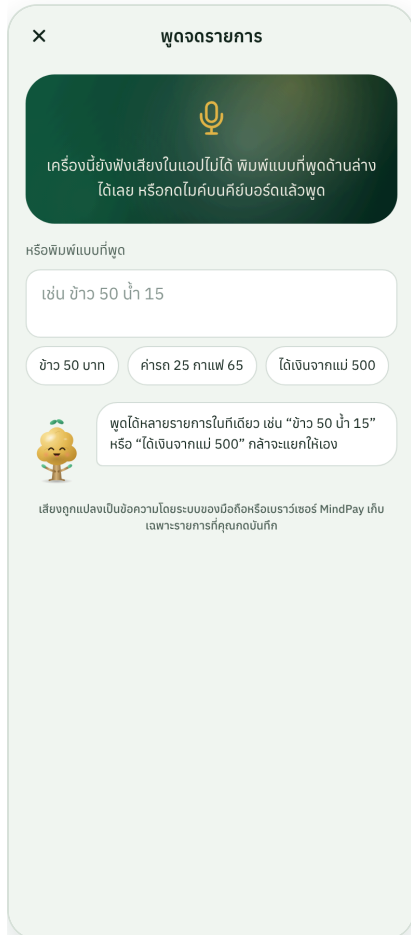
	ก่อน	บันทึก	หลัง
วันนี้ใช้ได้	฿461 / วัน	(11,678 – 500) ÷ 23 วันถึงสิ้นเดือน	฿486 / วัน

! ตัวอย่างนี้จงใจสลับภาพเป็นรายรับ

ในชีวิตจริงภาพเป็นรายจ่าย ภาพนี้สลับเพื่อสาธิตปุ่ม **≡** เท่านั้น ประเด็นที่ต้องเข้าใจคือ **ผู้ใช้เป็นคนตัดสินใจสุดท้าย** แอปเดาให้ แต่ถ้าเดาผิด ผู้ใช้แก้ไขได้ด้วยการแตะครั้งเดียว ตัวอย่างจริง: พูดว่า "เพื่อนโอนค่าข้าวคืน 50" แอปเข้าใจเป็น **รายจ่าย** หมวดยาอาหาร (เพราะไม่มีคำรายรับอย่าง "ได้เงิน" หรือ "คืนเงิน" และมีคำว่า "ข้าว") ทั้งที่จริงคือเงินที่เพื่อนคืน ผู้ใช้แตะ **≡** ให้เป็นรายรับได้ทันที (ถ้าพูดว่า "เพื่อนคืนเงิน 50" หรือ "เพื่อนโอนมาให้ 50" แอปจะเข้าใจเป็นรายรับเอง)

ทัวร์หน้าจอ: เครื่องที่ฟังเสียงไม่ได้

แอปต้องไม่ฟัง และยังคงได้ด้วยวิธีอื่น



เบราว์เซอร์ที่ไม่มี Web Speech API

เมื่อ `speechAvailable()` ตอบ false (เบราว์เซอร์ไม่มีตัวฟังเสียง หรือแอปรุ่นเก่าที่ยังไม่มีโมดูลเสียง) การ์ดไม่จะเปลี่ยนเป็น:

- ไอคอนไมค์เส้น (ไม่ใช่ปุ่ม กดไม่ได้)
- "เครื่องนี้ยังฟังเสียงในแอปไม่ได้ พิมพ์แบบที่พูดด้านล่างได้เลย หรือกดไมค์บนคีย์บอร์ดแล้วพูด"

ช่องพิมพ์และชิปตัวอย่างยังใช้ได้ครบ เพราะการ "เข้าใจประโยค" ไม่ขึ้นกับการฟังเสียง และคีย์บอร์ดของมือถือส่วนใหญ่มีปุ่มไมค์ของตัวเองที่พิมพ์เสียงลงช่องได้อยู่แล้ว

ข้อความเมื่อฟังไม่สำเร็จ

รหัสจากระบบ	แอปบอกผู้ใช้งาน
not-allowed	ต้องอนุญาตให้ใช้ไมโครโฟนก่อนนะ
no-speech, speech-timeout	ไม่ได้ยินเสียง ลองแตะไมค์แล้วพูดอีกครั้งนะ
network	ต้องต่ออินเทอร์เน็ตเพื่อแปลงเสียงเป็นข้อความ
audio-capture	ใช้ไมโครโฟนไม่ได้ตอนนี้ ลองใหม่อีกครั้ง
language-not-supported	เครื่องนี้ยังฟังภาษาไทยไม่ได้ พิมพ์แทนได้เลย
service-not-allowed	เครื่องนี้ยังไม่มีบริการแปลงเสียง พิมพ์แทนได้เลย
busy	ระบบฟังเสียงกำลังทำงานอยู่ ลองใหม่อีกสักครู่
(ปิดสวิตช์ถาวร)	ปิดสวิตช์ไมโครโฟนไว้ เปิดได้ในการตั้งค่าของมือถือ + ปุ่ม "เปิดการตั้งค่า"
(อื่น ๆ)	ฟังเสียงไม่สำเร็จ ลองอีกครั้ง หรือพิมพ์แทนได้เลย

หลักการ: จากประโยคเป็นรายการใน 6 ขั้นตอน

ไม่ใช่ AI ไม่ต้องใช้เน็ต เป็นกฎที่เขียนเองทั้งหมด จึงเร็ว ฟรี และทดสอบได้ทุกกรณี

ตัวแปลงเสียงของมือถือให้เราแค่ "ข้อความ" เช่น "เมื่อวานค่ารถสัปดาห์ครบ วันนี้ข้าว 50" งานของ FR-3 คือเปลี่ยนข้อความนั้นเป็นรายการที่บันทึกได้ ฟังก์ชัน `parseSpokenEntry` ทำ 6 ขั้นตอนตามลำดับ:

1. ทำให้เป็นมาตรฐาน

เลขไทย → เลขอารบิก · ตัดจุดภาคหลักพัน · ฿ → บาท · 7-11 → เซเว่น · ตัดคำลงท้าย ครบ ค่ะ นะ



2. ทำเครื่องหมายวัน

"เมื่อวานขึ้น" "เมื่อวาน" "วันนี้" กลายเป็นป้ายลบ ฐานารู ไว้ในข้อความ (จะรู้ว่าแต่ละรายการพูดหลังคำไหน)



3. เปลี่ยนคำตัวเลขเป็นเลข

"2k" → 2000 · "2 พันห้า" → สองพันห้า → 2500 · "15 ร้อย" → 1500 · "ห้าสิบ" → 50 (เฉพาะเมื่อชัดว่าเป็นตัวเลข)



4. หาจำนวนเงินทุกตัว

เลขที่ไม่ได้ตามด้วยหน่วย (จาน แก้ว ชื่น ...) อาจตามด้วย "บาท" และ "50 สตางค์"



5. แบ่งชื่อให้แต่ละจำนวน

ปกติ: ชื่อคือคำที่อยู่ก่อนเลข · ถ้าเริ่มประโยคด้วยเลข ("50 บาท ค่าข้าว"): ชื่อคือคำหลังเลข



6. ตัดสินรายการ

รายรับหรือรายจ่าย (คำรายรับ vs คำจ่ายเงิน) · ทำความสะอาดชื่อ (ตัด ชื่อ จ่าย กิน ได้เงิน) · หมวด · วัน

≈ เหมือนพนักงานจดออเดอร์ร้านอาหาร

ลูกค้าพูดรวดเร็ว "ข้าวผัดสองจาน น้ำเปล่าสิบบาท เมื่อวานยังไม่ได้จ่ายค่าค็อกกี้อีกสิบ" พนักงานที่เก่งจะ (1) ฟังให้ออกว่าตัวเลขไหนเป็นราคา ตัวไหนเป็นจำนวน (2) แยกเป็นรายการ (3) จำว่ารายการไหนของวันไหน ฟังก์ชันของเราทำแบบเดียวกัน แต่ใช้กฎที่เขียนไว้ชัดเจนแทนสามัญสำนึก

หลักการ: ขั้นที่ 1-3 ทำให้ข้อความเป็นมาตรฐาน

ทุกตัวอย่างในบทนี้รันผ่าน normalizeSpoken ของจริงแล้ว

กฎ	พิมพ์/พูดว่า	หลังทำให้เป็นมาตรฐาน
เลขไทย → เลขอารบิก	กาแฟ ๖๕	กาแฟ 65
ตัดจุลภาคหลักพัน	ค่าหอ 3,500 บาท	ค่าหอ 3500 บาท
จุดท้ายเลขที่ไม่ใช่ทศนิยม	ข้าว 50. น้ำ 15	ข้าว 50 น้ำ 15
฿ หน้าเลข / หลังเลข	฿50 ข้าว · ข้าว 50฿	50 ข้าว · ข้าว 50 บาท
7-11 คือร้าน	ขนม 7-11 45 บาท	ขนม เซเว่น 45 บาท
แต่ 711 ที่เป็นราคาไม่ใช่ร้าน	ค่าไฟ 711 บาท	ค่าไฟ 711 บาท
ตัดคำลงท้ายสุภาพ	ข้าว 50 บาทครับ · ข้าว 50 นะ	ข้าว 50 บาท · ข้าว 50
...แต่ไม่ตัดคำที่เป็นส่วนของคำ	ชนะ 50 · ข้าวเจ้า 50	ชนะ 50 · ข้าวเจ้า 50
คำบอกวันกลายเป็นป้าย	เมื่อวาน ข้าว 50	ข้าว 50 (วัน = -1)
k / เค = พัน	ขายของได้ 2k	ขายของได้ 2000
เลข 1 หลัก + คำหลัก	รองเท้า 2 พันห้า	รองเท้า 2500
สองคำหลักที่ต่อกันได้	สามพัน สองร้อย	3200
เลข × คำหลัก	ค่าเทอม 15 ร้อย · ค่ารถ 1.5 พัน · 2 พันครึ่ง	1500 · 1500 · 2500
คำตัวเลขท้ายคำ (หลายส่วน)	ข้าวห้าสิบ	ข้าว 50
คำตัวเลขที่อยู่เดี่ยว ๆ	ซานม สามสิบห้า · ข้าว ห้า	ซานม 35 · ข้าว 5
คำตัวเลขก่อน "บาท"	ข้าวห้า บาท	ข้าว 5 บาท
คำที่แค่ ดูเหมือน ตัวเลข	ห้าง 250 · ข้าวห้า	ห้าง 250 · ข้าวห้า (ไม่แปลง)

1 ทำไม 'ข้าวห้า' ไม่แปลง แต่ 'ข้าวห้าสิบ' แปลง

ภาษาไทยเขียนติดกันไม่มีเว้นวรรค ถ้าแปลงทุกคำที่มี "ห้า" คำว่า "ห้าง" "ห้าม" จะพัง กฎจึงแปลงคำตัวเลขก็ต่อเมื่อ **ชัดว่าเป็นตัวเลข**: อยู่เดี่ยว ๆ (มีช่องว่างหน้า-หลัง), ตามด้วย "บาท", หรือเป็นตัวเลข **หลายส่วน** ที่จบคำพอดี ("ห้าสิบ" มี 2 ส่วน: ห้า + สิบ) "ข้าวห้า" มีส่วนเดี่ยวและติดกับคำข้างหน้า จึงไม่แน่ใจ แอปเลือกไม่แปลง (ผลคือไม่มีรายการ ผู้ใช้เห็นกันที่ว่าต้องพูดใหม่)

หลักการ: อ่านเลขคำไทยแบบที่คนพูด

thaiNumberWords เข้าใจทั้งแบบเต็ม ('หนึ่งร้อยห้า' = 105) และแบบย่อภาษาพูด ('ร้อยห้า' = 150)

คำ	ค่า	เหตุผล
ห้าสิบ	50	ห้า × สิบ
ยี่สิบห้า	25	"ยี่" = 2 ใช้กับสิบ · 20 + 5
สิบเอ็ด	11	"เอ็ด" = 1 หลักหน่วยเสมอ
หนึ่งร้อยห้า	105	เริ่มด้วยเลข (หนึ่ง) = แบบเต็ม 100 + 5
ร้อยห้า	150	ภาษาพูดย่อ: เลขหลังร้อยโดยไม่มีเลขนำหน้า = หลักสิบ (5 × 10)
ร้อยครึ่ง	150	ครึ่ง = ครึ่งของค่าหลักล่าสุด (100 ÷ 2)
ร้อยนึ่ง	100	"นึ่ง" ถ้ายร้อยขึ้นไป = "หนึ่ง(ร้อย)" ไม่ใช่เลขเพิ่ม
ร้อยหนึ่ง · ร้อยเอ็ด	101 · 101	"หนึ่ง" หลังร้อย และ "เอ็ด" = หลักหน่วย (กรณีพิเศษ)
พันห้าร้อย	1,500	1,000 + 500
สองพันห้า	2,500	ภาษาพูดย่อ: เลขหลังพัน = หลักร้อย (5 × 100)
สองพันหนึ่ง	2,100	หลังพันขึ้นไป "หนึ่ง" ก็เป็นแบบย่อเหมือนเลขอื่น
พันนึ่ง	1,000	"นึ่ง" = หนึ่งพัน
ยี่สิบนึ่ง	21	หลังสิบ "นึ่ง" เป็นหลักหน่วย
แปดร้อยเก้าสิบเก้า	899	800 + 90 + 9
สามล้านห้าแสน	3,500,000	ล้าน = เก็บทุกอย่างก่อนหน้าเป็นหลักล้าน แล้วนับต่อ
ห้าง	null	"ง" ไม่ใช่คำตัวเลข → ไม่ใช่ตัวเลข

★ **กติกาแบบย่อในประโยคเดียว**

เลขตัวสุดท้ายที่ตามหลังคำหลักทันที: ถ้าคำหลักเป็น **พันขึ้นไป** → คูณหนึ่งในสิบของคำหลัก (สองพันห้า → 5 × 100) · ถ้าเป็น **ร้อย** และประโยคไม่ได้ขึ้นต้นด้วยเลข → คูณ 10 (ร้อยห้า → 50) · กรณีอื่นเป็นหลักหน่วยธรรมดา ยกเว้น "เอ็ด" "นึ่ง" และ "หนึ่งหลังร้อย" ที่เป็นหลักหน่วยเสมอ

หลักการ: ขั้นที่ 4–6 หาเงิน แบ่งซื้อ ตัดสินรายการ

ตัวอย่างทั้งหมดรันผ่าน parseSpokenEntry ของจริงแล้ว

ขั้นที่ 4: จำนวนเงินคือเลขแบบไหน

- เลขเต็มตัว (ทศนิยมได้ไม่เกิน 2 ตำแหน่ง) ไม่ใช่ส่วนหนึ่งของเลขที่ยาวกว่า และไม่ติดกับตัวอักษรอังกฤษ
- ไม่ตามด้วยหน่วย:** จาน แก้ว ขวด ชัน อัน ห่อ ถุง กล่อง ลูก คน ที่ ตัว คู่ เล่ม ใบ ครั้ง โมง นาที ชั่วโมง วัน เดือน ปี กิโล กก. ลิตร ชม ถ้วย เม็ด แผ่น ซอง แพ็ค หลอด คัน รอบ
- อาจตามด้วย "บาท" หรือ "บ." และ "xx สตางค์" (ข้าว 45 บาท 50 สตางค์ = ฿45.50)
- ต้องมากกว่า 0 และน้อยกว่า ฿10,000,000

ขั้นที่ 5: แบ่งซื้อ

ประโยค	แบบ	ผล
ข้าว 50 น้ำ 15	ซื้ออยู่ก่อนเลข	[ข้าว ฿50] [น้ำ ฿15]
50 บาท ค่าข้าว 30 บาท ค่าน้ำ	ขึ้นต้นด้วยเลข → ซื้ออยู่หลังเลข	[ค่าข้าว ฿50] [ค่าน้ำ ฿30]
ข้าว 50 บาท 30 ค่าน้ำ	เลขตัวสุดท้ายไม่มีชื่อข้างหน้า → ใช้คำหลังมัน	[ข้าว ฿50] [ค่าน้ำ ฿30]
ข้าว 50 น้ำ 15 ขนมน	"ขมน" ไม่มีราคา	[ข้าว ฿50] [น้ำ ฿15] (ขมนถูกข้าม)
ข้าวห้าสิบ สองร้อย	ตัวที่สองไม่มีชื่อ	[ข้าว ฿50] [อื่น ๆ ฿200] (ชื้อว่าง → ใช้ชื่อหมวด)

ขั้นที่ 6: รายรับหรือรายจ่าย

มองหา **คำรายรับ** (ได้เงิน ได้รับ รับเงิน เงินเข้า เงินเดือน ค่าจ้าง แม่ให้ พ่อให้ โอนมาให้ โอนเข้า รายรับ รายได้ ขายของได้ ได้ ค่ายขมน ได้ค่า ได้เงินคืน เงินคืน คืนเงิน) และ **คำจ่ายเงิน** (จ่าย ชื้อ โอนให้ โอนไป และ "คืนเงิน" ที่ขึ้นต้นประโยค) ถ้าไม่มีคำ รายรับ = รายจ่าย ถ้ามีทั้งคู่ **คำที่มาก่อนชนะ**

ประโยค	ผล	เพราะ
ได้เงินจากแม่ 500	รายรับ "เงินจากแม่" (เงินจากที่บ้าน)	มี "ได้เงิน" ไม่มีคำจ่าย
แม่ให้เงินซื้อขมน 100	รายรับ (เงินจากที่บ้าน)	"แม่ให้" มาก่อน "ซื้อ"
จ่ายค่าจ้างช่างแอร์ 800	รายจ่าย	"จ่าย" มาก่อน "ค่าจ้าง"
คืนเงินเพื่อน 200	รายจ่าย	"คืนเงิน" ขึ้นต้นประโยค = เราคืนให้เขา
เพื่อนคืนเงิน 200	รายรับ (รับโอน)	"คืนเงิน" อยู่หลังชื่อคน = เงินกลับมาหาเรา
ได้ค่ายขมน 100	รายรับ "ค่ายขมน" (เงินจากที่บ้าน)	"ได้ค่าย..." และตัดคำว่า "ได้" ออกจากชื่อ

ขั้นที่ 6 (ต่อ): ชื่อ หมวด และวัน

- **ชื่อ:** ตัดคำเชื่อมหน้า (และ กับ แล้วก็ อีก ต่อ) ตัดคำกริยาหน้า (ซื้อ จ่าย กิน เติม) ตัด "บาท" ถ้าย · รายรับตัด "ได้รับ ได้เงิน ได้ รับ" และเปลี่ยน "จาก..." เป็น "เงินจาก..." · ตัวอย่าง "ซื้อเสื้อ 390" → "เสื้อ", "เติมน้ำมัน 500" → "น้ำมัน" แต่ "เติมเงินโทรศัพท์" ยังเป็น "เติมเงินโทรศัพท์"
- **หมวด:** เทียบกับคำพูด 60 กว่าคำ (คำยาวกว่าเทียบก่อน: "ค่าน้ำ" = บิล ก่อน "น้ำ" = อาหาร) ถ้าไม่เจอใช้ suggestCategory ของ FR-1
- **วัน:** ใช้คำบอกวันตัวสุดท้ายที่พูด **ก่อน** ราคาของรายการนั้น ถ้าไม่มีเลยใช้คำบอกวันตัวแรกที่พูด (จึงพูดไว้ท้ายประโยค ก็ได้ "ข้าว 50 น้ำ 15 เมื่อวาน" → ทั้งสองเป็นเมื่อวาน) ไม่พูดเลย = วันนี้

อ่านโค้ด: voice.ts ตอนที่ 1 — ชนิดข้อมูลและเลขคำไทย

ไฟล์ src/domain/voice.ts มี 268 บรรทัด แบ่งอ่าน 3 ตอน ตอนนี้เป็นบรรทัด 1–90

ส่วนที่ 1: คอมเมนต์หัวไฟล์และชนิดข้อมูล (บรรทัด 1–30)

src/domain/voice.ts

บรรทัด 1–30

```

1 /**
2  * "พูดจ": turn what the user says (or types the way they would say it) into
3  * transactions. Pure functions, unit-tested in __tests__/voice.test.ts.
4  *
5  * "ข้าว 50 บาท"                -> ข้าว $50 (อาหาร)
6  * "ข้าวจ้าว น้ำส้ม"          -> ข้าว $50, น้ำ $15
7  * "ได้เงินจากแม่ 500"          -> เงินจากแม่ +$500 (รายรับ, เงินจากที่บ้าน)
8  * "เมื่อวาน ค่ารถ 40"         -> ค่ารถ $40, yesterday
9  * "ข้าว 2 จาน 100 บาท"        -> ข้าว 2 จาน $100 (2 is a quantity, not money)
10 *
11 * Speech recognizers usually write numbers as digits ("50"), but Thai number
12 * words ("ห้าสิบ", "สองพันห้า", "ร้อยครึ่ง") are understood too.
13 */
14 import { categoriesFor, getCategory, suggestCategory } from './categories';
15 import type { TxKind } from './types';
16
17 export interface SpokenItem {
18   title: string;
19   amountSatang: number;
20   kind: TxKind;
21   categoryKey: string;
22   /** 0 = today, -1 = yesterday, -2 = the day before. */
23   dayOffset: number;
24 }
25
26 export interface SpokenEntry {
27   items: SpokenItem[];
28   /** The text as understood (numbers as digits), shown back to the user. */
29   heard: string;
30 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/voice.ts:1

บรรทัด	ความหมาย
1–3	คอมเมนต์: "พูดจ" เปลี่ยนสิ่งที่ผู้ใช้พูด (หรือพิมพ์แบบที่พูด) เป็นรายการ เป็นฟังก์ชันล้วน มีทดสอบใน voice.test.ts
5–9	ตัวอย่าง 5 ประโยคที่เป็นสเปกของไฟล์นี้: ข้าว 50 บาท → ข้าว \$50 (อาหาร) · ข้าวจ้าว น้ำส้ม → 2 รายการ · ได้เงินจากแม่ 500 → รายรับ · เมื่อวาน ค่ารถ 40 → เมื่อวาน · ข้าว 2 จาน 100 บาท → 2 เป็นจำนวน ไม่ใช่เงิน
11–12	ตัวแปลงเสียงมักเขียนตัวเลขเป็นเลข ("50") แต่ไฟล์นี้เข้าใจเลขคำไทยด้วย ("ห้าสิบ", "สองพันห้า", "ร้อยครึ่ง")
14	ใช้ฟังก์ชันหมวดของ FR-1: หมวดตามประเภท, หาหมวด, เดาหมวด

บรรทัด	ความหมาย
15	ชนิด TxKind (income/expense)
17–24	SpokenItem = 1 รายการที่ได้จากคำพูด: ชื่อ, จำนวนสตางค์, ประเภท, หมวด, dayOffset (0 = วันนี้, -1 = เมื่อวาน, -2 = เมื่อวานขึ้น)
26–30	SpokenEntry = ผลของทั้งประโยค: รายการทั้งหมด และ heard = ข้อความที่เข้าใจ (ตัวเลขเป็นเลขแล้ว) ไว้แสดงกลับให้ผู้ใช้ดู (บรรทัดสีทองในการ์ดโมค)

i ทำไมใช้ dayOffset แทนวันที่จริง

ตัวเข้าใจประโยคไม่รู้ (และไม่ควรต้องรู้) ว่าวันนี้คือวันอะไร มันบอกแค่ "เมื่อวาน" = -1 หน้าจอจึงเอา -1 ไปบวกกับวันนี้ (addDays(today, item.dayOffset)) ทำให้ฟังก์ชันนี้เป็นฟังก์ชันล้วน ทดสอบได้โดยไม่ต้องตั้งนาฬิกา

ส่วนที่ 2: คำตัวเลขไทย (บรรทัด 32–41)

src/domain/voice.ts

บรรทัด 32–41

```

32 // -----
33 // Thai number words
34 // -----
35
36 const DIGIT_WORDS: Record<string, number> = {
37   ศูนย์: 0, หนึ่ง: 1, ับ: 1, เอ็ด: 1, สอง: 2, ยี่: 2, สาม: 3, สี่: 4, ห้า: 5, หก: 6, เจ็ด: 7, แปด: 8, เก้า: 9,
38 };
39 const MULTIPLIERS: Record<string, number> = { สิบ: 10, ร้อย: 100, พัน: 1000, หมื่น: 10000, แสน:
40   100000, ล้าน: 1000000 };
41 const NUMBER_WORD = 'ศูนย์|หนึ่ง|ับ|เอ็ด|สอง|ยี่|สาม|สี่|ห้า|หก|เจ็ด|แปด|เก้า|สิบ|ร้อย|พัน|หมื่น|แสน|ล้าน|ครั้ง';
42 const NUMBER_WORDS_RE = new RegExp(`(?:${NUMBER_WORD})+`, 'g');

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/voice.ts:32

บรรทัด	ความหมาย
36–38	DIGIT_WORDS คำของเลขหลักเดียว → คำ: ศูนย์ 0, หนึ่ง/ับ/เอ็ด 1, สอง/ยี่ 2, สาม 3 ... เก้า 9 (มีคำพูดอย่าง "ับ" และ "ยี่" ด้วย)
39	MULTIPLIERS คำหลัก → ตัวคูณ: สิบ 10, ร้อย 100, พัน 1,000, หมื่น 10,000, แสน 100,000, ล้าน 1,000,000
40	NUMBER_WORD รูปแบบที่จับคำตัวเลขทุกคำ คั่นด้วย (= "หรือ") รวม "ครั้ง"
41	NUMBER_WORDS_RE จับ "คำตัวเลขหลายคำติดกัน" ((?:...)+ = กลุ่มนี้ซ้ำ 1 ครั้งขึ้นไป) ธง g = หาทุกจุดในข้อความ เช่นใน "ข้าวห้าสิบ" จะจับ "ห้าสิบ"

ส่วนที่ 3: thaiNumberWords – แปลงคำเป็นเลข (บรรทัด 43–90)

src/domain/voice.ts

บรรทัด 43–90

```

43 /**
44  * "ห้าสิบ" -> 50, "ยี่สิบห้า" -> 25, "หนึ่งร้อยห้า" -> 105, "ร้อยห้า" -> 150 (spoken short form),
45  * "สองพันห้า" -> 2500, "ร้อยครั้ง" -> 150, "พันับ" -> 1000, "ร้อยับ" -> 101, "ยี่สิบับ" -> 21.

```

```

46  * Returns null when it is not a number.
47  */
48  export function thaiNumberWords(words: string): number | null {
49    const tokens = words.match(new RegExp(NUMBER_WORD, 'g'));
50    if (!tokens || tokens.join('') !== words) return null;
51    let total = 0;
52    let millions = 0;
53    let digit: number | null = null;
54    let lastMultiplier = 0;
55    let startsWithDigit = false;
56    let digitRightAfterMultiplier = false;
57    let digitWord = '';
58    tokens.forEach((tok, i) => {
59      if (tok in DIGIT_WORDS) {
60        // "ับ" right after ร้อย or more means "one of it" ("ร้อยับ" = 100), not another digit
61        // (after ับ it is a digit: "ยี่สิบับ" = 21).
62        if (tok === 'ับ' && lastMultiplier >= 100 && i === tokens.length - 1) return;
63        digit = DIGIT_WORDS[tok];
64        digitWord = tok;
65        if (i === 0) startsWithDigit = true;
66        digitRightAfterMultiplier = i > 0 && tokens[i - 1] in MULTIPLIERS;
67      } else if (tok === 'ครึ่ง') {
68        total += lastMultiplier ? lastMultiplier / 2 : 0;
69      } else if (tok === 'ล้าน') {
70        millions = (total + (digit ?? (total ? 0 : 1))) * 1_000_000;
71        total = 0;
72        digit = null;
73        lastMultiplier = 0;
74      } else {
75        const m = MULTIPLIERS[tok];
76        total += (digit ?? 1) * m;
77        digit = null;
78        lastMultiplier = m;
79      }
80    });
81    if (digit !== null) {
82      // Spoken short form: "สองพันห้า" = 2,500 and "ร้อยห้า" = 150 (but "หนึ่งร้อยห้า" = 105).
83      // "เอ็ด" is always the units digit ("ร้อยเอ็ด" = 101), and so is "หนึ่ง" after ร้อย ("ร้อยหนึ่ง" = 101);
84      // after พัน or more it is the short form like any digit ("สองพันหนึ่ง" = 2,100).
85      const one = digitWord === 'เอ็ด' || digitWord === 'ับ' || (digitWord === 'หนึ่ง' && lastMultiplier === 100);
86      const short = !one && digitRightAfterMultiplier && (lastMultiplier >= 1000 || (lastMultiplier === 100 && !startsWithDigit));
87      total += short ? digit * (lastMultiplier / 10) : digit;
88    }
89    return millions + total;
90  }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/voice.ts:43

ussกัค ความหมาย

43–47 คอมเมนต์ตัวอย่าง: ห้าสิบ 50, ยี่สิบห้า 25, หนึ่งร้อยห้า 105, ร้อยห้า 150 (แบบพูดย่อ), สองพันห้า 2,500, ร้อยครึ่ง 150, พันับ 1,000, ร้อยหนึ่ง 101, ยี่สิบับ 21 · ไม่ใช่ตัวเลขคืน null

48–49 รับข้อความ หั้นเป็น "โทเคน" (คำตัวเลขทีละคำ) เช่น "สองพันห้า" → ["สอง", "พัน", "ห้า"]

บรรทัด	ความหมาย
50	ตรวจสอบว่าเป็นตัวเลขล้วน: ถ้าหั่นไม่ได้ หรือเอาโทเคนมาต่อกันแล้วไม่เท่ากับข้อความเดิม (มีตัวอักษรอื่นปน เช่น "ห้า" ได้แก่ ["ห้า"]) ต่อกันได้ "ห้า" ≠ "ห้า" → null
51–57	ตัวแปรสถานะ: total ผลรวม (ต่ำกว่าล้าน), millions ส่วนหลักล้าน, digit เลขที่รอคูณ, lastMultiplier คำหลักล่าสุด, startsWithDigit ขึ้นต้นด้วยเลขไหม, digitRightAfterMultiplier เลขอยู่หลังคำหลักทันทีไหม, digitWord คำของเลขล่าสุด
58	วนทีละโทเคน (tok) พร้อมลำดับ (i)
59	กรณีเป็นเลข (อยู่ใน DIGIT_WORDS)
60–62	ขอยกเว้น: "นึ่ง" ที่เป็นโทเคนสุดท้ายหลังร้อยขึ้นไป แปลว่า "หนึ่ง(ร้อย/พัน)" ไม่ใช่เลขเพิ่ม → ข้าม (return ในฟังก์ชันลูกศร = ไปโทเคนถัดไป) แต่หลังสิบ "นึ่ง" เป็นเลข (ยี่สิบหนึ่ง = 21)
63–64	จำนวนและคำของเลข
65	ถ้าเป็นโทเคนแรก จำว่าขึ้นต้นด้วยเลข
66	จำว่าเลขนี้อยู่หลังคำหลักทันทีไหม
67–68	ครึ่ง: บวกครึ่งหนึ่งของคำหลักล่าสุด (ไม่มีคำหลักก่อนหน้า = บวก 0)
69–73	ล้าน: เอาทุกอย่างก่อนหน้า (total + เลขที่ค้าง หรือ 1 ถ้าไม่มีอะไรเลย) คูณล้าน เก็บใน millions แล้วเริ่มนับส่วนที่เหลือใหม่ (ทำให้ "สามล้านห้าแสน" ได้)
74–79	คำหลักอื่น (สิบ ร้อย พัน หมื่น แสน): บวก (เลขที่ค้าง หรือ 1 ถ้าไม่มี) × ตัวคูณ ล้างเลข จำคำหลักล่าสุด — "ร้อย" ลอย ๆ จึงได้ 100
81	จบแล้วยังมีเลขค้าง (เลขตัวท้าย) ต้องตัดสินใจว่าเป็นหลักไหน
82–84	คอมเมนต์อธิบายกติกาแบบพูดย่อ (ดูบทหลักการ)
85	one = เป็นหลักหน่วยแน่นอน: "เอ็ด", "นึ่ง", หรือ "หนึ่ง" ที่ตามร้อย
86	short = แบบย่อ: ไม่ใช่ one และเลขอยู่หลังคำหลักทันที และ (คำหลัก ≥ พัน หรือ (คำหลักเป็นร้อย และประโยคไม่ได้ขึ้นต้นด้วยเลข))
87	แบบย่อ → เลข × (คำหลัก ÷ 10) · ไม่ใช่ → บวกเลขตรง ๆ
89	ผล = ส่วนหลักล้าน + ส่วนที่เหลือ

ไล่โค้ดทีละโทเคน (trace)

"สองพันห้า"

โทเคน	บรรทัดที่ทำงาน	digit	total	lastMultiplier	หมายเหตุ
สอง	63–66	2	0	0	startsWithDigit = true
พัน	75–78	null	2,000	1,000	2 × 1,000
ห้า	63–66	5	2,000	1,000	อยู่หลังคำหลักทันที

โทเคน	บรรทัดที่ทำงาน	digit	total	lastMultiplier	หมายเหตุ
(จบ)	85-87	short = true (ค่าหลัก $\geq 1,000$) $\rightarrow 2,000 + 5 \times 100 = \mathbf{2,500}$			

"หนึ่งร้อยห้า" vs "ร้อยห้า"

	หนึ่งร้อยห้า	ร้อยห้า
หลังโทเคน "ร้อย"	total = $1 \times 100 = 100$	total = $1 \times 100 = 100$ (ไม่มีเลข ใช้ 1)
startsWithDigit	true (ขึ้นต้นด้วย "หนึ่ง")	false
เลขท้าย "ห้า"	short = false $\rightarrow 100 + 5 = \mathbf{105}$	short = true $\rightarrow 100 + 5 \times 10 = \mathbf{150}$

"สามล้านห้าแสน": สาม $\rightarrow$ digit 3 $\cdot$ ล้าน $\rightarrow$ millions = $3 \times 1,000,000$ แล้วเริ่มใหม่ $\cdot$ ห้า $\rightarrow$ digit 5 $\cdot$ แสน $\rightarrow$ total = $500,000 \cdot$ ผล = $3,000,000 + 500,000 = \mathbf{3,500,000}$

★ ลองเองใน VS Code

เปิด `src/domain/__tests__/voice.test.ts:47` (TC-61) เพิ่มบรรทัด `expect(thaiNumberWords('สามร้อยยี่สิบเอ็ด')).toBe(321);` แล้วรัน `npm test -- voice` ถ้าผ่าน แปลว่าโค้ดเข้าใจตัวเลขนี้แล้ว ลองคิดก่อนว่าโค้ดจะไล่อย่างไร: สาม $\rightarrow$ ร้อย (300) $\rightarrow$ ยี่ $\rightarrow$ สิบ (320) $\rightarrow$ เอ็ด (one = true $\rightarrow +1$)

อ่านโค้ด: voice.ts ตอนที่ 2 — คำ หมวด และชื่อ

บรรทัด 92–153: รายการคำที่ใช้ตัดสินว่า 'เป็นจำนวน' 'เป็นรายรับ' 'เป็นหมวดอะไร' และวิธีทำความเข้าใจ

≈ Regular expression (regex) อ่านยังงี้

ส่วนนี้มี regex เยอะ ให้อ่านแบบนี้: | = หรือ · ^ = ต้นข้อความ · \$ = ท้ายข้อความ · \s = ช่องว่าง · * = กี่ตัวก็ได้ (0 ขึ้นไป) · + = 1 ตัวขึ้นไป · ? = มีหรือไม่มีก็ได้ · (?: ...) = จัดกลุ่ม · (?! ...) = "ถ้าข้างหลังไม่ใช่..." · (?= ...) = "ถ้าข้างหลังคือ..." (มองไปข้างหน้าโดยไม่กินตัวอักษร) · [ก-๙] = ตัวอักษรไทยตัวใดก็ได้

ส่วนที่ 1: หน่วย คำรายรับ คำจ่ายเงิน (บรรทัด 92–112)

src/domain/voice.ts

บรรทัด 92–112

```

92 // -----
93 // Words
94 // -----
95
96 /** Words after a number that make it a quantity, not money ("ข้าว 2 จาน"). */
97 const UNIT = 'จาน|แก้ว|ขวด|ชิ้น|อัน|ห่อ|ถุง|กล่อง|ลูก|คน|ที่|ตัว|คู่|เล่ม|ใบ|ครั้ง|โมง|นาที|ชั่วโมง|วัน|เดือน|ปี|ทศ|ก|ค|ก\.\.?|
  ลิตร|ชาม|ถ้วย|เบ็ด|แผ่น|ซอง|แพ็ค|หลอด|คืบ|รอบ';
98 const INCOME_WORDS = ['ได้เงิน', 'ได้รับ', 'รับเงิน', 'เงินเข้า', 'เงินเดือน', 'ค่าจ้าง', 'แม่ให้', 'พ่อให้', 'โอนมาให้',
  'โอนเข้า', 'รายรับ', 'รายได้', 'ขายของได้', 'ได้ค่าขนม', 'ได้ค่า', 'ได้เงินคืน', 'เงินคืน', 'คืนเงิน'];
99 /**
100 * Paying words, when they come before any income word: "จ่ายค่าจ้างช่างแอร์" is spending, but
101 * "แม่ให้เงินซื้อขนม" is still money in. "คืนเงิน" pays back only at the start ("คืนเงินเพื่อน");
102 * after someone's name it is money coming back ("เพื่อนคืนเงิน").
103 */
104 const PAYING = /^(?:คืนเงิน)|จ่าย|ซื้อ|โอนให้|โอนไป/;
105
106 function isIncome(text: string): boolean {
107   const t = text.trim().replace(CONNECTORS, '');
108   const incomeAt = Math.min(...INCOME_WORDS.map(w => t.indexOf(w)).filter(i => i >= 0),
  Infinity);
109   if (incomeAt === Infinity) return false;
110   const pay = PAYING.exec(t);
111   return !pay || pay.index > incomeAt;
112 }

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/voice.ts:92

บรรทัด	ความหมาย
96–97	UNIT คำหน่วยที่ทำให้เลขข้างหน้าเป็น "จำนวน" ไม่ใช่เงิน: จาน แก้ว ขวด ชิ้น ... รอบ (35 คำ) "ก\.\.?" = ก หรือ คก. (จุดมีหรือไม่มีก็ได้)
98	INCOME_WORDS คำที่บอกว่าเป็นรายรับ 18 คำ เช่น ได้เงิน ได้รับ เงินเดือน แม่ให้ โอนมาให้ ขายของได้ เงินคืน
99–103	คอมเมนต์: คำจ่ายเงินที่มาก่อนคำรายรับชนะ ("จ่ายค่าจ้างช่างแอร์" = รายจ่าย) แต่ "แม่ให้เงินซื้อขนม" ยังเป็นรายรับ · "คืนเงิน" เป็นการจ่ายเฉพาะเมื่อขึ้นต้นประโยค ("คืนเงินเพื่อน") ถ้าอยู่หลังชื่อคนคือเงินคืนเข้ามา ("เพื่อนคืนเงิน")

บรรทัด	ความหมาย
104	PAYING: "คืนเงิน" ที่ต้นข้อความ (^) หรือ จ่าย หรือ ชื้อ หรือ โอนให้ หรือ โอนไป (ที่ไหนก็ได้)
106	isIncome รับข้อความของรายการ คืน true/false
107	ตัดช่องว่างหัวท้าย และตัดคำเชื่อมต้นข้อความ (และ กับ ...)
108	หาตำแหน่งของคำรายรับทุกคำ (indexOf, ไม่เจอได้ -1 แล้วกรองทิ้ง) เอาตำแหน่งที่ น้อยที่สุด (คำรายรับตัวแรก) ไม่มีเลย = Infinity
109	ไม่มีคำรายรับ → รายจ่าย (false)
110	หาคำจ่ายเงินตัวแรก
111	รายรับเมื่อ: ไม่มีคำจ่าย หรือคำจ่ายอยู่ หลัง คำรายรับ

ข้อความ	คำรายรับแรกที่	คำจ่ายแรกที่	ผล
แม่ให้เงินซื้อขนม	"แม่ให้" ตำแหน่ง 0	"ซื้อ" ตำแหน่ง 10 (นับวรรณยุกต์และสระเป็น 1 ตัวด้วย)	$10 > 0 \rightarrow$ รายรับ
จ่ายค่าจ้างช่างแอร์	"ค่าจ้าง" ตำแหน่ง 4	"จ่าย" ตำแหน่ง 0	$0 < 4 \rightarrow$ รายจ่าย
คืนเงินเพื่อน	"คืนเงิน" ตำแหน่ง 0	"คืนเงิน" ที่ต้นข้อความ ตำแหน่ง 0	ไม่มากกว่า → รายจ่าย
เพื่อนคืนเงิน	"คืนเงิน" ตำแหน่ง 6	ไม่มี ("คืนเงิน" ไม่ได้อยู่ต้น)	รายรับ

ส่วนที่ 2: คำเชื่อม และคำลงท้าย (บรรทัด 113–120)

src/domain/voice.ts	บรรทัด 113–120
<pre> 113 // "กับข้าว" (side dishes) and "ต่อเติม" are words, not "and" + something. 114 const CONNECTORS = /^(?:และ กับ(?:!ข้าว แกล้ม) แล้ว ก็ แล้ว อีก ต่อ(?:!เติม ภาษี ทะเบียน) , \+)+\s*/; 115 /** 116 * Polite endings at the end of a word ("ขอบคุณ"); a lone "นะ" only as its own word (so "ชนะ" 117 stays). 118 * Not after a leading vowel, where the letters belong to a word: "ข้าวเจ้า" keeps its "เจ้า". 119 */ 119 const POLITE = /^(^ ^[^ไ้]) (?:นะคะ นะครับ ครับผม ครับ คับ ค่ะ คะ จะ จ้า) (?=\s \$)/g; 120 const LONE_NA = /\sนะ(?:=\s \$)/g; </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub	VS Code ในเครื่อง: Ctrl+P → src/domain/voice.ts:113

บรรทัด	ความหมาย
113–114	CONNECTORS คำเชื่อมที่ต้นข้อความ (เข้ากันได้หลายคำ ตามด้วยช่องว่าง): และ, กับ (ยกเว้น "กับข้าว" "กับแกล้ม"), แล้วก็, แล้ว, ก็, อีก, ต่อ (ยกเว้น "ต่อเติม" "ต่อภาษี" "ต่อทะเบียน"), จุลภาค, + — คอมเมนต์ย้าว่า "กับข้าว" เป็นคำ ไม่ใช่ "กับ" + "ข้าว"
115–118	คอมเมนต์: คำลงท้ายสุภาพท้ายคำ ("ขอบคุณ") ส่วน "นะ" ตัดเฉพาะเมื่อเป็นคำเดี่ยว ("ชนะ" ต้องอยู่) และไม่ตัดถ้าข้างหน้าเป็นสระหน้า (แ ไ ใ) เพราะตัวอักษรนั้นเป็นส่วนของคำ: "ข้าวเจ้า" ต้องเก็บ "เจ้า" ไว้
119	POLITE: (ต้นข้อความ หรือตัวที่ไม่ใช่สระหน้า) ตามด้วย นะคะ/นะครับ/ครับผม/ครับ/คับ/ค่ะ/คะ/จะ/จ้า แล้วต้องตามด้วย ช่องว่างหรือจบข้อความ

บรรทัด	ความหมาย
120	LONE_NA: ช่องว่าง + "นะ" + (ช่องว่างหรือจบ)

ส่วนที่ 3: หมวดจากคำพูด (บรรทัด 122–145)

src/domain/voice.ts	บรรทัด 122–145
<pre> 122 /** Spoken things and their category (longest match wins: "ค่าน้ำ" is a bill, "น้ำ" is a drink). */ 123 const SPOKEN_CATEGORY: [string, string][] = [124 ['น้ำมัน', 'transport'], ['ค่ารถ', 'transport'], ['รถเมล์', 'transport'], ['วินมอเตอร์ไซด์', 'transport'], 125 ['มอเตอร์ไซด์', 'transport'], 126 ['แท็กซี่', 'transport'], ['รถไฟ', 'transport'], ['ค่าทางด่วน', 'transport'], ['เรือ', 'transport'], 127 ['ข้าว', 'food'], ['น้ำ', 'food'], ['ขนม', 'food'], ['กาแฟ', 'food'], ['ชาวม', 'food'], ['ชาเย็น', 128 'food'], ['หมูกระทะ', 'food'], 129 ['กล้วยเตี้ยว', 'food'], ['ส้มตำ', 'food'], ['ไก่', 'food'], ['ผลไม้', 'food'], ['ไอติม', 'food'], 130 ['ไอศกรีม', 'food'], 131 ['มะพร้าว', 'food'], ['อาหาร', 'food'], ['มื่อ', 'food'], ['ข้าวเที่ยง', 'food'], ['ข้าว 132 เย็น', 'food'], 133 ['เซเว่น', 'convenience'], ['ร้านสะดวกซื้อ', 'convenience'], 134 ['เสื้อ', 'shopping'], ['รองเท้า', 'shopping'], ['กางเกง', 'shopping'], ['กระเป๋า', 'shopping'], 135 ['ของใช้', 'shopping'], ['เครื่องสำอาง', 'shopping'], 136 ['ค่าหอ', 'bills'], ['ค่าเช่า', 'bills'], ['ค่าไฟ', 'bills'], ['ค่าน้ำ', 'bills'], ['ค่าเน็ต', 'bills'], 137 ['ค่าโทรศัพท์', 'bills'], ['เติมเงินโทรศัพท์', 'bills'], ['ค่ามือถือ', 'bills'], 138 ['หนัง', 'fun'], ['เกม', 'fun'], ['คอนเสิร์ต', 'fun'], ['คาราโอเกะ', 'fun'], 139 ['หนังสือ', 'study'], ['ค่าเทอม', 'study'], ['ถ่ายเอกสาร', 'study'], ['ปริญ', 'study'], ['เครื่องเขียน', 140 'study'], 141 ['ค่ายา', 'health'], ['ร้านยา', 'health'], ['หาหมอ', 'health'], ['คลินิก', 'health'], ['ฟิตเนส', 142 'health'], 143 ['เงินเดือน', 'salary'], ['ค่าจ้าง', 'salary'], ['ค่าขนม', 'allowance'], ['แม่', 'allowance'], ['พ่อ', 144 'allowance'], ['ที่บ้าน', 'allowance'], 145 ['พรีแลนซ์', 'part_time'], ['พาร์ทไทม์', 'part_time'], ['ขายของ', 'part_time'], 146] 147 .sort((a, b) => b[0].length - a[0].length) as [string, string][]; 148 149 export function spokenCategory(title: string, kind: TxKind): string { 150 const allowed = new Set(categoriesFor(kind).map((c) => c.key)); 151 for (const [word, key] of SPOKEN_CATEGORY) { 152 if (allowed.has(key) && title.includes(word)) return key; 153 } 154 return suggestCategory(title, kind); 155 } </pre>	
▶ เปิดใน VS Code (เว็บ)	GitHub
VS Code ในเครื่อง: Ctrl+P → src/domain/voice.ts:122	

บรรทัด	ความหมาย
122	คอมเมนต์: คำที่พูดกับหมวด คำยาวชนะ ("ค่าน้ำ" เป็นบิล, "น้ำ" เป็นเครื่องดื่ม)
123–136	รายการ [คำ, หมวด] 67 คู่: เดินทาง (น้ำมัน ค่ารถ รถเมล์ วินมอเตอร์ไซด์ ...), อาหาร (ข้าว น้ำ ขนม กาแฟ ชาวม หมูกระทะ ส้มตำ มาม่า ...), สะดวกซื้อ (เซเว่น), ซุปปิง (เสื้อ รองเท้า ...), บิล (ค่าหอ ค่าไฟ ค่าน้ำ ค่าเน็ต ...), บันเทิง (หนัง เกม ...), เรียน (หนังสือ ค่าเทอม ปริญ ...), สุขภาพ (ค่ายา หาหมอ ...), รายรับ (เงินเดือน ค่าจ้าง → เงินเดือน; ค่าขนม แม่ พ่อ ที่บ้าน → เงินจากที่บ้าน; พรีแลนซ์ ขายของ → งานพิเศษ)
137	เรียงคำยาวก่อน (b ยาว – a ยาว) ครั้งเดียวตอนโหลดไฟล์ แล้วบอก TypeScript ว่าเป็นรายการของคู่ข้อความ
139	spokenCategory รับชื่อและประเภท

บรรทัด	ความหมาย
140	ชุดหมวดที่ใช้ได้กับประเภทนี้ (Set ไว้เช็ครวด)
141–143	วนคำยาวไปสั้น: ถ้าหมวดใช้ได้กับประเภทนี้ และชื่อมีคำนี้ → ได้หมวดนั้นเลย
144	ไม่เจอ → ให้ suggestCategory ของ FR-1 ช่วย (คำของร้านค้าและชื่อแบรนด์)

ชื่อ	ถ้าเป็นรายจ่าย	ถ้าเป็นรายรับ	อธิบาย
ค่าน้ำ	bills	transfer_in	"ค่าน้ำ" (6 ตัว) เทียบก่อน "น้ำ"
น้ำมัน	transport	transfer_in	"น้ำมัน" ยาวกว่า "น้ำ"
ค่าขนม	food	allowance	รายจ่าย: หมวดเงินจากที่บ้านใช้ไม่ได้ จึงไปเจอ "ขนม" = อาหาร
แม่	other_expense	allowance	"แม่" เป็นคำของรายรับเท่านั้น
ชาไข่มุก	food	transfer_in	ไม่อยู่ในคำพูดของ FR-3 แต่ suggestCategory ของ FR-1 มีคำ "ชาไข่มุก"
ค่าจ้างช่างแอร์	other_expense	salary	"ค่าจ้าง" เป็นหมวดรายรับ ถ้าเป็นรายจ่ายจึงไม่ใช้

ส่วนที่ 4: ทำความสะอาดชื่อ (บรรทัด 147–153)

src/domain/voice.ts

บรรทัด 147–153

```

147 function cleanTitle(raw: string, kind: TxKind): string {
148   let t = raw.trim().replace(CONNECTORS, ' ').replace(/\s+/g, ' ').trim();
149   t = t.replace(/^(?:ชื่อ|จ่ายเงิน|จ่าย(?:=ค่า)|จ่าย|กิน|เติม(?:!เงิน))\s*/ , ' ');
150   if (kind === 'income') t = t.replace(/^(?:ได้เงินคืน/, 'เงินคืน').replace(/^(?:ได้รับ|ได้เงิน|ได้|รับ)\s*/ ,
    '').replace(/^(จาก/, 'เงินจาก');
151   t = t.replace(/\s*(?:บาท|บ.)$/ , ' ').trim();
152   return t;
153 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/voice.ts:147

บรรทัด	ความหมาย
147	cleanTitle รับข้อความดิบของรายการและประเภท
148	ตัดช่องว่าง ตัดคำเชื่อมต้นข้อความ ช่องว่างหลายตัวเหลือตัวเดียว
149	ตัดคำกริยาต้นชื่อ: ชื่อ, จ่ายเงิน, จ่าย (เมื่อตามด้วย "ค่า"), จ่าย, กิน, เติม (แต่ไม่ตัดถ้าเป็น "เติมเงิน") — "ชื่อเสือ" → "เสือ", "เติมน้ำมัน" → "น้ำมัน", "เติมเงินโทรศัพท์" คงเดิม
150	รายรับ: "ได้เงินคืน" → "เงินคืน" · ตัด ได้รับ/ได้เงิน/ได้/รับ ต้นชื่อ · ขึ้นต้น "จาก" → "เงินจาก" ("ได้เงินจากแม่" → "จากแม่" → "เงินจากแม่")
151	ตัด "บาท" หรือ "บ." ท้ายชื่อ
152	คืนชื่อที่สะอาดแล้ว

อ่านโค้ด: voice.ts ตอนที่ 3 — ทั้งประโยค

บรรทัด 155–268: normalizeMarked (ทำให้เป็นมาตรฐาน), normalizeSpoken และ parseSpokenEntry (หัวใจของ FR-3)

ส่วนที่ 1: ป้ายวัน (บรรทัด 155–163)

src/domain/voice.ts

บรรทัด 155–163

```
155 // -----
156 // Whole sentence
157 // -----
158
159 const DIGIT_NAMES = ['ศูนย์', 'หนึ่ง', 'สอง', 'สาม', 'สี่', 'ห้า', 'หก', 'เจ็ด', 'แปด', 'เก้า'];
160 /** Day words become these markers while parsing, so each item keeps its own day. */
161 const DAY_MARK = { '-2': ' รวันขึ้น ', '-1': ' รวัน ', '0': ' รวันนี้ ' } as const;
162 const DAY_MARK_RE = /(รวันขึ้น|รวัน|รวันนี้)/g;
163 const markOffset = (m: string) => (m === 'รวันขึ้น' ? -2 : m === 'รวัน' ? -1 : 0);
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/voice.ts:155

บรรทัด	ความหมาย
159	ชื่อเลข 0–9 แบบคำ (ใช้แปลง "2 พันห้า" → "สองพันห้า")
160–161	DAY_MARK: คำบอกวันจะถูกแทนด้วยป้ายพิเศษระหว่างแยกประโยค รวันขึ้น (−2), รวัน (−1), รวันนี้ (0) — ใช้ตัว ร เพราะคนไม่พูด/ไม่พิมพ์ จึงไม่ชนกับคำจริง
162	รูปแบบหาป้าย
163	แปลงชื่อป้ายเป็นตัวเลขวัน

ส่วนที่ 2: normalizeMarked — ทำความสะอาดข้อความ (บรรทัด 165–186)

src/domain/voice.ts

บรรทัด 165–186

```
165 function normalizeMarked(text: string): string {
166   let t = ` ${text} `
167   .replace(/[๐-๙]/g, (d) => String('๐๑๒๓๔๕๖๗๘๙'.indexOf(d)))
168   .toLowerCase()
169   .replace(/(\d),(?=\d{3}\b)/g, '$1')
170   .replace(/(\d)\.?(?!\d)/g, '$1 ')
171   .replace(/[!?,]/g, ' ')
172   .replace(/฿\s*(\d)/g, '$1')
173   .replace(/(\d)\s*฿/g, '$1 บาท ')
174   // 7-Eleven, but never the "711" inside an amount ("ค่าไฟ 711", "1711").
175   // "7 11" or "711" only when an amount follows (or the text ends): "ค่าไฟ 711 บาท" stays ฿711.
176   .replace(
177     /(^|[\^d.,])7\s*\s*11(?:\d|(^|[\^d.,])7\s*\s*eleven|(^|[\^d.,])7\s*11(?:=\s*\d|\s*$)/g,
178     (_, a?: string, b?: string, c?: string) => `${a} ?? b ?? c ?? `}เซเวน`,
179   )
180   .replace(/เซเวนอีเลฟเว่น|เซเวน/g, 'เซเวน')
181   .replace(POLITE, '$1 ')
```

```

182     .replace(LONE_NA, ' ');
183     t = t
184     .replace(/เมื่อวานขึ้น/g, DAY_MARK['-2'])
185     .replace(/เมื่อวานนี้|เมื่อวาน/g, DAY_MARK['-1'])
186     .replace(/วันนี้|ตอนนี้/g, DAY_MARK['0']);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/voice.ts:165`

บรรทัด	ความหมาย
166	เติมช่องว่างหน้าและหลัง (ทำให้กฎ "คำอยู่เดี่ยว ๆ" ใช้ได้กับคำแรกและคำสุดท้าย)
167	เลขไทย ๐-๙ → 0-9 (หาตำแหน่งในสตริง '๐๑๒...๙')
168	ตัวเล็ก (7-ELEVEN → 7-eleven)
169	จุดทศนิยมระหว่างเลขที่ตามด้วย 3 หลักพอดี (หลักพัน) → ตัดทิ้ง: 1,250 → 1250
170	จุดหลังเลขที่ไม่ได้ตามด้วยเลข (จบประโยค) → ช่องว่าง: "50." → "50 "
171	!?, → ช่องว่าง
172	"฿50" → "50"
173	"50฿" → "50 บาท"
174-179	เซเว่น: แปลงเป็นคำว่าเซเว่นเมื่อเป็น "7-11", "7 eleven"/"7-eleven", หรือ "7 11"/"711" ที่ตามด้วยจำนวนเงินหรือ จบข้อความ และต้องไม่ติดกับเลขข้างหน้า — "ค่าไฟ 711 บาท" ไม่แปลง (ตามด้วย "บาท" ไม่ใช่ตัวเลข) "1711" ไม่แปลง (มีเลขนำหน้า)
180	"เซเว่นอีเลฟเว่น" → "เซเว่น"
181-182	ตัดคำลงท้ายสุภาพ (เก็บตัวอักษรนำหน้าไว้ด้วย \$1) และ "นะ" เดี่ยว
183-186	คำบอกวัน → ป้าย: เมื่อวานขึ้น ก่อน (ยาวกว่า ไม่ขึ้น "เมื่อวาน" จะกินไปก่อน) แล้ว เมื่อวานนี้/เมื่อวาน แล้ว วันนี้/ตอนนี้

`src/domain/voice.ts`

บรรทัด 187-217

```

187 // "2k", "2 เค" -> 2000
188 t = t.replace(/(\d+(?:\.\d+)?)\s*(?:k|เค)(?=\s|$|บาท)/g, (_, n: string) =>
String(Math.round(Number(n) * 1000)));
189 // One digit before a multiplier joins the number words after it: "2 พันห้า" = สองพันห้า = 2,500,
190 // "5 ร้อยห้าสิบ" = 550.
191 t = t.replace(/(^|[\^d.]) (\d)\s*(?=\สิบ|ร้อย|พัน|หมื่น|แสน|ล้าน)/g, (_, pre: string, d: string) =>
` ${pre} ${DIGIT_NAMES[Number(d)]} `);
192 // "สามพัน สองร้อย" is one number when the second part is smaller: 3,200.
193 t = t.replace(
194   new RegExp(`(\สิบ|ร้อย|พัน|หมื่น|แสน|ล้าน)\s+(?=(?:${Object.keys(DIGIT_WORDS).join('|')})(\สิบ|ร้อย|พัน|
หมื่น|แสน))`, 'g'),
195   (m, a: string, next: string) => (MULTIPLIERS[next] < MULTIPLIERS[a] ? a : m),
196 );
197 // "15 ร้อย", "1.5 พัน", "2 พันครึ่ง" -> digits
198 t = t.replace(/(\d+(?:\.\d+)?)\s*(\สิบ|ร้อย|พัน|หมื่น|แสน|ล้าน)(\s*ครึ่ง)?/g, (_, n: string, w: string,
half?: string) => {
199   const m = MULTIPLIERS[w];
200   return ` ${Math.round(Number(n) * m + (half ? m / 2 : 0))} `;
201 });

```

```

202 // Thai number words become digits only when they are clearly a number: on their own, followed
    by
203 // "บาท", or a multi-part number at the end of a word ("ห้าห้าสิบลบาท"). So "ห้า" or "สี่แสน" stay words.
204 t = t.replace(NUMBER_WORDS_RE, (words, offset: number, whole: string) => {
205     const before = whole[offset - 1] ?? ' ';
206     const after = whole.slice(offset + words.length);
207     const endsHere = /^(?:\s|$)/.test(after);
208     const alone = /\s/.test(before) && endsHere;
209     const money = /\s*(?:บาท|บ\.)/.test(after);
210     const n = thaiNumberWords(words);
211     if (n === null || n === 0) return words;
212     const multiPart = (words.match(new RegExp(NUMBER_WORD, 'g')) ?? []).length >= 2;
213     if (!alone && !money && !(endsHere && multiPart)) return words;
214     return ` ${n} `;
215 });
216 return t.replace(/s+/g, ' ').trim();
217 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/voice.ts:187

บรรทัด	ความหมาย
187–188	"2k", "2 เค" (ตามด้วยช่องว่าง/จบ/บาท) → เลข × 1,000: "2k" → "2000", "1.5k" → "1500"
189–191	เลข 1 หลักหน้าคำหลัก → คำของเลขนั้น: "2 พันห้า" → "สองพันห้า" (จะได้อ่านแบบพุดย่อ = 2,500), "5 ร้อยห้าสิบลบาท" → "ห้าร้อยห้าสิบลบาท" (ต้องไม่ใช่ส่วนของเลขที่ยาวกว่า)
192–196	สองคำที่ต่อกันได้: "สามพัน สองร้อย" ลบช่องว่างกลางเมื่อคำหลักตัวหลัง เล็กกว่า ตัวหน้า (ร้อย < พัน) → "สามพัน สองร้อย" = 3,200 · ถ้าตัวหลังใหญ่กว่า ("ห้าสิบล สองร้อย") เป็น 2 จำนวน
197–201	เลข × คำหลัก (+ครั้ง): "15 ร้อย" → 1500, "1.5 พัน" → 1500, "2 พันครั้ง" → 2000 + 500 = 2500 (ปิดเป็นจำนวนเต็ม)
202–203	คอมเมนต์: คำตัวเลขไทยเปลี่ยนเป็นเลขเฉพาะเมื่อชัดว่าเป็นตัวเลข: อยู่เดี่ยว ๆ, ตามด้วย "บาท", หรือเป็นตัวเลขหลายส่วนที่จบคำ ("ห้าห้าสิบลบาท") ดังนั้น "ห้า" "สี่แสน" ยังเป็นคำ
204	หาคำตัวเลขทุกกลุ่ม ได้ (คำ, ตำแหน่ง, ข้อความทั้งหมด)
205–206	ตัวอักษรก่อนหน้า และข้อความหลังกลุ่มนี้
207	จบคำพอดีไหม (หลังเป็นช่องว่างหรือจบ)
208	อยู่เดี่ยว ๆ = หน้าเป็นช่องว่าง และจบคำ
209	ตามด้วย "บาท" หรือ "บ." ไหม
210–211	แปลงด้วย thaiNumberWords ถ้าไม่ใช่ตัวเลข หรือเป็น 0 → คงคำเดิม
212	มีกี่ส่วน ("ห้าสิบล" = 2, "ห้า" = 1)
213	ถ้าไม่อยู่เดี่ยว ไม่ตามด้วยบาท และไม่ใช้ (จบคำและหลายส่วน) → คงคำเดิม
214	ผ่านแล้ว → แทนด้วยเลขมีช่องว่างล้อม: "ห้าห้าสิบลบาท" → "ห้า 50 "
216	ช่องว่างหลายตัวเหลือตัวเดียว ตัดหัวท้าย

ส่วนที่ 3: normalizeSpoken (บรรทัด 219–224)

src/domain/voice.ts

บรรทัด 219–224

```
219 /** Numbers as digits: Thai digits, number words, "5 ร้อย", "2k", "1,250". dayOffset = the first  
day word said. */  
220 export function normalizeSpoken(text: string): { text: string; dayOffset: number } {  
221   const marked = normalizeMarked(text);  
222   const first = marked.match(/$(วนขึ้น|วน|วันนี้)$/);  
223   return { text: marked.replace(DAY_MARK_RE, ' ').replace(/\\s+/g, ' ').trim(), dayOffset: first ?  
     markOffset(first[1]) : 0 };  
224 }
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/voice.ts:219

บรรทัด	ความหมาย
219–221	ฟังก์ชันที่ export สำหรับใช้ภายนอก/เทสต์: ทำความสะอาดแบบมีป้าย
222	หาป้ายวันตัวแรก
223	คืนข้อความที่ลบป้ายออกแล้ว และวันของป้ายแรก (ไม่มี = 0)

ส่วนที่ 4: parseSpokenEntry – หาจำนวนเงิน (บรรทัด 226–241)

src/domain/voice.ts

บรรทัด 226–241

```
226 /** Understand one spoken (or typed) sentence. Items with no amount are left out. */  
227 export function parseSpokenEntry(input: string): SpokenEntry {  
228   const text = normalizeMarked(input);  
229   const marks = [...text.matchAll(DAY_MARK_RE)].map((m) => ({ at: m.index ?? 0, offset:  
     markOffset(m[1]) }));  
230   // A number, whole (never part of a longer one), not followed by a unit word ("2 จาน").  
231   const amountRe = new RegExp(`(\\d+(?:\\.\\d{1,2})?)(?!\\d\\.)(?!\\s*(?:${UNIT})(?![ก-๙]))  
     (?:\\s*(?:บาท|บ\\.)?(?:\\s*(\\d{1,2})\\s*สตางค์)?`, 'g');  
232   const found: { start: number; end: number; satang: number }[] = [];  
233   for (const m of text.matchAll(amountRe)) {  
234     // A number glued to latin letters or following another digit is not an amount on its own.  
235     const prev = text[(m.index ?? 0) - 1];  
236     if (prev && /[a-z0-9.]/.test(prev)) continue;  
237     const baht = Number(m[1]);  
238     const satang = Math.round(baht * 100) + (m[2] ? Number(m[2]) : 0);  
239     if (satang <= 0 || satang >= 1_000_000_000) continue;  
240     found.push({ start: m.index ?? 0, end: (m.index ?? 0) + m[0].length, satang });  
241   }
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/voice.ts:226

บรรทัด	ความหมาย
226–228	เข้าใจ 1 ประโยค รายการที่ไม่มีจำนวนเงินจะถูกข้าม · ทำความสะอาดแบบมีป้ายวัน
229	เก็บตำแหน่งและวันของป้ายทุกตัว เช่น [{at: 0, offset: -1}, {at: 15, offset: 0}]

บรรทัด	ความหมาย
230–231	amountRe รูปแบบจำนวนเงิน อ่านทีละก่อน: $(\backslash d+(?:\backslash .\backslash d\{1,2\})?)$ เลข (ทศนิยม 1–2 ตำแหน่งได้) $\cdot (?![\backslash d.])$ ไม่ใช่ส่วนของเลขที่ยาวกว่า $\cdot (?!\\s*(?:UNIT)(?![ก-๙]))$ ต้องไม่ตามด้วยคำหน่วยที่จบตรงนั้น ("ข้าว 2 จาน 100" $\rightarrow$ 2 ไม่ใช่เงิน) ถ้าหลังคำหน่วยยังมีตัวอักษรไทยต่อ ถือว่าไม่ใช่หน่วย เพื่อให้ "ข้าว 50 ที่ร้านป่า" ยังเป็นเงิน ๕0 (คำว่า "ที่" ในที่นี้ไม่ใช่หน่วย) $\cdot (?:\\s*(?:บาท บ\backslash .))?$ บาทมีหรือไม่มีก็ได้ $\cdot (?:\\s*(\backslash d\{1,2\})\\s*\text{สตางค์})?$ สตางค์มีหรือไม่มีก็ได้
232	รายการจำนวนเงินที่เจอ: ตำแหน่งเริ่ม ตำแหน่งจบ สตางค์
233	วนทุกจุดที่ตรงรูปแบบ
234–236	ถ้าตัวก่อนหน้าเป็นตัวอังกฤษ เลข หรือจุด ("a7", "1.5" ส่วนหลัง) ไม่นับ
237–238	บาท $\times$ 100 (ปิด) + สตางค์ที่พูดแยก
239	ต้องมากกว่า 0 และน้อยกว่า 1,000,000,000 สตางค์ (฿10 ล้าน)
240	เก็บไว้

ส่วนที่ 5: parseSpokenEntry – แบ่งชื่อและสร้างรายการ (บรรทัด 242–268)

src/domain/voice.ts

บรรทัด 242–268

```

242 const bare = (s: string) => s.replace(DAY_MARK_RE, ' ').trim().replace(CONNECTORS, ' ').trim();
243 // Amounts said first ("50 บาท ค่าข้าว 30 บาท ค่าน้ำ"): every title comes after its amount.
244 const amountFirst = found.length > 0 && !bare(text.slice(0, found[0].start));
245 const items: SpokenItem[] = [];
246 found.forEach((f, i) => {
247   let title = amountFirst
248     ? text.slice(f.end, found[i + 1]?.start ?? text.length)
249     : text.slice(i === 0 ? 0 : found[i - 1].end, f.start);
250   // Only the last amount has nothing before it ("ข้าว 50 บาท 30 ค่าน้ำ"): take the words after it.
251   if (!amountFirst && !bare(title) && i === found.length - 1) title = text.slice(f.end);
252   title = title.replace(DAY_MARK_RE, ' ');
253   const lower = title.toLowerCase();
254   const kind: TxKind = isIncome(lower) ? 'income' : 'expense';
255   const clean = cleanTitle(title, kind);
256   const categoryKey = spokenCategory(clean, kind);
257   // The day said last before this item's amount; if none was said before it, the first one
   said.
258   const before = marks.filter((m) => m.at < f.end).pop();
259   items.push({
260     title: clean || getCategory(categoryKey).label,
261     amountSatang: f.satang,
262     kind,
263     categoryKey,
264     dayOffset: before?.offset ?? marks[0]?.offset ?? 0,
265   });
266 });
267 return { items, heard: text.replace(DAY_MARK_RE, ' ').replace(/\\s+/g, ' ').trim() };
268 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P $\rightarrow$ src/domain/voice.ts:242

บรรทัด	ความหมาย
242	bare: ลบป้ายวันและคำเชื่อมออก ใช้ใช้คำว่า "มีคำจริง ๆ โหม"
243–244	amountFirst: มีจำนวนเงิน และก่อนจำนวนแรกไม่มีคำอะไรเลย ("50 บาท ค่าข้าว ...") → ชื่ออยู่หลังเลข
245–246	รายการผลลัพธ์ แล้ววนทีละจำนวนเงิน
247–249	ชื่อของจำนวนนี้: ถ้า amountFirst = ข้อความจากท้ายเลขนี้ถึงต้นเลขถัดไป · ไม่ขึ้น = ข้อความจากท้ายเลขก่อนหน้า (หรือต้นประโยค) ถึงต้นเลขนี้
250–251	กรณีพิเศษ: เลขตัวสุดท้ายไม่มีคำข้างหน้า ("ข้าว 50 บาท 30 ค่าน้ำ") → ใช้คำหลังมันแทน
252	ลบป้ายวันออกจากชื่อ
253–254	ตัวเล็ก แล้วตัดสิ้นรายรับ/รายจ่ายจากชื่อของรายการนี้เอง (ประโยคเดียวจึงมีทั้งรายรับและรายจ่ายได้)
255–256	ทำความเข้าใจชื่อ แล้วหาหมวด
257–258	วันของรายการนี้: ป้ายตัวสุดท้ายที่อยู่ก่อนท้ายเลขนี้
259–265	สร้างรายการ: ชื่อ (ว่าง → ชื่อหมวด), สตางค์, ประเภท, หมวด, วัน (ไม่มีป้ายก่อนหน้า → ป้ายตัวแรก → 0)
267	คืนรายการ และ heard = ข้อความที่ลบป้ายแล้ว (ใช้แสดงบรรทัดสีทอง)

ไล่โค้ดทั้งประโยค: "เมื่อวานค่ารถ 40 วันนี้ข้าว 50"

ขั้น	คำ (รันจริง)
หลัง normalizeMarked	"รู้วนรู้ ค่ารถ 40 รู้วันนี้รู้ ข้าว 50"
marks (บรรทัด 229)	วน ที่ตำแหน่ง 0 (−1) · วันนี้ ที่ตำแหน่ง 15 (0)
found (บรรทัด 233–241)	"40" ตำแหน่ง 12–14 = 4,000 สตางค์ · "50" ตำแหน่ง 29–31 = 5,000 สตางค์
amountFirst	ก่อน "40" มี "ค่ารถ" → false
รายการที่ 1	ชื่อ = ข้อความ 0–12 → "ค่ารถ" · ป้ายก่อนตำแหน่ง 14 = วน → ค่ารถ ฿40 เดินทาง เมื่อวาน
รายการที่ 2	ชื่อ = ข้อความ 14–29 → "ข้าว" · ป้ายก่อนตำแหน่ง 31 ตัวสุดท้าย = วันนี้ → ข้าว ฿50 อาหาร วันนี้
heard	"ค่ารถ 40 ข้าว 50"

ส่วน "ข้าว 50 น้ำ 15 เมื่อวาน": ป้ายวนอยู่ตำแหน่ง 15 หลังทั้งสองจำนวน → ไม่มีป้ายก่อนจำนวนไหนเลย → ใช้ป้ายตัวแรก → ทั้งสองรายการเป็นเมื่อวาน (TC-80)

อ่านโค้ด: หน้าจอ voice.tsx ตอนที่ 1 — ความจำและการกระทำ

ไฟล์ src/app/voice.tsx 346 บรรทัด บทนี้อ่านบรรทัด 1–178

ส่วนที่ 1: คอมเมนต์ import และค่าคงที่ (บรรทัด 1–43)

src/app/voice.tsx

บรรทัด 1–43

```
1 /**
2  * "พูด" (FR-1 by voice): tap the mic and say "ข้าว 50 บาท"; MindPay turns it
3  * into transactions (several in one sentence, income, yesterday), shows them
4  * for a quick look, and saves them with one tap. Where the phone or browser
5  * cannot listen, the same sentence can be typed (the keyboard's own mic works).
6  * Parsing: src/domain/voice.ts. Listening: src/services/speech(.web).ts.
7  */
8 import * as Haptics from 'expo-haptics';
9 import { LinearGradient } from 'expo-linear-gradient';
10 import { useEffect, useMemo, useRef, useState } from 'react';
11 import { Animated, Easing, KeyboardAvoidingView, Linking, Platform, Pressable, ScrollView,
12   TextInput, View } from 'react-native';
13 import { SafeAreaView } from 'react-native-safe-area-context';
14 import { useApp } from '../data/AppProvider';
15 import { VOICE_NOTE } from '../domain/achievements';
16 import { BUDDY_NAME } from '../domain/buddy';
17 import { getCategory } from '../domain/categories';
18 import { addDays, bkkDayKey, bkkTime, bkkToIso, relativeDayLabel } from '../domain/dates';
19 import { formatBaht } from '../domain/money';
20 import type { TxKind } from '../domain/types';
21 import { parseSpokenEntry, spokenCategory, type SpokenItem } from '../domain/voice';
22 import { askMicPermission, listen, speechAvailable } from '../services/speech';
23 import { BuddySays } from '../ui/Buddy';
24 import { Button, Card, Chip, IconButton, Ionicons, Money, Row, T } from '../ui/components';
25 import { inputBox, inputText } from '../ui/inputs';
26 import { Aurora, PulseRing, Reveal, useCelebrate, usePressSpring } from '../ui/effects';
27 import { useToast } from '../ui/feedback';
28 import { useNative, useReduceMotion } from '../ui/motion';
29 import { goBack, useLeaveWhenDone } from '../ui/nav';
30 import { fonts, radius, space, useTheme } from '../ui/theme';
31 import { HeaderDecor } from '../ui/halloween';
32
33 const EXAMPLES = ['ข้าว 50 บาท', 'ค่าธรรมเนียม 25 บาท', 'ได้เงินจากแม่ 500', 'เมื่อวาน หนูกระเพาะ 299'];
34
35 const SPEECH_ERRORS: Record<string, string> = {
36   'not-allowed': 'ต้องอนุญาตให้ใช้ไมโครโฟนก่อนนะ',
37   'no-speech': 'ไม่ได้ยินเสียง ลองแตะไมค์แล้วพูดอีกครั้งนะ',
38   'speech-timeout': 'ไม่ได้ยินเสียง ลองแตะไมค์แล้วพูดอีกครั้งนะ',
39   network: 'ต้องต่ออินเทอร์เน็ตเพื่อแปลงเสียงเป็นข้อความ',
40   'audio-capture': 'ใช้ไมโครโฟนไม่ได้ตอนนี้ ลองใหม่อีกครั้ง',
41   'language-not-supported': 'เครื่องนี้ยังฟังภาษาไทยไม่ได้ พิมพ์แทนได้เลย',
42   'service-not-allowed': 'เครื่องนี้ยังไม่มีการแปลงเสียง พิมพ์แทนได้เลย',
43   busy: 'ระบบฟังเสียงกำลังทำงานอยู่ ลองใหม่อีกสักครู่',
44 }
```

43 };

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/voice.tsx:1`

บรรทัด	ความหมาย
1–7	คอมเมนต์: "พุดจืด" (FR-1 ด้วยเสียง) และไม้พุด "ข้าว 50 บาท" แอปเปลี่ยนเป็นรายการ (หลายรายการ รายรับ เมื่อวาน) ให้ดูเว็บหนึ่ง แล้วบันทึกด้วยการแตะครั้งเดียว ที่ที่ฟังไม่ได้ก็พิมพ์ได้ (ไม้ของคีย์บอร์ดใช้ได้) · การเข้าใจอยู่ที่ voice.ts · การฟังอยู่ที่ speech(.web).ts
8–12	สั่นเครื่อง, ฟันไล่สี, hook, ชิ้นส่วน React Native (ภาพเคลื่อนไหว, เส้นโค้งเวลา, หลบคีย์บอร์ด, Linking เปิดหน้าต่างค่า, Platform, ปุ่ม, เลื่อน, ช่องพิมพ์, กล้อง), กับรอยบาก
13	useApp: addTx และวันนี้
14	VOICE_NOTE = "จดด้วยเสียง" ใส่ในบันทึกของทุกรายการจากหน้านี้
15–18	ชื่อน้องกล้า, หาหมวด, ฟังก์ชันวันเวลา, แสดงเงิน
19–20	ชนิด TxKind และ ตัวเข้าใจประโยค (parseSpokenEntry, spokenCategory, ชนิด SpokenItem)
21	บริการฟังเสียง 3 ตัว : ขอสิทธิ์ไมค์, ฟัง, มีตัวฟังใหม่ (Expo เลือกไฟล์ .web.ts ให้เองบนเว็บ)
22–30	น้องกล้าพูด, ชิ้นส่วน Design System, หน้าตาช่องกรอก, เอฟเฟกต์, toast, การตั้งค่าแอนิเมชัน, ออกจากหน้า, ธีม, ของตกแต่ง
32	ชิปตัวอย่าง 4 ประโยค ครอบคลุม: รายการเดียว, 2 รายการ, รายรับ, เมื่อวาน
34–43	SPEECH_ERRORS ตารางแปลงรหัสข้อผิดพลาดของระบบฟังเสียง → ประโยคภาษาไทยที่บอกว่าต้องทำอะไร (ดูตารางในบทกวีรหน้าจ่อ)

ส่วนที่ 2: แท่งเสียงที่เต้นตามเสียงพูด (บรรทัด 45–78)

src/app/voice.tsx

บรรทัด 45–78

```

45 /** Five bars that dance with the voice while listening. */
46 function VoiceBars({ level, active }: { level: Animated.Value; active: boolean }) {
47   const theme = useTheme();
48   const reduce = useReduceMotion();
49   const [wave] = useState(() => new Animated.Value(0));
50   useEffect(() => {
51     if (!active || reduce) return;
52     const loop = Animated.loop(Animated.timing(wave, { toValue: 1, duration: 900, easing:
Easing.linear, useNativeDriver: useNative }));
53     loop.start();
54     return () => loop.stop();
55   }, [active, reduce, wave]);
56   return (
57     <Row gap={6} style={{ height: 36 }} align="center">
58       {[0, 1, 2, 3, 4].map((i) => {
59         const phase = Animated.modulo(Animated.add(wave, i * 0.18), 1);
60         const bounce = phase.interpolate({ inputRange: [0, 0.5, 1], outputRange: [0.35, 1, 0.35]
});
61         const loud = level.interpolate({ inputRange: [0, 1], outputRange: [0.6, 1.6],
extrapolate: 'clamp' });
62         return (
63           <Animated.View

```

```

64         key={i}
65         style={{
66             width: 6,
67             height: 30,
68             borderRadius: 3,
69             backgroundColor: i % 2 ? theme.heroAccent : theme.heroInkSoft,
70             opacity: active ? 1 : 0.35,
71             transform: [{ scaleY: active && !reduce ? Animated.multiply(bounce, loud) : 0.35
        }],
72     }}
73     />
74 );
75 }}}
76 </Row>
77 );
78 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/voice.tsx:45

บรรทัด	ความหมาย
45–46	VoiceBars แท่ง 5 แท่งที่เดินตามเสียง รับ level (ความดัง 0–1 เป็นค่าภาพเคลื่อนไหว) และ active (กำลังฟังไหม)
47–49	สีรัศมี, ลดการเคลื่อนไหวไหม, ค่าคลื่น (0→1 วงซ้ำ)
50–55	เมื่อกำลังฟัง (และไม่ลดการเคลื่อนไหว) เล่นคลื่น 0→1 ใน 0.9 วินาทีวนไม่รู้จบ หยุดเมื่อเลิกฟัง
57–58	แถวสูง 36 วงสร้าง 5 แท่ง
59	แต่ละแท่งเหลื่อมเฟสกัน 0.18 (modulo 1 = วนกลับ 0) แท่งจึงขึ้นลงไม่พร้อมกันเหมือนคลื่น
60	เฟส → ความสูง 35% → 100% → 35%
61	ความดัง 0 → คูณ 0.6, ดัง 1 → คูณ 1.6 (พูดดังแท่งสูง)
63–72	แท่งกว้าง 6 สูง 30 มุมมน สีสลับทอง/ขาวจาง จางเมื่อไม่ฟัง ยัด-หดแนวตั้ง = คลื่น × ความดัง (ไม่ฟังหรือลดการเคลื่อนไหว = 35% 닝)

ส่วนที่ 3: ความจำของหน้า และรายการที่จะบันทึก (บรรทัด 80–114)

src/app/voice.tsx

บรรทัด 80–114

```

80 export default function VoiceEntry() {
81     const theme = useTheme();
82     const toast = useToast();
83     const leaveWhenDone = useLeaveWhenDone();
84     const celebrate = useCelebrate();
85     const { addTx, today } = useApp();
86     const [canListen] = useState(() => speechAvailable());
87     const [listening, setListening] = useState(false);
88     const [text, setText] = useState('');
89     const [notice, setNotice] = useState<string | null>(null);
90     const [removed, setRemoved] = useState<Set<string>>(new Set());
91     const [kinds, setKinds] = useState<Record<string, TxKind>>({});
92     const [busy, setBusy] = useState(false);
93     const [level] = useState(() => new Animated.Value(0));
94     const stop = useRef<() => void> | null>(null);
95     const mic = usePressSpring(0.92);

```

```

96
97 useEffect(() => () => stop.current?.(), []);
98
99 const parsed = useMemo(() => parseSpokenEntry(text), [text]);
100 const keyOf = (item: SpokenItem, i: number) => `${i}:${item.title}:${item.amountSatang}`;
101 const items = parsed.items
102   .map((item, i) => {
103     const key = keyOf(item, i);
104     const kind = kinds[key] ?? item.kind;
105     // Switched between income and expense: pick a category of the new kind.
106     return { key, item: kind === item.kind ? item : { ...item, kind, categoryKey:
spokenCategory(item.title, kind) } };
107   })
108   .filter(({ key }) => !removed.has(key));
109
110 function changeText(t: string) {
111   setText(t);
112   setRemoved(new Set());
113   setKinds({});
114 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/voice.tsx:80

บรรทัด	ความหมาย
80–85	component หลัก: ธีม, toast, ออกจากหน้าหลังเสร็จ, พลุ่, และ addTx กับวันนี้จากข้อมูลกลาง
86	canListen ถามครั้งเดียวตอนเปิดหน้าว่าเครื่องนี้ฟังเสียงได้ไหม
87	กำลังฟังอยู่ไหม
88	text ข้อความ (จากเสียงหรือที่พิมพ์) — แหล่งเดียวของทุกอย่างในหน้านี้
89	ข้อความแจ้งเตือน (เช่น ไม่ได้ยินเสียง)
90	removed ชุดกฎแฉของรายการที่ผู้ใช้กด ✕
91	kinds ประเภทที่ผู้ใช้สลับเอง (กฎแฉ → income/expense)
92–95	กำลังบันทึก, ค่าความดัง, ฟังก์ชันหยุดฟัง (เก็บใน ref), สปริงของปุ่มไมค์
97	ปิดหน้าแล้ว → หยุดฟัง (ไม่ปล่อยไมค์ค้าง)
99	parsed = parseSpokenEntry(text) คำนวณใหม่ทุกครั้งที่ข้อความเปลี่ยน (ระหว่างพูดก็เห็นรายการขึ้นทีละนิด)
100	กฎแฉของรายการ = ลำดับ: ชื่อ:จำนวน ใช้จำว่าผู้ใช้สลับ/ลบรายการไหน
101–104	วนรายการที่เข้าใจได้ หาประเภท: ถ้าผู้ใช้สลับไว้ใช้คำนั้น ไม่จันใช้ที่เดาได้
105–106	ถ้าสลับประเภท → คัดลอกรายการ เปลี่ยนประเภท และ หาหมวดใหม่ ของประเภทใหม่
108	ตัดรายการที่ผู้ใช้กด ✕ ออก
110–114	changeText : ข้อความใหม่ → ล้างการลบและการสลับทั้งหมด (เพราะรายการเปลี่ยนแล้ว กฎแฉเก่าใช้ไม่ได้)

i ทำไมไม่เก็บ 'รายการ' ไว้ใน state

หน้านี้เก็บแค่ ข้อความ + การแก้ไขของผู้ใช้ (removed, kinds) ส่วนรายการคำนวณจากข้อความทุกครั้ง ถ้าผู้ใช้แก้ไขข้อความในช่องพิมพ์ รายการก็ตามกันไปด้วยไม่ต้องเขียนโค้ดซิงค์ นี่คือหลัก "เก็บแหล่งความจริงให้น้อยที่สุด"

ส่วนที่ 4: แท็บ (บรรทัด 116–141)

src/app/voice.tsx

บรรทัด 116–141

```
116 async function toggleListening() {
117   if (listening) {
118     stop.current?.();
119     return;
120   }
121   const perm = await askMicPermission();
122   if (perm !== 'granted') {
123     setNotice(perm === 'blocked' ? 'ปิดสิทธิ์ไมโครโฟนไว้ เปิดได้ในการตั้งค่าของมือถือ' : SPEECH_ERRORS['not-allowed']);
124     return;
125   }
126   Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light).catch(() => {});
127   setNotice(null);
128   setListening(true);
129   stop.current = listen({
130     onText: (t) => changeText(t),
131     onError: (code) => {
132       setListening(false);
133       if (code !== 'aborted') setNotice(SPEECH_ERRORS[code] ?? 'ฟังเสียงไม่สำเร็จ ลองอีกครั้ง หรือพิมพ์แทนได้เลย');
134     },
135     onEnd: () => {
136       setListening(false);
137       level.setValue(0);
138     },
139     onVolume: (v) => level.setValue(v),
140   });
141 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/voice.tsx:116

บรรทัด	ความหมาย
116–120	toggleListening: ถ้ากำลังฟังอยู่ → สิ้นหยุด (ข้อความที่ได้ยินแล้วยังอยู่) แล้วจบ
121	ขอสิทธิ์ไมโครโฟน (ระบบจะถามผู้ใช้แค่ครั้งแรก ครั้งต่อไปตอบทันที)
122–125	ไม่ได้รับอนุญาต: ถ้าปิดถาวร (blocked) → "ปิดสิทธิ์ไมโครโฟนไว้ เปิดได้ในการตั้งค่าของมือถือ" ไม่งั้น → "ต้องอนุญาตให้ใช้ไมโครโฟนก่อนนะ" แล้วจบ
126–128	สั่นเบา ล้างข้อความแจ้ง ตั้งว่ากำลังฟัง
129–130	เริ่มฟัง เก็บฟังก์ชันหยุดไว้ · ได้ข้อความ → changeText (ทางเดียวกับการพิมพ์)
131–134	ข้อผิดพลาด: เลิกฟัง ถ้าไม่ใช้การยกเลิกเอง (aborted) แสดงข้อความไถ่จากตาราง (ไม่รู้จักรหัส → ข้อความกลาง)

บรรทัด	ความหมาย
135–138	ฟังจบ: เลิกฟัง และรีเซ็ตความดัง
139	ความดังเปลี่ยน → ส่งให้แก่งเสียง

ส่วนที่ 5: บันทึกทั้งหมด (บรรทัด 143–178)

src/app/voice.tsx

บรรทัด 143–178

```

143  async function saveAll() {
144    if (!items.length || busy) return;
145    setBusy(true);
146    try {
147      const time = bkkTime(new Date());
148      let saved = 0;
149      for (const { item, key } of items) {
150        await addTx({
151          kind: item.kind,
152          amountSatang: item.amountSatang,
153          categoryKey: item.categoryKey,
154          title: item.title,
155          note: VOICE_NOTE,
156          occurredAt: bkkToIso(addDays(today || bkkDayKey(new Date()), item.dayOffset), time),
157          source: 'manual',
158          status: 'confirmed',
159          slipRef: null,
160          slipImageHash: null,
161          ocrConfidence: null,
162          reviewFlags: [],
163        });
164        // Saved: take it off the list, so trying again after a failure never saves it twice.
165        saved += 1;
166        setRemoved((r) => new Set(r).add(key));
167      }
168      celebrate();
169      toast({ message: `บันทึก ${saved} รายการแล้ว` });
170      leaveWhenDone();
171    } catch {
172      toast({ message: 'บันทึกบางรายการไม่สำเร็จ รายการที่เหลือยังอยู่ในรายการ ลองอีกครั้งได้', tone: 'error' });
173    } finally {
174      setBusy(false);
175    }
176  }
177
178  const total = items.reduce((s, { item }) => s + (item.kind === 'income' ? item.amountSatang : -
  item.amountSatang), 0);

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/voice.tsx:143

บรรทัด	ความหมาย
143–145	saveAll: ไม่มีรายการหรือกำลังบันทึก → ไม่ทำ · ล็อกปุ่ม
147	เวลาเดียวกันทุกรายการ = เวลาตอนกดบันทึก (เวลาไทย)
148	บันทึกบันทึกสำเร็จ

บรรทัด	ความหมาย
149–163	วันที่จะรายการ addTx (ทางเดียวกับ FR-1): ประเภท, สตางค์, หมวด, ชื่อ, บันทึก "จดด้วยเสียง", เวลา = วันนี้ + dayOffset ตอนเวลานี้ (เมื่อวาน = วันเมื่อวานเวลาเดียวกัน), ที่มา manual, ยืนยันแล้ว , ไม่มีข้อมูลสลิป
164–166	บันทึกสำเร็จแล้วเอาออกจากรายการทันที (คอมเมนต์: ถ้าล้มเหลวกลางทางแล้วกดใหม่ จะไม่บันทึกซ้ำ) — แยกระหว่าง ตรวจ repository (TC-80)
168–170	ครบ: จุดพลุ toast "บันทึก N รายการแล้ว" ปิดหน้า
171–172	มีรายการพลาด: toast แดง "บันทึกบางรายการไม่สำเร็จ รายการที่เหลือยังอยู่ในรายการ ลองอีกครั้งได้"
173–175	ปลดล็อกปุ่มเสมอ
178	total ยอดรวมที่แสดงหัวรายการ: รายรับบวก รายจ่ายลบ

! ทำไม 'ตัดออกทีละรายการ' สำคัญ

สมมติพูด 3 รายการ รายการแรกบันทึกได้ รายการที่สองเน็ตหลุด ถ้าไม่ตัดรายการแรกออก ผู้ใช้กด "บันทึก" ใหม่ รายการแรกจะถูกบันทึก **ซ้ำ** เงินหายเกินจริง โค้ดบรรทัด 166 จึงตัดรายการที่สำเร็จออกทันที รอบใหม่จะบันทึกเฉพาะที่เหลือ

อ่านโค้ด: หน้าจอ voice.tsx ตอนที่ 2 — หน้าตา

บรรทัด 180–346 อ่านจากบนลงล่างตรงกับภาพหน้าจอ

ส่วนที่ 6: หัวหน้าและการ์ดไมค์ (บรรทัด 180–250)

src/app/voice.tsx

บรรทัด 180–250

```

180   return (
181     <SafeAreaView edges={['top', 'bottom']} style={{ flex: 1, backgroundColor: theme.bg }}>
182       <Row justify="space-between" style={{ paddingHorizontal: space.sm, paddingTop: space.sm }}>
183         <IconButton icon="close" label="ปิด" onPress={() => goBack()} />
184         <Text v="h3">พุดจรรยาการ</Text>
185         <HeaderDecor />
186       </Row>
187       <KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === 'ios' ? 'padding' :
undefined}>
188         <ScrollView contentContainerStyle={{ padding: space.lg, gap: space.lg, paddingBottom:
space.xxxl }} keyboardShouldPersistTaps="handled">
189           { /* The mic */ }
190           <Reveal zoom>
191             <View style={{ borderRadius: radius.xl, overflow: 'hidden' }}>
192               <LinearGradient
193                 colors={[theme.heroTop, theme.hero, theme.heroDeep]}
194                 start={{ x: 0, y: 0 }}
195                 end={{ x: 1, y: 1 }}
196                 style={{ paddingVertical: space.xl, paddingHorizontal: space.lg, alignItems:
'center', gap: space.md }}
197               >
198                 <Aurora cycles={listening ? 2 : 0} strength={0.3} seed={31} />
199                 {canListen ? (
200                   <>
201                     <View style={{ width: 104, height: 104, alignItems: 'center', justifyContent:
'center' }}>
202                       <PulseRing size={104} active={listening} times="always" color=
{theme.heroAccent} />
203                       <Animated.View style={{ transform: [{ scale: mic.scale }] }}>
204                         <Pressable
205                           onPress={toggleListening}
206                           onPressIn={mic.onPressIn}
207                           onPressOut={mic.onPressOut}
208                           accessibilityRole="button"
209                           accessibilityLabel={listening ? 'หยุดฟัง' : 'แตะแล้วพูด'}
210                           style={{
211                             width: 96,
212                             height: 96,
213                             borderRadius: 48,
214                             backgroundColor: listening ? theme.accent : theme.heroDeep,
215                             borderWidth: 3,
216                             borderColor: theme.heroAccent,
217                             alignItems: 'center',
218                             justifyContent: 'center',
219                             shadowColor: theme.accent,
220                             shadowOpacity: 0.5,
221                             shadowRadius: 16,
222                             shadowOffset: { width: 0, height: 4 },

```

```

223         elevation: 10,
224     }}
225     >
226     <Icons name={listening ? 'stop' : 'mic'} size={42} color={listening
? theme.heroDeep : theme.heroAccent} />
227     </Pressable>
228     </Animated.View>
229     </View>
230     <VoiceBars level={level} active={listening} />
231     <T v="body" color={theme.heroInk} center>
232         {listening ? 'กำลังฟัง... พูดได้เลย แล้วแตะอีกครั้งเมื่อพูดจบ' : 'แตะไมค์แล้วพูด เช่น "ข้าว 50
บาท"'}
233     </T>
234     </>
235     ) : (
236     <>
237         <Icons name="mic-outline" size={40} color={theme.heroAccent} />
238         <T v="body" color={theme.heroInk} center>
239             เครื่องนี้ยังฟังเสียงในแอปไม่ได้ พิมพ์แบบที่พูดด้านล่างได้เลย หรือกดไมค์บนคีย์บอร์ดแล้วพูด
240         </T>
241     </>
242     )}
243     {text ? (
244         <T v="h3" color={theme.heroAccent} center accessibilityRole="text">
245             "{parsed.heard || text}"
246         </T>
247         ) : null}
248     </LinearGradient>
249 </View>
250 </Reveal>

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/voice.tsx:180

บรรทัด	ความหมาย
181–186	กล่องนอกกันรอยบาท แถวหัว: ✕ ปิด, "พูดจดยรายการ", ของตกแต่ง
187–188	หลบคีย์บอร์ด (iOS) และส่วนเลื่อนได้
190–197	การ์ดไมค์ค่อย ๆ ขยาย มุมโค้ง พื้นไล่สีเขียว 3 ระดับของริม
198	ออโรราเคลื่อนไหว 2 รอบเฉพาะตอนฟัง
199–200	ถ้าฟังได้:
201–202	วงซีฟงรอบปุ่ม กระเพื่อมตลอดเวลาที่ฟัง
203–209	ปุ่มย่อตามสปริง กดแล้ว toggleListening ป้ายอ่านจอ "หยุดฟัง" / "แตะแล้วพูด"
210–224	ปุ่มกลม 96 ขอบทง 3 พื้น: ฟังอยู่ = ทอง, ไม่ฟัง = เขียวเข้ม มีเงาเรืองทง
226	ไอคอน ■ หยุด (ตอนฟัง) หรือไมค์
230	แท่งเสียง
231–233	คำแนะนำเปลี่ยนตามสถานะ
235–242	ถ้าฟังไม่ได้: ไอคอนไมค์เส้น และ "เครื่องนี้ยังฟังเสียงในแอปไม่ได้ พิมพ์แบบที่พูดด้านล่างได้เลย หรือกดไมค์บนคีย์บอร์ดแล้วพูด"

243–247 ถ้ามีข้อความ: บรรทัดทองในเครื่องหมายคำพูด = ข้อความที่เข้าใจ (heard) ถ้าว่างใช้ข้อความดิบ

ส่วนที่ 7: ข้อความแจ้ง และช่องพิมพ์ (บรรทัด 252–281)

src/app/voice.tsx

บรรทัด 252–281

```

252         {notice ? (
253             <Card tone="alt" style={{ paddingVertical: space.md }}>
254                 <Text v="small" color={theme.ink}>
255                     {notice}
256                 </Text>
257                 {notice.includes('การตั้งค่า') ? <Button label="เปิดการตั้งค่า" kind="soft" small onPress=
258                 (() => Linking.openSettings()) /> : null}
259             </Card>
260         ) : null}
261
262         {/* Type instead */}
263         <View style={{ gap: space.sm }}>
264             <Text v="small">หรือพิมพ์แบบที่พูด</Text>
265             <View style={[inputBox(theme), { paddingHorizontal: 0 }]}>
266                 <TextInput
267                     nativeID="voice-text"
268                     value={text}
269                     onChangeText={changeText}
270                     placeholder="เช่น ข้าว 50 น้า 15"
271                     placeholderTextColor={theme.inkFaint}
272                     accessibilityLabel="พิมพ์รายการแบบที่พูด"
273                     multiline
274                     style={[inputText(theme), { paddingHorizontal: 14, minHeight: 52,
275                     textAlignVertical: 'top' }]}
276                 />
277             </View>
278             <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle=
279             {{ gap: space.sm }} keyboardShouldPersistTaps="handled">
280                 {EXAMPLES.map((e) => (
281                     <Chip key={e} label={e} onPress={() => changeText(e)} />
282                 ))}
283             </ScrollView>
284         </View>

```

► เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/app/voice.tsx:252

252–259 ถ้ามีข้อความแจ้ง: การ์ดเทา ถ้าข้อความพูดถึง "การตั้งค่า" มีปุ่ม "เปิดการตั้งค่า" (Linking.openSettings เปิดหน้าตั้งค่าแอปของมือถือ)

261–263 หัวข้อ "หรือพิมพ์แบบที่พูด"

264–274 ช่องพิมพ์หลายบรรทัด id "voice-text" ค่า = text พิมพ์แล้ว changeText (ทางเดียวกับเสียง) ข้อความจาก "เช่น ข้าว 50 น้า 15"

276–280 แถวชิปตัวอย่างเลื่อนข้าง แตะแล้ว changeText(ตัวอย่าง)

ส่วนที่ 8: รายการที่จะบันทึก (บรรทัด 283–346)

src/app/voice.tsx

บรรทัด 283–346

```
283      /* What will be saved */
284      {items.length > 0 ? (
285        <View style={{ gap: space.sm }}>
286          <Row justify="space-between">
287            <T v="h3">จะบันทึก {items.length} รายการ</T>
288            <T v="small" color={total >= 0 ? theme.income : theme.ink}>
289              สบ {formatBaht(total, { sign: true, decimals: false })}
290            </T>
291          </Row>
292          {items.map(({ item, key }, i) => {
293            const cat = getCategory(item.categoryKey);
294            return (
295              <Reveal key={key} index={i} from={10} zoom>
296                <Card style={{ paddingVertical: space.md }}>
297                  <Row gap={space.md}>
298                    <View style={{ width: 40, height: 40, borderRadius: 20, backgroundColor:
299                      theme.surfaceAlt, alignItems: 'center', justifyContent: 'center' }}>
300                      <T v="h3">{cat.glyph}</T>
301                    </View>
302                    <View style={{ flex: 1, gap: 2 }}>
303                      <T v="body" numberOfLines={1} style={{ fontFamily: fonts.sansSemi }}>
304                        {item.title}
305                      </T>
306                      <T v="micro">
307                        {cat.label} · {relativeDayLabel(addDays(today || bkkDayKey(new
308                          Date()), item.dayOffset))}
309                      </T>
310                    </View>
311                    <Pressable
312                      onPress={() => setKinds((k) => ({ ...k, [key]: item.kind === 'income' ?
313                        'expense' : 'income' })))}
314                      accessibilityRole="button"
315                      accessibilityLabel={` ${item.kind === 'income' ? 'รายรับ' : 'รายจ่าย'} และ
316                        เพื่อสลับ`}
317                      hitSlop={6}
318                    >
319                      <Money
320                        satang={item.kind === 'income' ? item.amountSatang : -
321                          item.amountSatang}
322                        sign
323                        size="h3"
324                        color={item.kind === 'income' ? theme.income : theme.ink}
325                      />
326                      <T v="micro" style={{ textAlign: 'right' }}>
327                        {item.kind === 'income' ? 'รายรับ' : 'รายจ่าย'} ≠
328                      </T>
329                    </Pressable>
330                    <IconButton icon="close" label={`\`บันทึก ${item.title}\`} onPress={() =>
331                      setRemoved((r) => new Set(r).add(key))} />
332                  </Row>
333                </Card>
334              </Reveal>
335            );
336          })}
337          <Button label={`บันทึก ${items.length} รายการ`} kind="gold" icon="checkmark-done"
338            shine onPress={saveAll} loading={busy} />
339        )
340      ) : null}
```

```

332         </View>
333     ) : text.trim() ? (
334         <BuddySays mood="thinking">{`${BUDDY_NAME}ยังไม่เจอจำนวนเงินในประโยคนี้ ลองพูดชื่อรายการตามด้วยราคา
เช่น "ข้าว 50 บาท"}`</BuddySays>
335     ) : (
336         <BuddySays mood="happy">{`พูดได้หลายรายการในที่เดียว เช่น "ข้าว 50 น้ำ 15" หรือ "ได้เงินจากแม่ 500"
`${BUDDY_NAME}จะแยกให้เอง`}</BuddySays>
337     )}
338
339     <T v="micro" center>
340     เสียงถูกแปลงเป็นข้อความโดยระบบของมือถือหรือเบราว์เซอร์ MindPay เก็บเฉพาะรายการที่คุณกดบันทึก
341     </T>
342 </ScrollView>
343 </KeyboardAvoidingView>
344 </SafeAreaView>
345 );
346 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/voice.tsx:283`

บรรทัด	ความหมาย
284–291	ถ้ามีรายการ: หัว "จะบันทึก N รายการ" และ "รวม +฿..." สักรายรับเมื่อ ≥ 0
292–296	วนรายการ หาคหมวด แต่ละการ์ดค่อย ๆ โผล่
298–300	วงกลมไอคอนหมวด
301–308	ชื่อ (ตัวหนา บรรทัดเดียว) และ "หมวด · วัน" (วันนี้/เมื่อวาน/วันที่)
309–313	ก้อนเงินกดได้: แตะ → บันทึกประเภทตรงข้ามลง kinds ป้ายอ่านจ่อ "รายจ่าย แตะเพื่อสลับ" (E2E ใช้ป้ายนี้หาการ์ด)
315–323	เงินมีเครื่องหมาย (รายรับสีเขียว) และ "รายจ่าย ๕"/"รายรับ ๕"
325	ปุ่ม × "ไม่บันทึก [ชื่อ]" → ใส่กฎแฉลง removed
331	ปุ่มทอง "บันทึก N รายการ" มีแสงวิ่ง วงหมุนตอนบันทึก
333–334	มีข้อความแต่ไม่มีรายการ: น้องกล่ากรุ่นคิด "ยังไม่เจอจำนวนเงินในประโยคนี้ ลองพูดชื่อรายการตามด้วยราคา เช่น "ข้าว 50 บาท""
335–336	ยังไม่มีข้อความ: น้องกล่ายิ้ม แฉ่นำว่าพูดหลายรายการได้
339–341	ข้อความความเป็นส่วนตัว (ท้ายหน้าเสมอ)

อ่านโค้ด: ฟังเสียงบนมือถือ speech.ts ทั้งไฟล์

96 บรรทัด ห่อโมดูล expo-speech-recognition ให้ใช้งานและไม่ทำให้แอปพัง

≈ ใครแปลงเสียงเป็นข้อความ

MindPay ไม่ได้ แปลงเสียงเอง เราขอให้ "ล่าม" ที่มากับเครื่องทำ: Android ใช้บริการของ Google, iPhone ใช้ของ Apple, เว็บใช้ของเบราว์เซอร์ (Chrome/Edge/Safari) ไฟล์นี้คือ "คนประสานงานกับล่าม" บอกว่าฟังภาษาไทย ช่วยด้วยคำศัพท์เรื่องเงิน และส่งข้อความกลับมาให้หน้าจอ

ส่วนที่ 1: หาโมดูลแบบไม่บังคับ (บรรทัด 1–33)

src/services/speech.ts

บรรทัด 1–33

```
1 /**
2  * Speech to text on the phone ("พูดจด"), in Thai, with the phone's own speech
3  * recognizer (Google on Android, Apple on iOS) through expo-speech-recognition.
4  *
5  * The native module is looked up optionally: app builds made before it was
6  * added do not have it, and an over-the-air update must not crash them. There
7  * the voice screen simply offers typing instead (the keyboard's own mic works).
8  * The web version is speech.web.ts.
9  */
10 import { requireOptionalNativeModule } from 'expo';
11
12 interface Subscription {
13   remove(): void;
14 }
15
16 interface NativeSpeech {
17   start(options: Record<string, unknown>): void;
18   stop(): void;
19   abort(): void;
20   requestPermissionsAsync(): Promise<{ granted: boolean; canAskAgain?: boolean }>;
21   isRecognitionAvailable(): boolean;
22   addListener(event: string, listener: (e: never) => void): Subscription;
23 }
24
25 const Speech = requireOptionalNativeModule<NativeSpeech>('ExpoSpeechRecognition');
26
27 export interface ListenHandlers {
28   /** What was heard so far (`final` once the sentence is finished). */
29   onText(text: string, final: boolean): void;
30   onError(code: string): void;
31   onEnd(): void;
32   /** Loudness from about 0 (quiet) to 1 (loud), for the animation. */
33   onVolume?(level: number): void;
34 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/speech.ts:1

บรรทัด	ความหมาย
1-3	คอมเมนต์: แปลงเสียงเป็นข้อความบนมือถือ ภาษาไทย ด้วยตัวแปลงของระบบ (Google บน Android, Apple บน iOS) ผ่าน expo-speech-recognition
5-8	เหตุผลสำคัญ: โมดูลถูกหา "แบบไม่บังคับ" เพราะแอปที่ build ก่อนเพิ่มโมดูลนี้ไม่มีมัน และการอัปเดตแบบ over-the-air (EAS Update ส่งแต่โค้ด JavaScript) ต้องไม่ทำให้แอปเหล่านั้นพัง — ในเครื่องแบบนั้นหน้าพุดจจะเสนอให้พิมพ์แทน · เวอร์ชันเว็บอยู่ใน speech.web.ts
10	requireOptionalNativeModule จาก expo: หาโมดูลเนทีฟ ถ้าไม่มี คืน null (ไม่โยนข้อผิดพลาด)
12-14	ชนิด Subscription: สิ่งที่ได้จากการสมัครฟังเหตุการณ์ มีคำสั่ง remove() ยกเลิก
15-22	ชนิด NativeSpeech = คำสั่งที่โมดูลเนทีฟมี: เริ่ม, หยุด, ยกเลิก, ขอสิทธิ์ (ได้ granted และถามซ้ำได้ไหม), มีตัวแปลงใหม่, ฟังเหตุการณ์
24	Speech = โมดูลชื่อ 'ExpoSpeechRecognition' หรือ null
26-33	ListenHandlers สัญญาที่หน้าจอส่งมา: onChangeText (ข้อความที่ได้ยินถึงตอนนี้ และจบประโยคหรือยัง), onError (สหัสข้อผิดพลาด), onEnd (ฟังจบ), onVolume (ความดัง 0-1 สำหรับแอนิเมชัน ไม่บังคับ)

ส่วนที่ 2: คำช่วย มีตัวฟังใหม่ และขอสิทธิ์ (บรรทัด 35-55)

src/services/speech.ts บรรทัด 35-55

```

35  /** Words that help the recognizer with money talk. */
36  const HINTS = ['บาท', 'คำรด', 'คำข้าว', 'ข้าว', 'กาแฟ', 'ชานม', 'คำหอ', 'คำเน็ต', 'เงินเดือน', 'ได้เงินจากแม่',
37    'เซเว่น'];
38  export function speechAvailable(): boolean {
39    try {
40      return !!Speech && Speech.isRecognitionAvailable();
41    } catch {
42      return false;
43    }
44  }
45
46  export async function askMicPermission(): Promise<'granted' | 'denied' | 'blocked'> {
47    if (!Speech) return 'denied';
48    try {
49      const r = await Speech.requestPermissionsAsync();
50      if (r.granted) return 'granted';
51      return r.canAskAgain === false ? 'blocked' : 'denied';
52    } catch {
53      return 'denied';
54    }
55  }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → src/services/speech.ts:35

บรรทัด	ความหมาย
35-36	HINTS คำที่ช่วยตัวแปลงเสียงฟังเรื่องเงินแม่นขึ้น: บาท คำรด คำข้าว ข้าว กาแฟ ชานม คำหอ คำเน็ต เงินเดือน ได้เงินจากแม่ เซเว่น
38-44	speechAvailable: มีโมดูล และระบบบอกว่ามีบริการแปลงเสียง ถ้าเกิดข้อผิดพลาดใด ๆ ตอบ false (ปลอดภัยไว้ก่อน)

บรรทัด	ความหมาย
46–47	askMicPermission: ไม่มีโมดูล → denied
48–50	ขอสิทธิ์ (ระบบแสดงหน้าต่างถามผู้ใช้) ได้ → 'granted'
51	ไม่ได้: ถ้าระบบบอกว่า "ถามซ้ำไม่ได้แล้ว" (ผู้ใช้เลือกไม่ต้องถามอีก) → 'blocked' (ต้องไปเปิดในการตั้งค่า) ไม่ขึ้น 'denied'
52–54	ข้อผิดพลาด → denied

ส่วนที่ 3: listen – เริ่มฟัง (บรรทัด 57–96)

src/services/speech.ts

บรรทัด 57–96

```

57 /** Start listening; returns a function that stops (the text heard so far is kept). */
58 export function listen(h: ListenHandlers): () => void {
59   if (!Speech) {
60     h.onError('unavailable');
61     return () => {};
62   }
63   const subs: Subscription[] = [];
64   const done = () => subs.splice(0).forEach((s) => s.remove());
65   subs.push(
66     Speech.addListener('result', (e: { isFinal?: boolean; results?: { transcript?: string }[] })
=>
67       h.onText(e.results?.[0]?.transcript ?? '', !!e.isFinal),
68     ),
69     Speech.addListener('error', (e: { error?: string }) => h.onError(e.error ?? 'unknown')),
70     Speech.addListener('volumechange', (e: { value?: number }) => h.onVolume?.(Math.max(0,
Math.min(1, ((e.value ?? -2) + 2) / 12)))),
71     Speech.addListener('end', () => {
72       done();
73       h.onEnd();
74     }),
75   );
76   try {
77     Speech.start({
78       lang: 'th-TH',
79       interimResults: true,
80       continuous: false,
81       maxAlternatives: 1,
82       contextualStrings: HINTS,
83       volumeChangeEventOptions: { enabled: true, intervalMillis: 120 },
84     });
85   } catch {
86     done();
87     h.onError('unavailable');
88   }
89   return () => {
90     try {
91       Speech.stop();
92     } catch {
93       // Already stopped.
94     }
95   };
96 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/speech.ts:57

บรรทัด	ความหมาย
57–58	เริ่มฟัง ค้นฟังก์ชันสำหรับหยุด (ข้อความที่ได้ยินแล้วเก็บไว้)
59–62	ไม่มีโมดูล → แจ้งข้อผิดพลาด 'unavailable' ค้นฟังก์ชันว่าง
63–64	รายการการสมัครฟังเหตุการณ์ และ done() = ยกเลิกทั้งหมด (splice(0) เอาออกทั้งหมดแล้ววน remove)
65–68	เหตุการณ์ result : เอาข้อความของผลลัพธ์แรก ส่งให้ onText พร้อมบอกว่าจบประโยคหรือยัง
69	เหตุการณ์ error : ส่งรหัส (ไม่มี = 'unknown')
70	เหตุการณ์ volumechange : ระบบให้ค่าประมาณ -2 ถึง 10 แปลงเป็น 0–1 ด้วย $(\text{ค่า} + 2) \div 12$ และบีบให้อยู่ระหว่าง 0–1
71–74	เหตุการณ์ end : ยกเลิกการฟังเหตุการณ์ทั้งหมด แล้วแจ้ง onEnd
76–84	เริ่มฟัง : ภาษา th-TH, ส่งผลระหว่างพูด (interim) ด้วย, ฟังประโยคเดียวแล้วหยุดเอง (continuous: false), ขอคำตอบแบบเดียว, คำช่วย HINTS, ส่งความดังทุก 120 มิลลิวินาที
85–88	เริ่มไม่ได้ → ยกเลิกและแจ้ง 'unavailable'
89–95	ฟังก์ชันหยุดที่คืนไป: ส่ง stop (ถ้าหยุดไปแล้วก็ไม่ใช่เป็นไร)

อ่านโค้ด: ฟังเสียงบนเว็บ speech.web.ts ทั้งไฟล์

70 บรรทัด ใช้ Web Speech API ของเบราว์เซอร์ ส่งข้อมูลรูปแบบเดียวกับเวอร์ชันมือถือ หน้าจอจึงไม่ต้องรู้ว่ารันที่ไหน

src/services/speech.web.ts

บรรทัด 1-35

```

1  /**
2   * Speech to text on the web ("พูดจด"), with the browser's Web Speech API in
3   * Thai (Chrome and Edge on computers and Android; Safari on iPhone asks to use
4   * the mic and Siri's recognizer). Browsers without it get typing instead.
5   */
6  import type { ListenHandlers } from './speech';
7
8  type Recognition = {
9    lang: string;
10   interimResults: boolean;
11   continuous: boolean;
12   maxAlternatives: number;
13   onresult: ((e: { resultIndex: number; results: ArrayLike<ArrayLike<{ transcript: string }> & {
14     isFinal: boolean }> }) => void) | null;
15   onerror: ((e: { error: string }) => void) | null;
16   onend: (() => void) | null;
17   start(): void;
18   stop(): void;
19 };
20 const Ctor: (new () => Recognition) | undefined =
21   typeof window === 'undefined'
22     ? undefined
23     : ((window as unknown as { SpeechRecognition?: new () => Recognition;
24       webkitSpeechRecognition?: new () => Recognition }).SpeechRecognition ??
25       (window as unknown as { webkitSpeechRecognition?: new () => Recognition
26         }).webkitSpeechRecognition);
27
28 export type { ListenHandlers };
29
30 export function speechAvailable(): boolean {
31   return !!Ctor;
32 }
33
34 /** The browser asks for the mic itself when listening starts. */
35 export async function askMicPermission(): Promise<'granted' | 'denied' | 'blocked'> {
36   return Ctor ? 'granted' : 'denied';
37 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/speech.web.ts:1

บรรทัด ความหมาย

- 1-5 คอมเมนต์: Web Speech API ภาษาไทย (Chrome และ Edge บนคอม/Android; Safari บน iPhone ขอใช้ไมค์และใช้ตัวแปลงของ Siri) เบราว์เซอร์ที่ไม่มีก็พิมพ์แทน
- 6 ใช้ชนิด ListenHandlers จากไฟล์มือถือ (สัญญาเดียวกัน)

บรรทัด	ความหมาย
8-18	ชนิด Recognition = หน้าตาของวัตถุฟังเสียงของเบราว์เซอร์: ภาษา, ผลระหว่างพูด, ฟังต่อเนื่อง, จำนวนคำตอบ, ตัวรับผล/ข้อผิดพลาด, เริ่ม/หยุด
20-24	Ctor = ตัวสร้างของเบราว์เซอร์: ถ้าไม่มี window (ตอนสร้างหน้าเว็บล่วงหน้า) → undefined ไม่จำเป็นต้องใช้ SpeechRecognition หรือ webkitSpeechRecognition (ชื่อเก่าของ Chrome/Safari)
26	ส่งชนิดต่อให้ไฟล์อื่นใช้
28-30	มีตัวสร้าง = ฟังได้
32-35	บนเว็บ เบราว์เซอร์ขอสิทธิ์ไมค์เองตอนเริ่มฟัง จึงตอบ granted ถ้ามีตัวสร้าง

src/services/speech.web.ts

บรรทัด 37-70

```

37 export function listen(h: ListenHandlers): () => void {
38   if (!Ctor) {
39     h.onError('unavailable');
40     return () => {};
41   }
42   const rec = new Ctor();
43   rec.lang = 'th-TH';
44   rec.interimResults = true;
45   rec.continuous = false;
46   rec.maxAlternatives = 1;
47   rec.onresult = (e) => {
48     let text = '';
49     let final = true;
50     for (let i = 0; i < e.results.length; i++) {
51       text += e.results[i][0]?.transcript ?? '';
52       if (!e.results[i].isFinal) final = false;
53     }
54     h.onText(text, final);
55   };
56   rec.onerror = (e) => h.onError(e.error === 'not-allowed' || e.error === 'service-not-allowed' ?
'not-allowed' : e.error);
57   rec.onend = () => h.onEnd();
58   try {
59     rec.start();
60   } catch {
61     h.onError('unavailable');
62   }
63   return () => {
64     try {
65       rec.stop();
66     } catch {
67       // Already stopped.
68     }
69   };
70 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/speech.web.ts:37

บรรทัด	ความหมาย
37-41	ไม่มีตัวสร้าง → unavailable

บรรทัด	ความหมาย
42–46	สร้างตัวฟัง ตั้งภาษาไทย ผลระหว่างพูด ประโยคเดียว คำตอบเดียว
47–55	ได้ผล: ต่อข้อความทุกช่วงเข้าด้วยกัน (เบราร์เซอร์อาจแบ่งเป็นหลายช่วง) ถ้ามีช่วงไหนยังไม่จบ final = false แล้วส่ง onText
56	ข้อผิดพลาด: รวม not-allowed และ service-not-allowed เป็น 'not-allowed' (ข้อความเดียวกัน) ที่เหลือส่งรหัสตรง ๆ
57	จบ → onEnd
58–62	เริ่ม ถ้าฟัง → unavailable
63–69	ฟังก์ชันหยุด

★ Expo เลือกไฟล์ให้เองยังไ้

หน้าจอบันทึก import ... from '../services/speech' ไม่มี .web ตอน build เว็บ เครื่องมือ (Metro) จะหาไฟล์ speech.web.ts ก่อน ตอน build แอปจะใช้ speech.ts (บน A7) สองไฟล์จึงต้อง export ชื่อเดียวกัน (speechAvailable, askMicPermission, listen) และส่งข้อมูลรูปแบบเดียวกัน

i E2E ทดสอบเสียงยังไ้ทั้งที่ไม่มีไมค์

ทดสอบใส่ "ตัวฟังเสียงจำลอง" (FakeRecognition) แทน webkitSpeechRecognition ก่อนเปิดหน้า: เมื่อ start() มันจำภาษาที่ตั้งไว้ ส่ง "ข้าวมันไก่" (ยังไม่จบ) ที่ 0.3 วินาที และ "ข้าวมันไก่ 50 บาท" (จบ) ที่ 0.9 วินาที แล้วจบ ทดสอบจึงตรวจได้ทั้งว่า speech.web.ts ตั้งภาษา th-TH และหน้าจอบันทึกการ \$50 ([e2e/web-auth.mjs:826](#))

อ่านโค้ด: การตั้งค่าใน app.json

ปลั๊กอินที่ทำให้แอป Android/iOS มีสิทธิ์ใช้ไมโครโฟนและตัวแปลงเสียง

app.json

บรรทัด 71–80

```

71  [
72    "expo-speech-recognition",
73    {
74      "microphonePermission": "MindPay ใช้ไมโครโฟนเมื่อคุณแตะปุ่มไมค์เพื่อพูดจดยรายการ เช่น “ข้าว 50 บาท”",
75      "speechRecognitionPermission": "MindPay แปลงเสียงที่คุณพูดเป็นรายการรายรับรายจ่าย",
76      "androidSpeechServicePackages": [
77        "com.google.android.googlequicksearchbox"
78      ]
79    }
80  ]

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → app.json:71

บรรทัด ความหมาย

- | | |
|-------|--|
| 71–72 | ใช้ปลั๊กอิน expo-speech-recognition (ตอน build แอปจะใส่โค้ดเนกทีฟและสิทธิ์ให้) |
| 74 | ข้อความที่ระบบแสดงตอนขอใช้ไมโครโฟน: "MindPay ใช้ไมโครโฟนเมื่อคุณแตะปุ่มไมค์เพื่อพูดจดยรายการ เช่น “ข้าว 50 บาท”" (บอกเหตุผลชัดเจน ผู้ใช้จะคลำกดอนุญาต) |
| 75 | ข้อความขอใช้การแปลงเสียง (iOS): "MindPay แปลงเสียงที่คุณพูดเป็นรายการรายรับรายจ่าย" |
| 76–78 | Android: ระบุแพ็คเกจบริการแปลงเสียงของ Google |

! ทำไม APK เก่าไม่มีเสียง

โมดูลเนกทีฟต้องถูก build ลงไปในแอป แต่ EAS Update ส่งได้แค่ JavaScript แอปที่ build ก่อนเพิ่มโมดูลนี้ (ก่อน commit c0359d8 วันที่ 29 ก.ย.) จึงได้หน้าพูดจดยใหม่แต่ไม่มีโมดูล — speech.ts จึงหาโมดูลแบบไม่บังคับ และหน้าจอเสนอให้พิมพ์แทน (MT-12: "APK เก่าแสดงช่องพิมพ์แทน ไม่ crash") ผู้ใช้ต้องติดตั้ง APK ใหม่จาก GitHub Releases เพื่อใช้ไมค์ในแอป

การทดสอบ FR-3

ฉันพิสูจน์ยังงี้ว่าแอปเข้าใจประโยคถูก และหน้าจอบันทึกลงได้จริง

157/157

unit test ทั้งโปรเจกต์ผ่าน (รันซ้ำ 9 ๓.ค. 2569)

TC-59, 60, 61, 80

unit test ของ FR-3 (ประโยคตัวอย่าง
กว่า 60 ประโยค)

6 ข้อ

E2E voice ด้วยตัวฟังเสียงจำลอง + MT-
12 บนมือถือจริง

Unit test: voice.test.ts

src/domain/__tests__/voice.test.ts

บรรทัด 1-30

```
1 /**
2  * TC-59..TC-61: "พูดจ" - what the user says becomes transactions.
3  */
4  import { describe, expect, it } from 'vitest';
5  import { parseSpokenEntry, thaiNumberWords } from '../voice';
6
7  const one = (text: string) => {
8    const r = parseSpokenEntry(text);
9    expect(r.items).toHaveLength(1);
10   return r.items[0];
11 };
12
13 describe('Voice entry', () => {
14   it('TC-59 reads the usual sentence: item, amount, บาท (digits or Thai number words)', () => {
15     expect(one('ข้าว 50 บาท')).toMatchObject({ title: 'ข้าว', amountSatang: 5000, kind: 'expense',
16       categoryKey: 'food', dayOffset: 0 });
17     expect(one('ข้าวมันไก่ห้านาที')).toMatchObject({ title: 'ข้าวมันไก่', amountSatang: 5000 });
18     expect(one('กาแฟ 65')).toMatchObject({ title: 'กาแฟ', amountSatang: 6500, categoryKey: 'food'
19   });
20     expect(one('ค่ารถ ยี่สิบห้า')).toMatchObject({ title: 'ค่ารถ', amountSatang: 2500, categoryKey:
21       'transport' });
22     expect(one('เติมน้ำมัน 500 บาท')).toMatchObject({ title: 'น้ำมัน', amountSatang: 50000,
23       categoryKey: 'transport' });
24     expect(one('จ่ายค่าน้ำ 299')).toMatchObject({ title: 'ค่าน้ำ', amountSatang: 29900, categoryKey:
25       'bills' });
26     expect(one('ค่าน้ำ 120')).toMatchObject({ categoryKey: 'bills' });
27     expect(one('หนังสือ ๓๕๐ บาท')).toMatchObject({ title: 'หนังสือ', amountSatang: 35000, categoryKey:
28       'study' });
29     expect(one('เชvron 1,250.50 บาท')).toMatchObject({ amountSatang: 125050, categoryKey:
30       'convenience' });
31     expect(one('7-eleven 45')).toMatchObject({ amountSatang: 4500, categoryKey: 'convenience' });
32     expect(one('ข้าว 45 บาท 50 สตางค์')).toMatchObject({ amountSatang: 4550 });
33     expect(one('50 บาท ค่าข้าว')).toMatchObject({ title: 'ค่าข้าว', amountSatang: 5000, categoryKey:
34       'food' });
35   });
36
37   it('TC-60 several items, income, yesterday, quantities that are not money', () => {
38     const two = parseSpokenEntry('ข้าว 50 น้ำ 15').items;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/voice.test.ts:1

บรรทัด	ความหมาย
1-5	หัวไฟล์ TC-59 ถึง TC-61 และ import ตัวทดสอบกับสองฟังก์ชันที่ทดสอบ
7-11	ตัวช่วย one(text): เข้าใจประโยค แล้ว ยืนยันว่าได้ 1 รายการพอดี ค็นรายการนั้น (ทุกเทสต์ที่ใช้ one จึงตรวจจำนวนรายการไปด้วย)
14-15	TC-59 ประโยคปกติ: "ข้าว 50 บาท" → ข้าว 5,000 สตางค์ รายจ่าย อาหาร วันนี้ (toMatchObject = ต้องมีช่องเหล่านี้ตรง)
16	"ข้าวมันไก่ห้าสิบบาทครับ" → ข้าวมันไก่ 50 (เลขคำติดคำ + ตัดครับ)
17-18	กาแฟ 65 → อาหาร · ค่ารถ ยี่สิบห้า → 25 เดินทาง
19-21	"เติมน้ำมัน 500 บาท" → ชื่อ "น้ำมัน" เดินทาง · "จ่ายค่าเน็ต 299" → "ค่าเน็ต" บิล · ค่าน้ำ → บิล (ไม่ใช่เครื่องดื่ม)
22-24	เลขไทย ๓๕๐ · "เซเว่น 1,250.50 บาท" = 125,050 สตางค์ สะดวกซื้อ · "7-eleven 45"
25-26	"ข้าว 45 บาท 50 สตางค์" = 4,550 · "50 บาท ค่าข้าว" (พูดเงินก่อน)
29-30	TC-60 เริ่ม: "ข้าว 50 น้ำ 15" → 2 รายการ

src/domain/__tests__/voice.test.ts

บรรทัด 31-67

```

31   expect(two.map((i) => [i.title, i.amountSatang, i.categoryKey])).toEqual([
32     ['ข้าว', 5000, 'food'],
33     ['น้ำ', 1500, 'food'],
34   ]);
35   expect(parseSpokenEntry('ถ้วยเดียว 60 บาท และ ชาเย็น 35 บาท').items.map((i) =>
i.title)).toEqual(['ถ้วยเดียว', 'ชาเย็น']);
36   expect(one('ได้เงินจากแม่ 500')).toMatchObject({ title: 'เงินจากแม่', amountSatang: 50000, kind:
'income', categoryKey: 'allowance' });
37   expect(one('เงินเดือนเข้า 15000')).toMatchObject({ kind: 'income', categoryKey: 'salary' });
38   expect(one('ขายของได้ 2k')).toMatchObject({ kind: 'income', amountSatang: 20000 });
39   expect(one('เมื่อวาน ค่ารถ 40')).toMatchObject({ title: 'ค่ารถ', dayOffset: -1 });
40   expect(one('เมื่อวานขึ้น หมู่กระะ 299')).toMatchObject({ dayOffset: -2 });
41   expect(one('ข้าว 2 จาน 100 บาท')).toMatchObject({ title: 'ข้าว 2 จาน', amountSatang: 10000 });
42   expect(one('กาแฟ 3 แก้ว 195')).toMatchObject({ amountSatang: 19500 });
43   expect(one('เสื้อ 5 ร้อย')).toMatchObject({ title: 'เสื้อ', amountSatang: 50000, categoryKey:
'shopping' });
44   expect(one('รองเท้า สองพันห้า')).toMatchObject({ amountSatang: 25000 });
45 });
46
47 it('TC-61 Thai number words, and words that only look like numbers', () => {
48   expect(thaiNumberWords('ห้าสิบ')).toBe(50);
49   expect(thaiNumberWords('ยี่สิบห้า')).toBe(25);
50   expect(thaiNumberWords('สิบเอ็ด')).toBe(11);
51   expect(thaiNumberWords('หนึ่งร้อยห้า')).toBe(105);
52   expect(thaiNumberWords('ร้อยห้า')).toBe(150);
53   expect(thaiNumberWords('ร้อยครึ่ง')).toBe(150);
54   expect(thaiNumberWords('ร้อยมึง')).toBe(100);
55   expect(thaiNumberWords('พันห้าร้อย')).toBe(1500);
56   expect(thaiNumberWords('สองพันห้า')).toBe(2500);
57   expect(thaiNumberWords('แปดร้อยเก้าสิบเก้า')).toBe(899);
58   expect(thaiNumberWords('หนึ่งล้าน')).toBe(1_000_000);
59   expect(thaiNumberWords('ห้า')).toBeNull();
60   // "ห้า" (mall) and "สี่แยก" (junction) are words, not 5 and 4.
61   expect(one('ของใช้ในห้าง 250')).toMatchObject({ title: 'ของใช้ในห้าง', amountSatang: 25000 });

```

```

62     expect(one('ค่ารถไปสี่แยก 30')).toMatchObject({ amountSatang: 3000 });
63     // Nothing to record.
64     expect(parseSpokenEntry('สวัสดีครับ').items).toEqual([]);
65     expect(parseSpokenEntry('').items).toEqual([]);
66   });
67 });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/voice.test.ts:31

บรรทัด	ความหมาย
31–34	สองรายการต้องเป็น [ข้าว 5,000 อาหาร] และ [น้ำ 1,500 อาหาร]
35	"ก๋วยเตี๋ยว 60 บาท และ ชาเย็น 35 บาท" → ชื่อ ก๋วยเตี๋ยว และ ชาเย็น (ตัด "และ")
36–38	รายรับ: ได้เงินจากแม่ (เงินจากที่บ้าน) · เงินเดือนเข้า (เงินเดือน) · ขยายของได้ 2k = 200,000 สตางค์
39–40	วัน: เมื่อวาน = -1 · เมื่อวานขึ้น = -2
41–42	จำนวนไม่ใช่เงิน: "ข้าว 2 จาน 100 บาท" ชื่อ "ข้าว 2 จาน" \$100 · "กาแฟ 3 แก้ว 195"
43–44	"เสื้อ 5 ร้อย" = \$500 ซอปปิง · "รองเท้า สองพันห้า" = \$2,500
47–58	TC-61 เลขคำไทย 11 แบบ (ห้าสิบ ... หนึ่งในล้าน) และ "ห้าง" ต้องเป็น null
60–62	คำที่ดูเหมือนเลข: "ของใช้ในห้าง 250" และ "ค่ารถไปสี่แยก 30" ต้องได้จำนวนที่ถูกต้อง
63–65	ไม่มีอะไรให้จด: "สวัสดีครับ" และข้อความว่าง → ไม่มีรายการ

Unit test: TC-80 (ทดสอบการตรวจ repository และผู้ตรวจอิสระ)

src/domain/__tests__/audit.test.ts

บรรทัด 72–127

```

72 describe('TC-80 พุดจด', () => {
73   const items = (t: string) => parseSpokenEntry(t).items.map((i) => [i.title, i.amountSatang /
100, i.kind, i.dayOffset]);
74
75   it('keeps 711 inside an amount; 7-11 is the shop', () => {
76     expect(items('ค่าไฟ 711 บาท')).toEqual(['ค่าไฟ', 711, 'expense', 0]);
77     expect(items('ค่าเสื้อ 1711')).toEqual(['ค่าเสื้อ', 1711, 'expense', 0]);
78     expect(items('ค่าหอ 2,711 บาท')).toEqual(['ค่าหอ', 2711, 'expense', 0]);
79     expect(parseSpokenEntry('ขม 7-11 45 บาท').items[0]).toMatchObject({ amountSatang: 4500,
categoryKey: 'convenience' });
80   });
81
82   it('reads a digit followed by number words as one amount', () => {
83     expect(items('รองเท้า 2 พันห้า')).toEqual(['รองเท้า', 2500, 'expense', 0]);
84     expect(items('ค่าเทอม 3 พัน 2 ร้อย')).toEqual(['ค่าเทอม', 3200, 'expense', 0]);
85     expect(items('เสื้อ 5 ร้อยห้าสิบ')).toEqual(['เสื้อ', 550, 'expense', 0]);
86     expect(items('ค่ารถ 1.5 พัน')).toEqual(['ค่ารถ', 1500, 'expense', 0]);
87     // Two numbers where the second is bigger stay two items.
88     expect(items('ข้าวห้าสิบ สองร้อย').map((i) => i[1])).toEqual([50, 200]);
89   });
90
91   it('reads a trailing one as one', () => {
92     expect(thaiNumberWords('ร้อยหนึ่ง')).toBe(101);
93     expect(thaiNumberWords('ร้อยเอ็ด')).toBe(101);
94     expect(thaiNumberWords('ยี่สิบหนึ่ง')).toBe(21);

```

```

95     expect(thaiNumberWords('ร้อยมึง')).toBe(100);
96     expect(thaiNumberWords('พันมึง')).toBe(1000);
97     expect(thaiNumberWords('ร้อยห้า')).toBe(150);
98 });
99
100 it('pairs amounts said first with the words after them', () => {
101     expect(items('50 บาท ค่าข้าว 30 บาท ค่าน้ำ')).toEqual([
102         ['ค่าข้าว', 50, 'expense', 0],
103         ['ค่าน้ำ', 30, 'expense', 0],
104     ]);
105 });
106
107 it('paying words win over income words', () => {
108     expect(items('จ่ายค่าจ้างช่างแอร์ 800')[0][2]).toBe('expense');
109     expect(items('คืนเงินเพื่อน 200')[0][2]).toBe('expense');
110     expect(items('ได้เงินคืน 50')).toEqual(['เงินคืน', 50, 'income', 0]);
111     expect(items('ได้เงินจากแม่ 500')).toEqual(['เงินจากแม่', 500, 'income', 0]);
112 });
113
114 it('keeps words that only look like polite endings or connectors', () => {
115     expect(items('ข้าวเจ้า 50')[0][0]).toBe('ข้าวเจ้า');
116     expect(items('กั๊บบข้าว 60')[0][0]).toBe('กั๊บบข้าว');
117     expect(items('ข้าว 50 บาทครับ')[0][0]).toBe('ข้าว');
118 });
119
120 it('gives each item the day said before it', () => {
121     expect(items('เมื่อวานค่ารถ 40 วันนี้ข้าว 50')).toEqual([
122         ['ค่ารถ', 40, 'expense', -1],
123         ['ข้าว', 50, 'expense', 0],
124     ]);
125     expect(items('ข้าว 50 น้ำ 15 เมื่อวาน').map((i) => i[3])).toEqual([-1, -1]);
126 });
127 });

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/audit.test.ts:72

บรรทัด	ความหมาย
73	ตัวช่วยแปลงผลเป็น [ชื่อ, บาท, ประเภท, วัน] ให้เทียบง่าย
75–80	711 ในราคาต้องเป็นราคา (ค่าไฟ 711, ค่าเสื้อ 1711, ค่าหอ 2,711) แต่ "ขนม 7-11 45 บาท" = \$45 สะดวกซื้อ
82–89	เลขผสมคำ: 2 พันห้า = 2,500 · 3 พัน 2 ร้อย = 3,200 · 5 ร้อยห้าสิบ = 550 · 1.5 พัน = 1,500 · แต่ "ข้าวห้าสิบ สอง ร้อย" ยังเป็น 2 จำนวน [50, 200]
91–98	หนึ่งตัวท้าย: ร้อยหนึ่ง/ร้อยเอ็ด = 101 · ยี่สิบหนึ่ง = 21 · ร้อยหนึ่ง = 100 · พันหนึ่ง = 1,000 · ร้อยห้า = 150
100–105	พูดเงินก่อนชื่อหลายรายการ: "50 บาท ค่าข้าว 30 บาท ค่าน้ำ"
107–112	คำจ่ายชนะคำรายรับ: จ่ายค่าจ้างช่างแอร์ = รายจ่าย · คืนเงินเพื่อน = รายจ่าย · ได้เงินคืน = รายรับ "เงินคืน" · ได้เงินจากแม่ = รายรับ
114–118	คำที่ดูเหมือนคำลงท้าย/คำเชื่อม: ข้าวเจ้า, กั๊บบข้าว ต้องอยู่ครบ แต่ "บาทครับ" ตัด "ครับ"
120–126	วันของแต่ละรายการ: "เมื่อวานค่ารถ 40 วันนี้ข้าว 50" → -1 และ 0 · "ข้าว 50 น้ำ 15 เมื่อวาน" → ทั้งคู่ -1

E2E: พุดจดในเบราว์เซอร์จริง (ข้อ 19)

e2e/web-auth.mjs

บรรทัด 823–852

```
823 // 19. Voice entry: say "ข้าวมั่นไก่อ่ 50 บาท" (a stand-in for the browser's speech recognizer), or
    type it the same way
824 {
825   const { ctx, page } = await freshPage('voice');
826   await ctx.addInitScript(() => {
827     class FakeRecognition {
828       constructor() {
829         this.lang = '';
830         this.onresult = null;
831         this.onerror = null;
832         this.onend = null;
833       }
834       start() {
835         window.__speechLang = this.lang;
836         const say = (text, isFinal) => {
837           const result = [{ transcript: text }];
838           result.isFinal = isFinal;
839           this.onresult?.({ resultIndex: 0, results: [result] });
840         };
841         setTimeout(() => say('ข้าวมั่นไก่อ่', false), 300);
842         setTimeout(() => say('ข้าวมั่นไก่อ่ 50 บาท', true), 900);
843         setTimeout(() => this.onend?.(), 1000);
844       }
845       stop() {
846         setTimeout(() => this.onend?.(), 0);
847       }
848     }
849     for (const name of ['SpeechRecognition', 'webkitSpeechRecognition']) {
850       Object.defineProperty(window, name, { value: FakeRecognition, configurable: true,
851         writable: true });
852     });
853   });
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → e2e/web-auth.mjs:823

บรรทัด ความหมาย

823–826 เปิดเบราว์เซอร์ใหม่ และใส่สคริปต์ก่อนหน้าเว็บโหลด

827–848 **FakeRecognition** ตัวฟังเสียงจำลอง: start() จำภาษาที่ตั้งไว้ใน window.__speechLang, ส่ง "ข้าวมั่นไก่อ่" (ยังไม่จบ) ที่ 0.3 วินาที, "ข้าวมั่นไก่อ่ 50 บาท" (จบ) ที่ 0.9 วินาที, จบที่ 1 วินาที · stop() จบทันที

849–851 ใส่ตัวจำลองแทนทั้ง SpeechRecognition และ webkitSpeechRecognition

e2e/web-auth.mjs

บรรทัด 890–906

```
890   await button(page, 'จดด้วยเสียง').click();
891   check('Voice: the mic on the home screen opens "พุดจดยายการ"', await visible(page, 'แตะไม้แล้ว
    พุด'));
892   await button(page, 'แตะแล้วพุด').click();
893   const heard = await visible(page, 'จะบันทึก 1 รายการ', 6000);
894   await page.waitForTimeout(600);
895   await shot(page, '21-voice-heard');
896   // (The home screen stays underneath with its own amounts, so look inside the item itself.)
```

```

897     const itemText = await button(page, 'รายจ่าย แตะเพื่อสลับ').textContent().catch(() => '');
898     check('Voice: "ข้าวมันไก่ 50 บาท" said -> one item of ฿50 ready to save', heard &&
itemText.includes('฿50.00'), itemText);
899     check('Voice: the recognizer listens in Thai', (await page.evaluate(() =>
window.__speechLang)) === 'th-TH');
900     await page.fill('#voice-text', 'ค่ารถ 25 กาแฟ 65 ได้เงินจากแม่ 500');
901     check('Typed the same way: three items, the money from mom as income', (await visible(page,
'จะบันทึก 3 รายการ')) && (await visible(page, '฿500.00')));
902     await button(page, 'บันทึก 3 รายการ').click();
903     check('Voice: saved with a confirmation', await visible(page, 'บันทึก 3 รายการแล้ว'));
904     await visible(page, 'ยอดคงเหลือ', 8000);
905     check('Voice: the balance follows (-฿25 -฿65 +฿500)', (await balance()) === start - 2_500 -
6_500 + 50_000);
906     check('First voice entry earns the "พูดแล้วจด" badge', (await visible(page, 'ได้เหรียญใหม่')) &&
(await visible(page, 'พูดแล้วจด')));

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → e2e/web-auth.mjs:890

บรรทัด	ความหมาย
890–891	กดปุ่มลัด "จดด้วยเสียง" บนหน้าจอหลัก ต้องเห็น "แตะไมค์แล้วพูด"
892–895	กด "แตะแล้วพูด" ต้องเห็น "จะบันทึก 1 รายการ" ภายใน 6 วินาที ถ่ายภาพ 21-voice-heard
896–898	อ่านข้อความในก้อนเงินของรายการ (ปุ่ม "รายจ่าย แตะเพื่อสลับ") ต้องมี "฿50.00"
899	ตัวฟังต้องถูกตั้งเป็น th-TH
900–901	พิมพ์ "ค่ารถ 25 กาแฟ 65 ได้เงินจากแม่ 500" ต้องเห็น "จะบันทึก 3 รายการ" และ "฿500.00"
902–903	กด "บันทึก 3 รายการ" ต้องเห็น "บันทึก 3 รายการแล้ว"
904–905	ยอดคงเหลือต้องเปลี่ยน -฿25 -฿65 +฿500 พอดี
906	ได้เหรียญ "พูดแล้วจด" ครั้งแรก

MT-12: ทดสอบบนมือถือนจริง

ทำ	ต้องเห็น
APK ใหม่ (มีตัวฟังเสียง): แตะไมค์กองบนหน้าจอหลัก อนุญาตไมโครโฟน แล้วพูด "ข้าว 50 บาท น้ำ 15"	เห็น 2 รายการ ข้าว ฿50 และ น้ำ ฿15 กดบันทึก แล้วยอดลด ฿65
APK เก่า (ก่อนมีตัวฟังเสียง) ที่ได้อัปเดต OTA	แสดงช่องพิมพ์แทน ไม่ crash

FR-3 เชื่อมกับ FR อื่นยังไ้

FR-3 เป็น 'อีกประตูหนึ่ง' ของการจดยการ ผลลัพธ์ไปอยู่ที่เดียวกับ FR-1

FR	ความสัมพันธ์	ตรงไหนในโค้ด
FR-1	บันทึกผ่าน addTx ตัวเดียวกัน · ใช้ suggestCategory ของ FR-1 เป็นตัวสำรอง · แก่/ลบรายการจากเสียงได้ในหน้าจดยการปกติ	<code>src/app/voice.tsx:150</code> , <code>src/domain/voice.ts:144</code>
FR-2	ยอดคงเหลือ ภาพรวม และกราฟอัปเดตทันที · สรุปเดือนนับ "พูดจดย" จาก VOICE_NOTE	<code>src/domain/recap.ts:100</code>
FR-4	ต่างกันที่สถานะ: สลับเป็นแบบร่าง (AI อ่าน) แต่เสียงยืนยันทันที (ผู้ใช้เห็นรายการและกดเอง) · ทิ้งคู่ลดการพิมพ์	—
FR-5	รายการจากเสียงเข้าไปในข้อมูลที่โค้ชสรุป · เหยี่ย "พูดแล้วจดย" (จดยด้วยเสียงครั้งแรก) และการจดยด้วยเสียง 10 รายการที่ให้สกินน้องกล้า "ดีเจเสียงใส"	<code>src/domain/achievements.ts:113</code> , <code>src/domain/missions.ts:134</code>
FR-6	รายจ่ายจากเสียงเข้าค่าเฉลี่ย 7 วัน จำนวนวันเปลี่ยนทันที	<code>src/domain/runway.ts</code>

ทำไมฉันออกแบบแบบนี้

เหตุผลของการตัดสินใจแต่ละข้อ

ใช้ตัวแปลงเสียงของระบบ ไม่ใช่ AI ของเรา	ฟรี เร็ว ภาษาไทยดีอยู่แล้ว ไม่ต้องส่งเสียงไปเซิร์ฟเวอร์ของเรา (เป็นส่วนตัวกว่า) ไม่กินโควตา AI
เข้าใจประโยคด้วยกฎ ไม่ใช่ AI	ทำงานกันทีละขั้นตอน (ทุกตัวอักษร) ใช้ได้ออฟไลน์ ผลแน่นอนทดสอบได้ 60+ ประโยค และไม่มีโอกาส "แต่งตัวเลข"
ให้พิมพ์แบบที่พูดได้	ใช้ได้ทุกเครื่อง (รวมเครื่องที่ไม่มีตัวฟัง) และเป็นทางทดสอบที่ไม่ต้องใช้ไมค์
เห็นรายการก่อนบันทึก	เสียงอาจฟังผิด ผู้ใช้ต้องได้ตรวจ สลับ หรือเอาออกก่อนเงินเปลี่ยน
บันทึกเป็น "ยืนยันแล้ว"	ผู้ใช้เห็นทุกรายการและกดบันทึกเองแล้ว ไม่จำเป็นต้องยืนยันซ้ำแบบสลับ
dayOffset แทนวันที่	ตัวเข้าใจประโยคไม่ต้องรู้วันนี้ จึงเป็นฟังก์ชันล้นทดสอบง่าย
แปลงคำตัวเลขเฉพาะเมื่อชัด	ไม่ทำลายคำอย่าง "ห้า" "สี่แยก" ยอมพลาดบางกรณี ("ข้าวห้า") ดีกว่าเปลี่ยนคำผิด ๆ
คำจ่ายที่มาก่อนชนะคำรายรับ	ตรงกับวิธีพูดจริง: กริยาตัวแรกบอกทิศทางเงิน ("จ่ายค่าจ้าง" vs "แม่ให้เงินซื้อ")
หาโมดูลเสียงแบบไม่บังคับ	อัปเดต OTA ต้องไม่ทำให้แอปเฝ้าฟัง
ขอสิทธิ์ไมค์ตอนแต่ละครั้งแรก	ผู้ใช้เข้าใจว่าทำไมต้องใช้ (เพิ่งแตะไมค์) โอกาสอนุญาตสูงกว่าขอตอนเปิดแอป
ตัดรายการที่บันทึกแล้วออกทีละรายการ	กดบันทึกซ้ำหลังเน็ตหลุดจะไม่บันทึกซ้ำ

ข้อจำกัด และสิ่งที่จะทำต่อ

ทุกข้อทดสอบด้วยการรันโค้ดจริงแล้ว

ข้อจำกัด	ตัวอย่าง (ผลจริง)	แก้ไข
หน่วยที่มีค่าต่อท้ายไม่ถูกนับเป็นหน่วย	"ตัวหนัง 2 ที่นั่ง 360" → [ตัวหนัง 2] [ที่นั่ง 360] · "เบียร์ 3 ขวดใหญ่ 270" → [เบียร์ 3] [ขวดใหญ่ 270]	เพิ่มคำหน่วยยาว (ที่นั่ง) หรือยอมให้หน่วยมีคำขยาย (ใหญ่ เล็ก)
คำบอกวันที่พูด หลัง รายการ ใช้ได้เฉพาะกรณีพูดครั้งเดียวท้ายประโยค	"ค่าหอ 3500 เมื่อวาน ข้าว 60 วันนี้" → ทั้งสองรายการเป็นเมื่อวาน (ผู้พูดน่าจะหมายถึงข้าววันนี้)	ตรวจสอบรูปแบบ "รายการ เลข วัน" ทั้งประโยค แล้วจับคู่วันกับรายการก่อนหน้า
รายการที่ไม่มีราคาถูกข้ามเงียบ ๆ	"ข้าว 50 น้ำ 15 ขนมน" → 2 รายการ ขนมนหายไป	แสดงคำเตือน "ขนมน ยังไม่มีราคา"
"7 11" ที่ไม่ได้ตามด้วยราคากันที	"7 11 ขนมน 45" → 3 รายการ (฿7, ฿11, ฿45)	เพิ่มกฎ "7 11 + คำไทย" เป็นเซเว่น
เงินคืนที่ไม่มีคำรายรับ	"เพื่อนโอนค่าข้าวคืน 50" → รายการจ่าย อาหาร	เพิ่มคำ "คืน" ถ้ายประโยคเป็นสัญญาณรายรับ (ระวัง "ชื่อของคืน")
ชื่อยาวไม่ถูกตัดคำกริยากลางประโยค	"แม่ให้เงินซื้อขนมน 100" ซื้อทั้งประโยค	ตัดรูปแบบ "X ให้เงิน" → "เงินจาก X"
เวลาของรายการเป็นเวลาที่เกิดบันทึก	พูด "ข้าวเที่ยง 50" ตอน 19:00 → บันทึกเวลา 19:00	เข้าใจคำบอกเวลา (เช้า เที่ยง เย็น)
ภาษาอังกฤษ/ศัพท์ปนได้บางส่วน	ขึ้นกับคำสำคัญของ FR-1 (Starbucks, Grab)	เพิ่มคำศัพท์ หรือใช้ AI ช่วยเฉพาะประโยคที่กฎเข้าใจไม่ได้

คำถามที่อาจารย์อาจถาม พร้อมคำตอบ

ตอบได้ทุกข้อจากสิ่งที่อ่านในเล่มนี้

1. FR-3 ทำอะไร	ให้ผู้ใช้งาน (หรือพิมพ์แบบที่พูด) เช่น "ข้าว 50 บาท" แล้วแอปแยกเป็นรายการ แสดงให้ตรวจ และบันทึกด้วยการแตะครั้งเดียว ได้หลายรายการในประโยคเดียว
2. ใครแปลงเสียงเป็นข้อความ	ระบบของเครื่อง: Google บน Android, Apple บน iOS, Web Speech API บนเว็บ ผ่าน speech.ts / speech.web.ts MindPay ไม่ได้ส่งเสียงไปเซิร์ฟเวอร์ของเรา
3. ใช้ AI ใหม่	ไม่ การเข้าใจประโยคเป็นกฎใน voice.ts ทั้งหมด เร็ว ฟรี ออฟไลน์ได้ และทดสอบได้แน่นอน
4. "ร้อยห้า" ได้เท่าไร ทำไม	150 เพราะเป็นภาษาพูดย่อ เลขหลังร้อยโดยไม่มีเลขนำหน้า = หลักสิบ แต่ "หนึ่งร้อยห้า" = 105 (TC-61)
5. รู้ได้ยังไงว่า "ข้าว 2 จาน 100" ไม่ใช่ ๒2	เลขที่ตามด้วยคำหน่วย (จาน แก้ว ช้อน ... 35 คำ) ไม่นับเป็นเงิน จึงเหลือ 100 เป็นราคา และ "2 จาน" อยู่ในชื่อ
6. รู้ได้ยังไงว่าเป็นรายรับ	มีคำรายรับ (ได้เงิน แม่ให้ เงินเดือน ...) และไม่มีคำจ่าย (จ่าย ชื้อ โอนให้) อยู่ก่อนหน้า ถ้ามีทั้งคู่ คำที่มาก่อนชนะ
7. พูดหลายรายการแยกยังงัย	หาจำนวนเงินทุกตัว แล้วให้คำระหว่างเลขเป็นชื่อของเลขถัดไป (หรือคำหลังเลขถ้าประโยคขึ้นต้นด้วยเลข)
8. "เมื่อวาน" ทำงานยังงัย	แทนด้วยป้าย ฐานรู้ ในข้อความ รายการได้วันของป้ายตัวสุดท้ายก่อนราคาของมัน ไม่มีเลขใช้ป้ายแรก ตอนบันทึก วันจริง = วันนี้ + dayOffset
9. ถ้าเครื่องฟังเสียงไม่ได้	speechAvailable เป็น false หน้าจอซ่อนปุ่มไมค์ แสดงคำแนะนำให้พิมพ์หรือใช้ไมค์ของคีย์บอร์ด ทุกอย่างยังทำงาน
10. ทำไมแอป Android รุ่นเก่าไม่ฟัง	speech.ts ใช้ requireOptionalNativeModule ถ้าไม่มีโมดูลได้ null แทนการโยนข้อผิดพลาด
11. ขอสิทธิ์ไมค์เมื่อไร	ตอนแตะไมค์ครั้งแรกเท่านั้น ถ้าผู้ใช้ปิดถาวรจะมีปุ่มเปิดหน้าต่างคำขอของเครื่อง
12. รายการจากเสียงต่างจากสลิปยังงัย	เสียง = ยืนยันแล้วทันที (ผู้ใช้เห็นและกดเอง) · สลิป = แบบร่างจนกว่าผู้ใช้ยืนยัน (AI อ่าน)
13. ถ้าบันทึก 3 รายการแล้วเน็ตหลุดที่รายการที่ 2	รายการแรกถูกตัดออกแล้ว toast แจ้งบอกให้ลองใหม่ รายการที่เหลือยังอยู่ กดใหม่ไม่บันทึกซ้ำ (TC-80)
14. ทดสอบเสียงในเบราว์เซอร์ที่ไม่มีไมค์ยังงัย	E2E ใส่ FakeRecognition แทนตัวฟังของเบราว์เซอร์ ส่งข้อความกำหนดไว้ แล้วตรวจรายการ ภาษา th-TH และยอดเงิน
15. ทำไม "ห้า" ไม่กลายเป็น 5	thaiNumberWords ต้องประกอบด้วยคำตัวเลขทั้งหมด "ห้า" มี "ง" เกิน → null และตอนแปลงในประโยค ต้องชัดว่าเป็นตัวเลข (เดี่ยว ตามด้วยบาท หรือหลายส่วนจบคำ)
16. เก็บเสียงไว้ไหม	ไม่ เก็บเฉพาะรายการที่กดบันทึก (เขียนไว้ท้ายหน้าจอ)

แบบฝึกหัดเชิงความเข้าใจ

ทุกข้อตรวจคำตอบด้วยการรันโค้ดจริงแล้ว

1. thaiNumberWords ของ "ร้อยแปด", "หนึ่งร้อยแปด", "สามพันสอง", "ห้าร้อยเจ็ด", "หมื่นห้า" ได้เท่าไร
2. "กาแฟ 45 ขนมหึง 30 บาท" ได้รายการอะไรบ้าง
3. "ค่าเสีย 1711" ได้รายการอะไร ทำไมไม่เป็นเซเว่น
4. "เมื่อวานขึ้น ค่ารถ 40" ได้ dayOffset เท่าไร
5. "แม่ให้เงินซื้อขนม 100" เป็นรายรับหรือรายจ่าย เพราะอะไร
6. "ซื้อหนังสือ สามร้อยห้าสิบ บาท" บรรทัดสีทองแสดงว่าอะไร และได้รายการอะไร
7. "ค่าน้ำ 120 น้ำ 15" สองรายการได้หมวดอะไร
8. "เบียร์ 3 ขวด 270" กับ "เบียร์ 3 ขวดใหญ่ 270" ต่างกันยังไง

เฉลย

ข้อ	คำตอบ
1	180 (ย่อ) · 108 (เต็ม) · 3,200 (ย่อ หลังพัน) · 501 (เจ็ดเป็นหลักหน่วยเสมอ) · 15,000 (ย่อ หลังหมื่น)
2	2 รายการ: กาแฟ ฿45 อาหาร · ขนมหึง ฿30 อาหาร
3	ค่าเสีย ฿1,711 ซอปปิง — กฎเซเว่นต้องไม่มีเลขนำหน้า "711" (ที่นี่มี 1 นำหน้า) และ "711" ต้องตามด้วยราคาหรือจบประโยค
4	-2 (ค่ารถ ฿40 เดินทาง)
5	รายรับ (เงินจากที่บ้าน) เพราะคำรายรับ "แม่ให้" อยู่ตำแหน่ง 0 มาก่อนคำจ่าย "ซื้อ" ที่ตำแหน่ง 10
6	"ซื้อหนังสือ 350 บาท" → หนังสือ ฿350 การเรียน (ตัด "ซื้อ" ออกจากซื้อ)
7	ค่าน้ำ = บิล (ค่ายาวกว่าเทียบก่อน) · น้ำ = อาหาร
8	"3 ขวด" → [เบียร์ 3 ขวด ฿270] · "3 ขวดใหญ่" → "ขวด" มีคำต่อจึงไม่ใช่หน่วย → [เบียร์ ฿3] [ขวดใหญ่ ฿270] (ข้อจำกัด ผู้ใช้ กด × เอาออกได้)

สคริปต์นำเสนอ 1 นาที และคำศัพท์

อ่านออกเสียงได้เลย

“ สคริปต์ 1 นาที

"FR-3 คือพุดจด ผู้ใช้แตะไมค์สีกองบนหน้าหลักแล้วพุด เช่น คำรถ 25 กาแฟ 65 ได้เงินจากแม่ 500 ระบบของมือถือหรือเบราวเซอร์แปลงเสียงเป็นข้อความภาษาไทย แล้วฟังก์ชัน parseSpokenEntry ใน voice.ts แยกเป็น 3 รายการให้ทันที

มันเข้าใจเลขคำไทยแบบที่คนพุด เช่น ร้อยห้าคือ 150 สองพันห้าคือ 2,500 รู้ว่า ข้าว 2 จาน เลข 2 เป็นจำนวนไม่ใช่เงิน รู้ว่าได้เงินจากแม่เป็นรายรับ และเมื่อวานให้ลงวันเมื่อวาน ทั้งหมดเป็นกฎที่เขียนเอง ไม่ใช่ AI จึงเร็วและทดสอบได้กว่า 60 ประโยค

ผู้ใช้เห็นทุกรายการก่อนบันทึก สลับรายรับรายจ่ายหรือเอาออกได้ แล้วกดบันทึกครั้งเดียว รายการเข้าไปที่เดียวกับ FR-1 ยอดเงินและ Money Runway อัปเดตทันที

เครื่องที่ฟังเสียงไม่ได้ก็พิมพ์แบบที่พุดได้ แอปไม่ฟัง และเราไม่เก็บเสียง เก็บแค่รายการที่บันทึก ทดสอบด้วย TC-59 ถึง 61, TC-80 และ E2E ที่ใช้ตัวฟังเสียงจำลอง"

คำศัพท์ของ FR-3

คำ	ความหมาย
Speech-to-text	การแปลงเสียงพุดเป็นข้อความ
Web Speech API	ความสามารถของเบราวเซอร์ในการฟังเสียง (SpeechRecognition)
expo-speech-recognition	ไลบรารีที่เชื่อมต่อแปลงเสียงของ Android/iOS กับแอป Expo
native module	โค้ดภาษาของมือถือ (Kotlin/Swift) ที่ต้อง build ลงแอป
OTA update	อัปเดตแบบส่งเฉพาะ JavaScript ผ่านเน็ต (EAS Update) ไม่ต้องติดตั้งแอปใหม่
permission	สิทธิ์ที่แอปต้องขอผู้ใช้ เช่น ไมโครโฟน
interim result	ข้อความระหว่างพุดที่ยังไม่จบประโยค
parse	แยกและตีความข้อความให้เป็นข้อมูลที่มีโครงสร้าง
normalize	ทำข้อความให้เป็นรูปแบบมาตรฐาน (เลขอารบิก ตัดคำลงท้าย)
token	ชิ้นคำที่หั่นออกมา เช่น "สอง" "พัน" "ห้า"
regular expression	รูปแบบข้อความสำหรับค้นหา/แทนที่
lookahead (?= / ?!)	เงื่อนไขของ regex ที่ดูตัวอักษรข้างหน้าโดยไม่กินมัน
dayOffset	วันห่างจากวันนี้ (0, -1, -2)

คำ	ความหมาย
VOICE_NOTE	บันทึก "จดด้วยเสียง" ที่ติดกับรายการจากเสียง
FakeRecognition	ตัวฟังเสียงจำลองที่ใช้ในเทสต์ E2E